

Companion to Bird & de Moor, Algebra of Programming

§1. Programs

§1.1. Datatypes

§1.2. Natural numbers

§1.3. Lists

§1.4. Trees

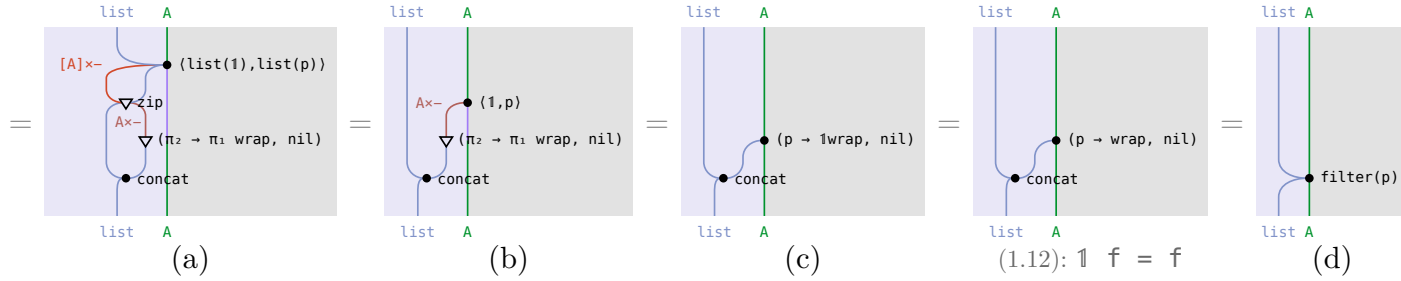
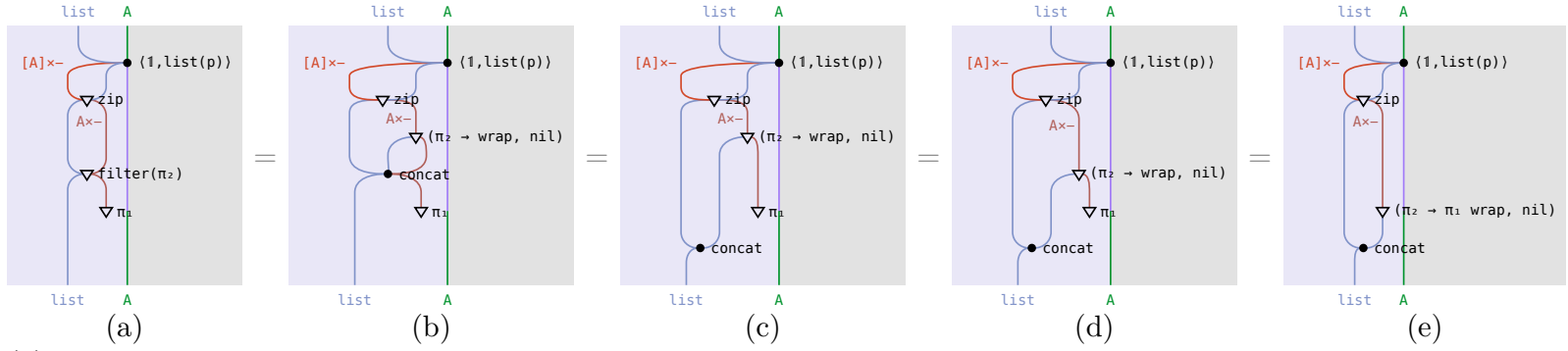
§1.5. Inverses

§1.6. Polymorphic functions

§1.7. Pointwise and point-free

$$\langle 1, \text{list}(p) \rangle \text{zip filter}(\pi_2) \text{list}(\pi_1) = \text{filter}(p) \quad (1.7a)$$

pairing each element with its answer under p ($\langle 1, \text{list}(p) \rangle \text{zip}$), keeping the pairs answered **true** ($\text{filter}(\pi_2)$) and dropping the answers ($\text{list}(\pi_1)$) is $\text{filter}(p)$



§2. Functions and Categories

§2.1. Categories

§2.2. Functors

§2.3. Natural transformations

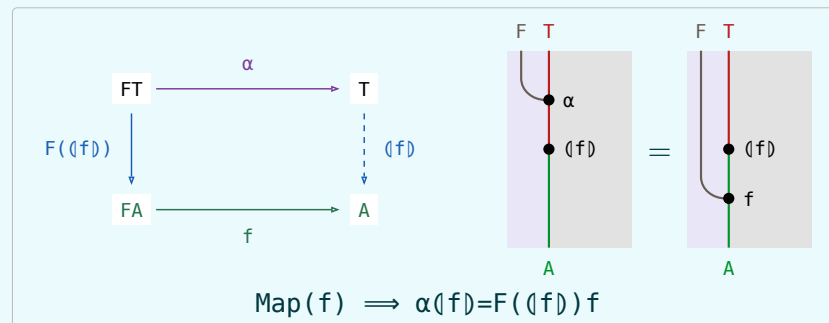
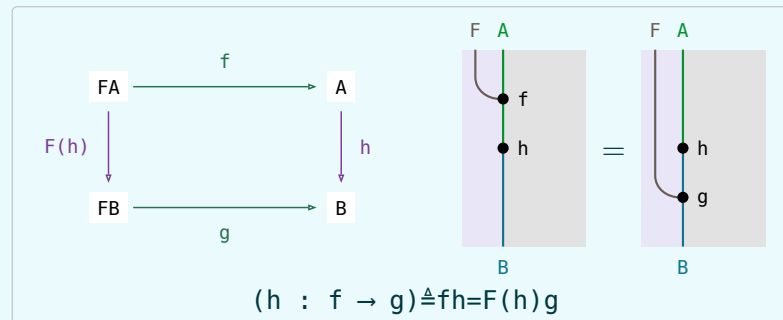
§2.4. Constructing datatypes

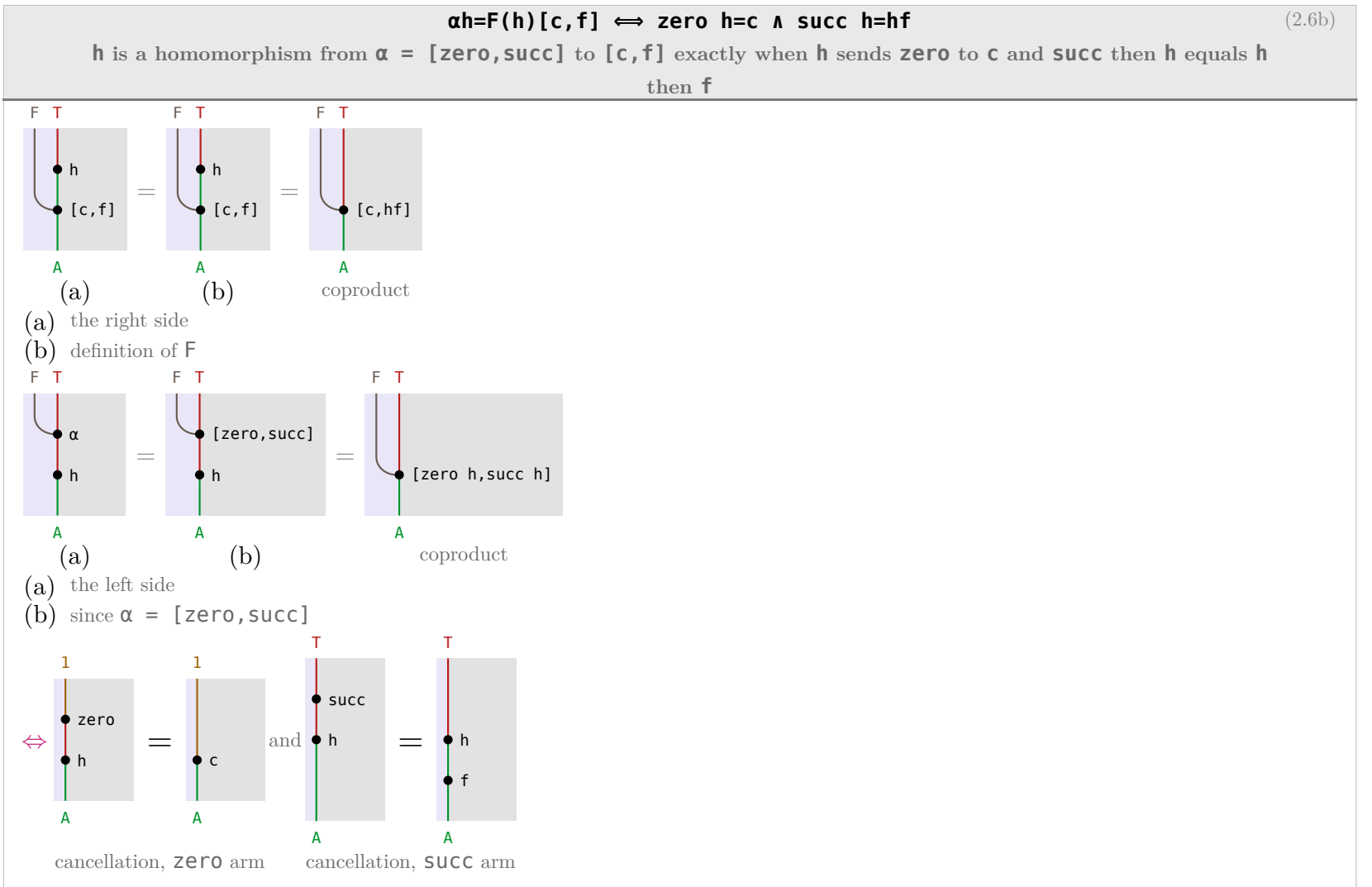
§2.5. Products and coproducts

§2.6. Initial algebras

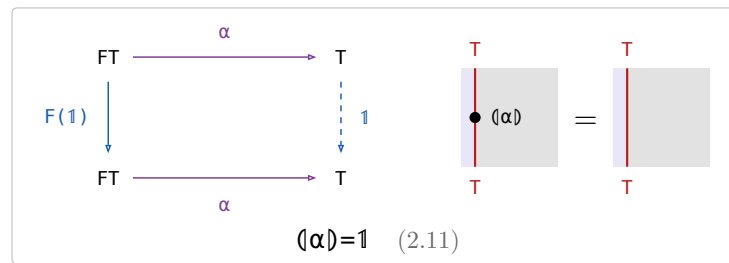
(2.6a)

An **F-algebra** is a map $f : F(A) \rightarrow A$; A is its **carrier**. An **F-homomorphism** from $f : F(A) \rightarrow A$ to $g : F(B) \rightarrow B$ is a map $h : A \rightarrow B$ with $f \circ h = F(h) \circ g$. The **initial algebra** $\alpha : F(T) \rightarrow T$ is the F-algebra with exactly one F-homomorphism $\llbracket f \rrbracket : T \rightarrow A$ to every F-algebra f .





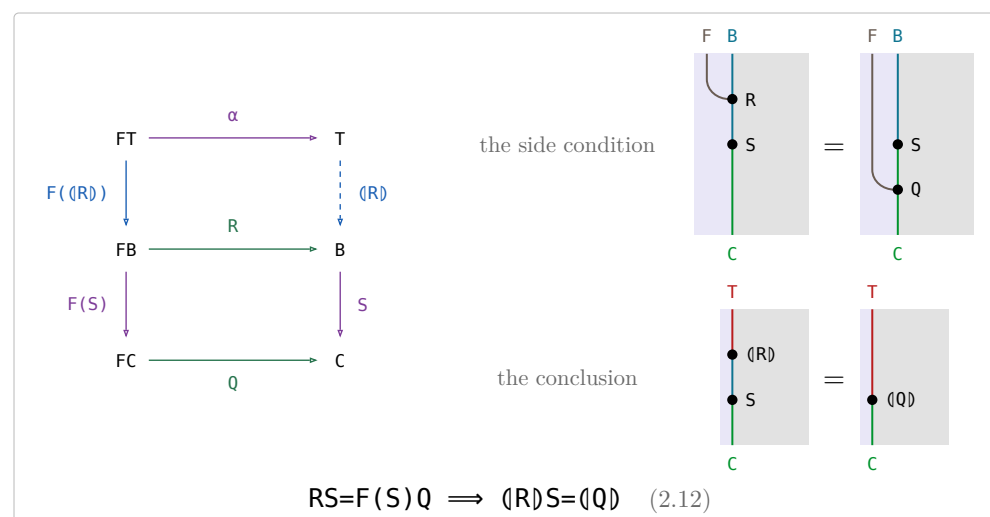
§2.6.1. Reflection



Taking a value apart with α and putting it straight back is doing nothing.

§2.6.2. Fusion: $(R:FB)$ absorb $S:R \rightarrow Q$ (in $\text{Alg}(F)$) and becomes (Q)

When $S : B \rightarrow C$ is an F -homomorphism from $R : F(B) \rightarrow B$ to $Q : F(C) \rightarrow C$, folding with R and then applying S is folding with Q .



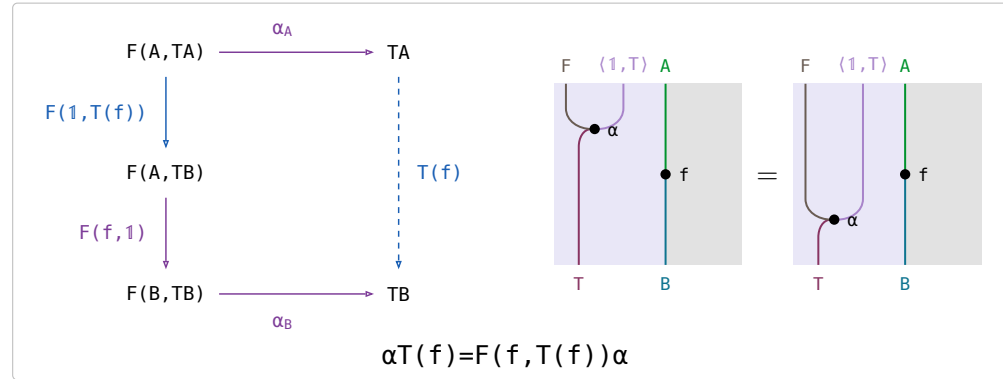
§2.7. Type functors

(2.7a)

Let F be a bifunctor taking both the parameter A and the recursive position TA , with an initial algebra $\alpha_A : F(A, TA) \rightarrow TA$ for every object A . Then T is a functor, acting on a map $f : A \rightarrow B$ by

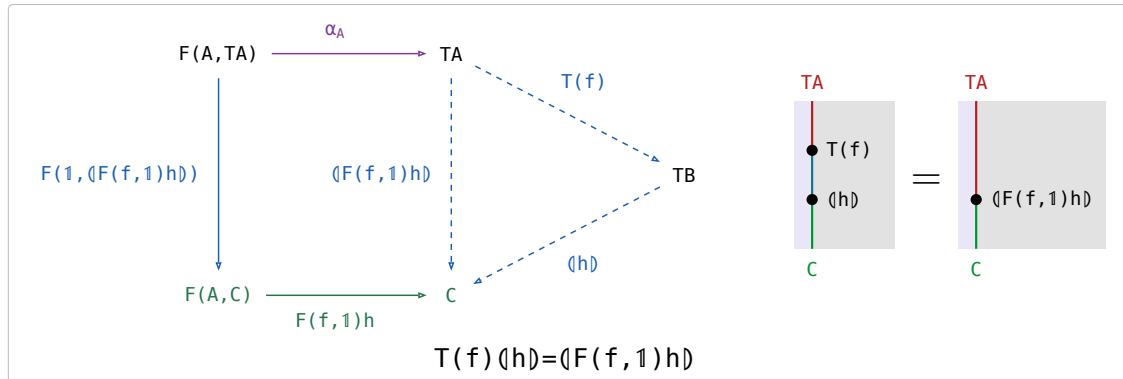
$$T(R) \triangleq (F(R, 1) \alpha)$$

(2.7b)



- $F : \mathcal{A} \times \mathcal{A} \rightarrow \mathcal{A}$ is a bifunctor and a wire is a unary functor, so the two arguments are packed first: $\langle 1, T \rangle : \mathcal{A} \rightarrow \mathcal{A} \times \mathcal{A}$ sends A to (A, TA) , and $F(\langle 1, T \rangle(A))$ is $F(A, TA)$.
- The picture is three wires — F , $\langle 1, T \rangle$, and the object — and the region between the first two is $\mathcal{A} \times \mathcal{A}$.
- α is then an ordinary natural transformation $F \circ \langle 1, T \rangle \Rightarrow T$: its bead eats the F and $\langle 1, T \rangle$ wires, and the T wire is born under it.
- $F(f, T(f))$ costs no notation. It is the bead f on the object wire with $\langle 1, T \rangle$ and F running past: $\langle 1, T \rangle$ is what turns f into the pair $(f, T(f))$, and F is what applies it.
- The law is the naturality of α , which is exactly the freedom to slide that f bead past it.

(2.7c)



(2.7d)

name	law	what it says
functor	$T(1) = 1$ and $T(R)T(S) = T(RS)$	Acting by the identity changes nothing, and two actions in a row are one action.
type functor fusion	$T(f)(fh) = (F(f, 1)h)$	A relator action followed by a fold is a single fold — the intermediate structure is never built. The side condition holds because F is a bifunctor — $F(R, 1)F(1, (Q)) = F(R, (Q)) = F(1, (Q))F(R, 1)$.
naturality of α	$\alpha T(f) = F(f, T(f)) \alpha$	Building and then mapping is the same as mapping the parts and then building, so α is natural from $G(R) = F(R, T(R))$ to T .

§3. Applications

§3.1. Banana-split

§3.1.1. Laws from Relation Algebra

§3.1.1.1. Adjunctions

(3.1.1.1a)

$F \dashv G$	F 's type	monad FG	$1 \in FG$	$GF \in 1$	F preserves $\sqcup$	G preserves $\cap$	$FGF=F$	$GFG=G$
$\triangleleft \dashv \triangleleft$	$A \rightarrow A \otimes A$	$\triangleleft \triangleright = 1$	$1 \in \triangleleft \triangleright$	$\triangleright \triangleleft \in 1$	$(R \sqcup S) \triangleleft = R \triangleleft \sqcup S \triangleleft$	$(R \cap S) \triangleright = R \triangleright \cap S \triangleright$	$\triangleleft \triangleright \triangleleft = \triangleleft$	$\triangleright \triangleleft \triangleright = \triangleright$
$\multimap \dashv \multimap$	$A \rightarrow I$	$\multimap \multimap = T$	$1 \in \multimap \multimap$	$\multimap \multimap \in 1$	$(R \sqcup S) \multimap = R \multimap \sqcup S \multimap$	$(R \cap S) \multimap = R \multimap \cap S \multimap$	$\multimap \multimap \multimap = \multimap$	$\multimap \multimap \multimap = \multimap$
$\circ \dashv \circ$	$(A \rightarrow B) \rightarrow (B \rightarrow A)$	$R \circ \circ = R$	$R \circ \circ = R$	$R \circ \circ = R$	$(R \sqcup S) \circ = S \circ \sqcup R \circ$	$(R \cap S) \circ = R \circ \cap S \circ$	$R \circ \circ = R$	$R \circ \circ = R$
$\multimap \triangleleft \dashv \triangleright \multimap$	$(X \otimes A \rightarrow Y) \rightarrow (X \rightarrow Y \otimes A)$	1	$(\multimap \triangleleft \otimes 1) = 1$ $(1 \otimes \triangleright \multimap) = 1$	$(1 \otimes \multimap \triangleleft) = 1$ $(\triangleright \multimap \otimes 1) = 1$	$=$	$=$	$=$	$=$
$\triangleright \multimap \dashv \multimap \triangleleft$	$(X \rightarrow Y \otimes A) \rightarrow (X \otimes A \rightarrow Y)$	1	$(1 \otimes \multimap \triangleleft) = 1$ $(\triangleright \multimap \otimes 1) = 1$	$(\multimap \triangleleft \otimes 1) = 1$ $(1 \otimes \triangleright \multimap) = 1$	$=$	$=$	$=$	$=$
$\Delta \dashv \cap$	$(A \rightarrow B) \rightarrow (A \rightarrow B)^2$	$R \cap R = R$	$R \cap R = R$	$R \cap S \in R$	$\Delta(R \sqcup S) = \Delta R \sqcup \Delta S$	$(R \cap T) \cap (S \cap U) = (R \cap S) \cap (T \cap U)$	$R \cap R = R$	$R \cap R = R$
$\sqcup \dashv \Delta$	$(A \rightarrow B)^2 \rightarrow (A \rightarrow B)$	$(R, S) \mapsto (R \sqcup S, R \sqcup S)$	$R \in R \sqcup S$	$R \sqcup R = R$	$(R \sqcup T) \sqcup (S \sqcup U) = (R \sqcup S) \sqcup (T \sqcup U)$	$\Delta(R \cap S) = \Delta R \cap \Delta S$	$R \sqcup R = R$	$R \sqcup R = R$
$\emptyset \dashv !$	$\{*\} \rightarrow (A \rightarrow B)$			$\emptyset \in R$				
$\cdot S \dashv /S$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	S/S	$R \in (RS)/S$	$(T/S) S \in T$	$(S \sqcup T)R = SR \sqcup TR$	$(R \cap R')/S = (R/S) \cap (R'/S)$	$((RS)/S)S = RS$	$((T/S)S)/S = T/S$
$S \cdot \dashv S \setminus$	$(B \rightarrow C) \rightarrow (A \rightarrow C)$	$S \setminus S$	$R \in S \setminus (SR)$	$S(S \setminus T) \in T$	$R(S \sqcup T) = RS \sqcup RT$	$S \setminus (R \cap R') = (S \setminus R) \cap (S \setminus R')$	$S(S \setminus (SR)) = SR$	$S \setminus (S(S \setminus T)) = S \setminus T$
$R \cap \dashv R \Rightarrow$	$(A \rightarrow B) \rightarrow (A \rightarrow B)$	$X \mapsto R \Rightarrow (X \cap R)$	$X \in R \Rightarrow (X \cap R)$	$R \cap (R \Rightarrow Y) \in Y$	$R \cap (X \sqcup Y) = (R \cap X) \sqcup (R \cap Y)$	$R \Rightarrow S \cap T = (R \Rightarrow S) \cap (R \Rightarrow T)$	$R \cap (R \Rightarrow (X \cap R)) = X \cap R$	$R \Rightarrow (R \cap (R \Rightarrow Y)) = R \Rightarrow Y$
$\mathcal{D} \dashv \cdot T$	$(A \rightarrow B) \rightarrow \text{Cor } A$	$R \mapsto (\mathcal{D}R)T$	$R \in (\mathcal{D}R)T$	$\mathcal{D}(AT) \in A$	$\mathcal{D}(R \sqcup S) = \mathcal{D}R \sqcup \mathcal{D}S$	$(A \cap B)T = AT \cap BT$	$\mathcal{D}((\mathcal{D}R)T) = \mathcal{D}R$	$(\mathcal{D}(AT))T = AT$
$\mathcal{R} \dashv T \cdot$	$(A \rightarrow B) \rightarrow \text{Cor } B$	$R \mapsto T(\mathcal{R}R)$	$R \in T(\mathcal{R}R)$	$\mathcal{R}(TA) \in A$	$\mathcal{R}(R \sqcup S) = \mathcal{R}R \sqcup \mathcal{R}S$	$T(A \cap B) = TA \cap TB$	$\mathcal{R}(T(\mathcal{R}R)) = \mathcal{R}R$	$T(\mathcal{R}(TA)) = TA$
$\cdot f \dashv \cdot f^\circ$	$(A \rightarrow B) \rightarrow (A \rightarrow C)$	$f f^\circ$	$\text{Map}(f) \Rightarrow 1 \in f f^\circ$	$f^\circ f \in 1$	$(S \sqcup T)R = SR \sqcup TR$	$\text{Simple}(g) \Rightarrow (X \cap Y)g^\circ = Xg^\circ \cap Yg^\circ$	$f f^\circ f = f$	$f^\circ f f^\circ = f^\circ$
$f^\circ \cdot \dashv f \cdot$	$(A \rightarrow C) \rightarrow (B \rightarrow C)$	$f f^\circ$	$\text{Map}(f) \Rightarrow 1 \in f f^\circ$	$f^\circ f \in 1$	$R(S \sqcup T) = RS \sqcup RT$	$f(X \cap Y) = fX \cap fY$	$f f^\circ f = f$	$f^\circ f f^\circ = f^\circ$
$i \dashv E$	$\text{Map} \dashv \text{Rel}$	E	$\frac{1}{\exists} : A \rightarrow EA$	$\exists : EB \rightarrow B$	—	—	$\frac{\exists}{\exists} = 1$	$\frac{R}{\exists} \exists = R$
$\cdot \exists \dashv \frac{\exists}{\exists}$	$\text{Map}(A, EB) \rightarrow (A \rightarrow B)$	1	$\frac{f \exists}{\exists} = f$	$\frac{R \exists}{\exists} = R$	—	—	$\frac{f \exists}{\exists} = f \exists$	$\frac{R \exists}{\exists} = R \exists$
$\frac{\exists}{\exists} \dashv \cdot \exists$	$(A \rightarrow B) \rightarrow \text{Map}(A, EB)$	1	$\frac{R \exists}{\exists} = R$	$\frac{f \exists}{\exists} = f$	—	—	$\frac{R \exists}{\exists} = R \exists$	$\frac{f \exists}{\exists} = f \exists$

§3.1.1.2. Composing adjunctions

(3.1.1.2a)

	$\cdot T$	$T \cdot$	$\cdot g$	$g \cdot$	Tn	$\circ$	$\circ \triangleleft$	Δ
$\cdot S$	$R/S_1 \ S_2 = R/S_2/S_1$	$S \setminus (R/W) = (S \setminus R)/W$	$R/(Sg) = (Rg^\circ)/S$	$\text{Map}(f) \Rightarrow f(R/S) = fR/S$	—	$(Y/S)^\circ = S^\circ \setminus (Y^\circ)$	$(RS)^\wedge = R^\wedge (S \otimes 1)$	$(RnR')/S = (R/S) \cap (R'/S)$
$S \cdot$	$S \setminus (R/W) = (S \setminus R)/W$	$S_1 \ S_2 \setminus R = S_2 \setminus (S_1 \setminus R)$	$S \setminus (Rg^\circ) = (S \setminus R)g^\circ$	$(g^\circ S) \setminus R = S \setminus (gR)$	—	$(S \setminus Y)^\circ = Y^\circ/S^\circ$	$((S \otimes 1)R)^\wedge = SR^\wedge$	$S \setminus (RnR') = (S \setminus R) \cap (S \setminus R')$
$\cdot f$	$\text{Map}(f) \Rightarrow R/fS = (R/S)f^\circ$	$T \setminus (Rf^\circ) = (T \setminus R)f^\circ$	$(RS)^\circ = S^\circ R^\circ$			$(RS)^\circ = S^\circ R^\circ$		$\text{Simple}(g) \Rightarrow (XnY)g^\circ = Xg^\circ nYg^\circ$
$f \cdot$	$\text{Map}(f) \Rightarrow f(R/S) = fR/S$	$(Tf^\circ) \setminus R = f(T \setminus R)$	—	$(RS)^\circ = S^\circ R^\circ$		$(RS)^\circ = S^\circ R^\circ$		$f(T_1 n T_2) = fT_1 n fT_2$
Rn	—	—	—	—	$R \Rightarrow S \Rightarrow T = RnS \Rightarrow T$	$(R \Rightarrow Y)^\circ = R^\circ \Rightarrow (Y^\circ)$	—	$R \Rightarrow SnT = (R \Rightarrow S) \cap (R \Rightarrow T)$
$\circ$	$(Y/T)^\circ = T^\circ \setminus (Y^\circ)$	$(T \setminus Y)^\circ = Y^\circ/T^\circ$	$(RS)^\circ = S^\circ R^\circ$	$(RS)^\circ = S^\circ R^\circ$	$(T \Rightarrow Y)^\circ = T^\circ \Rightarrow (Y^\circ)$	$R^\circ \circ = R$	$(R^\wedge)^\circ = (R^\circ \otimes 1)$ $(1 \otimes \triangleright \dashv \circ)$ both bends: the $\circ$ section's display	$(T_1 n T_2)^\circ = T_1^\circ n T_2^\circ$
$\circ \triangleleft$	$(RT)^\wedge = R^\wedge (T \otimes 1)$	$((T \otimes 1)R)^\wedge = TR^\wedge$	—	—	—	$(R^\wedge)^\circ = (R^\circ \otimes 1)$ $(1 \otimes \triangleright \dashv \circ)$ both bends: the $\circ$ section's display	—	—
u	$(S \cup T)R = SR \cup TR$	$R(S \cup T) = RS \cup RT$	$(S \cup T)R = SR \cup TR$	$R(S \cup T) = RS \cup RT$	$Rn(S \cup T) = RnS \cup RnT$	$(R \cup S)^\circ = S^\circ \cup R^\circ$		
$\emptyset$	$\emptyset R = \emptyset$	$R \emptyset = \emptyset$	$\emptyset R = \emptyset$	$R \emptyset = \emptyset$	$Rn \emptyset = \emptyset$	$\emptyset^\circ = \emptyset$		

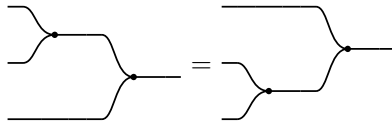
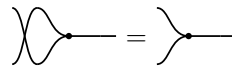
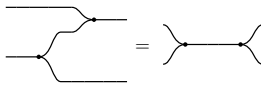
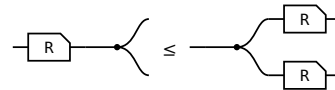
§3.1.1.3. Adjoint triples

(3.1.1.3a)

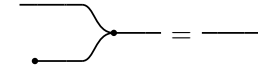
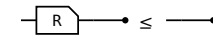
$$\begin{aligned} \cdot f \dashv \cdot f^\circ \dashv / f^\circ \\ f^\circ \cdot \dashv f \cdot \dashv f \setminus \end{aligned}$$

§3.1.1.4. Copy and merge

(3.1.1.4a)

 $\triangleright$ associative $\triangleright$ commutative

$$R \triangleleft \leq \triangleleft (R \otimes R)$$

 $\circ$ is its unit

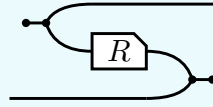
$$R \circ \leq \circ$$

Frobenius, one half — the other is its $\circ$ §3.1.1.5. $\circ$ is a contravariant 2-functor

(3.1.1.2a)

(3.1.1.5a)

$\circ$ is primitive, part of the data the first section lists. It turns both of R 's wires round, and the Frobenius structure **DRAWS** that — the picture and the formula below are R° , not its definition:



$$R^\circ = (\circ \triangleleft \otimes 1) (1 \otimes R \otimes 1) (1 \otimes \triangleright \multimap \circ)$$

where $\circ \triangleleft : \mathbb{I} \rightarrow a \otimes a$ opens a pair of wires out of nothing and $\triangleright \circ : b \otimes b \rightarrow \mathbb{I}$ closes one, so the input of R° is where the output of R was. Taking that formula as the definition would now be circular: $\triangleleft$ and $\multimap$ are $\triangleright^\circ$ and $\circ^\circ$. The laws of $\circ$ are that it is a contravariant 2-functor $\circ : \mathcal{C}^{\circ p} \rightarrow \mathcal{C}$:

- (i) $1^\circ = 1$ (ii) $(RS)^\circ = S^\circ R^\circ$ (iii) $(R \otimes S)^\circ = R^\circ \otimes S^\circ$ (iv) $R \leq S$ implies $R^\circ \leq S^\circ$

§3.1.1.6. Meet and composition

(3.1.1.6a)

semi-distributivity, and what supplies it	picture
$R (S \cap T) \sqsubseteq RS \cap RT$ — the lax copy law. Equality exactly when R is single valued: the Maps section's $F(R \cap S) = FR \cap FS$.	

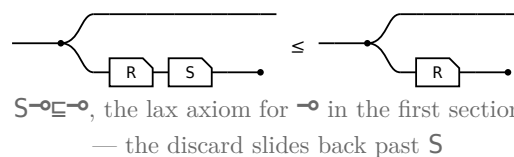
§3.1.1.7. Domain and range

(3.1.1.7a)

$\text{dom}(R) \triangleq 1 \cap R R^\circ$
$\text{Coreflexive}(X) \implies \text{dom}(R) \sqsubseteq X \iff R \sqsubseteq X R$
$\text{dom}(RS) \sqsubseteq \text{dom}(R)$
$\text{dom}(R \cap S) = 1 \cap S R^\circ$
$R \text{ entire} \iff \text{Entire}(R) \triangleq \text{dom}(R) = 1$
$R \text{ simple} \iff \text{Simple}(R) \triangleq R^\circ R \sqsubseteq 1$
$R \text{ a map} \iff \text{Map}(R) \triangleq \text{Entire}(R) \wedge \text{Simple}(R)$
$\text{Entire}(R) \wedge \text{Entire}(S) \implies \text{Entire}(RS)$ $\text{Simple}(f) \wedge \text{Simple}(g) \implies \text{Simple}(fg)$ $\text{Map}(f) \wedge \text{Map}(g) \implies \text{Map}(fg)$
$\text{Entire}(RS) \implies \text{Entire}(R)$

§3.1.1.8. Sliding the discard

(3.1.1.8a)



§3.1.1.9. Division

(3.1.1.5a)

$X \in R/S \iff XS \in R$ X is any x -to- y pairing; one that only pairs x with a y such that x admires everyone y hates lies inside R/S , and R/S is the largest such.	
$X \in S \setminus R \iff SX \in R$ The mirror — divide on the left when x comes first.	
$(R/S)S \in R$ There is a y such that x admires everyone y hates, and p is one of the people y hates — then x admires p too. Strict at $S=\emptyset$: R/S is everyone, $(R/S)S=\emptyset$.	
$S \setminus (S \setminus R) \in R$ The mirror.	
associate: $R/(S_1 S_2) = (R/S_2)/S_1$ A friend's enemy is two hops: divide by the far end first.	
$(S_1 S_2) \setminus R = S_2 \setminus (S_1 \setminus R)$ The mirror.	
maps: $f \setminus (R/S) = (fR)/S$ Rename x before or after dividing — the licence to write fR/S .	
$R/(fS) = (R/S) f^\circ$ Rename y : a map leaves a denominator as f° outside the box.	
$(R/S)(S/W) \in R/W$ Someone who admires all of a hate-set that already covers everyone z works for admires those people too.	
$1 \in R/R$ R/R runs admirer to admirer: each admires everyone they admire. Strict: two people who each admire only a and b admire each other's idols too, and still stay two people.	
$(R/R)(R/R) = R/R$ R/R is the preorder admires at least as much as , and a preorder is idempotent. Freyd writes $\sqsubseteq$; with $1 \in R/R$ above it is an equality.	
$(R/R)R = R$ Reaching p through someone whose idols x fully admires is reaching p directly, since x admires their own idols.	
$R/1 = R$ Dividing by 1 : p 's set is just $\{p\}$, so admiring all of it is admiring p .	
$R/(S_1 \cup S_2) = R/S_1 \cap R/S_2$ Admiring a combined hate-set is admiring each set in full.	
$S \setminus (R/W) = (S \setminus R)/W$ Which is why $S \setminus R/W$ needs no bracket.	

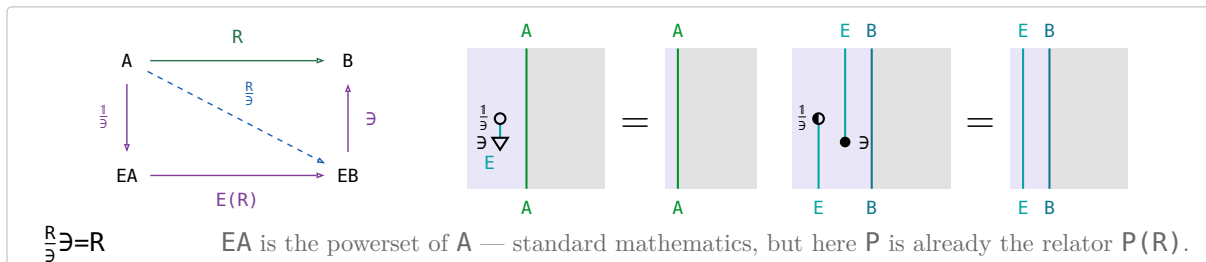
§3.1.1.10. Power allegories

(3.1.1.10a)

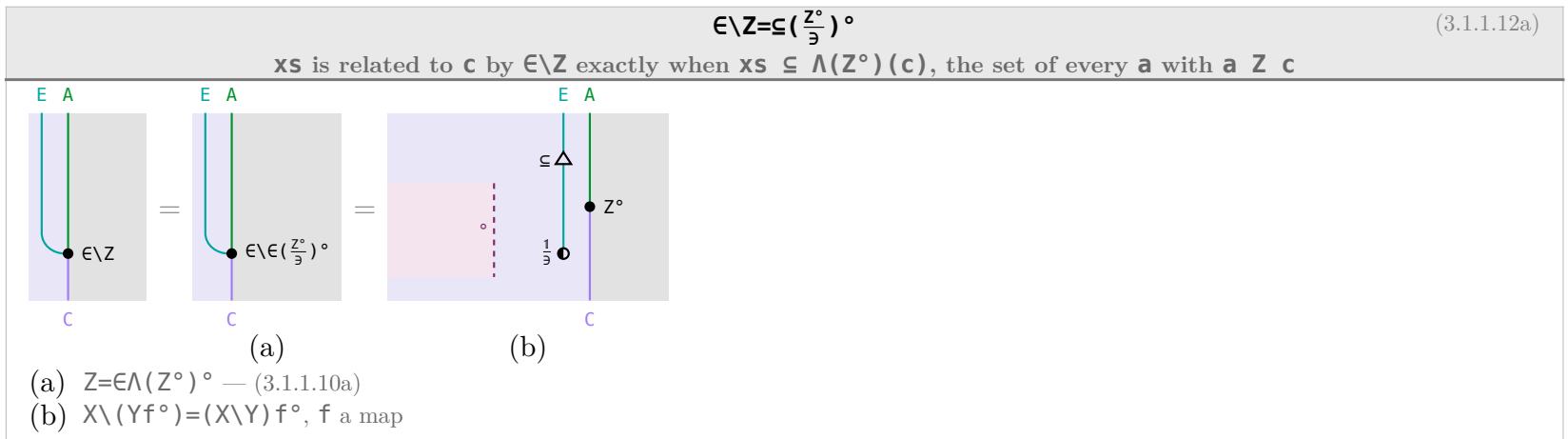
law	the reading
1 $\exists_R \square = R \square, \exists_R = \exists_R \square$	One $\exists$ per object, not per arrow.
2 $\exists$ is thick	Comprehension: every x has a set of exactly the people x admires.
3 $\frac{R}{\exists} : A \rightarrow EB$, for $R : A \rightarrow B$	convert a relation to a function. $a \frac{R}{\exists} = \{b \mid a R b\}$
4 $\text{Map}(\frac{R}{\exists})$	
5 $\frac{R}{\exists} \exists = R$	reading the chosen set back through $\exists$ returns the relation.
6 $\text{Simple}(F) \Rightarrow F \sqsubseteq \frac{F}{\exists}$	A partial choice of sets is inside the total one.
7 the singleton map is monic: $\frac{1}{\exists}$ $(\frac{1}{\exists})^\circ \sqsubseteq 1$ two points with the same one-person set are the same point.	The one-person set.
8 $\frac{\exists}{\exists} = 1$	Make the set of a set, then read it back one level down.
9 fusion: $\text{Map}(f) \Rightarrow \frac{fR}{\exists} = f \frac{R}{\exists}$	Naturality of the unit, $\text{Map}(f) \Rightarrow f \frac{1}{\exists} = \frac{1}{\exists} E(f)$. renaming a point and then taking its one-person set is taking the set and renaming inside it.
10 $\text{Map}(f) \wedge 1 \square = \exists \square \Rightarrow \frac{f}{\exists} = f \frac{1}{\exists}$	Rename first or take singletons first — the fusion row above at $R=1$.
11 $R \square = \exists \square \Rightarrow R / \exists S = \frac{R}{\exists} (\frac{S}{\exists})^\circ$	x and y match when R sends x and S sends y to the same set.
12 $E(R) \triangleq \frac{\exists R}{\exists}$	$E(R) : EA \rightarrow EB$, image of a set of A
13 $\frac{R}{\exists} = \frac{1}{\exists} E(R)$	$\frac{1}{\exists} : x \mapsto \{x\}$
14 $\frac{S}{\exists} E(R) = \frac{SR}{\exists}$	absorption — the monad's composition law, $\frac{S}{\exists} \diamond \frac{R}{\exists} = \frac{SR}{\exists}$, $\mathcal{A} \equiv \text{Kleisli}(E)$ in Relation Algebra
15 $\supseteq \triangleq \exists / \exists$	$xs \supseteq ys \iff \forall a. ys \exists a \rightarrow xs \exists a$
16 $\sqsubseteq \triangleq \supseteq^\circ$	$xs \sqsubseteq ys \iff ys \supseteq xs$

§3.1.1.11. $i \dashv E$: power transpose through singleton

(3.1.1.11a)

§3.1.1.12. $E \setminus$ as a composite

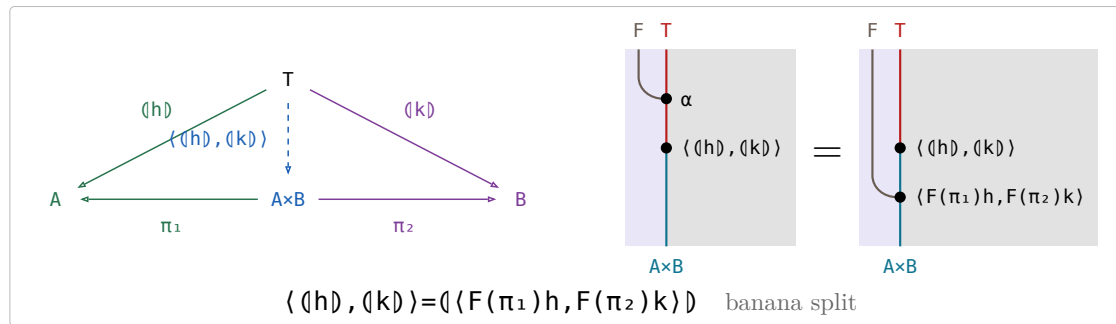
(3.1.1.12a)



(3.1.1.12b)

definition	type	note
$\text{listr } A ::= \text{nil} \mid \text{cons}(A, \text{listr } A)$	$\mathcal{A} \rightarrow \mathcal{A}$	The cons-lists over A , the datatype every row below folds.
$\text{sum} = ([\text{zero}, \text{plus}])$	$[A] \rightarrow A$	$\text{plus}(a, b) = a + b$.
$\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$	$[A] \rightarrow \text{Nat}$	π_2 drops the head and keeps the count of the tail, succ adds one for the head.
$\text{average} \triangleq (\text{sum}, \text{length}) \text{div}$	$[\text{Nat}] \rightarrow \text{Nat}$	$\text{div}(m, n) = m/n$, with $\text{div}(0, 0) = 0$ so average is total. Traverses the list twice.
banana-split law $\langle \langle h \rangle, \langle k \rangle \rangle = \langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$	$T \rightarrow A \times B$	Any fork of folds is a single fold, hence one traversal — F the base functor.
what it reduces to $\alpha \langle \langle h \rangle, \langle k \rangle \rangle = F(\langle \langle h \rangle, \langle k \rangle \rangle) \langle F(\pi_1)h, F(\pi_2)k \rangle$	$FT \rightarrow A \times B$	All that $\triangleq$ reduce leaves to check: the fork satisfies the defining equation.
the instance $(\text{sum}, \text{length}) = ([\text{zeros}, \text{pluss}])$	$[\text{Nat}] \rightarrow \text{Nat} \times \text{Nat}$	$\text{pluss}(a, (b, n)) = (a + b, n + 1)$, so average runs in one pass.
$\text{preds } n = [n, n-1, \dots, 1]$	$\text{Nat} \rightarrow [\text{Nat}]$	apply the earlier special case to write preds as $\langle k \rangle \pi_1$.

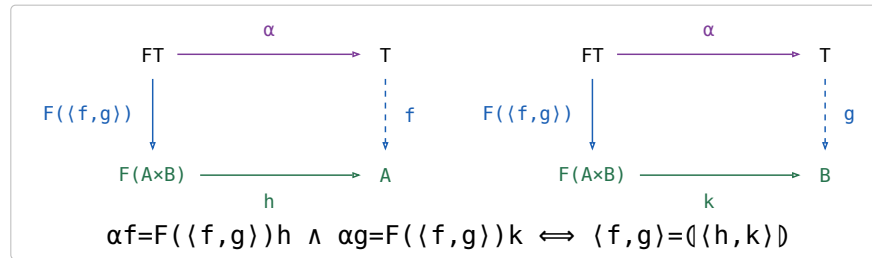
(3.1.1.12c)



§3.1.2. Fokkinga's mutual recursion theorem

- $F\text{-Alg}(\mathcal{A})$ — the algebras and their homomorphisms — has binary products whenever $\mathcal{A}$ does, and the forgetful $U : F\text{-Alg}(\mathcal{A}) \rightarrow \mathcal{A}$ creates them: the product of $h : FA \rightarrow A$ and $k : FB \rightarrow B$ is carried by $A \times B$, with structure $\langle F(\pi_1)h, F(\pi_2)k \rangle : F(A \times B) \rightarrow A \times B$.
- π_1 and π_2 are homomorphisms out of it, and $\langle p, q \rangle : X \rightarrow A \times B$ is a homomorphism **iff** p and q both are — substitute the structure and compare the two forks component by component.
- The banana-split law of (3.1.1.12b) IS that product's universal property read at the initial algebra: $\langle h \rangle$ and $\langle k \rangle$ are the unique homomorphisms to (A, h) and (B, k) , so their fork is the unique homomorphism into the product, which is $\langle \langle F(\pi_1)h, F(\pi_2)k \rangle \rangle$.
- Fokkinga's theorem is **strictly more general** and is not that product: in mutual recursion the two arrows leave $F(A \times B)$, not FA and FB , so each sees BOTH components. The product is the case where they factor as $F(\pi_1)h$ and $F(\pi_2)k$. What it says: **any algebra on a product carrier is folded by a fork**.

(3.1.2a)



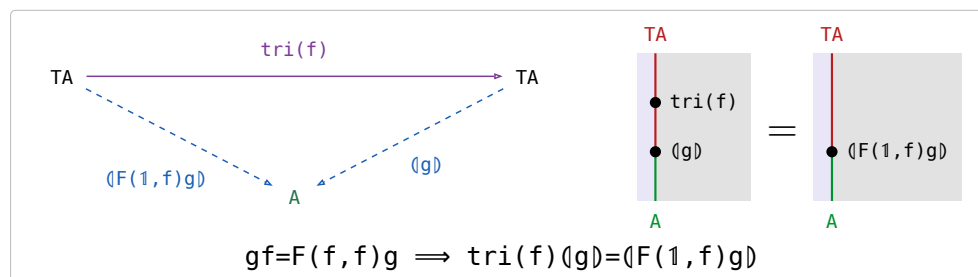
§3.2. Ruby triangles and Horner's rule

(3.2a)

stage	definition
informally	$\text{tri}(f)[a_0, a_1, \dots, a_i, \dots, a_n] = [a_0, f(a_1), \dots, f^i(a_i), \dots, f^n(a_n)]$
for cons-lists	$\text{tri}(f) = \langle [\text{nil}, (1 \times \text{listr}(f)) \text{ cons}] \rangle$
with the base functor named	$F(A, B) = 1 + A \times B$, $\alpha = [\text{nil}, \text{cons}]$, so $\text{tri}(f) = \langle F(1, \text{listr}(f)) \alpha \rangle$
abstractly	F a bifunctor with initial type (α, T) : $\text{tri}(f) = \langle F(1, T(f)) \alpha \rangle$

For the definition to make sense $f : A \rightarrow A$ is required, and then $\text{tri}(f) : TA \rightarrow TA$.

(3.2b)



Horner's rule is Horner's rule, generalised from cons-lists to any initial type. Fusion reduces it to $F(1, T(f))\alpha(g) = F(1, (g))F(1, f)g$.

§3.2.1. Depth of a tree

(3.2.1a)

the datatype	$\text{tree } A ::= \text{tip}(A) \mid \text{bin}(\text{tree } A, \text{tree } A)$, base functor $F(A,B)=A+B \times B$, initial type $([\text{tip}, \text{bin}], \text{tree})$
the fold	$\langle [g, h] \rangle$ is the unique f with $f(\text{tip}(a)) = g(a)$ and $f(\text{bin}(x, y)) = h(f(x), f(y))$
$\text{tree}(f)$	$\text{tree}(f) = \langle F(f, 1) \mid [\text{tip}, \text{bin}] \rangle$; pointwise $\text{tree}(f)(\text{tip}(a)) = \text{tip}(f(a))$ and $\text{tree}(f)(\text{bin}(x, y)) = \text{bin}(\text{tree}(f)(x), \text{tree}(f)(y))$
max	$\text{max} = \langle [1, \text{bmax}] \rangle$, where $\text{bmax}(a, b)$ is the larger of a and b
depths	$\text{depths} = \text{tree}(\text{zero}) \text{ tri}(\text{succ})$ — replaces every tip by its depth in the tree, zero the constant function returning 0 and succ the successor
depth	$\text{depth} = \text{depths } \text{max}$

§3.3. The TeX problem

§3.4. Conditions and conditionals

§3.5. Concatenation and currying

If φ is natural, i.e. $G(h \times id) \cdot \varphi = \varphi \cdot (F(h) \times id)$, then $h \cdot G(f) \cdot \varphi = f \iff f = \text{apply} \cdot ((h \cdot G(\text{apply}) \cdot \varphi) \times id)$ Theorem 3.1
 The unique solution of a two-argument recursion $f(a, x) = h(G(f)(a, x))$ built from a natural φ is **apply** composed with a one-argument fold, which is how **cat** — not itself a fold — is derived from one.

§4. Relations and Allegories

§4.1. Allegories

§4.2. Special properties of arrows

$R:A \rightarrow B$, $S:B \rightarrow A$, $1_A \sqsubseteq RS$, $SR \sqsubseteq 1_B \implies S=R^\circ$ and R is a map

Proposition 4.1

A relation with a two-sided partial inverse under composition — identity below one order, identity above the other — is that inverse's converse, and R is itself a map.

§4.3. Tabular allegories

(f,g) a tabulation of $R \implies k^\circ h \sqsubseteq R \iff \exists! \text{ map } m \text{ with } h=mf, k=mg$

Proposition 4.2

Every pair below R 's tabulating pair factors, uniquely, through a single map into the tabulation's apex.

§4.4. Locally complete allegories

§4.5. Boolean allegories

§4.6. Power allegories

§5. Datatypes in Allegories

§5.1. Relators

(5.1a)

Every hom-set of an allegory is a poset, so an allegory is a **locally posetal 2-category**: the 2-cell from R to S IS $R \subseteq S$. A **relator** $F : \mathcal{C} \rightarrow \mathcal{D}$ is a 2-functor between allegories:

$$F(1) = 1 \quad F(RS) = F(R)F(S) \quad R \subseteq S \implies F(R) \subseteq F(S)$$

Preserving $\circ$ is **not** asked for — $\circ$ is an identity-on-objects involution $\mathcal{C}^{\circ} \rightarrow \mathcal{C}$, no part of the 2-category.

(5.1b)

the statement

$\text{Map}(f) \implies \text{Map}(F(f))$ and $\text{Map}(f) \implies F(f^\circ) = F(f)^\circ$. Lemma 5.1

Over a **tabular** allegory a functor is a relator $\iff$ it preserves $\circ$. Theorem 5.1 the book's reverse direction (preserving $\circ \implies$ relator) needs the source to be **Rel**: the counterexample over a tabular allegory that is not **Rel** shows it can fail

$F(R^\circ) = F(R)^\circ$ for every R , so $F(R)^\circ$ needs no bracket.

Two relators agreeing on maps are equal. Corollary 5.1

$\text{Coreflexive}(X) \wedge \text{Coreflexive}(Y) \implies F(X \cap Y) = F(X) \cap F(Y)$ Ex 5.2

$F(R \cap S) \subseteq F(R) \cap F(S)$, and strictly. Ex 5.2

$\text{PreservesRecip}(F) \implies F(\text{dom}(R)) = \text{dom}(F(R))$ Ex 5.5

The **power relator** $P \multimap xs \ P(R) \ ys \iff (\forall a \in xs. \exists b \in ys. a \ R \ b) \wedge (\forall b \in ys. \exists a \in xs. a \ R \ b)$ — is where the fourth is strict: for $R = \{(a_1, b_1), (a_2, b_2)\}$ and $S = \{(a_1, b_2), (a_2, b_1)\}$ the pair $(\{a_1, a_2\}, \{b_1, b_2\})$ is in $P(R) \cap P(S)$, while $R \cap S = \emptyset$.

§5.2. Relational products

(5.1)

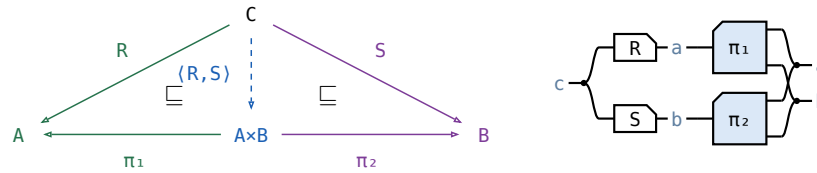
The **fork** of $R : \mathcal{C} \rightarrow \mathcal{A}$ and $S : \mathcal{C} \rightarrow \mathcal{B}$ is $\langle R, S \rangle \triangleq R \pi_1^\circ \cap S \pi_2^\circ$, where (π_1, π_2) is the tabulation of τ .

(5.2a)

$$\langle R, S \rangle \pi_1 = \text{dom}(S) R \quad (5.6)$$

$$\langle R, S \rangle \pi_2 = \text{dom}(R) S \quad (5.7)$$

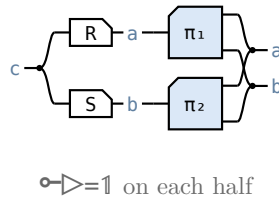
(5.2b)



A domain is coreflexive, so $\langle R, S \rangle \pi_1 \subseteq R$, with equality exactly when S is entire; for maps both triangles commute and $\langle f, g \rangle$ is unique. In **Rel**, $c \in \langle R, S \rangle (a, b)$ iff $c \ R \ a$ and $c \ S \ b$ — copy c , then R on one strand and S on the other, which is $\triangleleft (R \otimes S)$ on the right.

No $\circ$ survives the translation. $\pi_1 = 1 \otimes \bullet$ discards the second component, so $\pi_1^\circ = 1 \otimes \bullet$ **creates** one out of nothing, and $\cap$ is copy, run both, merge. Draw that and the created strands — the two dots with no left end, and the crossing they force — are merged against real ones, which is the monoid's unit law:

(5.2c)

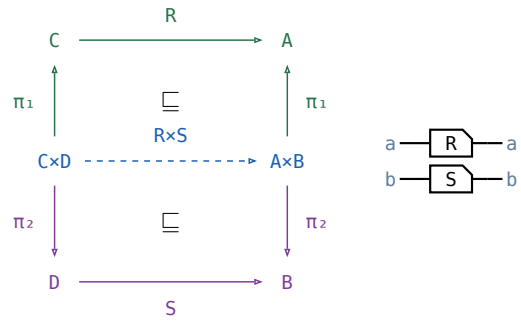


§5.2.1. Relational product $R \times S$

(5.2)

$R \times S \triangleq \langle \pi_1 R, \pi_2 S \rangle$, a relator in each argument but no longer a categorical product.

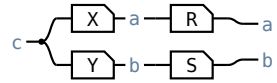
(5.2.1a)



Right-then-up is $(R \times S)\pi_1$, up-then-right is $\pi_1 R$, and $(R \times S)\pi_1 \sqsubseteq \pi_1 R$, equality when S is entire. In $\mathbf{Rel}$, $(c, d) (R \times S) (a, b)$ iff $c R a$ and $d S b$ — two strands side by side, no copy dot: $R \times S = R \otimes S$.

§5.2.2. Absorption

For $X : E \rightarrow C$ and $Y : E \rightarrow D$, $\langle X, Y \rangle (R \times S) = \langle XR, YS \rangle$. Both sides are this picture:



(5.3)

(5.2.2a)

the law	picture
1 $\langle X, Y \rangle (R, S)^\circ = XR^\circ \cap YS^\circ$	
2 $\langle R, S \rangle^\circ (U, V) \sqsubseteq (R^\circ U) \times (S^\circ V)$	
3 $\text{Map}(f) \Rightarrow f \langle R, S \rangle = \langle fR, fS \rangle$ f a map; it fails for an arbitrary arrow;	
4 $F(R \times S) \text{unzip}(F) \sqsubseteq \text{unzip}(F) (F(R) \times F(S))$ $\text{unzip}(F) \triangleq \langle F(\pi_1), F(\pi_2) \rangle$; only $\sqsubseteq$ for an arbitrary R, S	
5 $g = \text{curry}(f) \iff (g \times 1) \text{eval} = f$ reading $\times$ as the relational product, does $\mathbf{Rel}$ have exponentials?	

§5.3. Relational coproducts

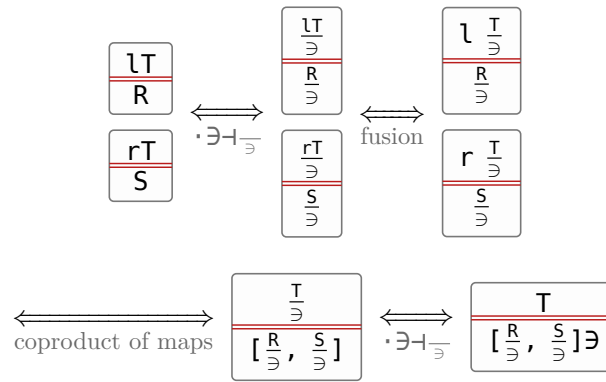
(5.3a)

the statement	picture
$[R, S] \triangleq l \circ R \cup r \circ S$ (5.9) The tape is the union — a particle entering at A+B takes exactly one branch — and the two mirrored boxes are what makes the branches disjoint.	
$[R, S] = [\frac{R}{\exists}, \frac{S}{\exists}] \exists$	
$R + S \triangleq [Rl, Sr]$ (5.10)	
$l[R, S] = R, r[R, S] = S$ $lT = R \wedge rT = S \Rightarrow T = [R, S]$	
$ll^\circ = 1, rr^\circ = 1$ Ex 5.12	
$lr^\circ = 0, rl^\circ = 0$ Ex 5.12	
$l^\circ l \cup r^\circ r = 1$ Ex 5.12	
$[U, V]^\circ [R, S] = U^\circ R \cup V^\circ S$ (5.11)	

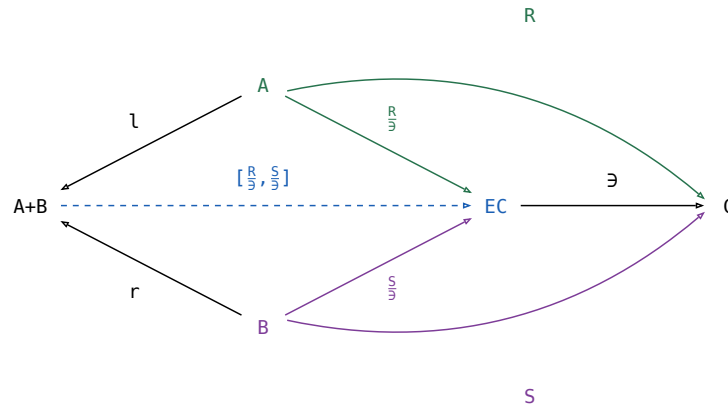
§5.3.1. $[R, S] \triangleq [\frac{R}{\exists}, \frac{S}{\exists}] \exists$

The universal-property row is not free: l, r were only ever asked to be a coproduct of **maps**. They stay one once every arrow is allowed because $\exists$ sends an arrow $A \rightarrow C$ to a map $A \rightarrow EC$ reversibly, so the map coproduct can be applied underneath it. For any $T : A+B \rightarrow C$,

(5.3.1a)



(5.3.1b)



(5.3a)

Nothing here holds only up to Ξ : every triangle commutes on the nose, which is the difference from the fork above. The border spells $[R, S] = [\frac{R}{\exists}, \frac{S}{\exists}] \exists$, and pushing $\exists$ into the union that $[\cdot, \cdot]$ on maps already is turns that back into the definition, $(\frac{R}{\exists} \cup \frac{S}{\exists}) \exists = \frac{R}{\exists} \cup \frac{S}{\exists}$.

(5.3.1c)

the law	picture
prove (5.11), and say why duality does not carry it over from the product law	
$(R+S) \cap [P, Q] [U, V]^\circ = (R \cap P U^\circ) + (S \cap Q V^\circ)$ $[P, Q] [U, V]^\circ$ is a full 2×2 of composites; $R+S$ is diagonal, so the meet cuts the two off-diagonal branches	

§5.4. The power relator $P(R)$

(5.4a)

For $R : A \rightarrow B$,

$$P(R) \triangleq (\epsilon \backslash R \epsilon) \cap (\exists R / \exists)$$

$$E(R) \triangleq \frac{\exists R}{\exists} = ((\exists R) / \exists) \cap (\exists / (\exists R))^\circ$$

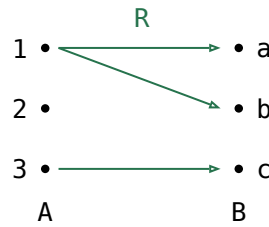
$$xs \ P(R) \ ys \iff (\forall a \in xs. \exists b \in ys. a \ R \ b) \wedge (\forall b \in ys. \exists a \in xs. a \ R \ b)$$

Every element of xs is related by R to some element of ys , and conversely.

(5.4b)

	in Rel	in words	$\text{im}(xs)$	partner
$(\exists R) / \exists$	$\forall y \in ys. \exists x \in xs. x \ R \ y$	$\forall y. \text{ some } xs \ R \ y$	$ys \subseteq \text{im}(xs)$	every y has a partner
$(\exists / (\exists R))^\circ$	$\forall y. (\exists x \in xs. x \ R \ y) \rightarrow y \in ys$	every y with some $xs \ R \ y$ is in ys	$ys \supseteq \text{im}(xs)$	ys leaves out no partner
$E(R)$	$ys = \{y \mid \exists x \in xs. x \ R \ y\}$	$ys = \text{every } y \text{ with some } xs \ R \ y$	$ys = \text{im}(xs)$	every y has a partner, and none is left out
$((\exists R^\circ) / \exists)^\circ$	$\forall x \in xs. \exists y \in ys. x \ R \ y$	$\forall x. x \ R \ \text{some } ys$	—	every x has a partner
$P(R)$	$\forall y \in ys. \exists x \in xs. x \ R \ y$ and $\forall x \in xs. \exists y \in ys. x \ R \ y$	$\forall y. \text{ some } xs \ R \ y$ and $\forall x. x \ R \ \text{some } ys$	—	every x and every y has a partner

(5.4c)



xs	$(\exists R) / \exists$	$(\exists / (\exists R))^\circ$	$E(R)$	$((\exists R^\circ) / \exists)^\circ$	$P(R)$
$\emptyset$	$\emptyset$	all 8 subsets of abc	$\emptyset$	all 8 subsets of abc	$\emptyset$
$\{1\}$	$\emptyset, a, b, ab$	ab, abc	ab	a, b, ab, ac, bc, abc	a, b, ab
$\{2\}$	$\emptyset$	all 8 subsets of abc	$\emptyset$	none	none
$\{3\}$	$\emptyset, c$	c, ac, bc, abc	c	c, ac, bc, abc	c
$\{1, 2\}$	$\emptyset, a, b, ab$	ab, abc	ab	none	none
$\{1, 3\}$	all 8 subsets of abc	abc	abc	ac, bc, abc	ac, bc, abc
$\{2, 3\}$	$\emptyset, c$	c, ac, bc, abc	c	none	none
$\{1, 2, 3\}$	all 8 subsets of abc	abc	abc	none	none

(5.4d)

law	the reading
$X \sqsubseteq P(R) \iff X \sqsubseteq \exists R \text{ and } X^\circ \sqsubseteq \exists R^\circ$	One containment, and the same one at R° — which is the definition read off the two divisions. Hence $P(R^\circ) = P(R)^\circ$, and $R \sqsubseteq S \implies P(R) \sqsubseteq P(S)$.
$P(1) = 1$	The straightness axiom verbatim: extensionality is P 's unit law.
$\text{Map}(f) \implies P(f) = E(f)$	In Rel, $xs \ P(f) \ ys \iff ys = \{f(a) \mid a \in xs\}$. The half at f° says every $a \in xs$ has its $f(a)$ on ys ; f has just the one image per a , so that already says ys contains everything xs reaches, which is the fraction's second half. For a map the two definitions coincide.
$P(RS) = P(R)P(S)$	$\sqsupseteq$ is the division cancellation laws. $\sqsubseteq$ is the one law in this section that is not a calculation: it needs a tabulation of $P(RS)$.

§5.4.1. Type relator

(5.4.1a)

Let F be a binary relator with initial type (α, T) , so T is a type functor. $F(R, S)$ is its action on a pair, and $F(X)$ abbreviates $F(1, X)$, the F of the reduce section. For every object A the initial algebra is $\alpha : F(A, TA) \rightarrow TA$, among the maps. T acts on an arrow $R : A \rightarrow B$ by

$$T(R) = (F(R, 1) \alpha)$$

(5.4.1b)

name	law	what it says
the defining equation	$T(R) = (F(R, 1) \alpha)$	Rebuild the structure with α , applying R to the parameter on the way.
type relator	$T(R)^\circ = T(R^\circ)$	A datatype acts on relations, not only on maps — the map of the converse is the converse of the map.

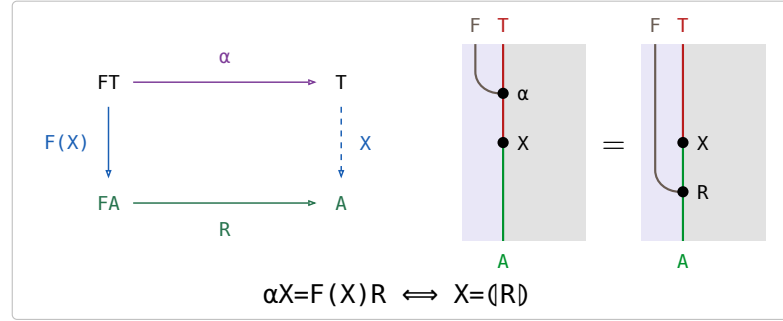
§5.5. Relational catamorphisms

(5.5a)

let F be a relator and has **initial algebra** $\alpha : F(T) \rightarrow T$ in the subcategory of functions. α is also initial in the allegory:

§5.5.1. The defining equation

(5.5.1a)



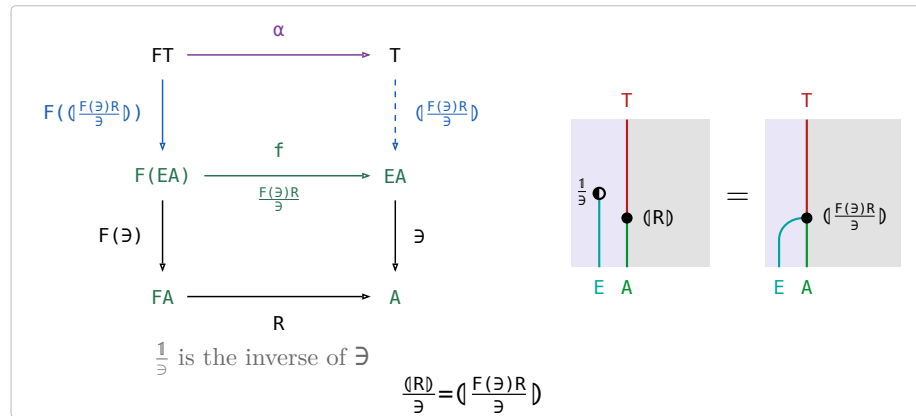
An F -algebra is a **weakened** natural transformation. A transformation $F \Rightarrow 1$ on $\mathcal{A}$ would need a component $FX \rightarrow X$ at every object and a commuting square at every arrow, but F -Algebra only need it works on T and A .

(5.5.1b)

$\llbracket [c, f] \rrbracket = \text{reduce}(c, f)$		
catamorphism: arrows of the form $\llbracket f \rrbracket$ are called catamorphisms, and these two examples reveal the notion to be a familiar idea in abstract clothing		
part	Nat	[A]
datatype	$\text{Nat} ::= \text{zero} \mid \text{succ Nat}$	$[A] ::= \text{nil} \mid \text{cons}(A, [A])$
base functor F	$F(X) = 1 + X$	$F(X) = 1 + A \times X$
initial algebra α	$\alpha = [\text{zero}, \text{succ}]$ $: 1 + \text{Nat} \rightarrow \text{Nat}$	$\alpha = [\text{nil}, \text{cons}]$ $: 1 + A \times [A] \rightarrow [A]$
the fold, pointwise	$\llbracket [c, f] \rrbracket (\text{zero}) = c$ $\llbracket [c, f] \rrbracket (\text{succ}(n)) = f(\llbracket [c, f] \rrbracket (n))$	$\llbracket [c, f] \rrbracket (\text{nil}) = c$ $\llbracket [c, f] \rrbracket (\text{cons}(a, x)) = f(a, \llbracket [c, f] \rrbracket (x))$

§5.5.2. $(R) = (\llbracket \frac{F(\exists)R}{\exists} \rrbracket) \exists$

(5.5.2a)



(5.12)

$$\begin{array}{c}
 \boxed{\frac{\alpha X}{F(X)R}} \iff \boxed{\frac{\alpha X}{\frac{F(X)R}{\exists}}} \iff \boxed{\frac{\alpha X}{\frac{F(\frac{X}{\exists})R}{\exists}}} \\
 \iff \boxed{\frac{\alpha \frac{X}{\exists}}{F(\frac{X}{\exists}) \frac{F(\exists)R}{\exists}}} \iff \boxed{\frac{\frac{X}{\exists}}{\llbracket \frac{F(\exists)R}{\exists} \rrbracket}} \iff \boxed{\frac{X}{\llbracket \frac{F(\exists)R}{\exists} \rrbracket \exists}}
 \end{array}$$

relator, fusion twice reduce of maps $\cdot \exists \dashv \frac{1}{\exists}$

the last two rows at $X := (R)$:

(5.5.2b)

$\alpha X = F(X)R \iff \frac{X}{\exists} = (\frac{F(\exists)R}{\exists})$

in a tabular allegory, with F a relator and $\alpha : F(T) \rightarrow T$ its initial algebra: a relation X out of T satisfies the fold equation $\alpha X = F(X)R$ of R exactly when $\frac{X}{\exists}$ is the fold of the map $\frac{F(\exists)R}{\exists}$

(a) α iso; $\alpha X = F(X)R$

(b) α^0 is a map because α is an iso, so it leaves Λ : $\frac{\alpha^0 F(X)R}{\exists} = \alpha^0 \frac{F(X)R}{\exists}$ — (3.1.1.10a)

(c) $X = \frac{X}{\exists}$ — (3.1.1.10a)

(d) $F(\frac{X}{\exists}) = F(\frac{X}{\exists})F(\exists)$ as F is a functor, and $F(\frac{X}{\exists})$ is a map (a relator sends maps to maps), so it leaves Λ — $\triangleq$ relator, (3.1.1.10a)

(e) fold uniqueness, $\frac{F(\exists)R}{\exists}$ being a map — (2.6a)

§5.6. Combinatorial functions

(5.6a)

definition	type	note
$[A] ::= \text{nil} \mid \text{cons}(A, [A])$	$A \rightarrow A$	The list type, under the short name it keeps.
$\text{list}(R) = (\text{nil}, (R \times 1) \text{cons})$	$[A] \rightarrow [B]$	The relator's action on $R : A \rightarrow B$: one R per element, the shape untouched.
$\text{subseq} = (\text{nil}, \text{cons} \cup \pi_2)$	$[A] \rightarrow [A]$	$xs \text{ subseq } ys$: ys is xs with elements dropped — cons keeps the head, π_2 drops it.
$\text{prefix} = (\text{nil}, \text{nil} \cup \text{cons})$ $\text{prefix} = \text{cat} \circ \pi_1 = \text{init}^*$	$[A] \rightarrow [A]$	ys is an initial segment of xs ; the first nil is where it stops early. $\text{init} \triangleq \text{snoc} \circ \pi_1$.
$\text{suffix} = \text{cat} \circ \pi_2 = \text{tail}^*$	$[A] \rightarrow [A]$	The dual, $\text{tail} \triangleq \text{cons} \circ \pi_2$; as a reduce it needs snoc -lists.
$\text{partition} = \text{concat}^0$	$[A] \rightarrow [[A]]$	This cat is restricted to $[A]^+ \times [A] \rightarrow [A]$, so ys is a list of non-empty segments of xs .
$\text{concat} = (\text{nil}, \text{cat})$	$[[A]] \rightarrow [A]$	Joins the segments back up, which is why its converse splits a list.
inits	$[A] \rightarrow [[A]]$	Implements $\frac{\text{prefix}}{\exists}$, listing the prefixes by increasing length.
tails	$[A] \rightarrow [[A]]$	Implements $\frac{\text{suffix}}{\exists}$ by decreasing length — the opposite order.

§5.6.1. $\frac{\text{subseq}}{\exists} = (\text{nil} \frac{1}{\exists}, (\frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2) \text{cup})$

(5.6.1a)

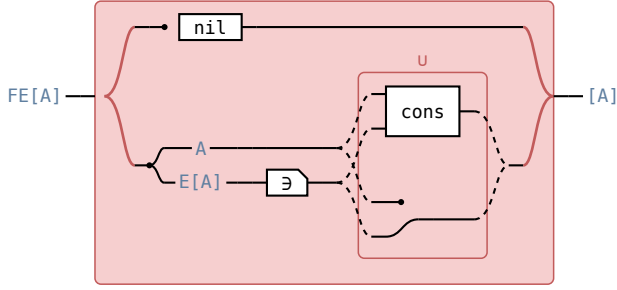
$\text{cup} \triangleq \frac{\pi_1 \exists \cup \pi_2 \exists}{\exists} : EA \times EA \rightarrow EA$, so $\frac{R \cup S}{\exists} = (\frac{R}{\exists}, \frac{S}{\exists}) \text{cup}$.

$$\frac{F(\exists)[\text{nil}, \text{cons } u \pi_2]}{\exists} = [\text{nil } \frac{1}{\exists}, \frac{(1 \times \exists)(\text{cons } u \pi_2)}{\exists}] \quad (5.6.1b)$$

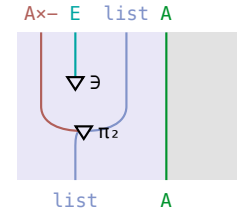
the set of lists the algebra builds is, from nothing, just **nil**, and from a head and a set of tails, every tail in the set with the head put on or left off — (5.12) at **subseq** = $([\text{nil}, \text{cons } u \pi_2])$, $\triangleq$ **subseq**.

circuit — the fork is $F([A]) = 1 + A \times [A]$: **nil** above, the pair below

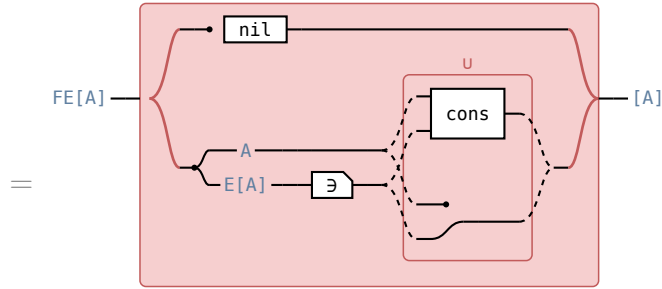
Hinze–Marsden



$$\frac{F(\exists)[\text{nil}, \text{cons } u \pi_2]}{\exists}$$

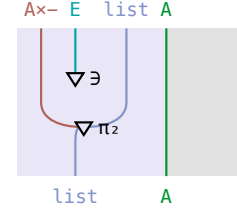


the π_2 operand of $\text{CONS } u \pi_2$ under the $1 \times \exists$ summand of $F(\exists)$, i.e. $(1 \times \exists) \pi_2$

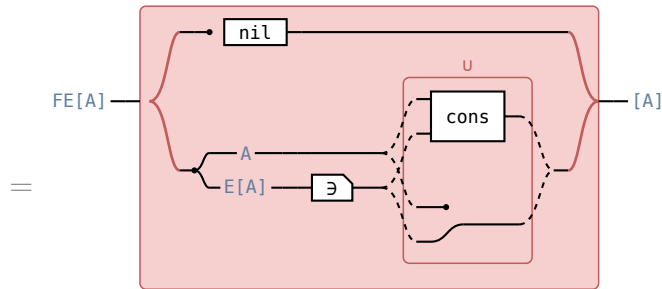


$$\frac{(1+1 \times \exists)[\text{nil}, \text{cons } u \pi_2]}{\exists}$$

$$F(X) = 1 + A \times X \triangleq \text{subseq}$$

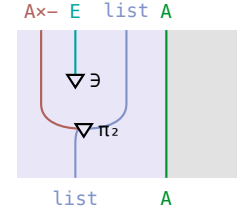


the same operand under $1 + 1 \times \exists$, whose $1 \times \exists$ summand it sits in

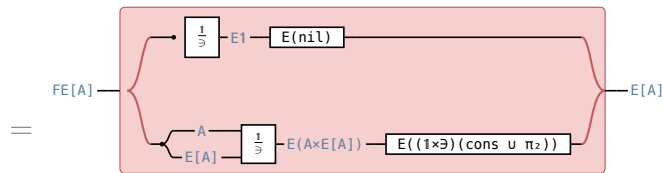


$$\frac{[\text{nil}, (1 \times \exists)(\text{cons } u \pi_2)]}{\exists}$$

$$R + S \triangleq [Rl, Sr], l[R, S] = R, r[R, S] = S \quad (5.3a)$$

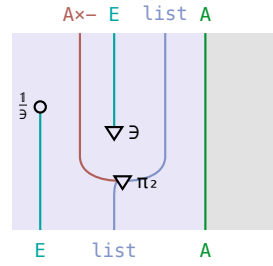


the π_2 operand of the second arm $(1 \times \exists)(\text{cons } u \pi_2)$

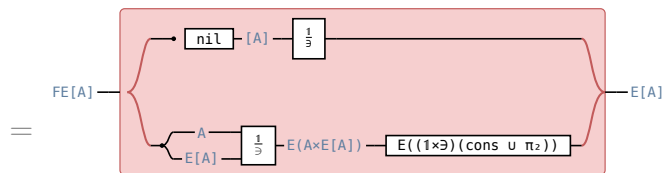


$$\frac{[\text{nil } \frac{1}{\exists}, \frac{(1 \times \exists)(\text{cons } u \pi_2)}{\exists}]}{\exists}$$

$$(5.3.1a) \text{ at } T := [\text{nil}, (1 \times \exists)(\text{cons } u \pi_2)]$$

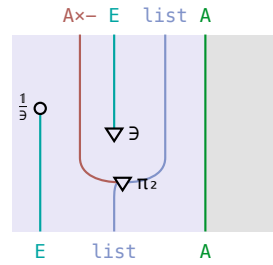


the π_2 operand under its $1 \times \exists$, the arm (5.6.1c)'s square rewrites, $(1 \times \exists) \pi_2 = \pi_2 \exists$



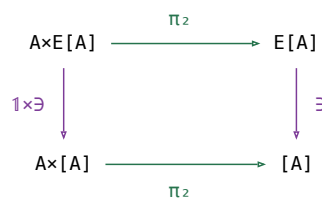
$$\frac{[\text{nil } \frac{1}{\exists}, \frac{(1 \times \exists)(\text{cons } u \pi_2)}{\exists}]}{\exists}$$

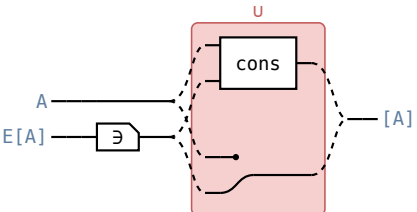
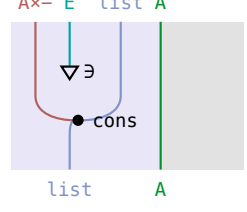
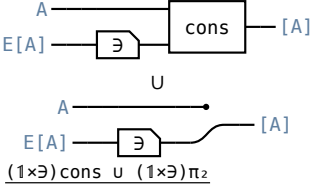
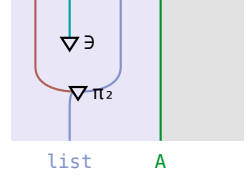
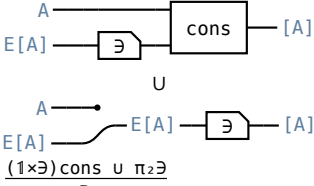
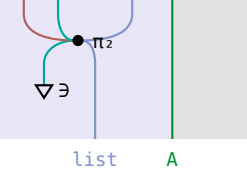
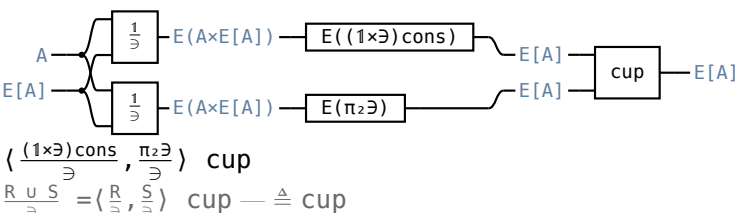
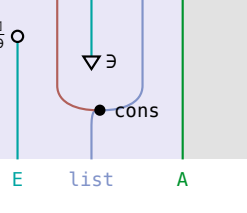
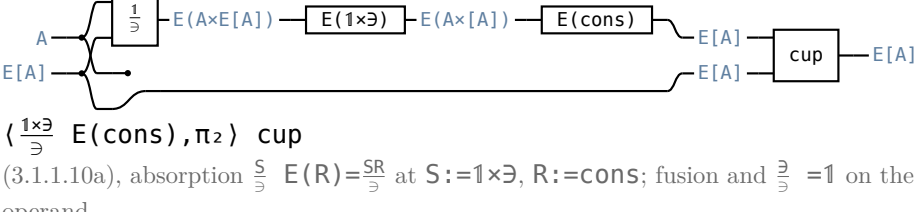
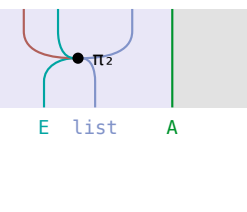
$$(3.1.1.10a), \frac{f}{\exists} = f \frac{1}{\exists} \text{ for } f \text{ a map, at } f := \text{nil}$$



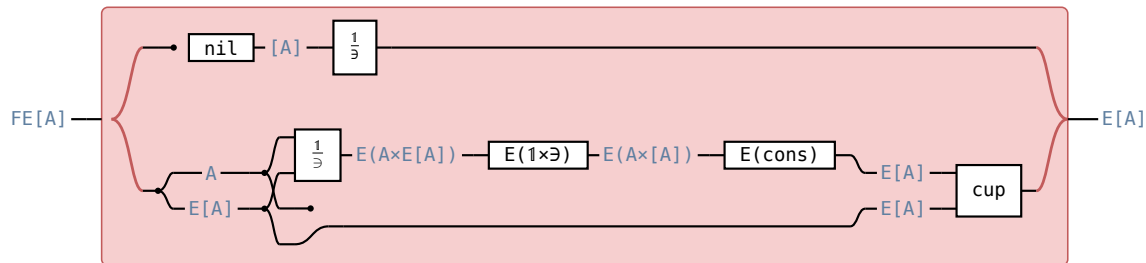
the same operand; the two rows differ only in the **nil** arm

(5.6.1c)



$\frac{(1 \times \exists)(\text{cons} \cup \pi_2)}{\exists} = \left(\frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \right) \text{cup}$ power transpose of join: the power transpose of the join of two relations is $\left(\frac{R}{\exists}, \frac{S}{\exists} \right) \text{cup}$, where cup is the function that returns the union of two sets	(5.6.1d)
circuit	Hinze–Marsden
 $\frac{(1 \times \exists)(\text{cons} \cup \pi_2)}{\exists}$ (5.6.1b)'s second branch	 the cons operand of cons $\cup$ π_2
 $\frac{(1 \times \exists)\text{cons} \cup (1 \times \exists)\pi_2}{\exists}$ $T(X_1 \cup X_2) = TX_1 \cup TX_2$ — (3.1.1.2a)	 the π_2 operand of $(1 \times \exists)\text{cons} \cup (1 \times \exists)\pi_2$
 $\frac{(1 \times \exists)\text{cons} \cup \pi_2 \exists}{\exists}$ $(1 \times \exists)\pi_2 = \pi_2 \exists$ — (5.2.1a) at π_2, an equality because 1 is entire	 the π_2 operand of $(1 \times \exists)\text{cons} \cup \pi_2 \exists$
 $\left(\frac{1 \times \exists}{\exists} \text{cons}, \frac{\pi_2 \exists}{\exists} \right) \text{cup}$ $\frac{R \cup S}{\exists} = \left(\frac{R}{\exists}, \frac{S}{\exists} \right) \text{cup} \triangleq \text{cup}$	 the cons operand under its $1 \times \exists$
 $\left(\frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \right) \text{cup}$ (3.1.1.10a), absorption $\frac{S}{\exists} E(R) = \frac{SR}{\exists}$ at $S := 1 \times \exists$, $R := \text{cons}$; fusion and $\frac{\exists}{\exists} = 1$ on the π_2 operand	 the π_2 operand, bare π_2

(5.6.1e)



$$\left[\frac{\text{nil}}{\exists}, \left(\frac{1 \times \exists}{\exists} E(\text{cons}), \pi_2 \right) \text{cup} \right]$$

which writes $P\text{cons}$; **cons** is a map, and there $P(\text{cons}) = E(\text{cons})$ — (5.4d).

§5.7. Lax natural transformations (LaT)

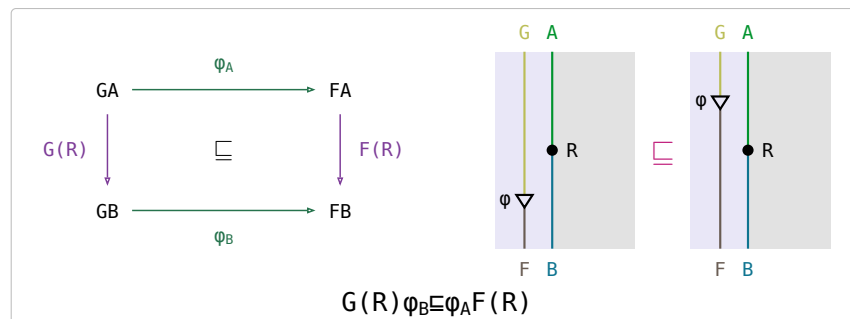
(5.7a)

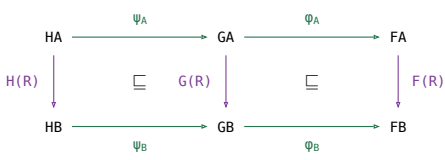
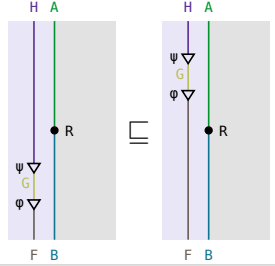
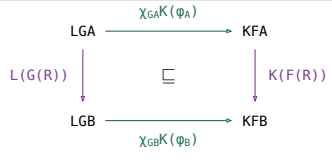
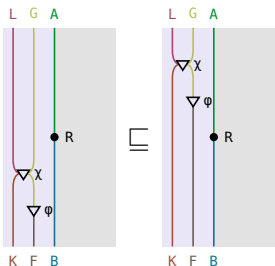
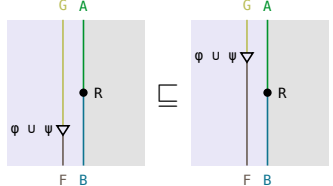
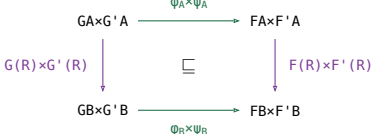
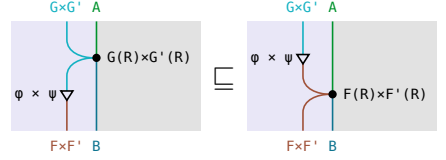
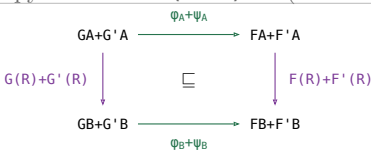
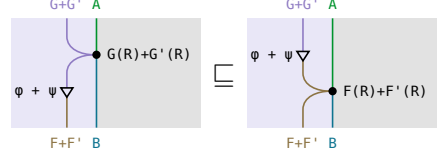
For relators $G, F : \mathcal{C} \rightarrow \mathcal{D}$ and components $\varphi_A : GA \rightarrow FA$, φ is **lax at** $R : A \rightarrow B$ when $G(R)\varphi_B \sqsubseteq \varphi_A F(R)$ — both sides $GA \rightarrow FB$, the left through the component at B , the right through the one at A .

$\varphi : G \Rightarrow F$ is a **lax natural transformation** (LaT) when it is lax at **every** R . (5.13)

Lax at every **map** already gives LaT, and at a map the inequation is an equality $G(f)\varphi = \varphi F(f)$: laxness is about relations only.

(5.7b)



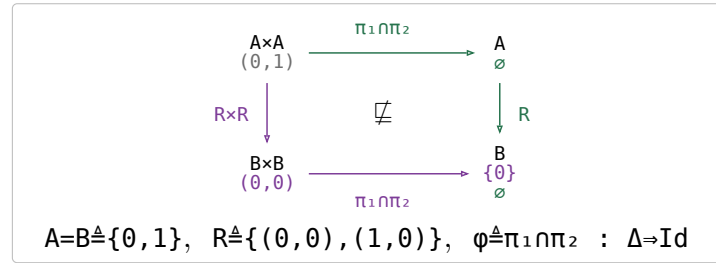
closed under	commutative diagram	string diagram
composition $\psi\phi$	 $H(R)\psi_B \sqsubseteq \psi_A G(R)$ and $G(R)\phi_B \sqsubseteq \phi_A F(R)$ give $H(R)(\psi_B \phi_B) \sqsubseteq (\psi_A \phi_A) F(R)$	
horizontal composition $\chi \circ \phi$	 $\phi : G \Rightarrow F$ with $G, F : \mathcal{C} \rightarrow \mathcal{D}$ and $\chi : L \Rightarrow K$ with $L, K : \mathcal{D} \rightarrow \mathcal{E}$ give $\chi \circ \phi : L \circ G \Rightarrow K \circ F$, the family $A \mapsto \chi_{GA} K(\phi_A)$ $L(G(R)) \chi_{GB} \sqsubseteq \chi_{GA} K(\phi_B) K(F(R))$ is χ lax at $G(R)$; then $K(G(R)\phi_B) \sqsubseteq K(\phi_A F(R))$ is K applied to ϕ's own inequation; $\chi := 1_K$ gives $K(\phi) : K \circ G \Rightarrow K \circ F$, K composed on the outside; $\phi := 1_G$ gives $\chi G : L \circ G \Rightarrow K \circ G$, G composed on the inside two candidates, $\chi_{GA} K(\phi_A)$ and $L(\phi_A) \chi_{FA}$, ordered by $\sqsubseteq$; this row is the first	
union $\phi \cup \psi$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ and $G(R)\psi_B \sqsubseteq \psi_A F(R)$ give $G(R)(\phi_B \cup \psi_B) \sqsubseteq (\phi_A \cup \psi_A) F(R)$	
a relator K $K(\phi)$	 $G(R)\phi_B \sqsubseteq \phi_A F(R)$ gives $K(G(R))K(\phi_B) \sqsubseteq K(\phi_A)K(F(R))$	
product $\phi \times \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi \times \psi : G \times G' \Rightarrow F \times F'$ $(R \times S)(U \times V) = (RU) \times (SV)$ and monotonicity in both slots — the row above's K applied to the inequation, run on a bifunctor; the fork is the derived case $\langle \phi, \psi \rangle = \langle \phi \times \psi \rangle$, and its ONE cost is the lax copy law $R \leq \langle \phi, \psi \rangle (R \times R)$ — (3.1.1.4a)	
coproduct $\phi + \psi$	 $\phi : G \Rightarrow F$ and $\psi : G' \Rightarrow F'$ give $\phi + \psi : G + G' \Rightarrow F + F'$ $(R + S)(U + V) = (RU) + (SV)$ and monotonicity in both slots; the co-fork is the derived case $[\phi, \psi] = (\phi + \psi) \nabla$, and ∇ costs nothing the step it would need is $(\phi_A F(R)) \cap (\psi_A F(R)) \sqsubseteq (\phi \cap \psi)_A F(R)$, the wrong direction of (3.1.1.6a) a counterexample: (5.7.1a)	
meet — fails $\phi \cap \psi$		

Each row is the general slide $R_A X \sqsubseteq X' R_B$ and $R_B Y \sqsubseteq Y' R_C$ give $R_A (XY) \sqsubseteq (X'Y') R_C$ at $R_A, R_B := G(R), F(R)$ and $X, X' := \phi_B, \phi_A$, quantified over R ; at $X' = X$ with R_A, R_B endorelations it is the monotonicity reading instead — the $\forall R$ sits outside the law, and is the only difference

§5.7.1. meet is not closed in LaT

counterexample — $\varphi, \psi : \mathbf{G} \Rightarrow \mathbf{F}$ lax natural does NOT give $\varphi \cap \psi$ lax natural

(5.7.1a)



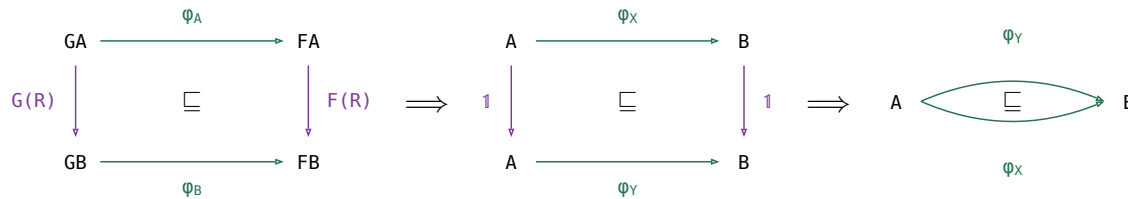
§5.7.2. Two stacked towers

(5.7.2a)

	lower — inside ONE allegory	upper — between allegories
0-cell	an object A	an allegory $\mathcal{C}$ — a whole lower tower
1-cell	a relation $R : A \rightarrow B$	a relator $F : \mathcal{C} \rightarrow \mathcal{D}$
2-cell	$R \sqsubseteq S$	a LaT $\varphi : \mathbf{G} \Rightarrow \mathbf{F} \triangleq \text{lax}$
order	$R \sqsubseteq S$	$\varphi \sqsubseteq \psi$ componentwise
union	$R \cup S$	$\varphi \cup \psi$, survives (5.7c)
product	$R \times S \triangleq \times$	$\varphi \times \psi$, survives; a TENSOR, not a categorical product; SYMMETRIC (5.2a) (5.7c)
coproduct	$R + S$	$\varphi + \psi$, survives; a BIPRODUCT (5.7c)
meet	$R \cap S$	$\varphi \cap \psi$ componentwise, fails (5.7.1a)
converse	R°	φ° , fails, oplax: $\varphi_A^\circ G(R) \sqsubseteq F(R) \varphi_B^\circ$
zero object	z with $1 \ z = \emptyset$	the constant relator at such a z , initial and terminal

§5.7.3. RES is a special case of LaT

(5.7.3a)



$G \triangleq \text{const } A$, $F \triangleq \text{const } B$ — the relators $X \rightarrow A$ and $X \rightarrow B$, each sending every arrow to $1 \triangleq \text{lax}$
 with $\mathcal{C}$ two objects and one non-identity arrow $X \rightarrow Y$, a LaT $\text{const } A \Rightarrow \text{const } B$ is the pair $\varphi_X, \varphi_Y : A \rightarrow B$ with $\varphi_Y \sqsubseteq \varphi_X$ — the datum $R \sqsubseteq S$

not every LaT is constant: $\exists : \mathbf{P} \Rightarrow \text{Id}$ (5.4d) $\pi_2 : (A \times -) \Rightarrow \text{Id}$ (5.6.1c) $\triangleleft : \text{Id} \Rightarrow \Delta$ (3.1.1.4a)

§6. Recursive Programs

§6.1. Digits of a number

(6.1a)

$$\text{val}^\circ = \text{embed}^\circ \text{wrap} \cup \text{op}^\circ - \times \text{Digit}(\text{val}^\circ) \text{snoc}$$

a number is read back into digits either as one nonzero digit, or by splitting off its last digit and reading back the rest

(a) (b) (c) (d) (e)

(a)	val°	the converse of val , which reads digits as a number
(b)	$([\text{wrap}, \text{snoc}]^\circ \text{Digit}^{++} (-\times \text{Digit}(\text{val})) [\text{embed}, \text{op}])^\circ$	val undoes wrap or snoc , reads the front with val and combines by embed or op , so its converse runs that backwards
(c)	$(\text{wrap}^\circ \text{embed})^\circ \cup (\text{snoc}^\circ - \times \text{Digit}(\text{val}) \text{op})^\circ$	undoing wrap or snoc first gives two cases: a decimal and its last digit read by val then op , or one digit read by embed
(d)	$\text{embed}^\circ \text{wrap} \cup \text{op}^\circ - \times \text{Digit}(\text{val})^\circ \text{snoc}$	the converse of each case is its steps undone in reverse: undo op , undo reading the front, then snoc ; or undo embed , then wrap
(e)	$\text{embed}^\circ \text{wrap} \cup \text{op}^\circ - \times \text{Digit}(\text{val}^\circ) \text{snoc}$	undoing the read of the front while keeping the last digit is val° on the front alone

(6.1b)

$$\text{op}^\circ(m, p) \iff \pi_1(p) = m/10 \wedge \pi_2(p) = m \% 10$$

op° splits a number into its quotient and remainder by 10

$$(\exists p. 0 < \pi_1 p \wedge \text{op}^\circ(m, p)) \iff 10 \leq m$$

op° gives a pair with a nonzero first component exactly at the numbers with two or more digits

$$(\exists d. \text{embed}^\circ(m, d)) \iff 0 < m \wedge m < 10$$

embed° gives a digit exactly at the one-digit numbers

§6.2. Least fixed points

(6.2a)

φ a **monotonic** mapping of the hom-set $A \rightarrow B$ into itself: $X \sqsubseteq Y \Rightarrow \varphi(X) \sqsubseteq \varphi(Y)$.

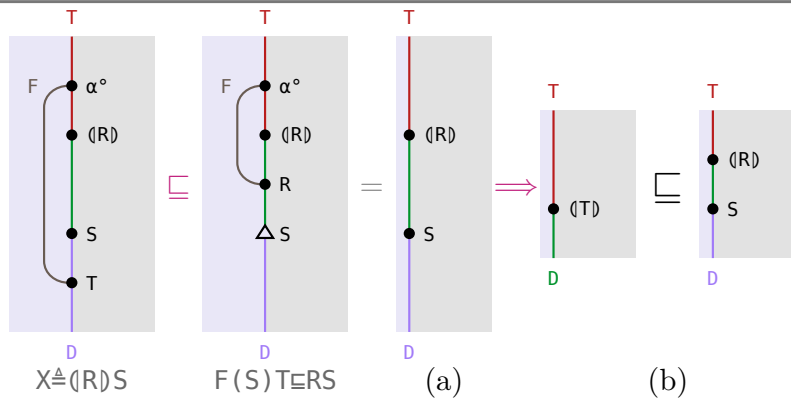
$(\mu X : \varphi(X))$ the least $X : A \rightarrow B$ with $\varphi(X) \sqsubseteq X$.

(6.2b)

the law	what it says
$\varphi(Y) \sqsubseteq Y \Rightarrow (\mu X : \varphi(X)) \sqsubseteq Y$	to bound $(\mu X : \varphi(X))$ above, exhibit one Y the body does not grow past — the half §6.3 and every chapter after it uses
$\text{Monotonic}(\varphi) \Rightarrow \varphi(\mu X : \varphi(X)) = (\mu X : \varphi(X))$	Knaster–Tarski : the least solution of $\varphi(X) \sqsubseteq X$ already solves $\varphi(X) = X$, so the least prefix point and the least fixed point are one relation
$\alpha^\circ F(X) R \sqsubseteq X \Rightarrow \langle R \rangle \sqsubseteq X$ (6.2)	a relation X that contains one unfolding of the fold's recursion at X contains the fold
$X \sqsubseteq \alpha^\circ F(X) R \Rightarrow X \sqsubseteq \langle R \rangle$ (6.3)	a relation X contained in one unfolding of the fold's recursion at X is contained in the fold

$$F(S)T \sqsubseteq RS \Rightarrow \langle T \rangle \sqsubseteq \langle R \rangle S \quad (6.2c)$$

if S followed by R absorbs T after $F(S)$, the fold of T is below the fold of R followed by S

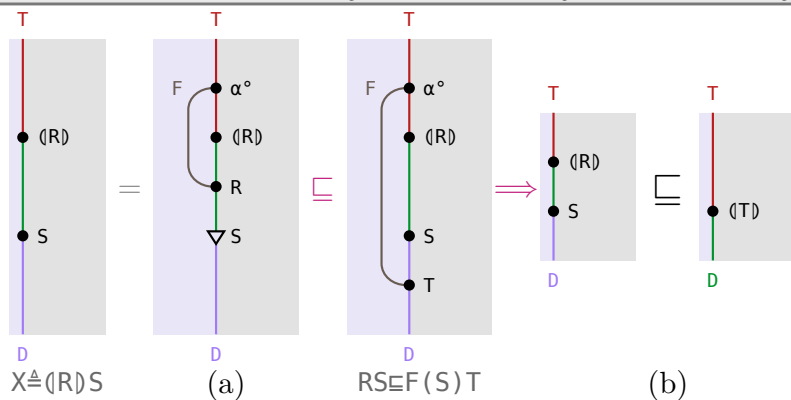


(a) $F(\langle R \rangle)R = \alpha(\langle R \rangle)$, $\alpha^\circ \alpha = 1$ — $\triangleq$ reduce

(b) $\langle R \rangle S$ is a prefix point of $\alpha^\circ F(-)T$, and $\langle T \rangle$ is the least one — (6.2) $\alpha^\circ F(X)T \sqsubseteq X \Rightarrow \langle T \rangle \sqsubseteq X$

$$RSEF(S)T \Rightarrow \langle R \rangle S \sqsubseteq \langle T \rangle \quad (6.2d)$$

if R followed by S is absorbed by $F(S)$ followed by T , the fold of R followed by S is below the fold of T



(a) $\alpha(\langle R \rangle) = F(\langle R \rangle)R$, $\alpha^\circ \alpha = 1$ — $\triangleq$ reduce

(b) $\langle R \rangle S$ is a postfix point of $\alpha^\circ F(-)T$, and $\langle T \rangle$ is the greatest one — (6.3) $X \sqsubseteq \alpha^\circ F(X)T \Rightarrow X \sqsubseteq \langle T \rangle$

§6.3. Hylomorphisms

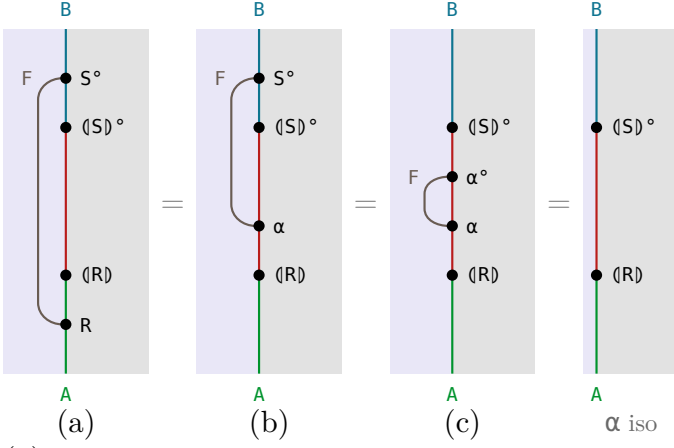
$$(\mathbb{S})^\circ(\mathbb{R}) = (\mu X : S^\circ F(X)R)$$

(6.3a)

hylomorphism theorem: a hylomorphism is the least fixed point of a certain recursion equation

$$S^\circ F((\mathbb{S})^\circ(\mathbb{R}))R = (\mathbb{S})^\circ(\mathbb{R})$$

hylomorphism theorem: a prototypical ‘divide and conquer’ scheme — the term S° represents the decomposition stage, $F((\mathbb{S})^\circ(\mathbb{R}))$ the stage of solving the subproblems recursively, and R the recombination stage; $R : FA \rightarrow A$, $S : FB \rightarrow B$, $\alpha : FT \rightarrow T$ initial



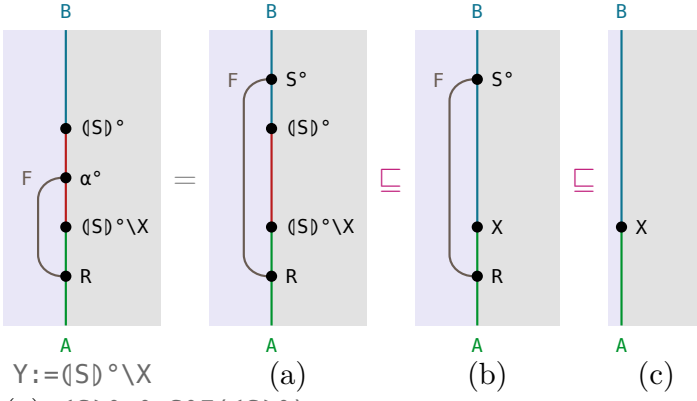
(a) the body at $(\mathbb{S})^\circ(\mathbb{R})$

(b) $F((\mathbb{R}))R = \alpha(\mathbb{R}) \triangleq \text{reduce}$

(c) $(\mathbb{S})^\circ \alpha^\circ = S^\circ F((\mathbb{S})^\circ) \triangleq \text{reduce, (5.1b)}$

$$S^\circ F(X)R \sqsubseteq X \implies (\mathbb{S})^\circ(\mathbb{R}) \sqsubseteq X$$

hylomorphism theorem: by Knaster–Tarski, the hylomorphism $(\mathbb{S})^\circ(\mathbb{R})$ is included in X if X satisfies the associated recursion inequation

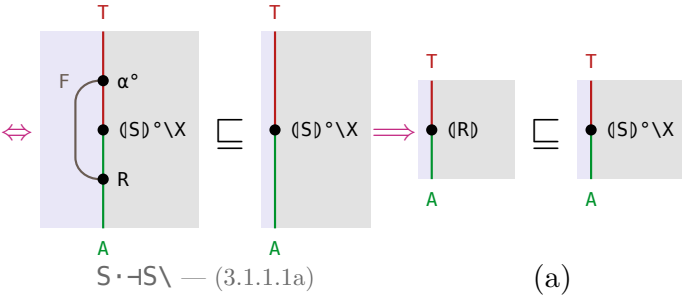


$Y := (\mathbb{S})^\circ \setminus X$

(a) $(\mathbb{S})^\circ \alpha^\circ = S^\circ F((\mathbb{S})^\circ)$

(b) $(\mathbb{S})^\circ((\mathbb{S})^\circ \setminus X) \sqsubseteq X \text{ — (3.1.1.1a)}$

(c) $S^\circ F(X)R \sqsubseteq X$



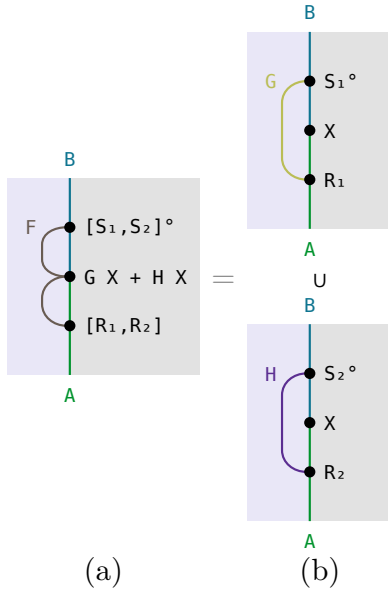
$S \cdot \neg S \setminus \text{ — (3.1.1.1a)}$

(a)

(a) $(\mathbb{R}) = (\mu X : \alpha^\circ F(X)R) \triangleq \text{reduce, (6.2b), (3.1.1.1a)}$

$$(\forall x. \forall y. \forall X. F(X)=G(X)+H(X)) \Rightarrow ([S_1, S_2]^\circ \circ ([R_1, R_2])^\circ) = (\mu X : S_1^\circ G(X) R_1 \cup S_2^\circ H(X) R_2) \quad (6.3b)$$

when both algebras are case splits over the same sum, the recursion runs each case on its own and unites the results

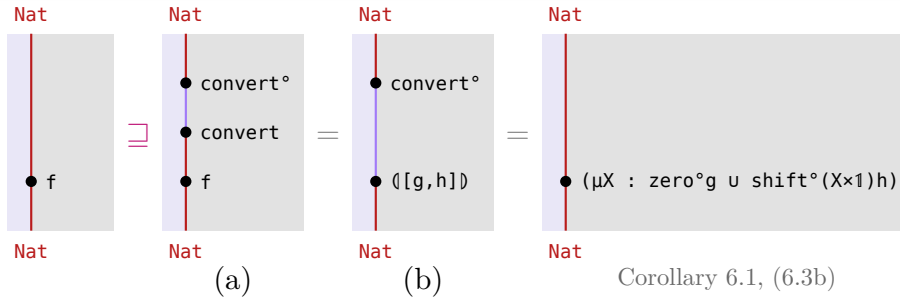


(a)	$[S_1, S_2]^\circ (G(X)+H(X)) [R_1, R_2]$	the body of Theorem 6.2 at $F(X)=G(X)+H(X)$
(b)	$S_1^\circ G(X) R_1 \cup S_2^\circ H(X) R_2$	$[S, T] = (S+T) \nabla$ on both sides gives $\nabla^\circ (S_1^\circ G(X) R_1 + S_2^\circ H(X) R_2) \nabla$; the + stands for the codiagonals ∇°, ∇ , which are not drawn $\nabla^\circ (P+Q) \nabla = P \cup Q$ — coproduct

§6.4. Fast exponentiation and modulus computation

$$\text{zero } f=g \wedge \text{shift } f=(f \times 1)h \Rightarrow (\mu X : \text{zero}^\circ g \cup \text{shift}^\circ (X \times 1)h) \sqsubseteq f \quad (6.4a)$$

once **f** sends **zero** to **g** and turns **shift** into **h**, the recursion that halves the argument at each step computes **f**



Corollary 6.1, (6.3b)

- (a) convert simple
(b) fusion: $\text{zero } f=g, \text{shift } f=(f \times 1)h$

$$(\mu X : \text{zero}^\circ \text{one} \cup \text{shift}^\circ (X \times 1) \text{op}(a)) \sqsubseteq \text{exp}(a) \quad (6.4b)$$

the recursion that halves the exponent at each step computes **a** to the power **b**

$\text{zero } \text{exp}(a)=\text{one}$

a to the power 0 is 1

$\text{shift } \text{exp}(a)=(\text{exp}(a) \times 1) \text{op}(a)$

a to the power $2n+d$ is $\text{op}(a)$ of **a** to the power **n** and **d**

$$(\mu X : \text{zero}^\circ \text{zero} \cup \text{shift}^\circ (X \times 1) \text{op}(b)) \sqsubseteq \text{mod}(b) \quad (6.4c)$$

the recursion that halves **a** at each step computes the remainder of **a** by **b**

$\text{zero } \text{mod}(b)=\text{zero}$

0 mod **b** is 0

$\text{shift } \text{mod}(b)=(\text{mod}(b) \times 1) \text{op}(b)$

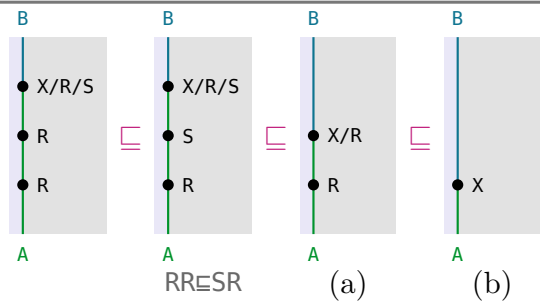
$(2a+d) \text{ mod } b$ is $\text{op}(b)$ of **a mod b** and **d**

§6.5. Unique fixed points

$$\text{inductive}(S) \wedge R \sqsubseteq SR \Rightarrow \text{inductive}(R)$$

(6.5a)

R is inductive when $R \sqsubseteq SR$ for an inductive S



- (a) division by S
(b) division by R

$$\text{member} = \emptyset$$

(6.5b)

a constant relator records no elements, so its membership is empty

$$\text{member} = [\text{member}, \text{member}]$$

a member of $F+G$ is a member of whichever summand the value is in

$$\text{member} = \pi_1 \text{ member} \cup \pi_2 \text{ member}$$

a member of $F \times G$ is a member of either component

$$\text{inductive}(\text{Smember}) \wedge X = SF(X)R \wedge Y = SF(Y)R \Rightarrow X = Y$$

(6.5c)

when S **member** is inductive, two solutions X, Y of $X = SF(X)R$ are equal

$$\text{inductive}(\text{Smember}) \wedge \text{Entire}(S) \wedge \text{Entire}(R) \wedge SF(X)R \sqsubseteq X \Rightarrow \text{Entire}(X)$$

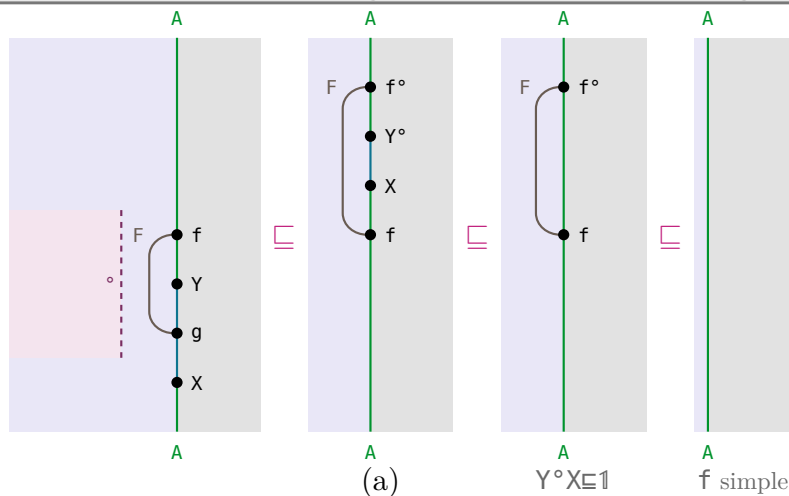
(6.5d)

when S **member** is inductive and S, R are entire, a pre-fixed point of $X \mapsto SF(X)R$ is entire

$$\text{inductive}(\text{gmember}) \wedge \text{Map}(g) \wedge \text{Map}(f) \wedge X = gF(X)f \Rightarrow \text{Map}(X)$$

(6.5e)

when g **member** is inductive and f, g are maps, the solution of $X = gF(X)f$ is a map

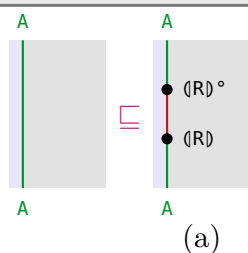


- (a) $X = gF(X)f$, g simple

$$\text{inductive}(R^\circ \text{member}) \wedge 1 \sqsubseteq R^\circ R \Rightarrow 1 \sqsubseteq (R)^\circ (R)$$

(6.5f)

when $R^\circ \text{member}$ is inductive and R is surjective, the fold (R) is surjective

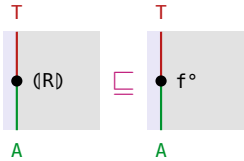


- (a) (6.5d) at $S = R^\circ$

$$\text{Map}(f) \wedge Rf \sqsubseteq F(f)\alpha \wedge 1 \sqsubseteq R^\circ R \Rightarrow f^\circ = \langle R \rangle$$

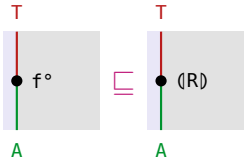
(6.5g)

a map f with $Rf \sqsubseteq F(f)\alpha$, for a surjective R , has the fold $\langle R \rangle$ as its converse



(a)

(a) shunting f , fusion, $Rf \sqsubseteq F(f)\alpha$



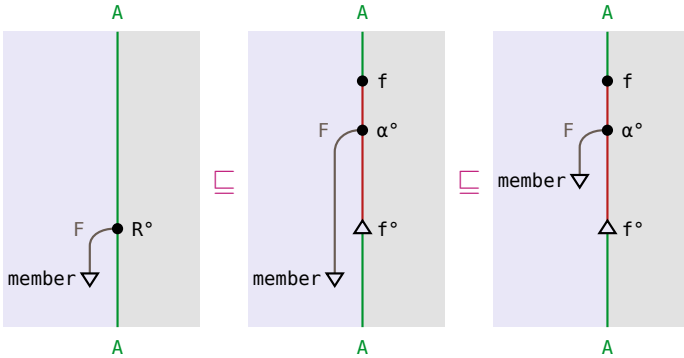
(a)

(a) (6.5f) and $\langle R \rangle f \sqsubseteq 1$

$$\text{Map}(f) \wedge Rf \sqsubseteq F(f)\alpha \Rightarrow R^\circ \text{member} \sqsubseteq f\alpha^\circ \text{member } f^\circ$$

(6.5h)

$R^\circ \text{member}$ is below $\alpha^\circ \text{member}$ conjugated by f , so it is inductive when $\alpha^\circ \text{member}$ is



(a)

(b)

(a) $Rf \sqsubseteq F(f)\alpha$, shunting

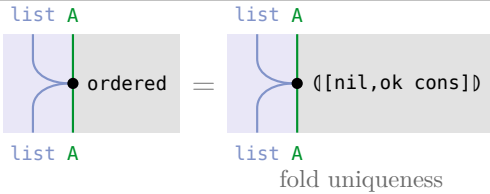
(b) member lax natural

§6.6. Sorting by selection

$$\text{ordered} = \langle [\text{nil}, \text{ok cons}] \rangle$$

(6.6a)

a list is sorted exactly when the fold that rebuilds it passes **ok** at every **cons**, i.e. each head is **R**-below every element after it

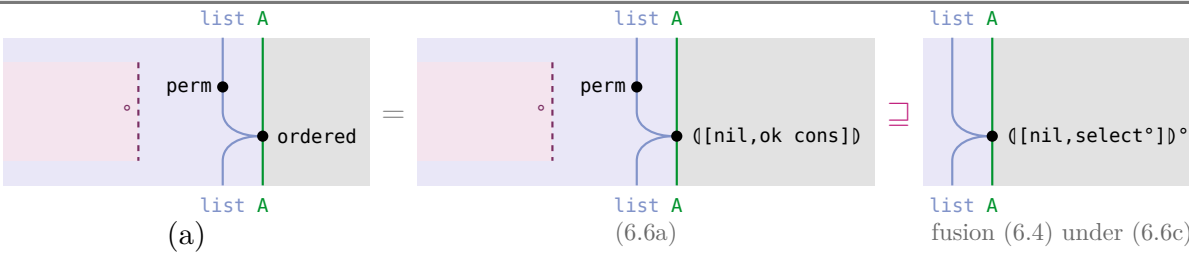


fold uniqueness

$$\text{select}^\circ \sqsubseteq \text{ok cons perm} \Rightarrow \langle [\text{nil}, \text{select}^\circ] \rangle^\circ \sqsubseteq \text{perm ordered}$$

(6.6b)

every output of unfolding the input by **select** is a sorted permutation of it



(a)

(6.6a)

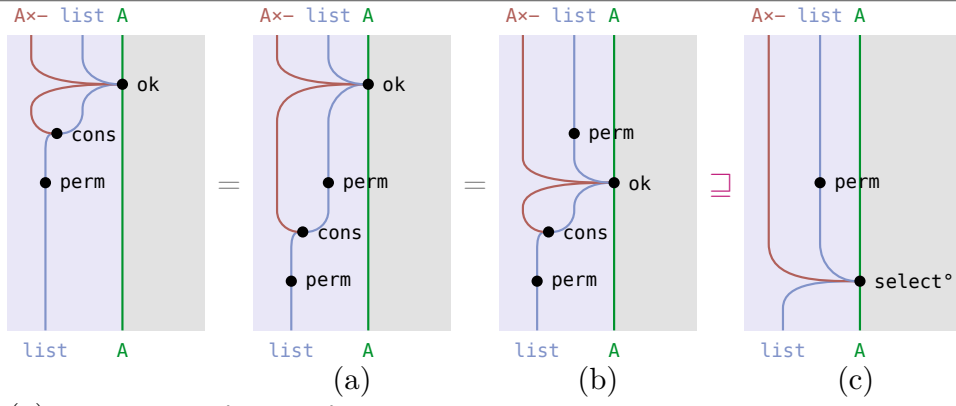
fusion (6.4) under (6.6c)

(a)

$\text{perm}^\circ = \text{perm}$, $\text{ordered}^\circ = \text{ordered}$

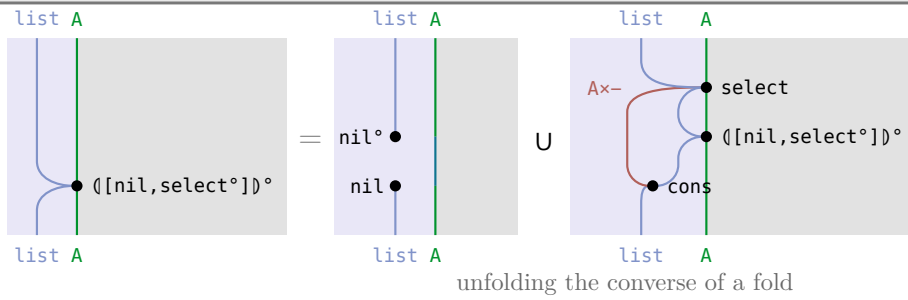
$$\text{select}^\circ \sqsubseteq \text{ok cons perm} \Rightarrow (1 \times \text{perm}) \text{select}^\circ \sqsubseteq \text{ok cons perm} \quad (6.6c)$$

permuting the tail and then undoing **select** lands among the **ok** conses of a permutation



$$([nil, \text{select}^\circ])^\circ = nil^\circ nil \vee \text{select } (1 \times ([nil, \text{select}^\circ])^\circ) \text{cons} \quad (6.6d)$$

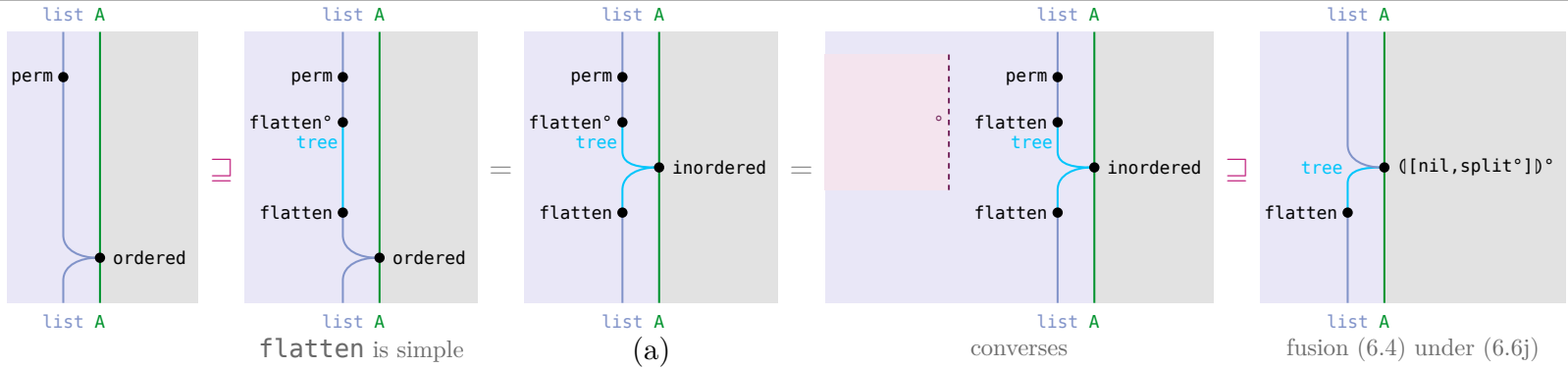
selection sort returns **[]** on **[]**, and otherwise selects **(a,y)**, sorts **y** and conses **a** back on



$$(\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)) \wedge \text{split}^\circ \sqsubseteq \text{check}' \text{ join perm} \Rightarrow \quad (6.6e)$$

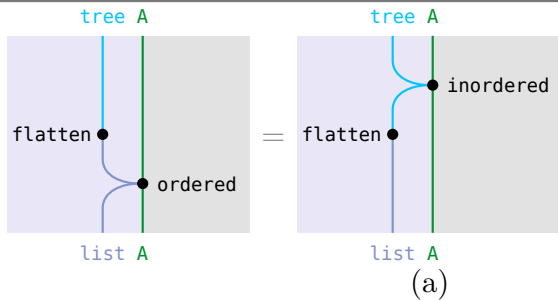
$$([nil, \text{split}^\circ])^\circ \text{flatten} \sqsubseteq \text{perm ordered}$$

every output of unfolding the input by **split** into a tree and flattening that tree is a sorted permutation of the input



$$(\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)) \Rightarrow \text{flatten ordered} = \text{inordered flatten} \quad (6.6f)$$

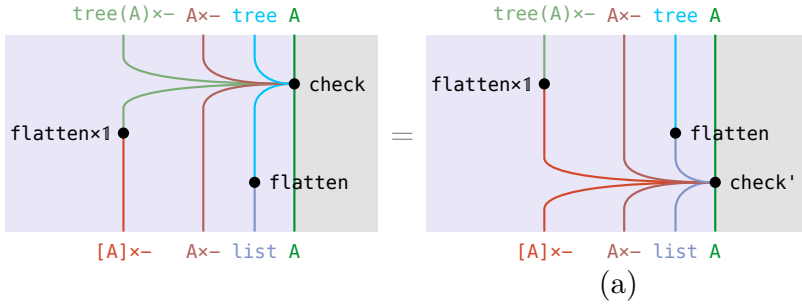
flattening a tree and then testing the list for order is the same as testing the tree with **inordered** and then flattening



check (flatten×(1×flatten))=(flatten×(1×flatten))check'

(6.6g)

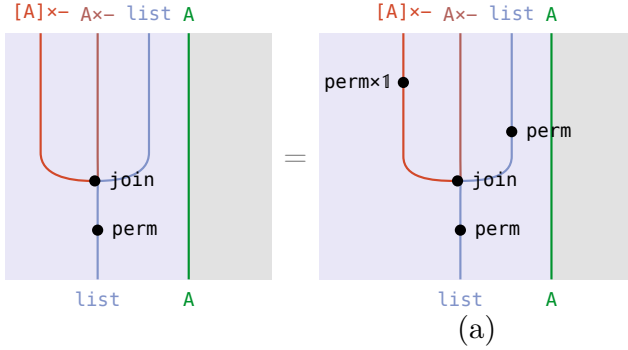
testing a fork with **check** and then flattening both subtrees equals flattening both subtrees and then testing with **check'**, since **intree** of a tree is **inlist** of its flattening



join perm=(perm×(1×perm))join perm

(6.6h)

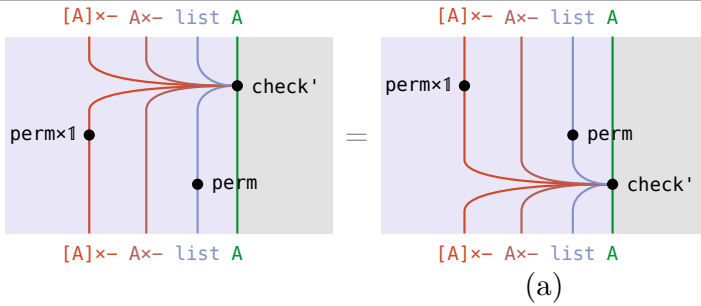
permuting the list $x ++ [a] ++ y$ is the same as first permuting x and y and then permuting the joined list



check' (perm×(1×perm))=(perm×(1×perm))check'

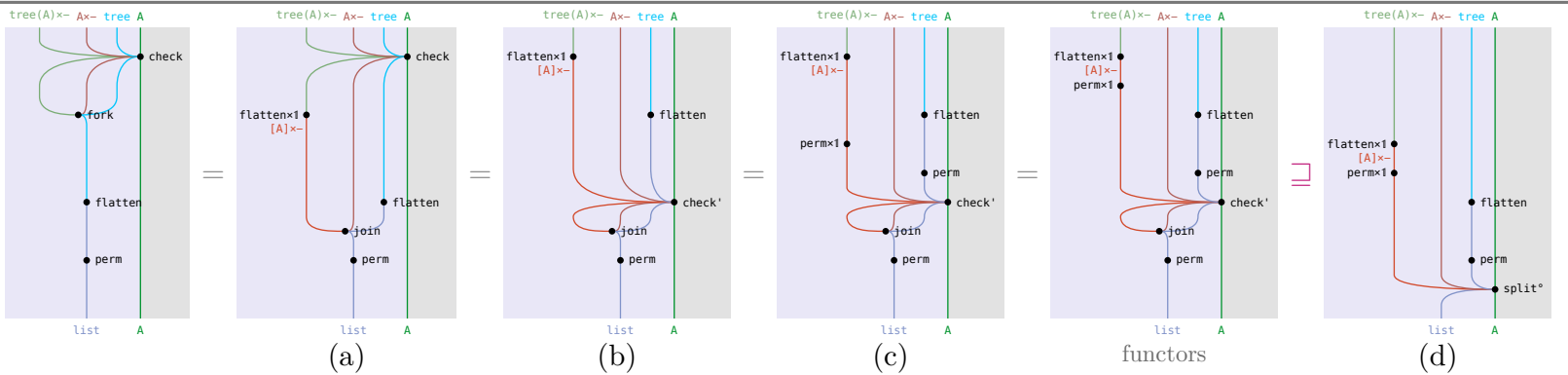
(6.6i)

testing a fork of lists with **check'** and then permuting both parts equals permuting both parts and then testing, since permuting a list leaves its members unchanged



split°⊆check' join perm ⇒ ((flatten perm)×(1×(flatten perm)))split°⊆check fork flatten perm (6.6j)

undoing **split** and then flattening and permuting both parts lands among the **checked** forks whose flattening is permuted



(b) $\text{check } F(\text{flatten}) = F(\text{flatten})\text{check'}$

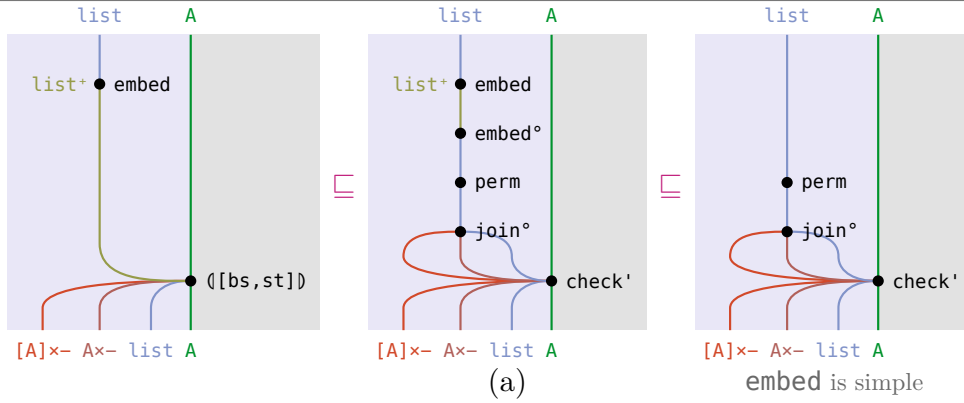
(c) $\text{join perm} = F(\text{perm})\text{join perm}, \text{check' } F(\text{perm}) = F(\text{perm})\text{check'}$

(d) $\text{split}^\circ \subseteq \text{check' join perm}$

$$\text{bs}\text{Ewrap perm join}^\circ\text{check}' \wedge (1 \times (\text{perm join}^\circ\text{check}')) \text{stEcons perm join}^\circ\text{check}' \Rightarrow \text{embed}([bs, st]) \text{E} \quad (6.6k)$$

perm join[°]check'

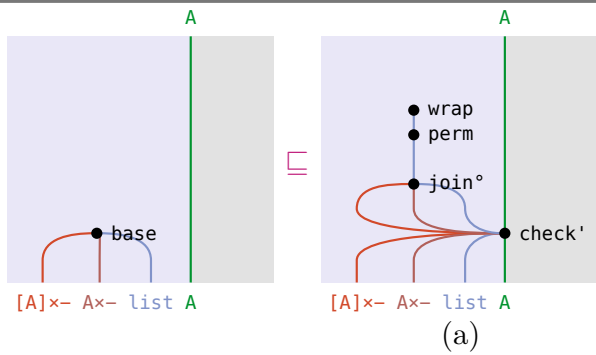
turning a list into a non-empty list and folding it with any $[bs, st]$ meeting the two conditions below gives only **checked triples** (x, a, y) whose $\text{join } x \# [a] \# y$ is a permutation of the list



baseEwrap perm join[°]check'

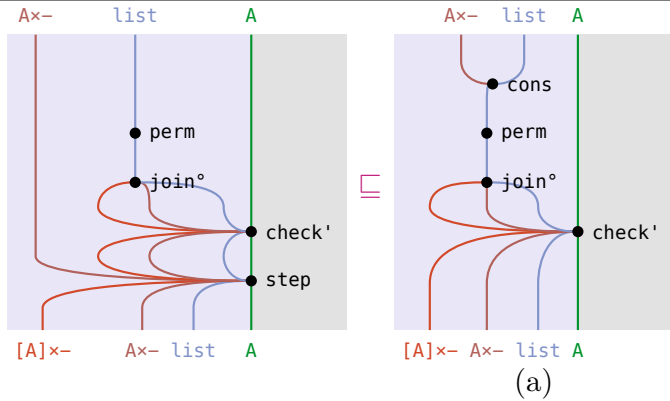
(6.6l)

base(a) is a **checked triple** whose **join** is a permutation of the one-element list $[a]$



$$(\forall a. \forall b. \text{leb}(a, b) = \text{true} \Rightarrow R(a, b)) \wedge (\forall a. \forall b. \text{leb}(a, b) = \text{false} \Rightarrow R(b, a)) \Rightarrow (1 \times (\text{perm join}^\circ\text{check}')) \text{stepEcons perm join}^\circ\text{check}' \quad (6.6m)$$

if (x, b, y) is a **checked triple** for a permutation of l , then $\text{step}(a, (x, b, y))$ is a **checked triple** for a permutation of $[a] \# l$

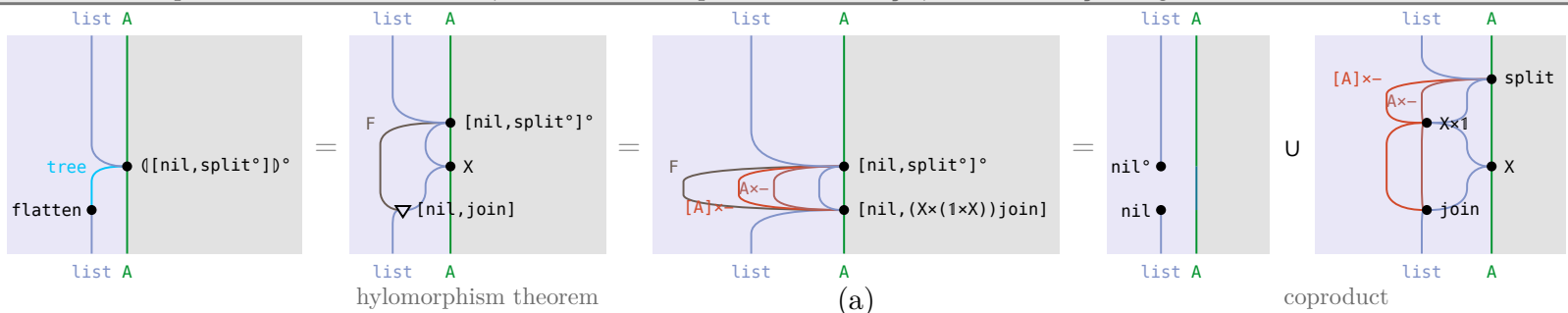


$$([nil, \text{split}^\circ])^\circ \text{flatten} = nil^\circ nil \cup \text{split}$$

(6.6n)

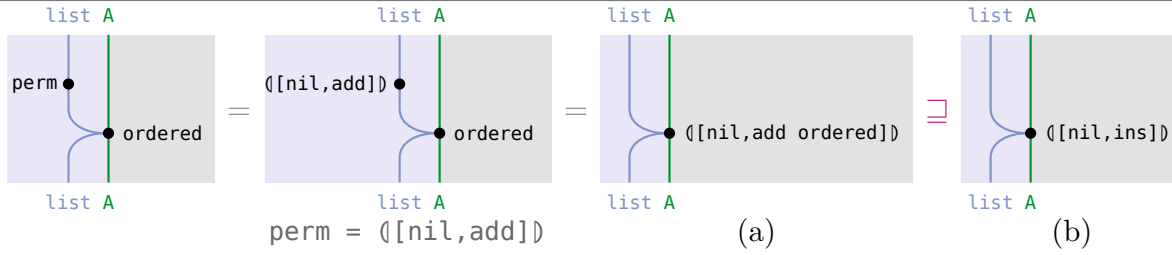
$$((([nil, \text{split}^\circ])^\circ \text{flatten}) \times (1 \times ([nil, \text{split}^\circ])^\circ \text{flatten})) \text{join}$$

quicksort returns $[]$ on $[]$, and otherwise splits into (x, a, y) , sorts x and y and joins them around a



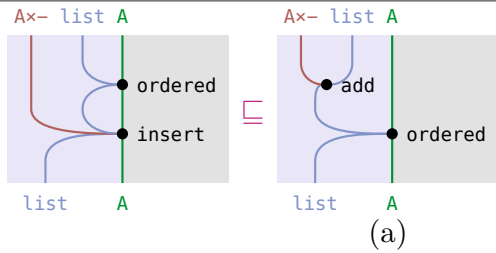
$$(1 \times \text{ordered}) \text{ins} \sqsubseteq \text{add ordered} \Rightarrow ([\text{nil}, \text{ins}]) \sqsubseteq \text{perm ordered} \quad (6.6o)$$

for any **ins** that, on an ordered list, returns only ordered results of **add**, folding with **[nil, ins]** gives only sorted permutations



$$(\forall a. \forall b. \text{leb}(a,b)=\text{true} \Rightarrow R(a,b)) \wedge (\forall a. \forall b. \text{leb}(a,b)=\text{false} \Rightarrow R(b,a)) \wedge (\forall a. \forall b. \forall c. R(a,b) \Rightarrow R(b,c) \Rightarrow R(a,c)) \Rightarrow (1 \times \text{ordered}) \text{insert} \sqsubseteq \text{add ordered} \quad (6.6p)$$

inserting **a** into an ordered list puts **a** somewhere in it and keeps it ordered



(a) **insert** splices **a** in, and **R** transitive and total

§6.7. Closure

$$R^* = (\mu X : 1 \cup XR) \quad (6.7a)$$

closing **R** by composing it on the left and closing it by composing on the right give the same relation **R***

$$1 \sqsubseteq X \wedge XX \sqsubseteq X \Rightarrow R \sqsubseteq X \Leftrightarrow R^* \sqsubseteq X \quad (6.7b)$$

for a preorder **X**, **X** contains **R** exactly when it contains **R***

$$1 \sqsubseteq R^* \quad (6.7c)$$

R* is reflexive

	(a)	1	
$\sqsubseteq$	(b)	$1 \cup RR^*$	union
$=$	(c)	R^*	fixed point, Thm 6.1

$$R \sqsubseteq R^* \quad (6.7d)$$

R* contains **R**

	(a)	R	
$=$	(b)	R	identity
$\sqsubseteq$	(c)	RR^*	$1 \sqsubseteq R^* \Rightarrow (6.7c)$
$\sqsubseteq$	(d)	R^*	fixed point

$$R^* R^* \sqsubseteq R^* \quad (6.7e)$$

R* is transitive, because **R*/R*** is a prefixed point of $X \mapsto 1 \cup RX$

	(a)	$(1 \cup R(R^*/R^*))R^*$	
$=$	(b)	$R^* \cup R(R^*/R^*)R^*$	$(P \cup Q)T = PT \cup QT$ — composition distributes over $\cup$
$\sqsubseteq$	(c)	$R^* \cup RR^*$	$(S/S)S \sqsubseteq S$
$\sqsubseteq$	(d)	R^*	$RR^* \sqsubseteq R^* \Rightarrow (6.7d)$

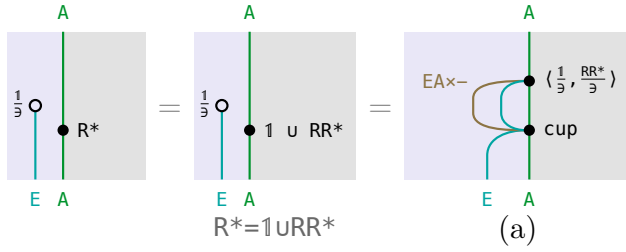
$$1 \sqsubseteq X \wedge XX \sqsubseteq X \wedge R \sqsubseteq X \Rightarrow R^* \sqsubseteq X \quad (6.7f)$$

a reflexive transitive **X** containing **R** is a prefixed point of $X \mapsto 1 \cup RX$, so it contains **R***

	(a)	$1 \cup RX$	
$\sqsubseteq$	(b)	$X \cup XX$	$1 \sqsubseteq X, R \sqsubseteq X$
$\sqsubseteq$	(c)	X	$XX \sqsubseteq X$

$$\frac{R^*}{S} = \left(\frac{1}{S}, \frac{RR^*}{S} \right) \text{cup} \quad (6.7g)$$

the set of R^* -successors of a is a itself joined with the R^* -successors of its R -successors



$$(a) \ \Lambda(R \cup S) = \langle \Lambda(R), \Lambda(S) \rangle \text{cup}$$

$$R - \emptyset = R \quad (6.7h)$$

taking nothing away leaves R

$$R \cup (S - R) = R \cup S$$

beside R , only the part of S outside R adds anything

$$R - (S \cup T) = R - S - T$$

taking away S and then T takes away $S \cup T$

$$(R \cup S) - T = (R - T) \cup (S - T)$$

taking T away from a union takes it away from each part

$$\text{monotonic}(\varphi) \wedge \text{monotonic}(\psi) \implies (\mu X : \varphi(\psi(X))) = \varphi(\mu Y : \psi(\varphi(Y))) \quad (6.7i)$$

the least fixed point of φ after ψ is φ applied to the least fixed point of ψ after φ

$$SR^* = (\mu X : S \cup XR) \quad (6.7j)$$

S followed by any number of R steps is the least X containing S and closed under a further R step

$$R^*S = (\mu X : S \cup RX)$$

any number of R steps followed by S is the least X containing S and closed under an R step in front

$$\theta(\emptyset, S) = SR^* \quad (6.7k)$$

started with nothing found and S to explore, θ computes SR^*

	(a)	$\theta(\emptyset, S)$	
=	(b)	$\emptyset \cup (\mu X : S \cup (XR - \emptyset))$	definition of θ
=	(c)	$(\mu X : S \cup XR)$	$R - \emptyset = R$ — (6.7h)
=	(d)	SR^*	(6.7j)

$$\theta(P, \emptyset) = P \quad (6.7l)$$

with nothing left to explore, θ returns what it has found

	(a)	$\theta(P, \emptyset)$	
=	(b)	$P \cup (\mu X : \emptyset \cup (XR - P))$	definition of θ
=	(c)	$P \cup \emptyset$	$\emptyset$ is a prefixed point: $\emptyset R - P = \emptyset$
=	(d)	P	$P \cup \emptyset = P$

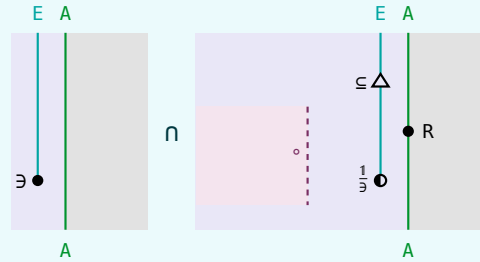
§7. Optimisation Problems

§7.1. Minimum and maximum

(7.1a)

For $R : A \rightarrow A$, $\text{est}(R) \triangleq \exists n (\epsilon \setminus R^\circ) : EA \rightarrow A$ $X \in \epsilon \setminus R^\circ \iff EX \in R^\circ$, and ϵX runs $y \rightarrow xS \rightarrow x$, member first, so its pair (y, x) is in R° exactly when $x R y$; at $R \triangleq \leq$ that is $x \leq y$ for every $y \in xs$, the least — $\epsilon \setminus \leq$ would give $y \leq x$, the greatest.

$\text{est}(R) = \exists n \subseteq (\frac{R}{S})^\circ$ x is the $\text{est}(R)$ of xs when $x \in xs$ and $xs \subseteq \Lambda(R)(x)$, the set of every y with $x R y$ — (3.1.1.12a) at $Z \triangleq R^\circ$



$xs (\text{est}(R)) x \iff x \in xs \wedge (\forall y \in xs. x R y)$ the same predicate under the same letter, $\min R$, so $\text{est}(R) = \min(R^\circ)$ once R is an arrow;

$xs (\text{est}(R)) x \iff (x \text{ in } xs) \text{ and all } x R \setminus : xs \text{ in } q$

$\text{est}(R) = \exists n \text{ all } R^\circ$ all $R \triangleq \epsilon \setminus R$, q 's all; the chains below keep it written $\epsilon \setminus$

$E(R) \triangleq \frac{\exists R}{S} : EA \rightarrow EB$, $xs E(R) ys \iff ys = \{y \mid \exists x \in xs. x R y\}$ the image of xs , (3.1.1.10a)

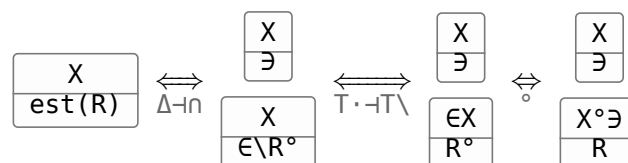
$P(R) : EA \rightarrow EB$, $xs P(R) ys \iff (\forall x \in xs. \exists y \in ys. x R y) \wedge (\forall y \in ys. \exists x \in xs. x R y)$ every x and every y has a partner, (5.4b)

(7.1b)

the law	what it says
$X \in \text{est}(R) \iff X \in \exists$ and $X^\circ \exists \in R$	in the set, and below every element of it
$\frac{1}{S} (\epsilon \setminus R) = R$	bounding a singleton is bounding its element
$\frac{S}{S} (\epsilon \setminus R) = S^\circ \setminus R$	bound S 's image without building the set
$\text{union} \triangleq \frac{\exists \exists}{S} : E(EA) \rightarrow EA$	flattens a set of sets
$\text{union} (\epsilon \setminus R) = \epsilon \setminus (\epsilon \setminus R)$	bound a union by bounding each member set
$\frac{1}{S} \text{est}(R) = 1 n R^\circ$ (7.4)	a singleton's minimum is its element, where R is reflexive
$\frac{S}{S} \text{est}(R) = S n (S^\circ \setminus R^\circ)$	$\frac{S}{S} \text{est}(R)$ at $S := 1$ an S -value that points to every S -value
$\frac{S}{S} \text{est}(R n S^\circ S) = \frac{S}{S} \text{est}(R)$	only R between values S gives one argument counts — context
$E(S) \text{est}(R) = (\exists S) n ((\exists S)^\circ \setminus R^\circ)$ (7.7)	the same for the image of a set $\frac{S}{S} \text{est}(R)$ at $S := \exists S$
$\text{Map}(f) \implies P(f) \text{est}(R) = \text{est}(f R f^\circ) f$	shunt a function through a minimum
$P(S) \text{est}(R) = (\exists S) n (\epsilon \setminus (S R^\circ))$ (7.9) R reflexive	fusion with the power relator $\exists$ is the only proof here that tabulates
$P(S) \text{est}(R) \subseteq (\exists S) n (\epsilon \setminus (S R^\circ))$	the half of the row above that costs nothing
$P(\text{est}(R)) \text{est}(R) \subseteq \text{union est}(R)$ R transitive	a minimum in each set, then a minimum of those
$R R \in R \implies P(\text{est}(R)) \text{est}(R) = P(\text{dom}(\text{est}(R))) \text{union est}(R)$ (7.12)	the same as an equality, once empty sets are dropped

§7.1.1. $X \in \text{est}(R) \iff X \in \exists$ and $X^\circ \exists \in R$

(7.1.1a)



§7.1.2. $\frac{1}{S} (\epsilon \setminus R) = R$

(7.1)

$$\frac{X}{\frac{1}{\exists} (\epsilon \backslash R)} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \frac{\frac{1}{\exists} X}{\epsilon \backslash R} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \frac{\epsilon \frac{1}{\exists} X}{R} \xleftrightarrow{\circ} \frac{(\frac{1}{\exists} \exists)^\circ X}{R} \xleftrightarrow{i \cdot \neg E} \frac{X}{R}$$

§7.1.3. $\frac{S}{\exists} (\epsilon \backslash R) = S^\circ \backslash R$

$\frac{S}{\exists}$ gathers the S -image of a point into one set and $\epsilon \backslash R$ asks that every member of that set be R -related to the target, so the set cancels and $S^\circ \backslash R$ asks it of the S -image directly.

(7.2)

$$\frac{X}{\frac{S}{\exists} (\epsilon \backslash R)} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \frac{\frac{S}{\exists} X}{\epsilon \backslash R} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \frac{\epsilon \frac{S}{\exists} X}{R} \xleftrightarrow{\circ} \frac{(\frac{S}{\exists} \exists)^\circ X}{R} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \frac{S^\circ X}{R} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \frac{X}{S^\circ \backslash R}$$

§7.1.4. **union** $(\epsilon \backslash R) = \epsilon \backslash (\epsilon \backslash R)$

(7.3)

$$\frac{X}{\text{union } (\epsilon \backslash R)} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \frac{\text{union}^\circ X}{\epsilon \backslash R} \xleftrightarrow{T^\circ \cdot \neg T^\circ} \frac{\epsilon \text{union}^\circ X}{R} \xleftrightarrow{\circ} \frac{(\text{union} \exists)^\circ X}{R} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}} \frac{(\exists \exists)^\circ X}{R} \xleftrightarrow{\circ, T^\circ \cdot \neg T^\circ} \frac{X}{\epsilon \backslash (\epsilon \backslash R)}$$

§7.1.5. $\frac{S}{\exists} \text{est}(R) = S_n(S^\circ \backslash R^\circ)$

(7.5)

$$\frac{X}{\frac{S}{\exists} \text{est}(R)} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \frac{\frac{S}{\exists} X}{\text{est}(R)} \xleftrightarrow{\Delta \neg n} \frac{\frac{S}{\exists} X}{\exists} \xleftrightarrow{f^\circ \cdot \neg f^\circ} \frac{\frac{X}{\frac{S}{\exists} \exists}}{\frac{S}{\exists} (\epsilon \backslash R^\circ)} \xleftrightarrow{\cdot \exists \neg \frac{S}{\exists}, (7.2)} \frac{\frac{X}{S}}{\frac{X}{S^\circ \backslash R^\circ}} \xleftrightarrow{\Delta \neg n} \frac{X}{S_n(S^\circ \backslash R^\circ)}$$

§7.1.6. $\frac{S}{\exists} \text{est}(R) = \frac{S}{\exists} \text{est}(R_n S^\circ S)$

(7.6)

$$\frac{X}{\frac{S}{\exists} \text{est}(R_n S^\circ S)} \xleftrightarrow{(7.5), \circ} \frac{X}{S_n(S^\circ \backslash (R^\circ \cap S^\circ S))} \xleftrightarrow{\Delta \neg n, T^\circ \cdot \neg T^\circ} \frac{\frac{X}{S}}{\frac{S^\circ X}{R^\circ \cap S^\circ S}} \xleftrightarrow{\Delta \neg n, S^\circ \cdot \text{monotone}} \frac{\frac{X}{S}}{\frac{S^\circ X}{R^\circ}} \xleftrightarrow{T^\circ \cdot \neg T^\circ, \Delta \neg n} \frac{X}{S_n(S^\circ \backslash R^\circ)} \xleftrightarrow{(7.5)} \frac{X}{\frac{S}{\exists} \text{est}(R)}$$

§7.1.7. **P(f)** $\text{est}(R) = \text{est}(f R f^\circ)$ **f**

(7.8)

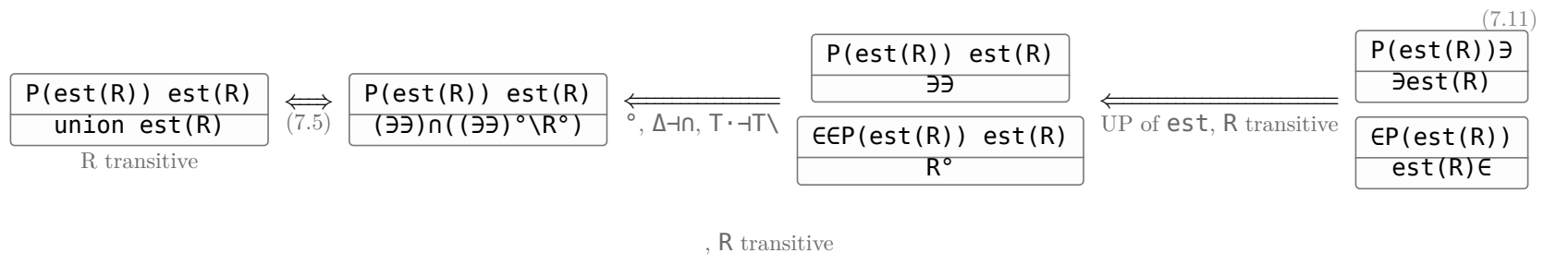
$$\frac{\text{P(f) est}(R)}{f \text{ a map}} \xrightarrow{P=E \text{ on maps, (7.5)}} \frac{(\exists f) n((\exists f)^\circ \backslash R^\circ)}{\circ, T^\circ \cdot \neg T^\circ, f^\circ \cdot \neg f^\circ} \frac{(\exists f) n(\epsilon \backslash (f R^\circ))}{\text{modular law, f simple}} \frac{(\exists n((\epsilon \backslash (f R^\circ)) f^\circ)) f}{\cdot f \neg f^\circ, \circ, \text{est}} \text{est}(f R f^\circ) f$$

§7.1.8. **P(S)** $\text{est}(R) \in (\exists S) n(\epsilon \backslash (S R^\circ))$

(7.10)

$$\frac{\text{P(S) est}(R)}{(\exists S) n(\epsilon \backslash (S R^\circ))} \xleftrightarrow{\Delta \neg n, T^\circ \cdot \neg T^\circ} \frac{\text{P(S) est}(R)}{\exists S} \xleftrightarrow{\text{UP of est}} \frac{\text{P(S) } \exists}{\exists S} \xleftrightarrow{\text{UP of est}} \frac{\text{EP(S) est}(R)}{S R^\circ} \xleftrightarrow{\text{UP of est}} \frac{\text{EP(S) }}{S \epsilon}$$

§7.1.9. **P(est(R))** $\text{est}(R) \in \text{union est}(R)$



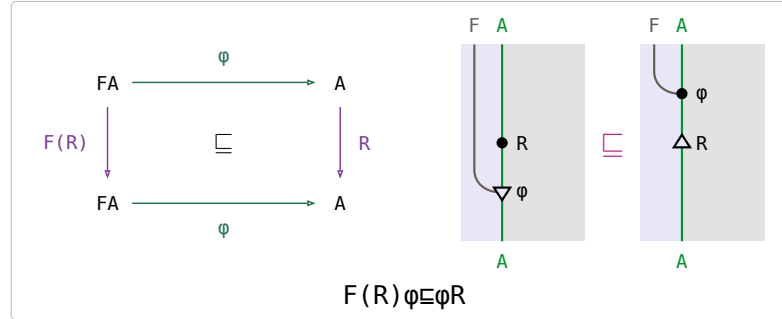
§7.2. Monotonic algebras

(7.2a)

An F -algebra $\varphi : FA \rightarrow A$ is **monotonic on** $R : A \rightarrow A$ when it is lax at R at $G := F$, $F := \text{Id}$: $F(R) \varphi \sqsubseteq \varphi R$. R is an **endorelation**, so $B = A$ and the two components φ_A, φ_B are the one arrow φ — an algebra, not a family.

For a map $f : FA \rightarrow A$ that is $f \circ F(R) f \in R$ (3.1.1.1a)'s $f^\circ \cdot \dashv f^\circ$ at $X := F(R)f, Y := R$, equivalently $F(R) \sqsubseteq f R f^\circ \cdot f^\circ \dashv f^\circ$ then $f^\circ \cdot \dashv f^\circ$, . . .
 $(\leq x \leq) + \sqsubseteq + \leq$ — addition on Nat is monotonic on $\leq$, which at the point level reads $c = a + b \wedge a \leq a' \wedge b \leq b' \implies c \leq a' + b'$.

(7.2b)



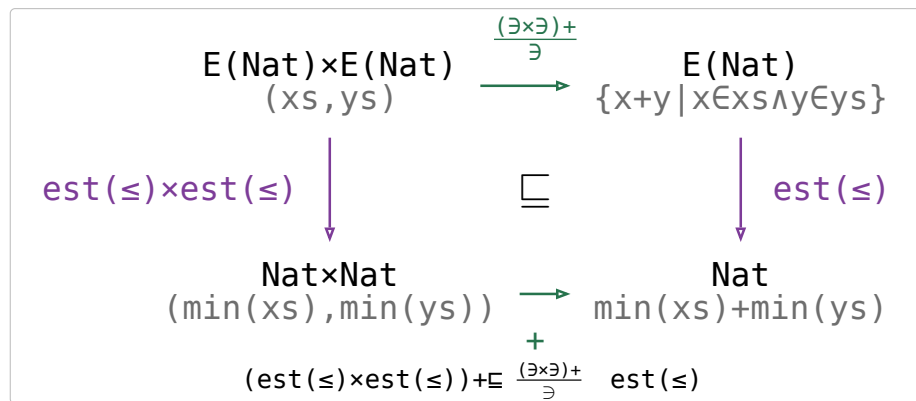
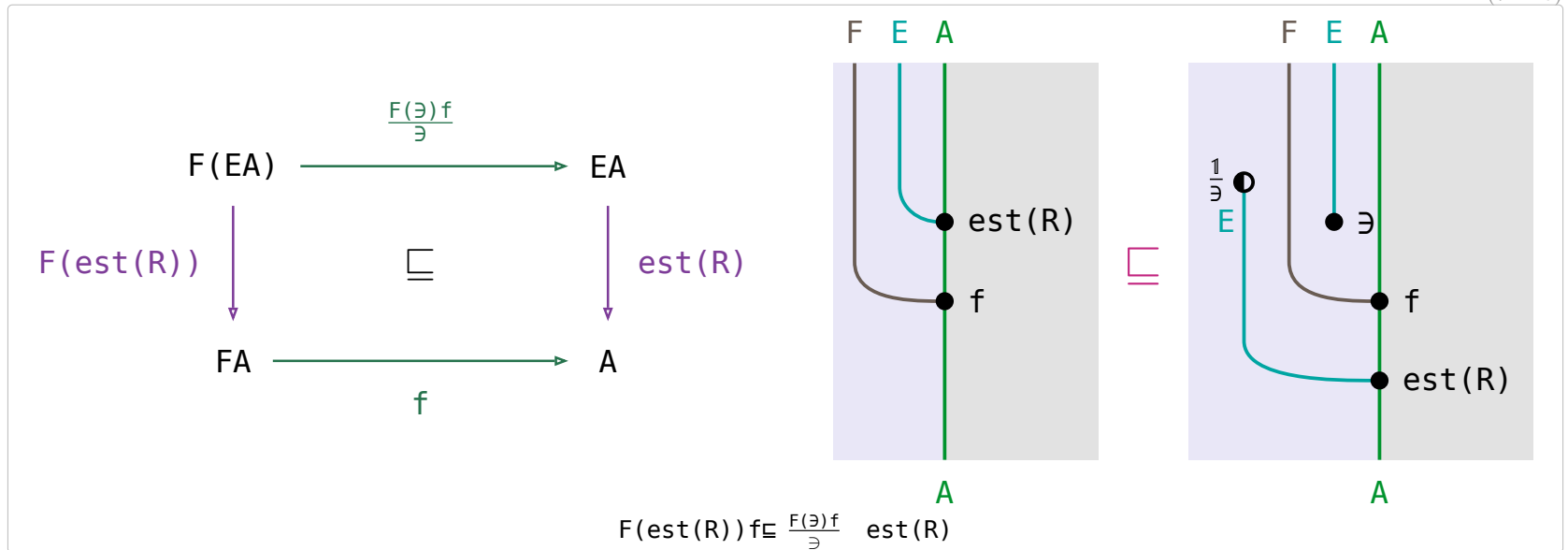
§7.2.1. Function f is monotonic on R iff it distributes over R

(7.2.1a)

$f : FA \rightarrow A$ **distributes over** R if $F(\text{est}(R)) f \sqsubseteq \frac{F(\exists)f}{\exists} \text{est}(R)$.

$+$ distributes over $\leq$, at the point level $\min(xs) + \min(ys)$ is a least element of $\{x + y \mid x \in xs \wedge y \in ys\}$ for xs, ys non-empty and $\min \triangleq \text{est}(\leq)$.

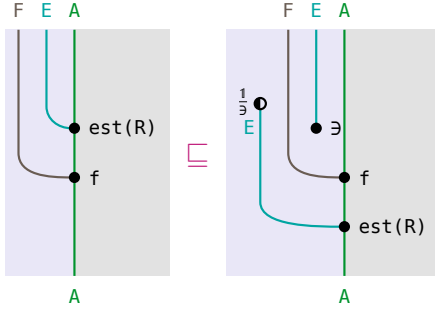
(7.2.1b)



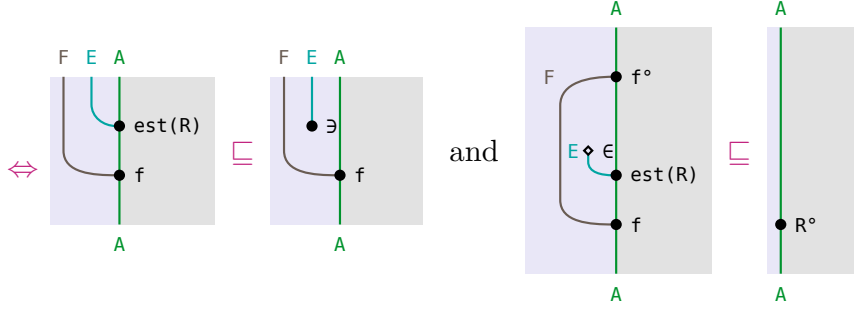
$$f^\circ F(R) f \sqsubseteq R \iff F(\text{est}(R)) f \sqsubseteq \frac{F(\exists) f}{\exists} \text{est}(R)$$

Theorem 7.1

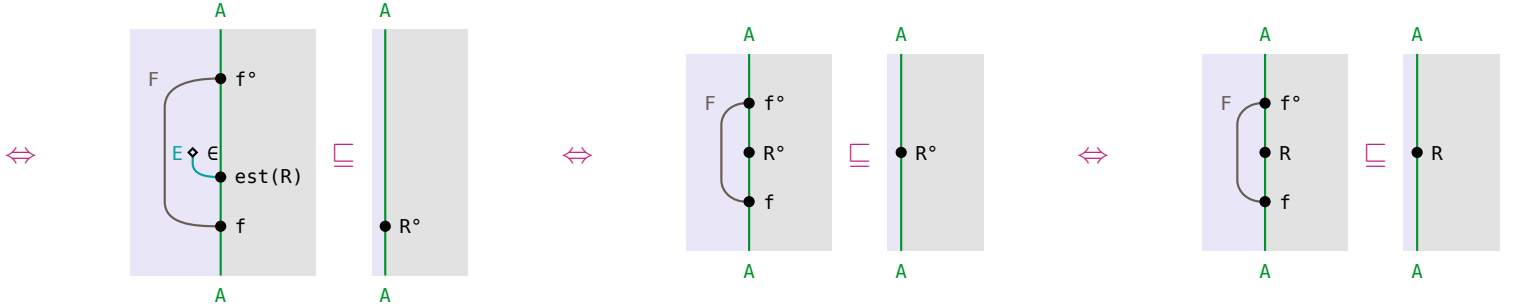
function f is monotonic over R if and only if it distributes over R ; f a map, R reflexive



f distributes over R — $\triangleq$ distributes — the fraction bent as (3.1.1.11a)



(7.5) splits the bound in two, (3.1.1.9a) moving $(F(\exists)f)^\circ$ across



$\text{est}(R) \sqsubseteq \exists$ — $\triangleq$ est — so the first conjunct drops

$(F(\exists)f)^\circ = f^\circ F(\epsilon)$ — $\triangleq$ $^\circ$ — and ϵ
 $\text{est}(R) = R^\circ$ — $\triangleq$ est , R reflexive

both sides conversed — $F(R^\circ)^\circ = F(R)$,
 (5.1b)

§7.2.3. $\text{takewhile}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \rightarrow \text{nil})])$

(7.2.3a)

name	definition	type	example	in words
F	$FX = 1 + A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$		nothing, or a head and a tail
nil, cons	$[A] ::= \text{nil} \mid \text{cons}(A, [A])$ $\triangleq \text{subseq}$	$1 \rightarrow [A],$ $A \times [A] \rightarrow [A]$	$\text{cons}(3, [1, 2]) = [3, 1, 2]$	the empty list; a head onto a tail
α	$[\text{nil}, \text{cons}]$	$F[A] \rightarrow [A]$		both constructors as one map
p	a coreflexive	$A \rightarrow A$	$p \triangleq \text{even} \rightarrow 2 \text{ p } 2$, and $3 \notin \text{dom}(p)$	$\{(a, a) \mid a \text{ passes the test}\}$
R	$\text{length} \leq \text{length}^\circ$, a preorder	$[A] \rightarrow [A]$	$[1] \text{ R } [1, 2]$	$xs \text{ R } ys \iff \text{length}(xs) \leq \text{length}(ys)$
$\rightarrow \text{nil}$	the constant nil — the second nil of prefix	$A \times [A] \rightarrow [A]$	$(\rightarrow \text{nil})(3, [1, 2]) = \text{nil}$	drop the pair, return nil
prefix	$([\text{nil}, \rightarrow \text{nil} \cup \text{cons}]) \triangleq \text{subseq}$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ prefix } [3, 1]$	$xs \text{ prefix } ys \iff \exists zs. xs = ys \# zs$ — at each cons, stop or keep the head
S	$[\text{nil}, \rightarrow \text{nil} \cup (p \times 1) \text{ cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) \text{ S } [4, 2]$, and $(3, [2]) \text{ S } \text{nil}$ only	prefix's algebra with one extra p — stop, or keep a head that passes p
$(g \rightarrow X, Y)$	X where g is defined and Y where it is not	$A \rightarrow B$		the test picks the branch

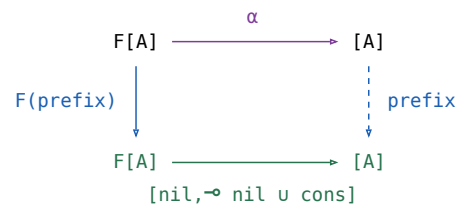
$\text{nil} \text{ R } \tau$, $\text{nil} \text{ R}^\circ = \text{nil}$ nil is the shortest list — below every list, and above only itself, so it loses every $\text{est}(\text{R}^\circ)$

prefix = $([\text{nil}, \rightarrow \text{nil} \cup \text{cons}])$

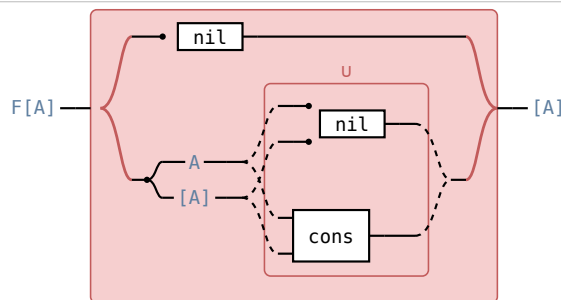
(7.2.3b)

the fold whose algebra, at each **cons**, stops with **nil** or keeps the head: $xs \text{ prefix } ys \iff \exists zs. xs = ys \# zs$

the defining square $\alpha \text{ prefix} = F(\text{prefix})[\text{nil}, \rightarrow \text{nil} \cup \text{cons}]$ — build the list and then take a prefix, or take a prefix of the tail and then rebuild with the algebra

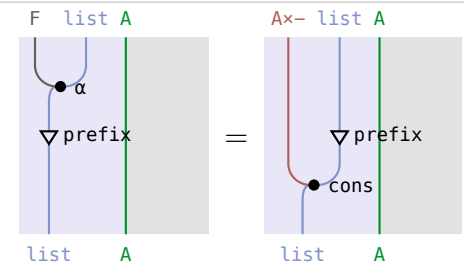


$F = 1 + A \times -$, so $F(\text{prefix}) = 1 + 1 \times \text{prefix}$: the head passes, the fold recurses on the tail alone



the algebra $[\text{nil}, \rightarrow \text{nil} \cup \text{cons}] : F([A]) \rightarrow [A]$ as a circuit: nil on the 1 branch; on a pair (a, ys') two outputs, nil and $\text{cons}(a, ys')$

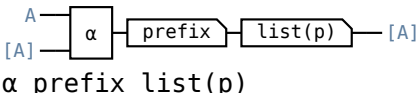
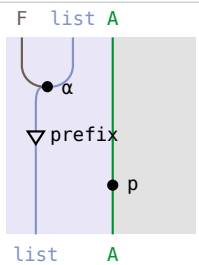
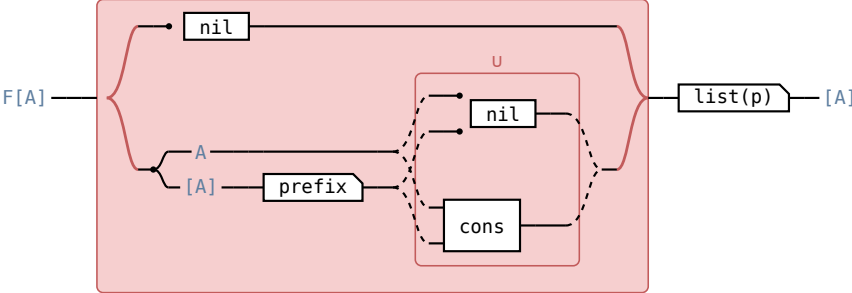
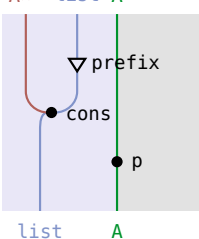
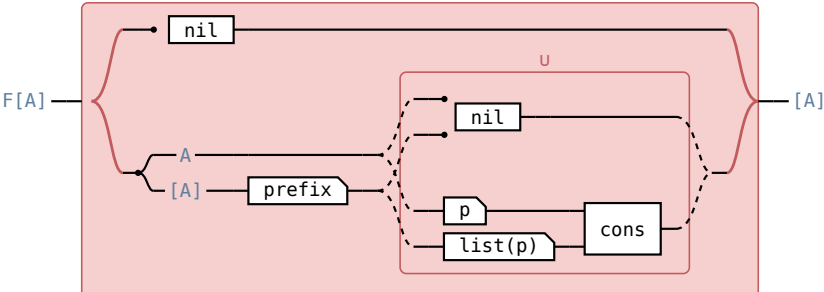
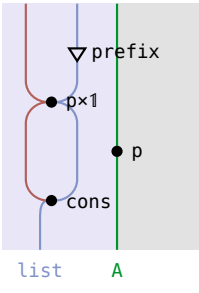
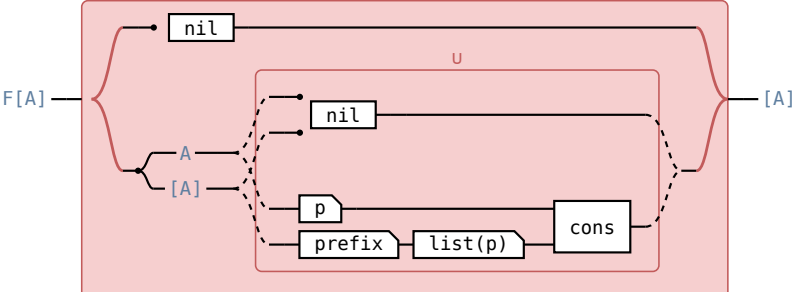
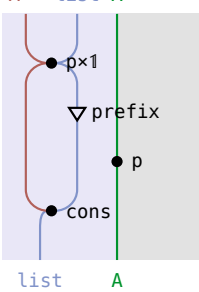
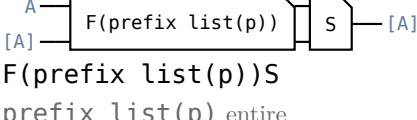
Hinze–Marsden



the **cons** operand of $\rightarrow \text{nil} \cup \text{cons}$; $\rightarrow \text{nil}$ makes a constant and draws nothing

$\text{nil} \text{ prefix } ys \iff ys = \text{nil}$
 $\text{cons}(a, xs') \text{ prefix } ys \iff ys = \text{nil} \vee \exists ys'. xs' \text{ prefix } ys' \wedge ys = \text{cons}(a, ys')$

a right fold: α° peels, $F(\text{prefix})$ recurses, the algebra runs on the way back; init^* ($\triangleq \text{subseq}$) is the tail-recursive form
 lax natural only, $\text{list}(\text{R}) \text{ prefix} \sqsubseteq \text{prefix}$
 $\text{list}(\text{R})$: $\text{list}(\text{R})$ first needs an R-image of every element, **prefix** first may have dropped the ones without

$\alpha \text{prefix list}(p) = F(\text{prefix list}(p))S$ (7.2.3c)	
building the list and then keeping a p -passing prefix of it is keeping one of the tail first, and then building with S this same diagram is subseq 's: algebra $[\text{nil}, \pi_2 \cup \text{cons}]$, type $[A] \rightarrow [A]$	
circuit — the fork is $F([A]) = 1 + A \times [A]$: nil above, the pair below	Hinze–Marsden
 $\alpha \text{prefix list}(p)$	 the cons branch alone, without $1 +$ or $\multimap \text{nil}$
 $F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup \text{cons}] \text{list}(p)$ defining equation	 the cons operand of $\multimap \text{nil} \cup$ cons
 $F(\text{prefix}) [\text{nil}, \multimap \text{nil} \cup (p \times \text{list}(p)) \text{cons}]$ $\text{list}(p)$ through cons	 the $(p \times \text{list}(p))$ cons operand of $\multimap \text{nil} \cup$ $(p \times \text{list}(p))$ cons
 $[\text{nil}, \multimap \text{nil} \cup (p \times (\text{prefix list}(p))) \text{cons}]$ relator, prefix entire	
 $F(\text{prefix list}(p))S$ prefix list(p) entire	

$\triangleq$ **reduce** reads that off as $\text{prefix list}(p) = (S)$. fusion cannot: $\text{list}(p)$ is not entire, $(1 \times \text{list}(p)) \multimap \text{nil} \sqsubseteq \multimap \text{nil}$, and no algebra meets the side condition. ,

$(1 \times R^\circ)(\multimap \text{nil} \cup (p \times 1)\text{cons}) \sqsubseteq (\multimap \text{nil} \cup (p \times 1)\text{cons})R^\circ$ (7.2.3d)	
shortening the tail and then taking the step lands inside taking the step and then shortening the result	
circuit — the cons branch of $F(R^\circ)S \sqsubseteq SR^\circ$	reason
$(1 \times R^\circ)(\multimap \text{nil} \cup (p \times 1)\text{cons})$	
$(1 \times R^\circ) \multimap \text{nil} \cup (p \times R^\circ)\text{cons}$	each operand is reached on its own (3.1.1.1a) — and $(1 \times R^\circ)$ $(p \times 1)$ is p and R° on the pair's two strands at once
$\multimap \text{nil} \cup (p \times R^\circ)\text{cons}$	$\multimap$ is the greatest arrow into 1 , so $(1 \times R^\circ) \multimap \sqsubseteq \multimap$
$\multimap \text{nil} \cup (p \times 1)\text{cons} R^\circ$	$\text{cons length} = (1 \times \text{length})\pi_2$ succ with succ monotone — a shorter tail makes a shorter list
$\multimap \text{nil} R^\circ \cup (p \times 1)\text{cons} R^\circ$	$\text{nil } R^\circ = \text{nil} \triangleq \text{takewhile}$ — so the constant branch may carry the R° the other one already has
$(\multimap \text{nil} \cup (p \times 1)\text{cons})R^\circ$	one R° past the join is the two inside it (3.1.1.1a)

the nil branch, which no row above draws, is $\text{nil} \sqsubseteq \text{nil } R^\circ$.

$\frac{\text{S}}{\text{S}} \text{ est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]$ the longest of the lists the algebra allows is the cons where the head passes p, and nil where it does not	
formula	reason
$ \begin{array}{c} A \\ [A] \quad \frac{1}{\text{S}} \text{ EF}[A] \text{ E}(S) \text{ E}[A] \text{ est}(R^\circ) [A] \end{array} $	
$ \begin{array}{c} = \\ F[A] \text{ } \left\{ \begin{array}{l} \frac{1}{\text{S}} \text{ E1} \text{ E}(\text{nil}) \text{ E}[A] \text{ est}(R^\circ) \\ \frac{A}{[A]} \frac{1}{\text{S}} \text{ E}(A \times [A]) \text{ E}(\neg \text{nil} \cup (p \times 1) \text{cons}) \text{ E}[A] \text{ est}(R^\circ) \end{array} \right\} [A] \end{array} $ $[\frac{\text{nil}}{\text{S}} \text{ est}(R^\circ), \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{\text{S}} \text{ est}(R^\circ)]$	coproduct of maps
$ \begin{array}{c} = \\ F[A] \text{ } \left\{ \begin{array}{l} \text{nil} \\ \frac{A}{[A]} \frac{1}{\text{S}} \text{ E}(A \times [A]) \text{ E}(\neg \text{nil} \cup (p \times 1) \text{cons}) \text{ E}[A] \text{ est}(R^\circ) \end{array} \right\} [A] \end{array} $ $[\text{nil}, \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{\text{S}} \text{ est}(R^\circ)]$	singleton, R° reflexive
$ \begin{array}{c} = \\ F[A] \text{ } \left\{ \begin{array}{l} \text{nil} \\ \frac{A}{[A]} \frac{1}{\text{S}} \text{ E}(A \times [A]) \text{ E}(\neg \text{nil} \cup (p \times 1) \text{cons}) \text{ E}[A] \text{ est}(R^\circ) \end{array} \right\} [A] \end{array} $ $[\text{nil}, \frac{\neg \text{nil} \cup (p \times 1) \text{cons}}{\text{S}} \text{ est}(R^\circ)]$	$\text{nil } R = \top$

the set is $\{\text{nil}\}$ where p fails on the head and $\{\text{nil}, \text{cons}(a, xs)\}$ where it holds, and nil loses the second — $\triangleq \text{est}$ at a two-element set.

(7.2.3f)

name	definition	type	example	in words
S	$[\text{nil}, \neg \text{nil} \cup (p \times 1) \text{cons}] \triangleq \text{takewhile}$	$F[A] \rightarrow [A]$	$(4, [2]) \text{ S } [4, 2]$, and $(3, [2]) \text{ S nil only}$	prefix's algebra with one extra p — stop, or keep a head that passes p

$\text{takewhile} \triangleq \frac{\text{prefix list}(p)}{\text{S}} \text{ est}(R^\circ)$ takewhile = $([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})])$ takewhile: takewhile(p)(x) returns the longest prefix of x with the property that all its elements satisfy p; the catamorphism is the standard implementation.	
$ \begin{array}{c} \text{list } A \\ \frac{1}{\text{S}} \text{ E} \text{ } \nabla \text{ prefix} \\ \bullet p \\ \bullet \text{est}(R^\circ) \end{array} $ $\text{list } A \triangleq \text{est}$ (a) greedy theorem at R° — (7.2.3d)	$ \begin{array}{c} \text{list } A \\ \frac{1}{\text{S}} \text{ E} \text{ } \nabla (S) \\ \bullet \text{est}(R^\circ) \end{array} $ (7.2.3c) $ \begin{array}{c} \text{list } A \\ \frac{1}{\text{S}} \text{ E} \text{ } \nabla (\frac{\text{S}}{\text{S}} \text{ est}(R^\circ)) \end{array} $ (a) $ \begin{array}{c} \text{list } A \\ \frac{1}{\text{S}} \text{ E} \text{ } \nabla ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})]) \end{array} $ (7.2.3e)

$\text{takewhile}(p)^\circ \text{ takewhile}(p) \sqsubseteq \text{prefix}^\circ \text{ prefix} \sqcap R \sqcap R^\circ \sqsubseteq 1$
 $\text{takewhile}(p) \sqsubseteq \text{prefix list}(p)$ and $(\text{prefix list}(p))^\circ \text{ takewhile}(p) \in R$ — (7.5) at $\text{est}(R^\circ)$ — and two prefixes of one list of equal length are equal, so **takewhile**(p) is simple: the longest, not a longest.

$\frac{\text{prefix list}(p)}{\text{S}} \text{ est}(R^\circ)$ entire
 nil is always a p -prefix and R is connected on the prefixes of one list, so the longest exists.

$X \sqsubseteq Y, X \text{ entire}, Y \text{ simple} \implies X = Y$

$([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \neg \text{nil})])$ is a reduce of maps, hence entire — what turns the $\sqsubseteq$ above into the heading's $=$.

§7.2.4. $\text{mss} = ([\text{zero}(1, \frac{1}{\text{S}}), ((1 \times \pi_1) \oplus, ((1 \times \pi_1) \oplus \frac{1}{\text{S}}, \pi_2 \pi_2) \text{ cup})]) \pi_2 \text{ est}(\geq)$

(7.2.4a)

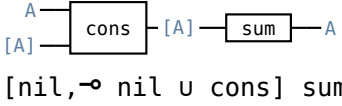
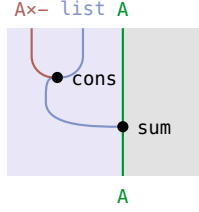
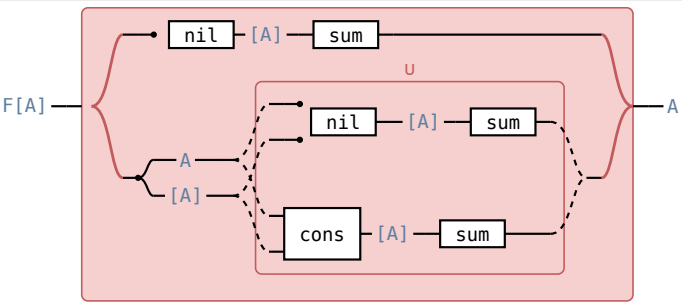
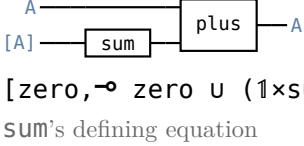
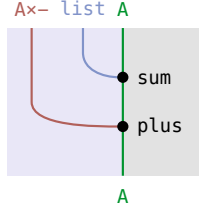
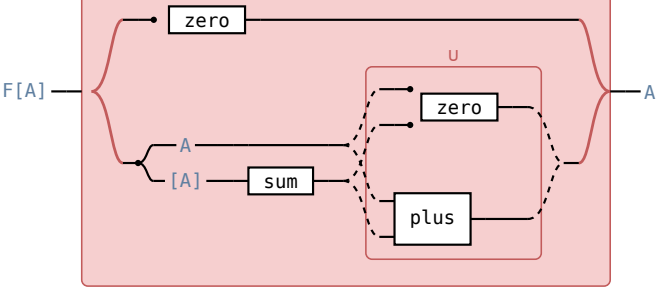
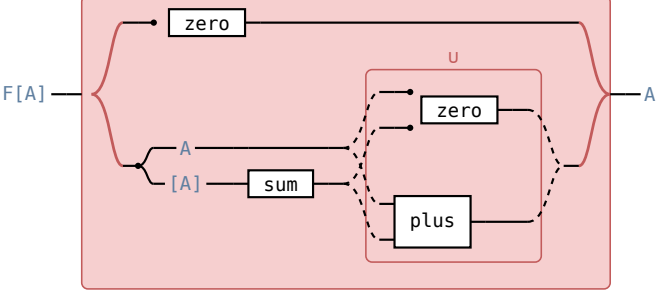
$FX=1+AX$, $\alpha \triangleq [\text{nil}, \text{cons}]$, $\text{sum} = ([\text{zero}, \text{plus}])$ and $\text{segment} = \text{suffix prefix}$ from (3.1.1.12b) and (7.2.5e).

$\text{head} \triangleq \text{cons} \circ \pi_1$, $\text{wrap} \triangleq (\mathbb{1}, \rightarrow \text{nil}) \text{ cons}$ the head of a list and the one-element list, beside $\triangleq \text{subseq}$'s $\text{tail} \triangleq \text{cons} \circ \pi_2$

$\oplus \triangleq \frac{\rightarrow \text{zero} \cup \text{plus}}{\exists}$ $\text{est}(\geq)$ the set at (a, b) is $\{0, a+b\}$, so $\oplus$ is the larger of the two,

$\text{mss} = \frac{\text{suffix}}{\exists} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \text{est}(\geq)$ (7.2.4b)	
maximum segment sum problem: using segment=suffix prefix , the specification is expressed in this form	
formula — one wire from [A] to A, its type written along it	reason
$[A] \xrightarrow{\frac{1}{\exists}} E[A] \xrightarrow{E(\text{segment sum})} EA \xrightarrow{\text{est}(\geq)} A$	
$= [A] \xrightarrow{\frac{1}{\exists}} E[A] \xrightarrow{E(\text{suffix prefix sum})} EA \xrightarrow{\text{est}(\geq)} A$	segment=suffix prefix (7.2.5e), $\triangleq \text{mss}$
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\text{prefix sum}) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	absorption (3.1.1.10a) — $\text{frac}(S, \exists)$ $E(R) = \text{frac}(SR, \exists)$ at $S := \text{suffix}, R := \text{prefix}$ sum
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{\text{prefix sum}}{\exists}) \xrightarrow{EEA} \text{union} \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	$\frac{R}{\exists} \exists = R$, union $= E(\exists)$ (3.1.1.10a)'s $\text{frac}(R, \exists) \exists = R$ at $R := \text{prefix sum}$ and $E(R) \triangleq \text{frac}(\exists R, \exists)$; (7.1b)'s union $\triangleq \text{frac}(\exists \exists, \exists)$; the middle equality is $\triangleq$ relator's $F(RS) = F(R)F(S)$ at $F := E$
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{\text{prefix sum}}{\exists}) \xrightarrow{EEA} E(\text{est}(\geq)) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	(7.1b), the sets non- empty
$= [A] \xrightarrow{\frac{1}{\exists}} E(\text{suffix}) \xrightarrow{E[A]} E(\frac{\text{prefix sum}}{\exists} \text{est}(\geq)) \xrightarrow{EA} \text{est}(\geq) \xrightarrow{A}$	relator $\triangleq$ relator — $F(RS) = F(R)F(S)$ at $F := E$

The **union** step is (7.1b)'s $P(\text{est}(R)) \text{ est}(R) = P(\text{dom}(\text{est}(R))) \text{ union est}(R)$ — every suffix has the empty prefix, so **Dom** is $\mathbb{1}$ here — and $P(f) = E(f)$ at the map it is applied to ((5.4d)).

$[nil, \multimap nil \text{ u } cons] \text{sum} = F(\text{sum})S$ (7.2.4c)	
fusion: prefix expressed as a catamorphism on cons-lists, $([nil, \multimap nil \text{ u } cons])$, and this is the fusion condition that expresses prefix sum as a catamorphism	
circuit — the fork is the bracket's case split $F([A]) = 1 + A \times [A]$: nil above, the pair and its u below	Hinze–Marsden
 $[nil, \multimap nil \text{ u } cons] \text{sum}$	 the cons operand of $\multimap nil \text{ u } cons$
 $[nil \text{ sum}, \multimap nil \text{ sum } \text{u } cons \text{ sum}]$ coproduct of maps, composition over U	
 $[zero, \multimap zero \text{ u } (1 \times \text{sum}) \text{ plus}]$ sum's defining equation	 the $(1 \times \text{sum})$ plus operand of $\multimap zero \text{ u } (1 \times \text{sum})$ plus
 $[zero, (1 \times \text{sum}) (\multimap zero \text{ u } plus)]$ $(1 \times \text{sum}) \multimap = \multimap$, sum entire	
 $F(\text{sum}) [zero, \multimap zero \text{ u } plus]$ relator	

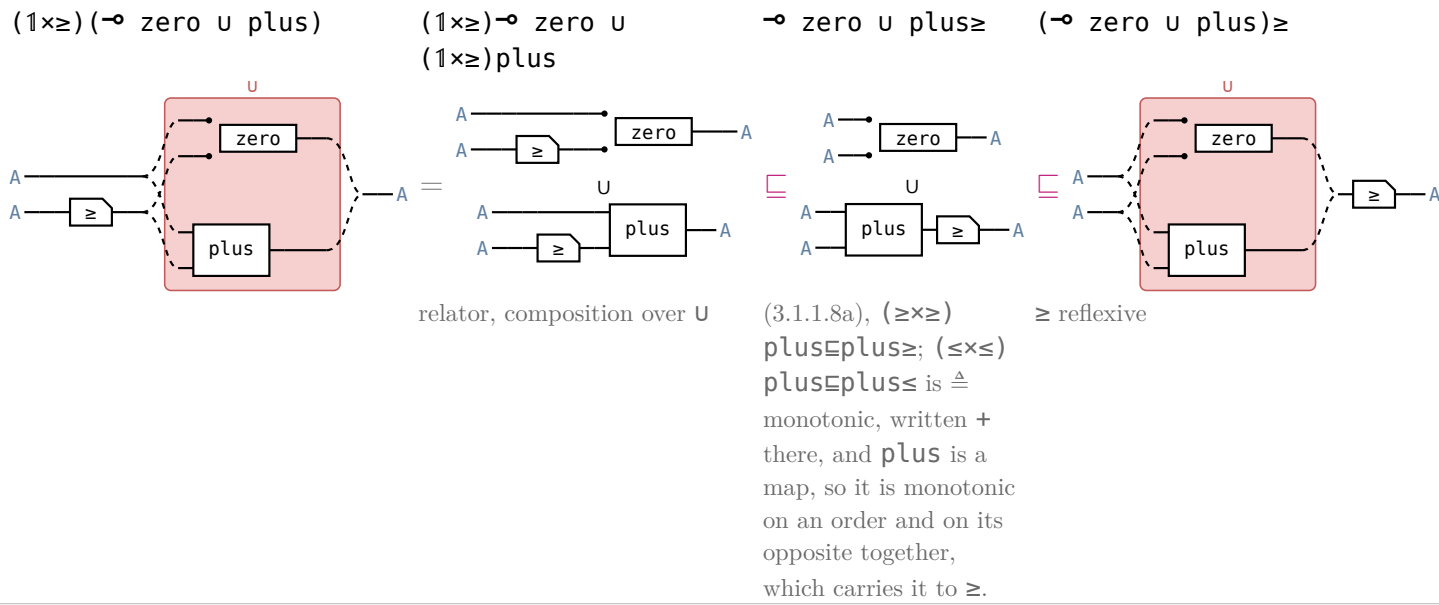
fusion at $R := [nil, \multimap nil \text{ u } cons]$, $S := \text{sum}$: the side condition, so **prefix sum** $= ([zero, \multimap zero \text{ u } plus])$. **prefix** is the reduce, **sum** the map fused into it — the intermediate list is gone.

$$(1 \times \geq)(\neg \text{zero} \cup \text{plus})E(\neg \text{zero} \cup \text{plus})\geq$$

(7.2.4d)

monotonic algebra: an **F**-algebra **S** is monotonic on a relation **R** if $F(R)SESR$ — the **plus** branch of $F(\geq)SE\geq$;
the **zero** branch is $\text{zero}E\text{zero}\geq$

circuit — the head above, the running sum below; the tape is the **u**



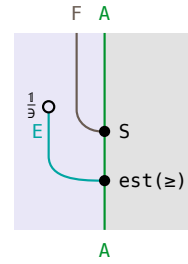
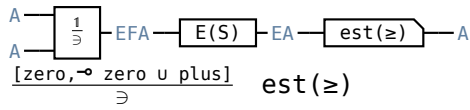
$$\frac{S}{\geq} \text{est}(\geq) = [\text{zero}, \oplus]$$

(7.2.4e)

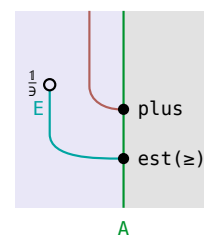
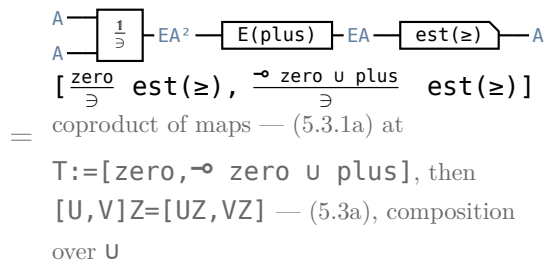
the largest sum the algebra offers is zero from nothing and, from a head and a running sum, the larger of zero and the head added to it

circuit — the tape is the coproduct:
zero's branch above, **plus**'s below

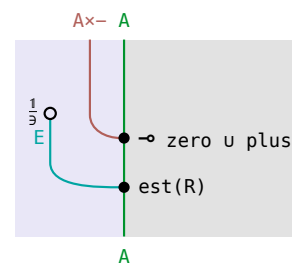
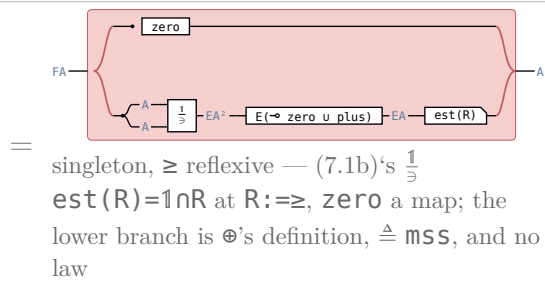
Hinze–Marsden



the **zero** arm of the bracket



the **plus** operand of the lower arm's $\neg \text{zero} \cup \text{plus}$, under its $1\% \ni E(\dots)$ and $\text{est}(\geq)$



with (7.2.4d) the greedy theorem gives $([\text{zero}, \oplus]) \sqsubseteq \frac{\text{prefix sum}}{\geq} \text{est}(\geq)$ — and Exercise 7.40 makes it an equality.

$[\text{nil}, \text{cons}] (\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle \rangle) \rangle) = F(\langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle \rangle) \rangle) k \quad (7.2.4f)$ $k \triangleq [\text{zero} \langle 1, \frac{1}{\supset} \rangle, \langle w, \langle w \frac{1}{\supset}, \pi_2 \pi_2 \rangle \text{cup} \rangle], w \triangleq (1 \times \pi_1) \oplus: \text{the value at the whole list, paired with the set of the values at its suffixes, runs } k\text{'s recursion.}$	
the equation at that branch	why
$\text{nil}(\langle g, \frac{\text{suffix}}{\supset} E(g) \rangle) = \text{zero} \langle 1, \frac{1}{\supset} \rangle$	nil has one suffix, itself, and $g(\text{nil}) = \text{zero}$, so the set is the singleton $\{\text{zero}\}$
$\text{cons}(\langle g, \frac{\text{suffix}}{\supset} E(g) \rangle) = (1 \times \langle g, \frac{\text{suffix}}{\supset} E(g) \rangle) \langle w, \langle w \frac{1}{\supset}, \pi_2 \pi_2 \rangle \text{cup} \rangle$	$g(\text{cons}(a, x)) = a \oplus (g(x))$, which is w reading $g(x)$ off π_1 ; and the suffixes of $\text{cons}(a, x)$ are $\text{cons}(a, x)$ itself, whose value is that same w , together with those of x , which $\pi_2 \pi_2$ carries — so the two sets meet at cup

$\text{mss} = \langle k \rangle \pi_2 \text{est}(\geq) \quad \text{Exercise 7.40}$	
maximum segment sum problem: $\frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle \rangle)$ expressed as the catamorphism $\langle k \rangle$, hence mss implemented by a linear-time algorithm, $\oplus \triangleq \frac{\text{zero} \cup \text{plus}}{\supset} \text{est}(\geq) \triangleq \text{mss}; k \text{ and } w \triangleq (7.2.4f).$	
(a) greedy theorem at $R: \geq$ — (7.2.4d), (7.2.4e), Exercise 7.39 (b) $\langle k \rangle = \langle \langle [\text{zero}, \oplus] \rangle, \frac{\text{suffix}}{\supset} E(\langle \langle [\text{zero}, \oplus] \rangle \rangle) \rangle \triangleq \text{reduce}, (7.2.4f)$	

one fold builds the $n+1$ running maxima and the final $\text{est}(\geq)$ reads them in one more pass, so mss is linear.

§7.2.5. $\text{filter}(p) = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)])$

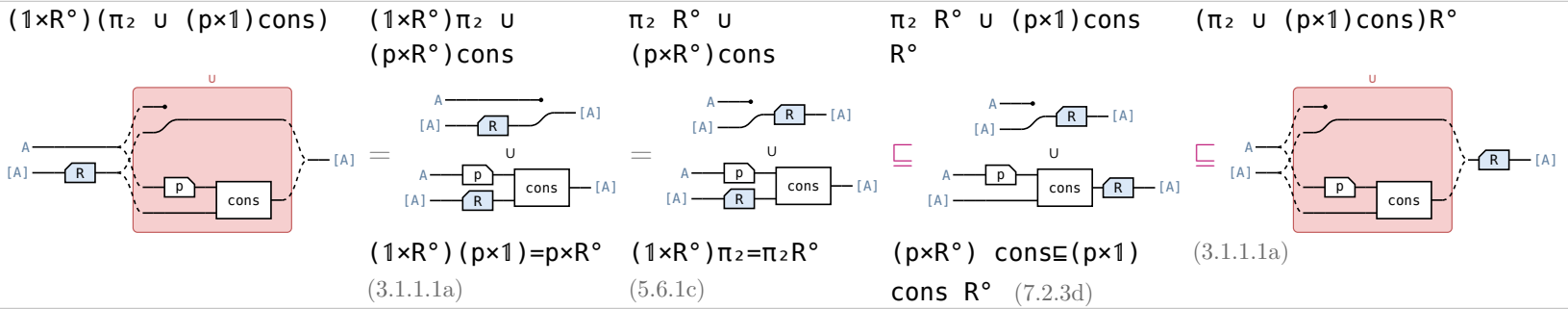
(7.2.5a)

name	definition	type	example	in words
F, α, p, R	as in $\triangleq \text{takewhile}$			
π_2	π_2 where $\triangleq \text{takewhile}$ has $\rightarrow \text{nil}$	$A \times [A] \rightarrow [A]$	$\pi_2(3, [1, 2]) = [1, 2]$	drop the head, keep the tail
subseq	$([\text{nil}, \pi_2 \cup \text{cons}]) \triangleq \text{subseq}$	$[A] \rightarrow [A]$	$[3, 1, 2] \text{ subseq } [3, 2]$	$xs \text{ subseq } ys \iff ys \text{ is } xs \text{ with elements dropped}$ — at each cons , drop the head or keep it
S	$[\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$	$F[A] \rightarrow [A]$	$(4, [2]) S [2]$ and $(4, [2]) S [4, 2]$, but $(3, [2]) S [2]$ only	subseq's algebra with one extra p — drop the head, or keep a head that passes p
$1 \sqsubseteq \pi_2 R \text{ cons}^\circ$				the tail is one shorter than the cons , so π_2 loses the $\text{est}(R^\circ)$ at every step — where $\triangleq \text{takewhile}$'s loser is nil

$$(1 \times R^\circ)(\pi_2 \cup (p \times 1) \text{ cons}) \sqsubseteq (\pi_2 \cup (p \times 1) \text{ cons}) R^\circ$$

(7.2.5b)

shortening the tail and then taking the step lands inside taking the step and then shortening the result

formula — the cons branch of $F(R^\circ) S \sqsubseteq S R^\circ$; reason under each circuitthe nil branch: $\text{nil} \sqsubseteq \text{nil} R^\circ$.

$\frac{S}{S} \text{ est}(R^\circ) = [\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)]$ the longest of the lists the algebra allows is the cons where the head passes p, and the tail where it does not	
formula	reason
$[\frac{\text{nil}}{S} \text{ est}(R^\circ), \frac{\pi_2 \cup (p \times 1) \text{ cons}}{S} \text{ est}(R^\circ)]$	$S = [\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}] \triangleq \text{filter}$ — and the $\frac{\%}{S}$ of a coproduct of maps is the coproduct of their $\frac{\%}{S}$ (5.3.1a)
$[\text{nil}, \frac{\pi_2 \cup (p \times 1) \text{ cons}}{S} \text{ est}(R^\circ)]$	$\text{nil} \frac{\%}{S}$ is the singleton $\{\text{nil}\}$, and $\text{est}(R^\circ)$ of a singleton is its element because R° is reflexive $\triangleq \text{est}$
	$\frac{\pi_2 \cup (p \times 1) \text{ cons}}{S} = \left(\frac{\pi_2}{S}, \frac{(p \times 1) \text{ cons}}{S} \right) \text{ cup} \triangleq \text{cup}$
	$1 \sqsubseteq \pi_2 R \text{ cons}^\circ \triangleq \text{filter}$ — $x s \ R \text{ cons}(a, x s)$, so $\text{est}(R^\circ)$ returns the cons where p a puts it in the set and xs where the set is $\{x s\} \triangleq \text{est}$

the head is dropped, not the whole tail: that is the one place π_2 shows against (7.2.3e)'s $\rightarrow \text{nil}$.

name	definition	type	example	in words
S	$[\text{nil}, \pi_2 \cup (p \times 1) \text{ cons}]$ $\triangleq \text{filter}$	$F[A] \rightarrow [A]$	$(4, [2]) \ S \ [2]$ and $(4, [2]) \ S \ [4, 2]$, but $(3, [2]) \ S \ [2]$ only	subseq's algebra with one extra p — drop the head, or keep a head that passes p

$\text{filter} = ([\text{nil}, (\pi_1 p \rightarrow \text{cons}, \pi_2)])$ filter: filter(p) x returns the longest subsequence of x with the property that all its elements satisfy p; the catamorphism is the standard program; R a preorder.	
(a) $\text{subseq list}(p) = (S)$ — (7.2.3c)	(b) greedy theorem at R° — (7.2.5b)
(7.2.5e)	(7.2.5c)

the catamorphism is entire and **filter(p)** simple, so $\sqsubseteq$ is $=$.

$(\text{subseq list}(p))^\circ (\text{subseq list}(p)) \cap R \cap R^\circ \sqsubseteq 1$ fails — two **p**-subsequences of one list can be of equal length and different — so §7.2.3's uniqueness argument does not transfer. What survives $\text{est}(R^\circ)$ is the one keeping **exactly** the passing elements: a subsequence that drops a passing element is beaten by the one that keeps it.

definition	type	note
$\text{segment} = \text{suffix prefix}$	$[A] \rightarrow [A]$	A contiguous stretch of xs : a suffix, then a prefix of that.
$\text{filter}(p) \triangleq \frac{\text{subseq list}(p)}{\ni} \text{est}(R^\circ)$	$[A] \rightarrow [A]$	The longest subsequence of xs whose every element passes p . $\text{est}(R^\circ)$ is $\triangleq \text{est}$
$R \triangleq \text{length} \leq \text{length}^\circ$	$[A] \rightarrow [A]$	The preorder filter and takewhile maximise over: the longer list wins. $\geq \triangleq \leq^\circ$
$\text{takewhile}(p) \triangleq \frac{\text{prefix list}(p)}{\ni} \text{est}(R^\circ)$	$[A] \rightarrow [A]$	The same with prefix for subseq : the longest prefix whose every element passes p .
$\text{mss} \triangleq \frac{\text{segment sum}}{\ni} \text{est}(\geq)$	$[A] \rightarrow A$	Maximum segment sum. $\text{segment} = \text{suffix prefix}$ splits it into $\frac{\text{prefix sum}}{\ni} \text{est}(\geq)$ on each suffix.

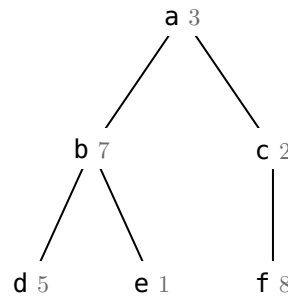
§7.3. Planning a company party

(7.3a)

definition	type	note
$\text{tree } A ::= \text{node } (A, [\text{tree } A])$	$\mathcal{A} \rightarrow \mathcal{A}$	The company hierarchy: an employee, and the list of subtrees under them.
$F(A, B) = A \times [B]$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor tree folds: an employee beside the recursive position, one layer deep.
rating	$A \rightarrow \text{Int}$	What one employee is worth as a guest.
$\text{cost} = \text{list}(\text{rating}) \text{sum}$	$[A] \rightarrow \text{Int}$	What a guest list is worth.
$R = \text{cost} \leq \text{cost}^\circ$	$[A] \rightarrow [A]$	The preorder the guest list is maximised over.
$\text{choose} \triangleq \pi_1 \cup \pi_2$	$[A] \times [A] \rightarrow [A]$	Takes one of the two parties a subtree returns.
$\text{include} = (1 \times (\text{list}(\pi_2) \text{concat})) \text{cons}$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that invites the root, which puts every immediate subtree's root out. A map.
$\text{exclude} = (1 \times (\text{list}(\text{choose}) \text{concat})) \pi_2$	$F(A, [A] \times [A]) \rightarrow [A]$	The party that leaves the root out, so each subtree is free to choose. Not a map.
$S \triangleq (\text{include}, \text{exclude})$	$F(A, [A] \times [A]) \rightarrow [A] \times [A]$	The algebra: one step returns both parties of a subtree at once.
$\text{party} = (S) \text{choose}$	$\text{tree}(A) \rightarrow [A]$	Every guest list the president's ruling allows.
the specification $\frac{\text{party}}{\exists} \text{est}(R^\circ)$	$\text{tree}(A) \rightarrow [A]$	A guest list of greatest total conviviality.

§7.3.1. include and exclude

(7.3.1a)



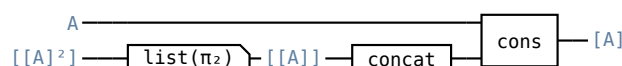
the small grey number is the employee's rating

(7.3.1b)

node	the algebra's input $A \times [[A] \times [A]]$	include	exclude
d, e, f	$(d, [])$	$[d]$	$[]$
b	$(b, ([[d], []], ([e], []]))$	$[b]$	$[d, e], [d], [e], []$
c	$(c, ([[f], []]))$	$[c]$	$[f], []$
a	$(a, ([[b], \dots], ([c], \dots)))$	$[a] \# \text{exclude}(b) \# \text{exclude}(c)$	$\text{choose}(b) \# \text{choose}(c)$

with those ratings, $\text{est}(R^\circ)$ keeps $([b]=7, [d, e]=6)$ at **b**, where **include** wins, and $([c]=2, [f]=8)$ at **c**, where **exclude** wins, so at the root **include**=[a,d,e,f]=17 beats **exclude**=[b,f]=15, and **choose** takes 17.

(7.3.1c)



(7.3.1d)

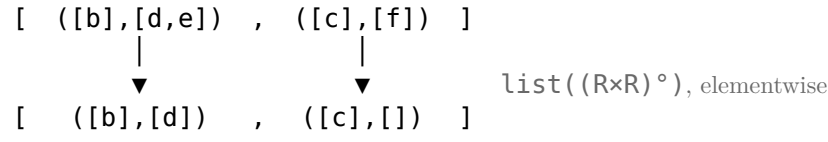


(7.3.1e)

list(choose) [([d],[]),([e],[])]
 1st item ([d],[]) choose→[d] or []
 2nd item ([e],[]) choose→[e] or []
 2×2=4 combinations: [[d],[e]] [[d],[]] [[],[e]] [[],[]]
 concat flattens: [d,e] [d] [e] []

§7.3.2. list((R×R)[°])

(7.3.2a)



([b],[d,e])
 | — exclude: the best party in b's subtree when b does not come
 | — include: the best party in b's subtree when b comes

(7.3.2b)

element	1st component include	2nd component exclude
1st, ([b], [d,e])	[b] R [°] [b] 7 ≥ 7, reflexivity	[d,e] R [°] [d] 6 ≥ 5, and ([b],[d,e]) is the pair est (R [°]) keeps at b
2nd, ([c],[f])	[c] R [°] [c] 2 ≥ 2, reflexivity	[f] R [°] [] 8 ≥ 0, and ([c],[f]) is the pair est (R [°]) keeps at c

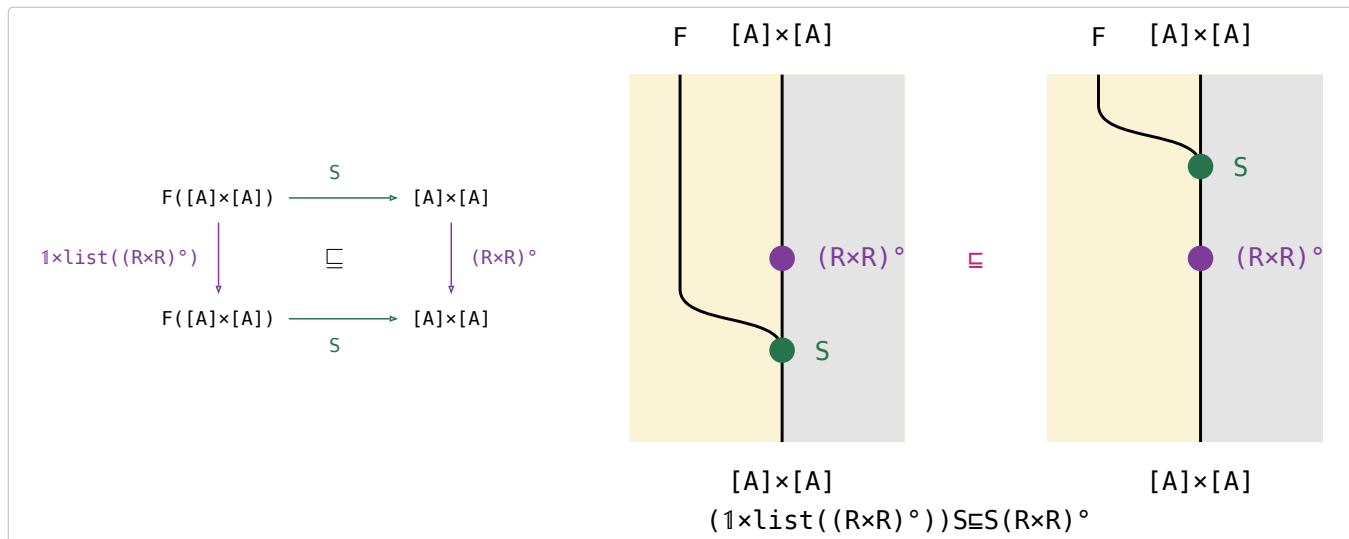
the first component of $1 \times \text{list}((R \times R)^\circ)$ is the root employee.

$R \triangleq \text{cost} \leq \text{cost}^\circ$ is a preorder, so the components that do not move are related by reflexivity — $(R \times R)^\circ$ demands a relation in **both** components, it cannot skip one.

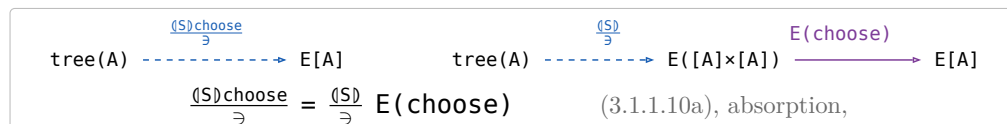
R compares cost only, not contents: $[f] R^\circ []$ is legal though $[f]$ has an element $[]$ does not. That is why the three leaves of §7.3.3's proof all reduce to “cost is a sum”.

§7.3.3. $(1 \times \text{list}((R \times R)^\circ)) S \sqsubseteq S(R \times R)^\circ$ — $S : F([A] \times [A]) \rightarrow [A] \times [A]$ monotonic on $(R \times R)^\circ$

(7.3.3a)

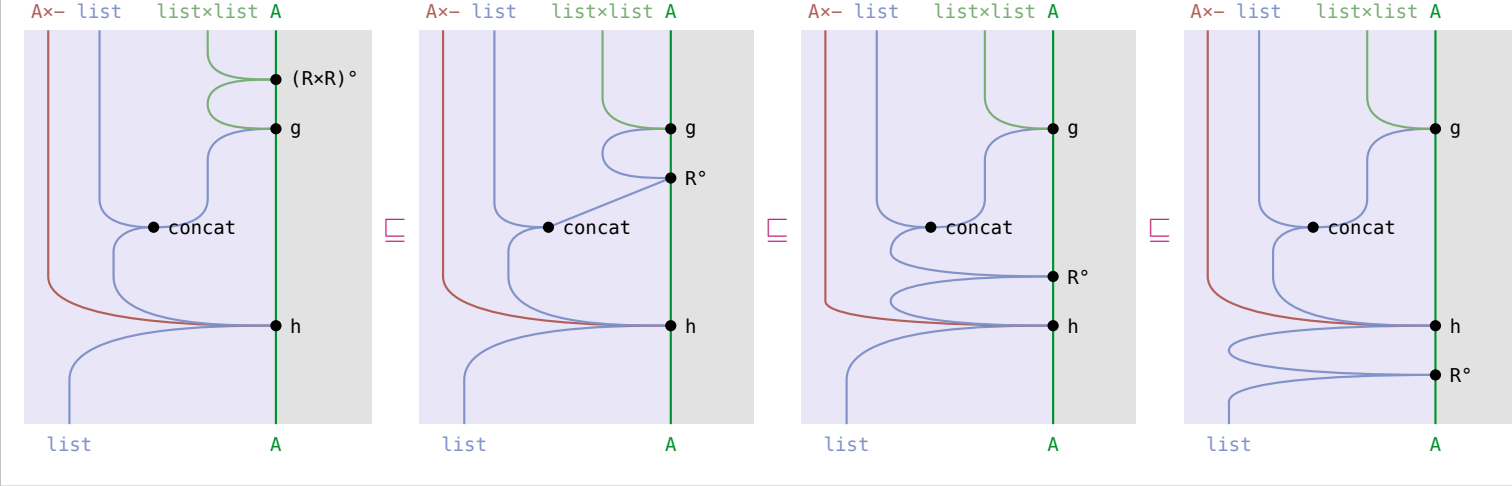


(7.3.3b)



$$\begin{aligned}
S &\triangleq (\text{include}, \text{exclude}) \\
(1 \times \text{list}((R \times R)^\circ)) S &\sqsubseteq S(R \times R)^\circ \\
(1 \times \text{list}((R \times R)^\circ)) \text{include} &\sqsubseteq \text{include } R^\circ \\
(1 \times \text{list}((R \times R)^\circ)) \text{exclude} &\sqsubseteq \text{exclude } R^\circ
\end{aligned}
\tag{7.3.3c}$$

bettering both parties of every subtree before the node's algebra runs gets no further than running it first and bettering the two parties it returns,



$g : (R \times R)^\circ g \sqsubseteq g R^\circ$
 $g := \pi_2$ is $(R \times R)^\circ \pi_2 = (\text{dom}(\pi_1 R^\circ)) \pi_2 R^\circ \sqsubseteq \pi_2 R^\circ$, $g := \pi_1$ its mirror $(\text{dom}(\pi_2 R^\circ)) \pi_1 R^\circ \sqsubseteq \pi_1 R^\circ$ — 1 and 4 of (5.2.2a), then
 $\text{dom} \sqsubseteq 1$; $g := \text{choose} \triangleq \pi_1 \cup \pi_2$ is the union of the two (5.7c). **list** monotonic, $\triangleq$ relator
concat $\text{list}(R^\circ) \text{concat} \sqsubseteq \text{concat } R^\circ$
 a LEAF: no law above it. **cost** is a sum, so $\text{cost}(\text{concat}(xss)) = \text{sum}(\text{list}(\text{cost})(xss))$ and a cheaper part makes a cheaper whole.
h $(1 \times R^\circ) h \sqsubseteq h R^\circ$
 a LEAF: $\text{cost}(\text{cons}(a, xs)) = \text{rating}(a) + \text{cost}(xs)$, so a cheaper tail makes a cheaper list. For $h := \pi_2$ it is an EQUALITY
 $(1 \times R^\circ) \pi_2 = (\text{dom}(\pi_1)) \pi_2 R^\circ = \pi_2 R^\circ : \pi_1$ is a map, hence entire, so $\text{dom}(\pi_1) = 1$ — (3.1.1.7a)

$(1 \times (\text{list}(g) \text{ concat})) h$	g	h
include	π_2	cons
exclude	choose	π_2

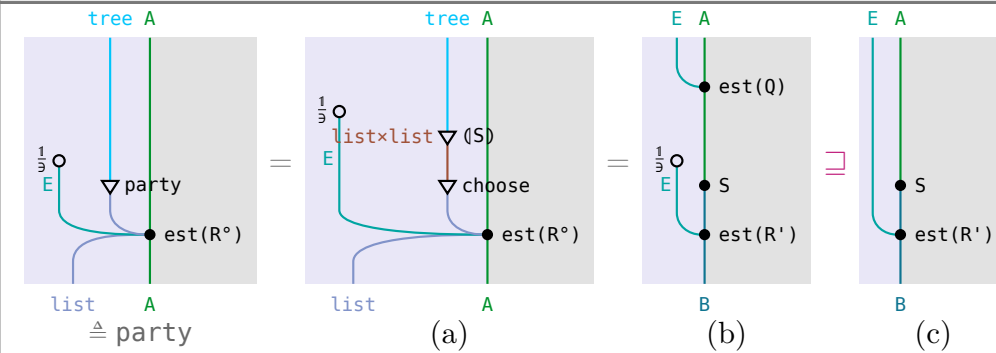
(7.3.3d)

$R \triangleq \{(\emptyset, \emptyset)\} : \{0, 1\} \rightarrow \{0, 1\}$	not entire — undefined at 1
$(R \times R) \text{choose}$ at $(\emptyset, 1)$	nothing — $R \times R$ needs BOTH components related, and R is undefined at 1
$\text{choose } R$ at $(\emptyset, 1)$	0 — choose picks the first component 0, and $R \emptyset \emptyset$ holds
$((\emptyset, 1), \emptyset)$	in choose R , not in $(R \times R) \text{choose}$
$(R \times R) \text{choose} = \text{choose } R$	iff R is entire — $\text{dom} \sqsubseteq 1$ is the g row's only inequality step, and it is an equality iff $\text{dom}(R) = 1$

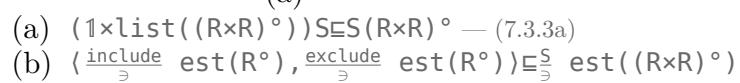
§7.3.4. The derivation

$$(\text{list}(\text{include}, \pi_2 \text{ list}(\frac{\text{choose}}{3} \text{ est}(R^\circ)) \text{ concat})) \frac{\text{choose}}{3} \text{ est}(R^\circ) \sqsubseteq \frac{\text{party}}{3} \text{ est}(R^\circ)
\tag{7.3.4a}$$

the best of every guest list the president allows is one pass up the tree, each subtree handing up its best party with its boss in and its best with the boss out, and **choose** taking the better of the two at the root



- (a) $\text{party} \triangleq (S) \text{ choose} \triangleq \text{party}$
 (b) $(S) \text{ choose} = (S) \frac{\text{choose}}{3} E(\text{choose})$ — (7.3.3b)
 (c) $(R \times R)^\circ \text{choose} \sqsubseteq \text{choose } R^\circ$ — (7.3.3c)



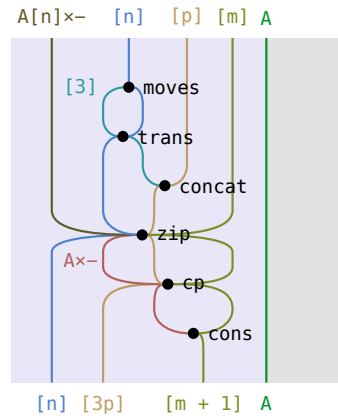
§7.4. Shortest paths on a cylinder

(7.4a)

	on $L = \text{list}^+$, 11.3.3	on Vec	note
F	$F(A, X) = A + A \times X$	$F(A, A[m]) = A[1] + A[m+1]$	A square alone, or in front of the path so far.
the input	$L(N\ A)$	$A[m][n] \triangleq (A[n])[m]$	m columns of n squares, outermost index first — the matrix itself.
a path	$L\ A$, a non-empty list	$A[1] \triangleq A$, $A[m+1] \triangleq A \times A[m]$	Through m columns, m squares: the count is in the type. A one-square path is the square; $\text{Vec}(m)$ is the functor $X \mapsto X[m]$.
the paths into a row	$E(L\ A)$	$A[p][m]$, $p = 3^{m-1}$	Three moves per column crossed, so this count is in the type too; a set forgets it.
α	$[\text{wrap}, \text{cons}]$ $F(A, LA) \rightarrow LA$	$\text{wrap} = 1 : A \rightarrow A[1]$ $\text{cons} : A \times A[m] \rightarrow A[m+1]$	wrap is an identity; cons is only an iso, so it stays a bead.
moves	$NA \rightarrow E(NA)$	$X[n] \rightarrow X[3][n]$	$\text{moves}(x) = (\text{up}(x), x, \text{down}(x))$, a $3 \times n$ matrix.
trans	$E(NX) \rightarrow N(EX)$	$X[3][n] \rightarrow X[n][3]$	The transpose.
union	$E(EX) \rightarrow EX$	$\text{concat} : X[j][k] \rightarrow X[jk]$	The j rows laid end to end; j is 3 inside gen and n inside paths .
zip	$F(NA, NB) \rightarrow NF(A, B)$	$F(A[n], B[n]) \rightarrow F(A, B)[n]$	Unchanged.
cp	$F(A, EB) \rightarrow E(F(A, B))$	$A \times B[p] \rightarrow (A \times B)[p]$	The square paired with each of the p candidates.
$\text{est}(R) \triangleq \exists n (\in \mathbb{N} \setminus \mathbb{R}) X \rightarrow X$		$X[k] \rightarrow X$, $\exists : X[k] \rightarrow X$	A component that is R -below every component; $\exists$ is “is some component” and $\in$ its converse.
$R \triangleq \text{sum} \leq \text{sum}^\circ$	$L\ \text{Nat} \rightarrow L\ \text{Nat}$	$\text{Nat}[m] \rightarrow \text{Nat}[m]$	The cost of a path, which the cheapest minimises — one order per length m .
gen	$F(1, \text{moves trans } N(\text{union}))$ $\text{zip } N(\text{cp } P(\alpha))$ $F(NA, N(E(LA)))$ $\rightarrow N(E(LA))$	$F(1, \text{moves trans } \text{Vec}(n)(\text{concat}))$ $\text{zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$ $F(A[n], A[n][p][m]) \rightarrow A[n][3p][m+1]$	The same composite without α ; the type shown is the cons side, the wrap side is $1 : A[n] \rightarrow A[n]$.
$\langle \text{gen} \rangle$	$L(N\ A) \rightarrow N(E(L\ A))$	$A[m][n] \rightarrow A[n][p][m]$	Columns in, rows out.
$\text{paths} \triangleq \langle \text{gen} \rangle$ concat	$\langle \text{gen} \rangle$ setify union $L\ N\ \text{Nat} \rightarrow E(L\ \text{Nat})$	$A[m][n] \rightarrow A[np][m]$	The n rows of p paths laid end to end: every path of the cylinder, in one row.
Q , 7.4.3	$F(1, \text{moves trans } N(\text{est}(R)))$ $\text{zip } N(\alpha)$	$F(1, \text{moves trans } \text{Vec}(n)(\text{est}(R)))$ $\text{zip}, F(A[n], A[n][m]) \rightarrow A[n][m+1]$	The cheapest of three; no p anywhere. $\langle Q \rangle : A[m][n] \rightarrow A[n][m]$ has the type of a transpose.
the specification paths $\text{est}(R)$	$L\ N\ \text{Nat} \rightarrow L\ \text{Nat}$	$A[m][n] \rightarrow A[m]$, $\text{est}(R) : X[np] \rightarrow X$	A cheapest path from the entry side to the exit side.

§7.4.1. $\text{gen} \triangleq (1 \times (\text{moves trans } \text{Vec}(n)(\text{concat}))) \text{ zip } \text{Vec}(n)(\text{cp } \text{Vec}(3p)(\text{cons}))$

(7.4.1a)



(7.4.1b)

a row is the outer index, a cell one path

$u = ((1,2,3,4), C)$

$C = 5$

6

7

8

$(1 \times (\text{moves trans Vec}(n)(\text{concat})))$

$\text{moves}(C)$

$= 6\ 7\ 8\ 5$

5 6 7 8

8 5 6 7

$\text{trans}(\text{that})$

$= 6\ 5\ 8$

7 6 5

8 7 6

5 8 7

$\text{Vec}(n)(\text{concat})(\text{that}) = \text{that}$

$\text{zip}(\text{that})$

$= (1, 6\ 5\ 8)$

(2, 7 6 5)

(3, 8 7 6)

(4, 5 8 7)

$\text{Vec}(n)(\text{cp})(\text{that})$

$= 1\ 6\ 2\ 7\ 3\ 8\ 4\ 5$

1 5 2 6 3 7 4 8

1 8 2 5 3 6 4 7

$: A[n] \times A[n]$

one candidate per row, one square long:

a column of squares

1 keeps the column, the candidates move down, unmoved, up: a $3 \times n$ matrix

$: A[3][n]$

the transpose: row k gets rows $k-1, k, k+1$

$: A[n][3]$

$\text{concat}: X[3][1] \rightarrow X[3]$ is 1

each row: its square, and its candidates

$: (A \times A[3])[n]$

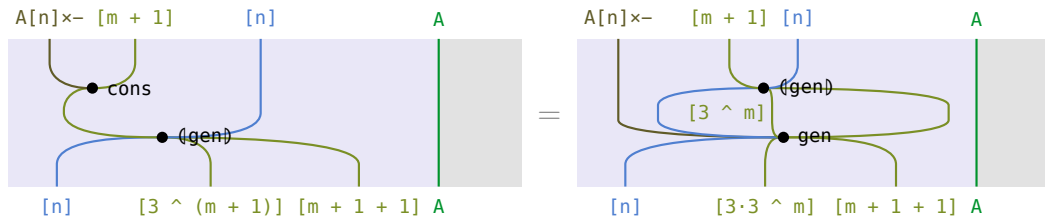
cp pairs the square with each candidate:

row k of that is the k -th 3×2 block, left to right; each line a two-square path

$: A[n][3][2]$

§7.4.2. gen is an F-algebra; $(\text{gen}): \text{cons } (\text{gen}) = (1 \times (\text{gen})) \text{gen}$

(7.4.2a)



the $[m+1]$ bracket puts the column back on the matrix, then the fold reads all of it

the fold reads the rest under F , then one gen puts the column in front

(7.4.2b)

$xs = 1\ 2\ 3\ 4$

5 6 7 8

$: A[m][n], m=2$

$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})(5,6,7,8))$

$(\text{gen})(5,6,7,8)$

$= 5$

6

7

8

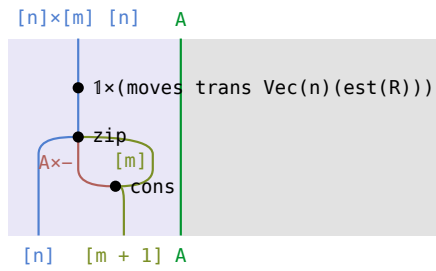
one column: each square its own one-square path

$: A[n][1][1]$

both halves are in (7.4.1b): the tail folds to the column 5 6 7 8, one path per row and each of them one square long, and gen on it is that walk.

§7.4.3. $Q \triangleq (1 \times (\text{moves trans Vec}(n)(\text{est}(R)))) \text{ zip}, A[n] \times A[n][m] \rightarrow A[n][m+1]$

(7.4.3a)



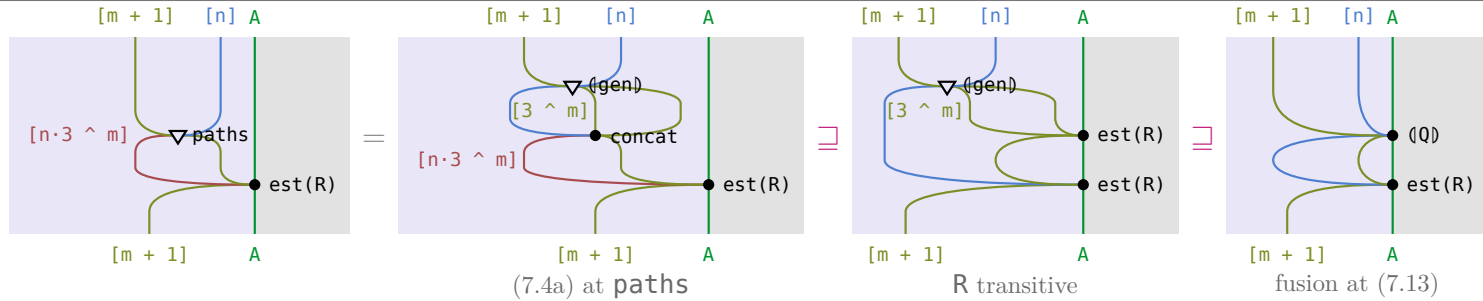
(7.4.3b)

$u = ((1,2,3,4), C)$: $A[n] \times A[n][m]$
 $C = 5$: one path per row, one square long
 6
 7
 8
 $(1 \times (\text{moves trans Vec}(n)(\text{est}(R))))$: 1 keeps the column, the paths move down, unmoved, up: a $3 \times n$ matrix
 $\text{moves}(C)$
 $= 6 \ 7 \ 8 \ 5$
 $5 \ 6 \ 7 \ 8$
 $8 \ 5 \ 6 \ 7$: $A[3][n][m]$
 $\text{trans}(\text{that})$: the transpose: row k gets rows $k-1, k, k+1$
 $= 6 \ 5 \ 8$
 $7 \ 6 \ 5$
 $8 \ 7 \ 6$
 $5 \ 8 \ 7$: $A[n][3][m]$
 $\text{Vec}(n)(\text{est}(R))(\text{that})$: the cheapest into each row – chosen BEFORE
 $= 5 \ 5 \ 6 \ 5$: $A[n][m]$ the new square is put in front
 $\text{zip}(\text{that})$: each row: its square, and the one predecessor that survived; the $[m+1]$ bracket is cons
 $= 1 \ 5$
 $2 \ 5$
 $3 \ 6$
 $4 \ 5$: $A[n][m+1]$

§7.4.4. paths $\text{est}(R) \sqsubseteq (\mathcal{Q}) \text{est}(R)$

$$(\forall i. 1 \in R) \wedge (\forall i. R R \in R) \wedge (\forall i. (1 \times R) \text{cons} \sqsubseteq \text{cons } R) \Rightarrow (\mathcal{Q}) \text{est}(R) \sqsubseteq \text{paths est}(R) \quad (7.4.4a)$$

a cheapest of all n paths of the cylinder is beaten by the greedy fold's one path per row and then a cheapest of those n , which costs $O(n \times m)$.



$$S \text{SES} \Rightarrow \text{Vec}(j)(\text{est}(S)) \text{est}(S) \sqsubseteq \text{concat est}(S), R \text{ transitive}$$

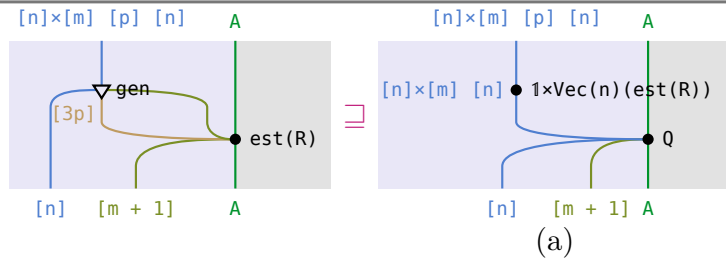
a cheapest of each of the j rows and then a cheapest of those j is a cheapest of all j entries laid end to end.

$$(\forall i. 1 \in R) \wedge (\forall i. R R \in R) \wedge (\forall i. (1 \times R) \text{cons} \sqsubseteq \text{cons } R) \Rightarrow (\mathcal{Q}) \sqsubseteq (\mathcal{Q} \text{gen}) \text{Vec}(n)(\text{est}(R)), R \text{ reflexive, transitive and monotonic}$$

the one path per row the greedy fold keeps is one of the p that $(\mathcal{Q} \text{gen})$ generates for that row, and a cheapest of them.

$$R R \in R \wedge (1 \times R) \text{cons} \sqsubseteq \text{cons } R \Rightarrow (1 \times \text{Vec}(n)(\text{est}(R))) \mathcal{Q} \sqsubseteq \text{gen Vec}(n)(\text{est}(R)) \quad (7.13)$$

choosing a cheapest of each square's p paths before the column is extended is no better than extending first and choosing among the $3p$.



$$(7.13) \text{ on Vec: } (1 \times R) \text{cons} \sqsubseteq \text{cons } R \Rightarrow (1 \times \text{est}(R)) \text{cons} \sqsubseteq \text{cp Vec}(k)(\text{cons est}(R)), R \text{ monotonic}$$

putting the new square in front of every one of the p candidates and then choosing a cheapest is beaten by choosing a cheapest first and putting the square in front of that one.

(7.4.3a)

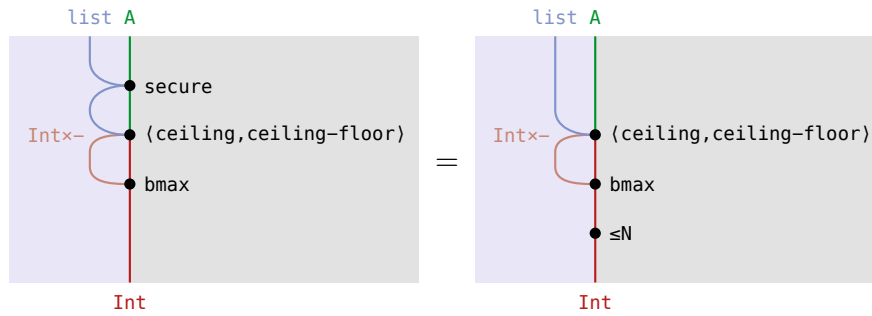
§7.5. The security van problem

(7.5a)

definition	type	example
$R \triangleq \text{length} \leq \text{length}^\circ$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$, and not back: $1 \leq 2$.
$\text{ceiling} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\geq)$	$[A] \rightarrow \text{Int}$	$\text{ceiling}[a, b]$ is the largest of 0, a , $a+b$.
$\text{floor} \triangleq \frac{\text{prefix sum}}{\ni} \text{est}(\leq)$	$[A] \rightarrow \text{Int}$	$\text{floor}[a, b]$ is the smallest of 0, a , $a+b$, so $[a, b]$ carries ceiling-floor cash.
secure the coreflexive on x with $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$	$[A] \rightarrow [A]$	secure keeps $[a]$ exactly when $-N \leq a \leq N$.
ok the coreflexive on (a, xs) with xs non-empty and $[a] \# \text{head}(xs)$ secure	$A \times [[A]] \rightarrow A \times [[A]]$	ok keeps $(a, [[b], [c]])$ exactly when $[a, b]$ is secure .
$\text{new} \triangleq (\text{wrap} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{new}(a, [[b], [c]]) = [[a], [b], [c]]$.
$\text{glue} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } (\text{cons} \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{glue}(a, [[b], [c]]) = [[a, b], [c]]$.
$\text{old} \triangleq (1 \times \text{cons}^\circ) \text{ assocl } ((\text{cons } \text{secure}) \times 1) \text{ cons}$	$A \times [[A]] \rightarrow [[A]]$	$\text{old}(a, [[b], [c]]) = [[a, b], [c]]$ when $[a, b]$ is secure , and nothing otherwise.
partition $\triangleq ([\text{nil}, \text{new } u \text{ glue}])$ cutting the transactions into non-empty segments is one pass along them that either opens a segment for the transaction in hand or puts it on the front of the segment already open	$[A] \rightarrow [[A]]$	partition $[a, b]$ gives $[[a], [b]]$ and $[[a, b]]$.
$S \triangleq [\text{nil}, \text{new } u \text{ old}]$	$F[[A]] \rightarrow [[A]]$	$S(a, [[b], [c]])$ gives $[[a], [b], [c]]$, and $[[a, b], [c]]$ when $[a, b]$ is secure .
$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow$ partition list (secure) $= (S)$ cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new } u \text{ old}]$	$[A] \rightarrow [[A]]$	Both take $[a, b]$ to $[[a], [b]]$ — each $[a]$ is secure — and to $[[a, b]]$ when $[a, b]$ is.
$H \triangleq (\text{head } \text{prefix}^\circ \text{ head}^\circ) u (\text{nil}^\circ \text{ nil})$	$[[A]] \rightarrow [[A]]$	$[[a], [b, c]]$ to $[[a, b], [c]]$: $[a]$ is a prefix of $[a, b]$.
$R; H \triangleq Rn(R^\circ \Rightarrow H)$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$ by $ R $, and $[[a], [b, c]]$ to $[[a, b], [c]]$ by RnH .
$ R \triangleq Rn \neg R^\circ$	$[[A]] \rightarrow [[A]]$	$[[a, b, c]]$ to $[[a], [b, c]]$: $1 < 2$. The strict part R splits into: $R; H = R u (RnH)$.
the specification $\frac{\text{partition list}(\text{secure})}{\ni} \text{est}(R)$	$[A] \rightarrow [[A]]$	$[a, b]$ gives $[[a, b]]$ when $[a, b]$ is secure , and $[[a], [b]]$ otherwise.

§7.5.1. **secure prefix** $\Rightarrow$ **prefix secure**

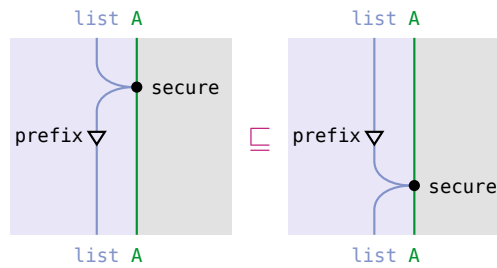
(7.5.1a)



$$\text{secure}(\text{ceiling}, \text{ceiling-floor}) \text{bmax} = (\text{ceiling}, \text{ceiling-floor}) \text{bmax} (\leq N)$$

a stretch passes **secure** before the test exactly where the test's own value passes $\leq N$ after it, which is $\triangleq$ **secure**'s $\text{bmax}(\text{ceiling } x, \text{ceiling } x - \text{floor } x) \leq N$

Exercise 7.49



$$\text{secure prefix} \sqsubseteq \text{prefix secure}$$

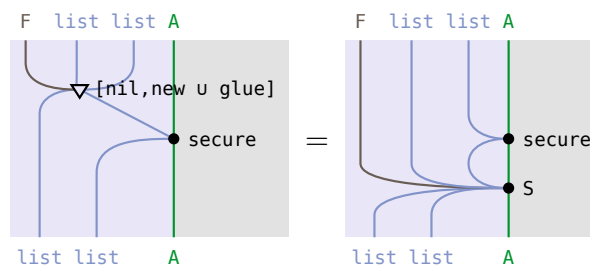
every pair **secure** then **prefix** gives, **prefix** then **secure** gives too — a prefix of a secure stretch is itself secure, which is the prefix-closure
B&dM p.185 names

§7.5.2. partition list(secure)=([nil,new u old])

(7.5.2a)

definition	Hinze–Marsden
new $\triangleq$ (wrap×1) cons wrap makes the transaction a segment on its own, and cons puts that segment at the front of the schedule: the van is called	
glue $\triangleq$ (1×cons°) assocl (cons×1) cons cons° splits the schedule into its first segment and the rest, the first cons puts the transaction at the front of that segment, and the second puts the segment back	
old $\triangleq$ (1×cons°) assocl ((cons secure)×1) cons glue with secure on the segment the transaction has just joined: the van stays away only where the longer segment still passes	

(7.5.2b)



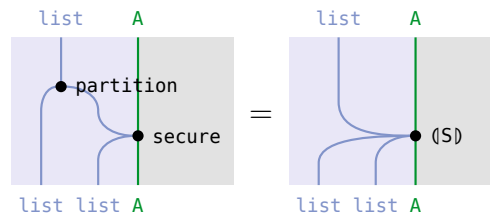
$$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow [\text{nil}, \text{new } u \text{ glue}] \text{list}(\text{secure}) = F(\text{list}(\text{secure})) S$$

testing every segment for security after the algebra has run is testing it before it ran: a segment **glue** has extended is secure exactly where **old** would have let it be, and the one transaction **new** sets out on its own is secure whatever it is

(7.5.1a)

$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$ Exercise 7.51	
cutting the transactions into non-empty segments every way and keeping the cuts whose every segment is secure is the one fold whose algebra is $[\text{nil}, \text{new} \cup \text{old}]$	
step	Hinze–Marsden
partition list(secure) the specification: cut the transactions every way, then keep the cuts whose every segment is secure	
$= \langle [\text{nil}, \text{new} \cup \text{glue}] \rangle \text{list}(\text{secure})$ partition is the fold that either opens a segment for the transaction in hand or puts it on the front of the segment already open	
$= \langle [\text{nil}, \text{new} \cup \text{old}] \rangle$ fusion: the condition above moves $\text{list}(\text{secure})$ inside that fold, which is what turns glue into old	

(7.5.2c)



$$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow \text{partition list}(\text{secure}) = \langle S \rangle$$

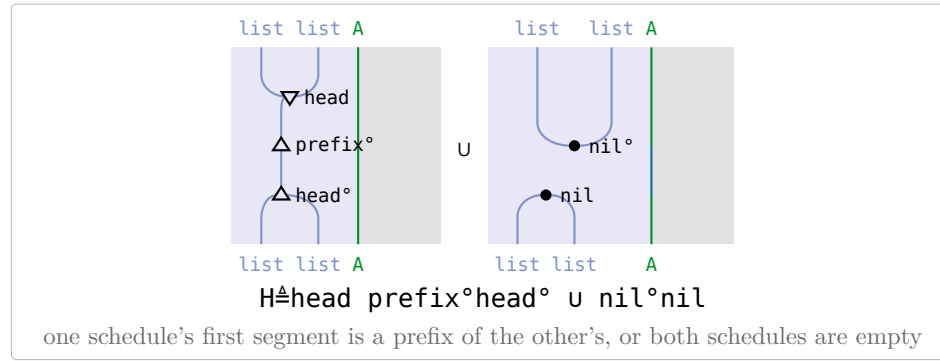
cutting the transactions every way and then keeping the cuts whose every segment is secure is one pass along them that either calls the van or extends the open segment while it stays secure

§7.5.3. $(1 \times R) \text{new} \sqsubseteq (\text{new} \cup \text{old}) R$

$(1 \times R) \text{new} \sqsubseteq (\text{new} \cup \text{old}) R$ (7.14)	
calling the van for the transaction on a no-longer schedule gets no further than calling it on this one and shortening the schedule afterwards	
step	Hinze–Marsden
$(1 \times R) \text{new}$ the left side of (7.14) $\triangleq \text{secure}$	
$= (\text{wrap} \times R) \text{cons}$ $\text{new} \triangleq (\text{wrap} \times 1) \text{cons}$ — (7.5.2a) — and $(\text{wrap} \times 1) (1 \times R) = (\text{wrap} \times R)$	
$\sqsubseteq \text{new} R$ cons is monotonic on R — $(1 \times R) \text{cons} \sqsubseteq \text{cons} R$, consing onto a no-longer schedule leaves it no longer — and $\text{new} \sqsubseteq \text{new} \cup \text{old}$	

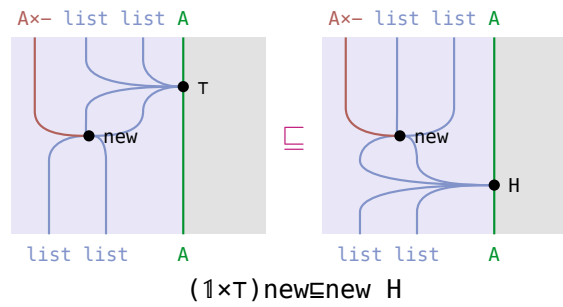
§7.5.4. $H \triangleq (\text{head prefix} \circ \text{head}) \cup (\text{nil} \circ \text{nil})$

(7.5.4a)



§7.5.5. $(1 \times (R;H)) \text{new} \sqsubseteq (\text{new } u \text{ old})(R;H)$

(7.18)



whatever schedule the van is called on, the result's first segment is the one transaction $[a]$, so any two results of **new** on that transaction stand in H

$(1 \times (R;H)) \text{new} \sqsubseteq (\text{new } u \text{ old})(R;H)$

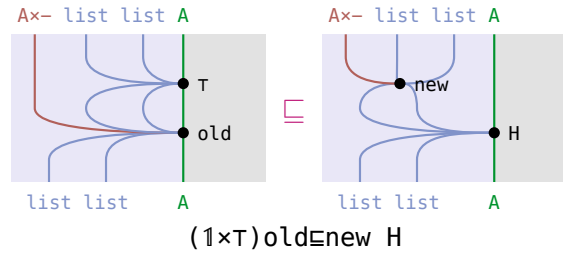
(7.16)

calling the van for the transaction on a $R;H$ -better schedule gets no further than calling it on this one and bettering the whole schedule afterwards

step	Hinze–Marsden
$(1 \times (R;H)) \text{new}$ the left side of (7.16)	
$\sqsubseteq (1 \times R) \text{new}$ $R;H \sqsubseteq R \text{ — } \triangleq \text{secure}$	
$\sqsubseteq \text{new } R \cap \text{new } H$ (7.14) as far as its new R line — (7.14) — and (7.18)	
$= \text{new } (R \cap H)$ new is a map, and a map distributes over $\cap$	
$\sqsubseteq \text{new } (R;H)$ $X \cap Y \sqsubseteq X;Y$, and $\text{new} \sqsubseteq \text{new } u \text{ old}$	

§7.5.6. $(1 \times (R;H)) \text{old} \sqsubseteq (\text{new } u \text{ old})(R;H)$

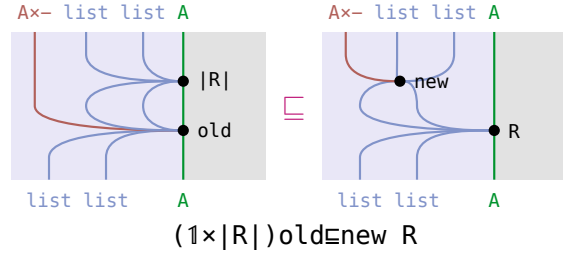
(7.19)



$$(1 \times \tau) \text{old} \sqsubseteq \text{new H}$$

old leaves the transaction **[a]** at the front of the first segment, and **[a]** is what **new** makes that segment, so the two first segments are **prefix**-related whatever the schedules were

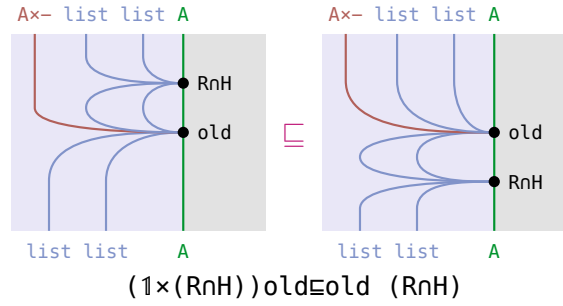
(7.20)



$$(1 \times |R|) \text{old} \sqsubseteq \text{new R}$$

old keeps the schedule's length, so a strictly shorter one still comes out no longer than the one the van's own segment lengthens

(7.21)



$$(1 \times (RnH)) \text{old} \sqsubseteq \text{old (RnH)}$$

on a tie the one first segment is a prefix of the other, so prefix-closure of **secure** — Exercise 7.49 — lets **old** fire on this side too, and it keeps both the length and the prefix

$(1 \times (R;H)) \text{old} \sqsubseteq (\text{new } u \text{ old})(R;H)$ (7.17)	
gluing the transaction onto a better schedule for the rest gets no further than gluing it on, or calling the van, and bettering the whole schedule after,	
circuit — the old branch of each union	Hinze–Marsden
the old operand of new u old , in every row	
$(1 \times R) \text{old } u (1 \times (R;H)) \text{old}$ $R;H = R \text{ } u (R;H) \text{ — } \triangleq \text{ secure, } u \text{ distributes,}$	
$\sqsubseteq$ $\text{new } (R;H) \text{ } u \text{ old } (R;H)$ (7.19) and (7.20) on $ R $ — (7.19), (7.20) — and (7.21) on $R;H$ — (7.21)	
$\sqsubseteq$ $(\text{new } u \text{ old})(R;H)$ $X \text{ } n Y \sqsubseteq X; Y, \text{ converses}$	

§7.5.7. The derivation

$(\forall a. \text{secureP}(\text{amount})(N, [a])) \Rightarrow ([\text{nil}, (\text{ok} \rightarrow \text{glue}, \text{new})]) \sqsubseteq \frac{\text{partition list}(\text{secure})}{3} \text{est}(R)$ (7.5.7a)	
the fewest secure segments the transactions can be cut into are one pass along them, the next transaction glued onto the open segment wherever that segment stays secure and the van called where it does not	
$\triangleq \text{secure}$	
(a) $\text{partition list}(\text{secure}) = \{S\} \text{ — } \triangleq \text{secure, fusion}$	(b) $\text{old} \sqsubseteq \text{new } (R;H)^\circ$
(b) greedy theorem at $R;H$ — (7.17), (7.16), (7.18)	

§8. Thinning Algorithms

§8.1. Thinning

(8.1)

For $Q : A \rightarrow A$, $\text{thin}(Q) \triangleq (\exists/\exists) \cap (\epsilon \setminus (Q^\circ \epsilon)) : EA \rightarrow EA$ (8.1).

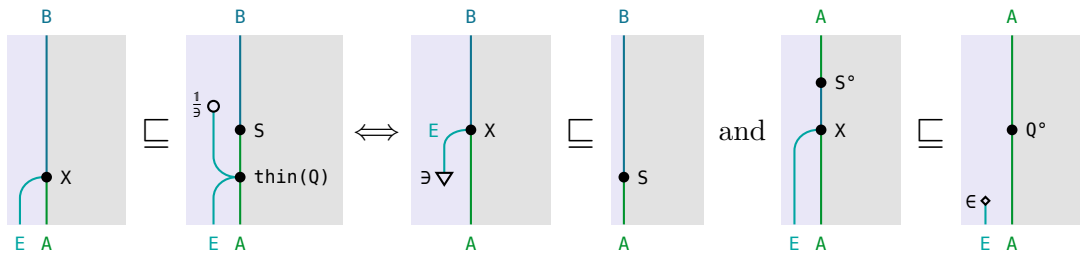
$ys \text{ thin}(Q) \ xs \iff xs \subseteq ys \wedge (\forall a \in ys. \exists b \in xs. b \ Q \ a)$ the same $^\circ$ as $\triangleq \text{est}$: ϵX runs $a \rightarrow ys \rightarrow xs$, member of ys first, and $Q^\circ \epsilon$ runs $a \rightarrow b \rightarrow xs$, so $(a, b) \in Q^\circ$ reads $b \ Q \ a$; at $Q^\circ \leq$ every $a \in ys$ keeps some $b \leq a$ in xs , the end $\text{est}(\leq)$ picks

(8.1a)

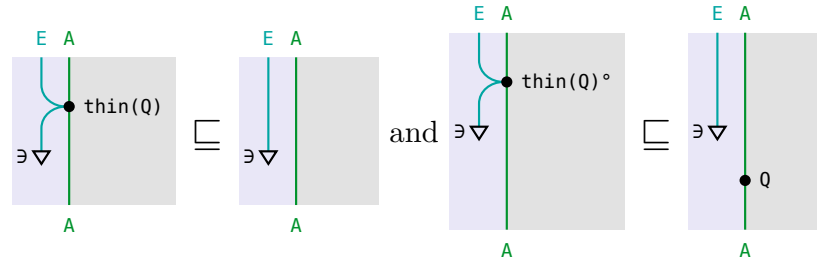
the law	what it says
$X \in \frac{S}{\exists} \text{ thin}(Q) \iff X \in S$ and $S^\circ X \in Q^\circ \epsilon$	everything kept is an S -value, and every S -value has a Q -lower bound among the kept ones
$\text{thin}(Q) \exists \epsilon \exists$	everything a thinning keeps was in the set
$\text{thin}(Q)^\circ \exists \epsilon \exists Q$	every element of the set has a Q -lower bound among the kept ones
$Q \in R \implies \text{thin}(Q) \in \text{thin}(R)$	the fewer pairs Q relates, the fewer subsets count as thinnings
$1 \in Q \implies 1 \in \text{thin}(Q)$ $Q \in Q \implies \text{thin}(Q) \text{ thin}(Q) \in \text{thin}(Q)$	keeping everything is always a legal thinning
$Q \in R \wedge 1 \in Q \wedge R \in R \implies \text{est}(R) = \text{thin}(Q) \text{ est}(R)$ $Q \in R, 1 \in Q, R \in R$ — (8.1.2a); weaker than the book's “both preorders”: Q transitive is never used and $1 \in R$ follows	thin-introduction: thinning first cannot lose an R -minimum
$\text{thin}(Q) \supseteq \text{est}(Q) \frac{1}{\exists}$ (8.2) — (8.2)	thin-elimination: keeping one element is a thinning, but its domain is the sets $\text{est}(Q)$ is defined on
$\frac{S}{\exists} \text{ thin}(Q) \supseteq \frac{S}{\exists} \text{ est}(R) \frac{1}{\exists}$ (8.3), $R \cap (S^\circ S) \in Q$ — thin-elimination variant	the usable variant: R need only refine Q between values S gives one argument
$P(\text{thin}(Q)) \text{ union} \in \text{union thin}(Q)$ (8.4)	thinning each member set is a thinning of the union

§8.1.1. $X \in (S^\circ \exists) \text{ thin}(Q) \iff X \in S$ and $S^\circ X \in Q^\circ \epsilon$

(8.1.1a)



(8.1.1b)



(8.1.1a) at $X \triangleq \text{thin}(Q)$, $S \triangleq \exists$: the left side is $\text{thin}(Q) \in \text{thin}(Q)$, since $\exists \circ \exists = \wedge(\exists) = 1$

§8.1.2. $\text{est}(R) = \text{thin}(Q) \text{ est}(R)$ given $Q \in R, 1 \in Q, R \in R$

$\text{est}(R) \sqsubseteq \text{thin}(Q) \text{ est}(R)$ (8.1.2a)
the $\sqsubseteq$ half: keeping everything is a thinning — $1 \sqsubseteq Q$

(a) $1 \sqsubseteq \text{thin}(Q)$ — (8.1a)

$\text{thin}(Q) \text{ est}(R) \sqsubseteq \exists$ (8.1.2b)
the $\sqsubseteq$ half, first condition of the UP of **est** at $X \triangleq \text{thin}(Q) \text{ est}(R)$ — (7.1.1a)

(a) $\text{est}(R) \sqsubseteq \exists$ — (7.1b)

(b) $\text{thin}(Q) \sqsubseteq \exists$ — (8.1.1a)

$\in \text{thin}(Q) \text{ est}(R) \sqsubseteq R^\circ$ (8.1.2c)
the $\sqsubseteq$ half, second condition — $Q \sqsubseteq R$, R transitive

(a) $\in \text{thin}(Q) \sqsubseteq Q^\circ \in$ — (8.1.1a)

(b) $\in \text{est}(R) \sqsubseteq R^\circ$ — (7.1b)

(c) R transitive

$\text{est}(Q) \frac{1}{3} \sqsubseteq \text{thin}(Q)$ (8.2)
the singleton holding a Q -least member of a set is a thinning of that set

$\text{est}(Q) \frac{1}{3} \sqsubseteq \exists$
every member of the singleton is a member of the set

(a) $\triangleq \text{thin}, \exists/\exists$ half

(b) $\frac{1}{3} \sqsubseteq 1$ — (3.1.1.10a); $\text{est}(Q) \sqsubseteq \exists$ — (7.1b)

$\in \text{est}(Q) \frac{1}{3} \sqsubseteq Q^\circ \in$
the singleton's member stands in Q to every member of the set

(a) $\triangleq \text{thin}, \in \setminus (Q^\circ \in)$ half

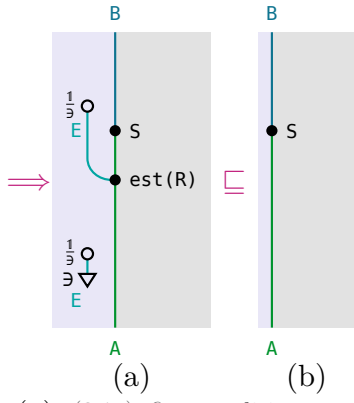
(b) $\in \text{est}(Q) \sqsubseteq Q^\circ$ — (7.1.1a)

(c) $\frac{1}{3} \sqsubseteq \in$ — (3.1.1.10a)

$$\mathbf{RnS^\circ S \sqsubseteq Q \Rightarrow \frac{S}{\exists} \text{est}(R) \frac{1}{\exists} \sqsubseteq \frac{S}{\exists} \text{thin}(Q)} \quad (8.3)$$

given $\mathbf{Rn}(S^\circ S) \sqsubseteq Q$, Q a preorder

$\frac{S}{\exists} \text{est}(R) \frac{1}{\exists} \ni \sqsubseteq S$
every member of the singleton is a value S returns

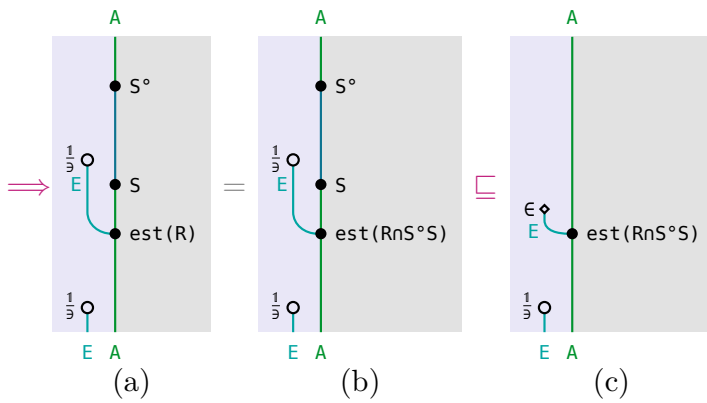


(a) (8.1a), first condition

(b) $\frac{1}{\exists} \ni = 1$ — (3.1.1.10a); $\frac{S}{\exists} \text{est}(R) \sqsubseteq S$ — (7.1b)

$$\mathbf{RnS^\circ S \sqsubseteq Q \Rightarrow S^\circ \frac{S}{\exists} \text{est}(R) \frac{1}{\exists} \sqsubseteq Q^\circ \in}$$

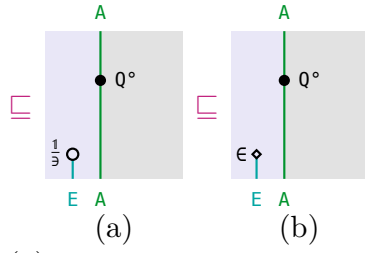
the singleton's member stands in Q to every value S returns at the same argument



(a) (8.1a), second condition

(b) $\frac{S}{\exists} \text{est}(R) = \frac{S}{\exists} \text{est}(RnS^\circ S)$ — (7.1b)

(c) $S^\circ \frac{S}{\exists} \sqsubseteq \in$ — (3.1.1.10a)



(a) $\in \text{est}(RnS^\circ S) \sqsubseteq (RnS^\circ S)^\circ$ — (7.1.1a); $\mathbf{Rn}(S^\circ S) \sqsubseteq Q$

(b) $\frac{1}{\exists} \sqsubseteq \in$ — (3.1.1.10a)

Theorem 8.1

$$Q \sqsubseteq Q \wedge \text{MonoAlg}(S, Q) \Rightarrow \langle \frac{F(\exists)S}{\exists} \text{thin}(Q) \rangle \sqsubseteq \langle \frac{QS}{\exists} \text{thin}(Q) \rangle$$

thinning at every step of the reduce is a thinning of the whole candidate set — S monotonic on Q , Q a preorder

(8.1a) at $X \triangleq \langle \frac{F(\exists)S}{\exists} \text{thin}(Q) \rangle$, $\langle QS \rangle$ for its S

(a) fusion, hylomorphism theorem
 (b) $S^\circ F(Q^\circ) \sqsubseteq Q^\circ S^\circ$ — (7.2b), (5.1b)
 (c) $S^\circ F(\epsilon) \sqsubseteq \epsilon$ — (3.1.1.10a)
 (d) $\epsilon \text{thin}(Q) \sqsubseteq Q^\circ \epsilon$ — $\triangleq \text{thin}$, (3.1.1.1a)

Corollary 8.1

$$Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge Q \sqsubseteq Q \wedge R \sqsubseteq R \wedge \text{MonoAlg}(S, Q) \Rightarrow \langle \frac{F(\exists)S}{\exists} \text{thin}(Q) \rangle \text{est}(R) \sqsubseteq \langle \frac{QS}{\exists} \text{est}(R) \rangle$$

the thinning fold refines the optimisation problem itself — S monotonic on Q , $Q \sqsubseteq R$, both preorders

(a) thinning theorem
 (b) $\text{est}(R) = \text{thin}(Q) \text{est}(R)$ — (8.1a)

§8.2. Paths in a layered network

(8.2a)

$F(A, X) = A + A \times X$, $L = \text{list}^+$ with initial algebra $\alpha \triangleq [\text{wrap}, \text{cons}] : F(A, LA) \rightarrow LA$.
 $\text{wrap} \triangleq \langle \text{wrap}, \text{zero} \rangle$, $\text{cons}w(a, (xs, n)) = (\text{cons}(a, xs), \text{wt}(a, \text{head}(xs)) + n)$.
 $\text{cost} \triangleq \langle [\text{wrap}, \text{cons}w] \rangle \pi_2$, $\langle [\text{wrap}, \text{cons}w] \rangle = \langle 1, \text{cost} \rangle$, $R \triangleq \text{cost} \leq \text{cost}^\circ$.
 $Q \triangleq Rn(\text{head head}^\circ)$, $S \triangleq F(1, \exists)\alpha$, $\frac{F(\exists, 1)}{\exists} = 1 + \text{cpl}$, $\frac{F(1, \exists)}{\exists} = 1 + \text{cpr}$, $\text{step} \triangleq \text{cpr } P(\text{cons}) \text{est}(R)$.

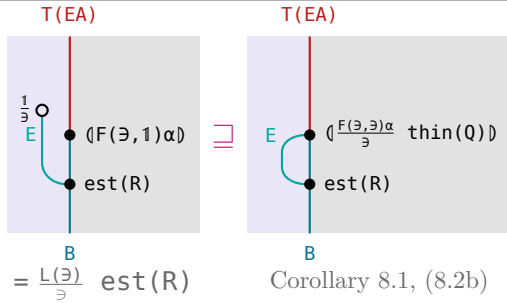
(8.2b)

the law	what it says
$F(\exists, Q) \alpha \sqsubseteq F(\exists, 1) \alpha Q$	$F(\exists, 1) \alpha$ is monotonic on Q ; on R it is not, since the next edge can cost arbitrarily much
$S \text{ head} \sqsubseteq [1, \pi_1] \quad S \triangleq F(1, \exists) \alpha$	$S \text{ head}$ is simple, which gives $Rn(S^\circ S) \sqsubseteq Q$: between two paths S builds from one argument, equal cost and equal head already means Q

$$Q \sqsubseteq R \wedge 1 \sqsubseteq Q \wedge QQ \sqsubseteq Q \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge \text{MonoAlg}(F(\exists, 1)\alpha, Q) \wedge Rn(F(1, \exists)\alpha)^\circ F(1, \exists)\alpha \sqsubseteq Q \Rightarrow \downarrow \frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) \quad (8.2c)$$

$$\text{est}(R) \sqsubseteq \text{est}(R) \sqsubseteq \frac{F(\exists, 1)\alpha}{\exists} \text{est}(R)$$

a least-cost path in a layered network, as a fold over the layers



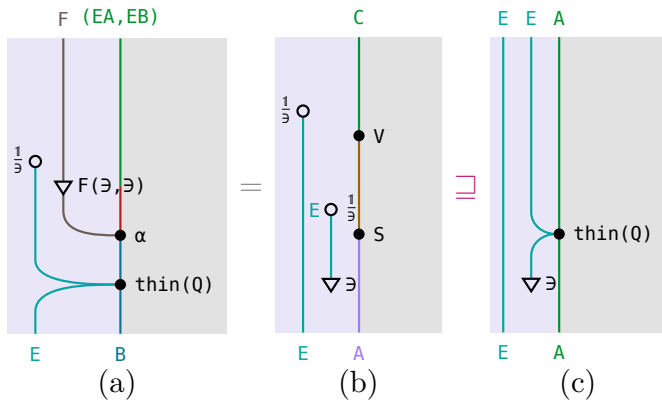
$\sqsubseteq$ (8.2d) under $\downarrow$ $\sqsupset$ monotonic: the whole chain runs inside the reduce, and the $\text{est}(R)$ behind it never moves.

$$Rn(F(1, \exists)\alpha)^\circ F(1, \exists)\alpha \sqsubseteq Q \Rightarrow \frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) \text{est}(R) \sqsubseteq \frac{F(\exists, \exists)\alpha}{\exists} \text{thin}(Q) \quad (8.2c)$$

thinning the algebra of a layered network costs no more than taking the program's two cases

$$\frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) P(\text{thin}(Q)) \text{union} \sqsubseteq \frac{F(\exists, \exists)\alpha}{\exists} \text{thin}(Q)$$

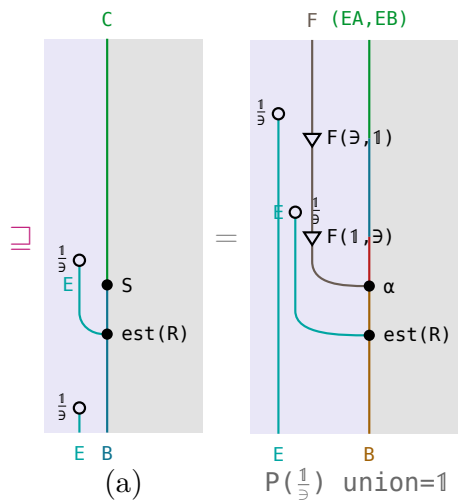
thinning each set the algebra's parts build, then taking the union, is below thinning the one set the whole algebra builds



- (a) the algebra of (8.2c) row 2
 (b) $\frac{F(\exists, 1)F(1, \exists)\alpha}{\exists} = \frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) \text{union}$
 (c) $\text{union thin}(Q) \sqsupseteq P(\text{thin}(Q)) \text{union} \text{ — (8.1a)}$

$$Rn(F(1, \exists)\alpha)^\circ F(1, \exists)\alpha \sqsubseteq Q \Rightarrow \frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) \text{est}(R) \sqsubseteq \frac{F(\exists, 1)}{\exists} P(\frac{F(1, \exists)\alpha}{\exists}) P(\text{thin}(Q)) \text{union}$$

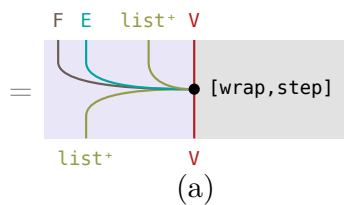
choosing an R-least member of each of those sets is below thinning each and taking the union



- (a) $\frac{S}{\exists} \text{thin}(Q) \sqsupseteq \frac{S}{\exists} \text{est}(R) \frac{1}{\exists} \text{ — (8.1a), (8.2b)}$

$$\text{cp } P(\alpha) \text{est}(\text{path}) = [\text{wrap}, \text{step}]$$

at the layered network, choosing an R-least path from each case is the program's two cases $[\text{wrap}, \text{step}]$



- (a) α a map; $\frac{F(1, 1)}{\exists} P(\alpha) \text{est}(R) = [\text{wrap}, \text{step}] \text{ — } \triangle \text{ cost}$

$$= \frac{F(\exists, 1)}{\exists} = 1 + \text{cpl} \text{ — } \triangle \text{ cost}$$

§8.3. Implementing thin

(8.3a)

$\text{setify} : [A] \rightarrow EA$, $\text{cup} : EA \times EA \rightarrow EA$, $\text{cp}(F) \triangleq \frac{F(\exists)}{\exists}$, $\text{listcp} : F(L) \rightarrow LF$, $\text{sort}(P) \triangleq \text{setify}^\circ \text{ ordered } P$ for P a connected preorder.

$\text{thinlist}(Q)$ is any $\text{thinlist}(Q) \sqsubseteq \text{subseq}$ with $\text{thinlist}(Q) \text{ setify} \sqsubseteq \text{setify thin}(Q)$; one is $\langle [\text{nil}, \text{bump } Q] \rangle$, $\text{bump } Q (a, []) = [a]$, $\text{bump } Q (a, [b] \# xs) = (b \text{ } Q \text{ } a \rightarrow [a] \# xs, a \text{ } Q \text{ } b \rightarrow [b] \# xs, [a] \# [b] \# xs)$.

Binary thinning data: $S = (f_1 p_1) \cup (f_2 p_2)$ with p_1, p_2 coreflexive; Q a preorder with $Q \sqsubseteq R$ and both $f_1 p_1, f_2 p_2$ monotonic on Q ; P a connected preorder with both f_1, f_2 monotonic on P ; $g_i \triangleq \text{list}(f_i) \text{ filter}(p_i)$.

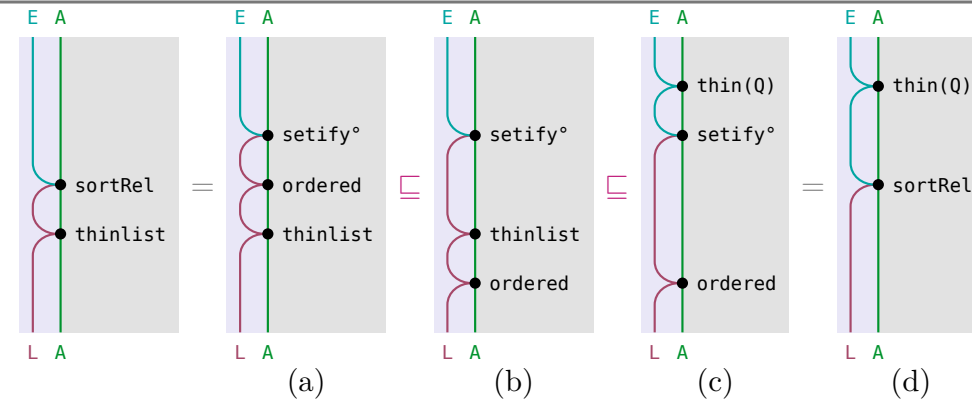
(8.3b)

the law	what it says
$\text{thinlist}(Q) \text{ } xs = [\text{minlist}(Q) \text{ } xs]$ (8.5), Q connected, xs non-empty	what thinning should come to when it can: one element
$\text{sort}(P) \text{ thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{ sort}(P)$ (8.6) — (8.6)	thinning a sorted list is a thinning of the set — this is what $\text{thinlist}(Q) \sqsubseteq \text{subseq}$ buys
$\text{sort}(P) \text{ minlist}(Q) \sqsubseteq \text{est}(Q)$ (8.7)	a minimum of the sorted list is a minimum of the set
$\text{sort}(f P f^\circ) \text{ list}(f) \sqsubseteq P(f) \text{ sort}(P)$ (8.8)	shunt a function through a sort
$\text{sort}(P) \text{ filter}(p) \sqsubseteq E(p) \text{ sort}(P)$ (8.9), p coreflexive	filtering a sorted list sorts the restricted set
$(\text{sort}(P) \times \text{sort}(P)) \text{ merge}(P) \sqsubseteq \text{cup sort}(P)$ (8.10)	merging two sorted lists sorts their union
$F(\text{sort}(P)) \text{ listcp} \sqsubseteq \text{cp}(F) \text{ sort}(FP)$ (8.11), F linear	listcp is the list implementation of the cartesian product $\text{cp}(F)$

Map(setify) $\wedge$ Coreflexive(ordered) $\wedge$ thinlist $\sqsubseteq$ subseq $\wedge$ ordered subseq $\sqsubseteq$ subseq ordered $\wedge$ thinlist (8.6)

setify $\sqsubseteq$ setify thin(Q) $\Rightarrow$ sortRel thinlist $\sqsubseteq$ thin(Q) sortRel

a thinning of the sorted list lists a thinning of the set — P a connected preorder, $\text{thinlist}(Q) \sqsubseteq \text{subseq}$.



(a) $\text{sort}(P) \triangleq \text{setify}^\circ \text{ ordered } P \text{ — } \triangleq \text{thinlist}$

(b) $\text{ordered } P \text{ thinlist}(Q) \sqsubseteq \text{thinlist}(Q) \text{ ordered } P \text{ — } \triangleq \text{thinlist}$

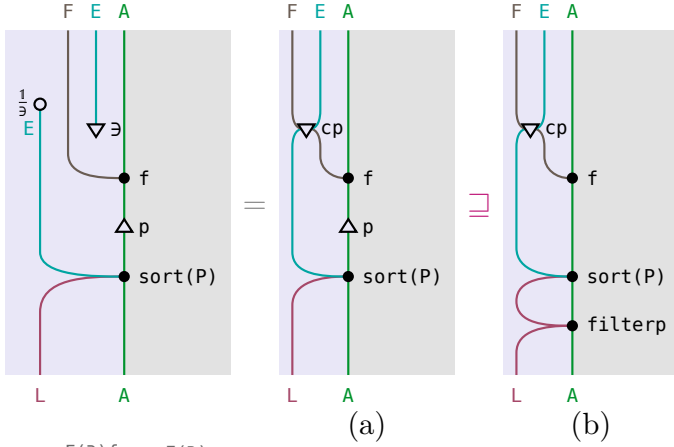
(c) $\text{thinlist}(Q) \text{ setify} \sqsubseteq \text{setify thin}(Q) \text{ — } \triangleq \text{thinlist}$, (3.1.1.7a), (3.1.1.3a)

(d) $\text{sort}(P) \triangleq \text{setify}^\circ \text{ ordered } P \text{ — } \triangleq \text{thinlist}$

Lemma 8.1

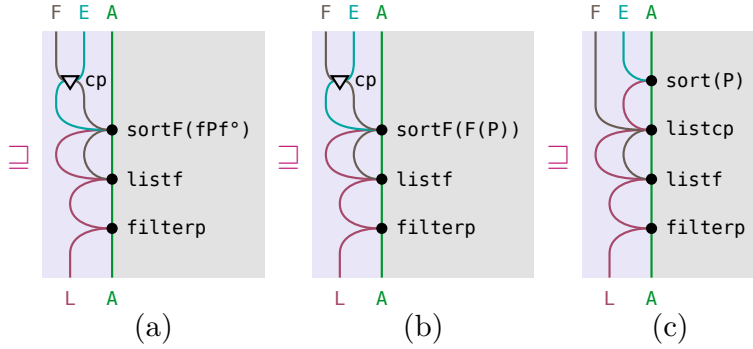
$\text{Map}(f) \wedge (\forall X. \forall Y. X \sqsubseteq Y \Rightarrow \text{sortF}(X) \sqsubseteq \text{sortF}(Y)) \wedge \text{MonoAlg}(f, P) \wedge$
 $\text{sortF}(fP f^\circ) \text{listf} \sqsubseteq P(f) \text{sort}(P) \wedge \text{sort}(P) \text{filterp} \sqsubseteq E(p) \text{sort}(P) \wedge F(\text{sort}(P)) \text{listcp} \sqsubseteq \text{cp}$
 $\text{sortF}(F(P)) \Rightarrow F(\text{sort}(P)) \text{listcp} \text{listf} \text{filterp} \sqsubseteq \frac{F(\exists)fp}{\exists} \text{sort}(P)$
 one sorted list built from sorted arguments, instead of a set built and then sorted — $f : FA \rightarrow A$ monotonic
 on P , p coreflexive, F linear.

$\text{Map}(f) \wedge \text{sort}(P) \text{filterp} \sqsubseteq E(p) \text{sort}(P) \Rightarrow \text{cp } P(f) \text{sort}(P) \text{filterp} \sqsubseteq \frac{F(\exists)fp}{\exists} \text{sort}(P)$
 sorting the f -images of the product and filtering by p is below sorting the set $F(\exists)fp$ builds



- (a) $\frac{F(\exists)fp}{\exists} = \frac{F(\exists)}{\exists} E(fp) \text{ — (3.1.1.10a), } \triangleq \text{thinlist}$
 $E(f) = P(f)$, f a map — (5.4d)
 (b) $\text{sort}(P) \text{filterp} \sqsubseteq E(p) \text{sort}(P) \text{ — (8.3b)}$

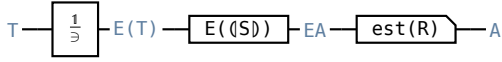
$\text{Map}(f) \wedge (\forall X. \forall Y. X \sqsubseteq Y \Rightarrow \text{sortF}(X) \sqsubseteq \text{sortF}(Y)) \wedge \text{MonoAlg}(f, P) \wedge \text{sortF}(fP f^\circ) \text{listf} \sqsubseteq P(f) \text{sort}(P) \wedge$
 $F(\text{sort}(P)) \text{listcp} \sqsubseteq \text{cp} \text{sortF}(F(P)) \Rightarrow F(\text{sort}(P)) \text{listcp} \text{listf} \text{filterp} \sqsubseteq \text{cp } P(f) \text{sort}(P) \text{filterp}$
 combining the arguments' sorted lists by listcp , mapping f and filtering is below sorting the f -images and filtering



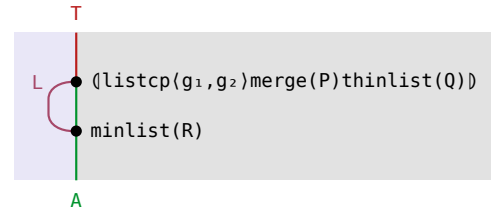
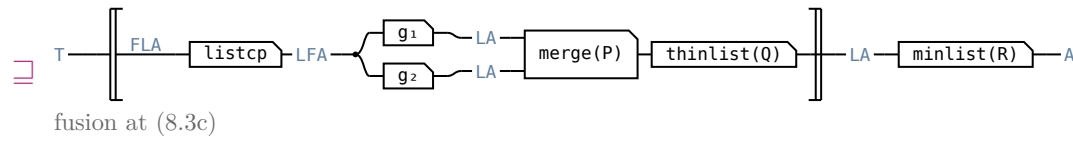
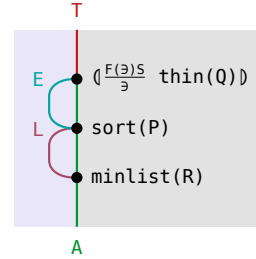
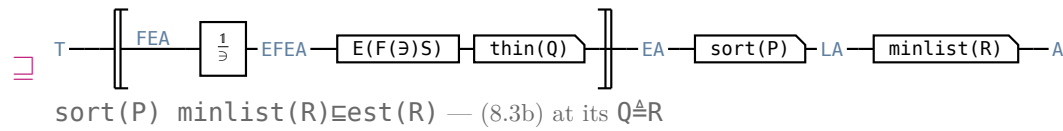
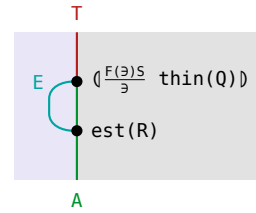
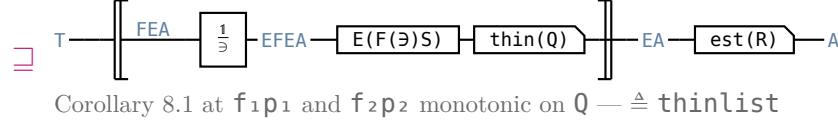
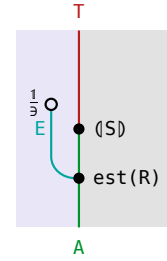
- (a) $\text{sort}(fP f^\circ) \text{listf} \sqsubseteq P(f) \text{sort}(P) \text{ — (8.3b)}$
 (b) $FP \sqsubseteq fP f^\circ \text{ — (7.2b), } \triangleq \text{thinlist}$
 (c) $F(\text{sort}(P)) \text{listcp} \sqsubseteq \text{cp}(F) \text{sort}(FP) \text{ — (8.3b)}$

$\text{Map}(f_1) \wedge \text{Map}(f_2) \wedge Q \subseteq R \wedge 1 \in Q \wedge Q \subseteq Q \wedge R \circ R \subseteq R \wedge \text{MonoAlg}(f_1 \text{ } p_1, Q) \wedge \text{MonoAlg}(f_2 \text{ } p_2, Q)$ Theorem 8.2
 $\wedge (\forall X. \forall Y. X \subseteq Y \Rightarrow \text{sortF}(X) \subseteq \text{sortF}(Y)) \wedge \text{MonoAlg}(f_1, P) \wedge \text{MonoAlg}(f_2, P) \wedge \text{sortF}(f_1$
 $P f_1^\circ) \text{listf}_1 \subseteq P(f_1) \text{sort}(P) \wedge \text{sortF}(f_2 P f_2^\circ) \text{listf}_2 \subseteq P(f_2) \text{sort}(P) \wedge$
 $\text{sort}(P) \text{filterp}_1 \subseteq E(p_1) \text{sort}(P) \wedge \text{sort}(P) \text{filterp}_2 \subseteq E(p_2) \text{sort}(P) \wedge F(\text{sort}(P)) \text{listcp} \subseteq \text{cp}$
 $\text{sortF}(F(P)) \wedge (\text{sort}(P) \times \text{sort}(P)) \text{merge}(P) \subseteq \text{cup } \text{sort}(P) \wedge$
 $\text{sort}(P) \text{thinlist}(Q) \subseteq \text{thin}(Q) \text{sort}(P) \wedge \text{sort}(P) \text{minlist}(R) \subseteq \text{est}(R) \wedge g_1 = \text{listf}_1 \text{ filterp}_1 \wedge$
 $g_2 = \text{listf}_2 \text{ filterp}_2 \wedge S = f_1 \text{ } p_1 \cup f_2 \text{ } p_2 \Rightarrow (\text{listcp}(g_1, g_2) \text{merge}(P) \text{thinlist}(Q))$
 $\text{minlist}(R) \subseteq \frac{(S)}{S} \text{est}(R)$
 a fold on sorted lists of partial solutions, thinned at every step — at $\triangleq \text{thinlist}$'s binary thinning data.

circuit



Hinze–Marsden

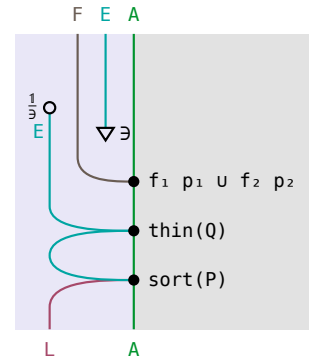
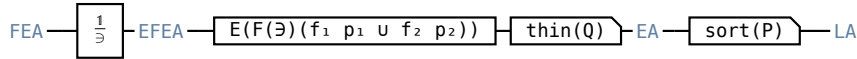


$$\begin{aligned}
& \text{Map}(f_1) \wedge \text{Map}(f_2) \wedge (\forall X. \forall Y. X \sqsubseteq Y \Rightarrow \text{sortF}(X) \sqsubseteq \text{sortF}(Y)) \wedge \text{MonoAlg}(f_1, P) \wedge \text{MonoAlg}(f_2, P) \wedge \quad (8.3c) \\
& \text{sortF}(f_1 \text{ Pf}_1^\circ) \text{listf}_1 \sqsubseteq \text{P}(f_1) \text{sort}(P) \wedge \text{sortF}(f_2 \text{ Pf}_2^\circ) \text{listf}_2 \sqsubseteq \text{P}(f_2) \text{sort}(P) \wedge \\
& \text{sort}(P) \text{filterp}_1 \sqsubseteq \text{E}(p_1) \text{sort}(P) \wedge \text{sort}(P) \text{filterp}_2 \sqsubseteq \text{E}(p_2) \text{sort}(P) \wedge \text{F}(\text{sort}(P)) \text{listcp} \sqsubseteq \text{cp} \text{sortF}(\text{F}(P)) \\
& \wedge (\text{sort}(P) \times \text{sort}(P)) \text{merge}(P) \sqsubseteq \text{cup} \text{sort}(P) \wedge \text{sort}(P) \text{thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{sort}(P) \Rightarrow \\
& \text{F}(\text{sort}(P)) \text{listcp}(\text{listf}_1 \text{ filterp}_1, \text{listf}_2 \text{ filterp}_2) \text{merge}(P) \text{thinlist}(Q) \sqsubseteq \frac{\text{F}(\exists)(f_1 \text{ p}_1 \cup f_2 \text{ p}_2)}{\exists} \\
& \text{thin}(Q) \text{sort}(P)
\end{aligned}$$

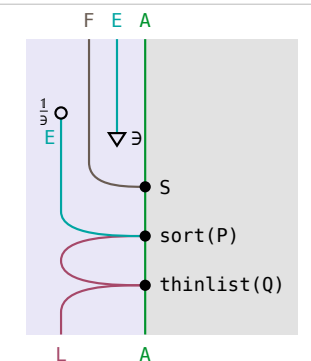
sorting the candidate set is what turns the thinning algebra into an algebra on lists — the side condition of binary thinning theorem's last step.

circuit

Hinze–Marsden



$$\begin{aligned}
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)S)} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \text{sort}(P) \text{thinlist}(Q) \sqsubseteq \text{thin}(Q) \text{sort}(P) \text{---} (8.3b)
\end{aligned}$$



$$\begin{aligned}
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)f_1 \text{ p}_1)} \text{---} \text{EA} \text{---} \text{cup} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& = \\
& \left\langle \frac{\text{F}(\exists)f_1 \text{ p}_1}{\exists}, \frac{\text{F}(\exists)f_2 \text{ p}_2}{\exists} \right\rangle \text{cup} \text{sort}(P) \text{thinlist}(Q) \\
& S = (f_1 \text{ p}_1) \cup (f_2 \text{ p}_2) \text{---} \triangle \text{thinlist, then } \triangle \text{cup}
\end{aligned}$$

$$\begin{aligned}
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)f_1 \text{ p}_1)} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{merge}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)f_2 \text{ p}_2)} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{merge}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \left\langle \frac{\text{F}(\exists)f_1 \text{ p}_1}{\exists}, \frac{\text{F}(\exists)f_2 \text{ p}_2}{\exists} \right\rangle (\text{sort}(P) \times \text{sort}(P)) \text{merge}(P) \text{thinlist}(Q) \\
& (\text{sort}(P) \times \text{sort}(P)) \text{merge}(P) \sqsubseteq \text{cup} \text{sort}(P) \text{---} (8.3b)
\end{aligned}$$

$$\begin{aligned}
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)f_1 \text{ p}_1)} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{LA} \text{---} \text{merge}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \text{FEA} \text{---} \boxed{\frac{1}{3}} \text{---} \text{EFEA} \text{---} \boxed{\text{E}(\text{F}(\exists)f_2 \text{ p}_2)} \text{---} \text{EA} \text{---} \text{sort}(P) \text{---} \text{LA} \text{---} \text{merge}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \left\langle \frac{\text{F}(\exists)f_1 \text{ p}_1}{\exists} \text{sort}(P), \frac{\text{F}(\exists)f_2 \text{ p}_2}{\exists} \text{sort}(P) \right\rangle \text{merge}(P) \text{thinlist}(Q) \\
& (X, Y) (\text{sort}(P) \times \text{sort}(P)) = (X \text{sort}(P), Y \text{sort}(P)) \text{---} (5.2.2a)
\end{aligned}$$

$$\begin{aligned}
& \text{FEA} \text{---} \boxed{\text{F}(\text{sort}(P))} \text{---} \text{FLA} \text{---} \text{listcp} \text{---} \text{LFA} \text{---} \left\langle \text{listf}_1 \text{ filterp}_1, \text{listf}_2 \text{ filterp}_2 \right\rangle \text{---} \text{LA} \text{---} \text{merge}(P) \text{---} \text{thinlist}(Q) \text{---} \text{LA} \\
& \text{F}(\text{sort}(P)) \text{listcp} \langle g_1, g_2 \rangle \text{merge}(P) \text{thinlist}(Q) \\
& \text{Lemma 8.1 at } f_1, p_1 \text{ and at } f_2, p_2, \text{ then } X \langle g_1, g_2 \rangle \sqsubseteq \langle Xg_1, Xg_2 \rangle \text{---} (5.2.2a); g_i \triangle \text{list}(f_i) \\
& \text{filter}(p_i) \text{---} \triangle \text{thinlist}
\end{aligned}$$

§8.4. The knapsack problem

(8.4a)

$\text{vol}, \text{wt} : \text{Item} \rightarrow \text{Real}, \text{value} \triangle \text{list}(\text{vol}) \text{ sum}, \text{weight} \triangle \text{list}(\text{wt}) \text{ sum}.$

$\text{subseq} = ([\text{nil}, \text{cons}] \cup [\text{nil}, \pi_2])$, within w the coreflexive on xs with weight $xs \leq w, 0 \leq w$.

$R \triangle \text{value} \geq \text{value}^\circ, Q \triangle Rn(\text{weight} \leq \text{weight}^\circ), P \triangle R, .$

$\text{FA} = 1 + \text{Item} \times A, \text{listcp} = \text{wrap} + \text{cpr}, g_1 \triangle \text{list}([\text{nil}, \text{cons}]) \text{ filter}(\text{within } w) = [\text{list}(\text{nil}), h_1],$
 $g_2 \triangle \text{list}([\text{nil}, \pi_2]) = [\text{list}(\text{nil}), h_2].$

$h_1 \triangle \text{list}(\text{cons}) \text{ filter}(\text{within } w), h_2 \triangle \text{list}(\pi_2).$

(8.4b)

the law	what it says
$(1 \times R) \text{ (cons (within } w)) \sqsubseteq \text{cons (within } w) R$ FALSE	not monotonic on R: a selection of greater value need not still fit once one more item goes in
$(1 \times Q) \text{ (cons (within } w)) \sqsubseteq \text{cons (within } w) Q$ $(1 \times Q) \pi_2 \sqsubseteq \pi_2 Q$,	both halves are monotonic on Q once ties in value are broken by weight

$$\begin{aligned} & 0 \leq w \wedge (\forall i. 0 \leq \text{wt}(i)) \wedge (\forall X. \forall Y. X \subseteq Y \implies \text{sortF}(X) \subseteq \text{sortF}(Y)) \wedge \\ & \text{sortF}([\text{nil}, \text{cons}]R[\text{nil}, \text{cons}]^\circ) \text{listf}_1 \subseteq P([\text{nil}, \text{cons}]) \text{sortP} \wedge \\ & \text{sortF}([\text{nil}, \pi_2]R[\text{nil}, \pi_2]^\circ) \text{listf}_2 \subseteq P([\text{nil}, \pi_2]) \text{sortP} \wedge \text{sortP} \text{filterp}_1 \subseteq E(\text{within}) \text{sortP} \wedge \\ & F(\text{sortP}) \text{listcp} \subseteq \text{cp} \text{sortF}(F(R)) \wedge (\text{sortP} \times \text{sortP}) \text{mergeP} \subseteq \text{cup} \text{sortP} \wedge \text{sortP} \\ & \text{thinlist} \subseteq \text{thin}(Q(\text{vol}, \text{wt})) \text{sortP} \wedge \text{sortP} \text{minlist} \subseteq \text{est}(R) \implies (\text{listcp}(\text{listf}_1 \text{filterp}_1, \text{listf}_2) \text{mergeP} \\ & \text{thinlist}) \text{minlist} \subseteq \underbrace{\text{subseq within}}_3 \text{est}(R) \end{aligned}$$

the knapsack problem, as a fold that thins the packings kept at each item

[illegible]

§8.5. The paragraph problem

(8.5a)

```

Line=list+ Word, Para=list+ Line, FA=Word+Word×A, listcp=wrap+cpr.
new(a,xs)=[a]#xs, glue(a,xs)=[a]#head(xs)#tail(xs), partition≐([wrap wrap,new u glue]) : list+
Word→Para.

width≐([length,(length×1) plus succ]), 0≤length a, fits w the coreflexive on a line x with width x≤w, ok w the
coreflexive on [x]#xs with width x≤w.

white w x=w-width x, collect≐list(sqr) sum, waste w≐init list(white w) collect.

R≐(waste w)≤(waste w)°, Q≐Rn(head head°), P≐T, .

g1≐list([wrap wrap,new]), g2≐list([wrap wrap,glue]) filter(ok w), start≐wrap wrap wrap, h1≐list(new),
h2≐list(glue) filter(ok w).

```

(8.5b)

the law	what it says
$(1 \times R) \text{ glue} \sqsubseteq \text{glue } R \quad \text{FALSE}$	glue is not monotonic on R: the waste of a paragraph depends on its whole first line, so no greedy algorithm solves this
$(1 \times Q) \text{ new} \sqsubseteq \text{new } Q$ $(1 \times Q) (\text{glue } (ok \ w)) \sqsubseteq \text{glue } (ok \ w) Q$ $\text{cons monotonic on } \text{collect} \sqsubseteq \text{collect}^\circ, .$	both halves are monotonic on Q once ties in waste are broken by the first line
merge $T = \text{cat}$; $P \triangleq \text{head prefix head}^\circ$ also serves	T needs no sorting at all, and prefix is a linear order on first lines of paragraphs of one input

(8.5c)

$(\forall a. 0 \leq \text{len}(a)) \wedge (\forall a. \text{len}(a) \leq w) \wedge (\forall X. \forall Y. X \sqsubseteq Y \Rightarrow \text{sortF}(X) \sqsubseteq \text{sortF}(Y)) \wedge \text{sortF}([wrap$
 $wrap,new]T[wrap wrap,new]^\circ) \text{listf}_1 \sqsubseteq P([wrap wrap,new]) \text{sortP} \wedge \text{sortF}([wrap wrap,glue]T[wrap$
 $wrap,glue]^\circ) \text{listf}_2 \sqsubseteq P([wrap wrap,glue]) \text{sortP} \wedge \text{sortP filterp}_2 \sqsubseteq E(ok) \text{sortP} \wedge F(\text{sortP}) \text{listcp} \sqsubseteq \text{cpr}$
 $\text{sortF}(F(T)) \wedge (\text{sortP} \times \text{sortP}) \text{mergeP} \sqsubseteq \text{cup sortP} \wedge \text{sortP thinlist} \sqsubseteq \text{thin}(Q(\text{len},w)) \text{sortP} \wedge \text{sortP}$
 $\text{minlist} \sqsubseteq \text{est}(R) \Rightarrow ([\text{listcp}(\text{listf}_1, \text{listf}_2 \text{ filterp}_2) \text{ mergeP thinlist}] \text{minlist} \sqsubseteq \frac{\text{partition fits}}{3} \text{est}(R))$
 a paragraph laid out as a fold that thins the layouts kept at each word

circuit	Hinze–Marsden
$\text{list}^*(\text{Word}) \xrightarrow{\frac{1}{3}} E(\text{list}^*(\text{Word})) \xrightarrow{E(\text{partition fits})} E(\text{list}^*(\text{list}^*(\text{Word}))) \xrightarrow{\text{est}(R)} \text{list}^*(\text{list}^*(\text{Word}))$ $\frac{\text{partition list}^*(\text{fits } w)}{3} \text{est}(R)$	
$\text{list}^*(\text{Word}) \xrightarrow{\frac{1}{3}} E(\text{list}^*(\text{Word})) \xrightarrow{E([wrap wrap,new] u [wrap wrap,glue]ok)} E(\text{list}^*(\text{list}^*(\text{Word}))) \xrightarrow{\text{est}(R)} \text{list}^*(\text{list}^*(\text{Word}))$ $= \frac{([wrap wrap,new] u ([wrap wrap,glue] (ok \ w)))}{3} \text{est}(R)$ fusion, every word fits on a line by itself.	
$\text{list}^*(\text{Word}) \xrightarrow{\frac{1}{3}} E(\text{list}^*(\text{Word})) \xrightarrow{E([wrap wrap,new] u [wrap wrap,glue]ok)} E(\text{list}^*(\text{list}^*(\text{Word}))) \xrightarrow{\text{est}(R)} \text{list}^*(\text{list}^*(\text{Word}))$ $= \frac{([wrap wrap,new] u ([wrap wrap,glue] (ok \ w)))}{3} \text{est}(R)$ the algebra as $(f_1 p_1) \cup (f_2 p_2)$, $p_1 \triangleq 1 \text{ — } \triangleq \text{thinlist}$.	
$\text{listcp} \langle g_1, g_2 \rangle \text{ cat thinlist}(Q) \text{ minlist}(R)$ binary thinning theorem, at $P \triangleq T$ with merge $T = \text{cat}$, Q from (8.5b)	
$([start, cpr \langle h_1, h_2 \rangle \text{ cat thinlist}(Q)])$ $= \text{minlist}(R)$ $\text{listcp} = \text{wrap} + \text{cpr}$, g_i along the coproduct — $\triangleq \text{partition}$	

FA=(City×City)+(City×A), the base functor of cons-lists of length at least two; listcp=wrap+cpr; tc : City×City→Real, neither positive nor symmetric.

$$\text{cost}(x_s, y_s) = \text{outcost } x_s + \text{incost } y_s, \quad \text{outcost}[a_0, \dots, a_n] = \text{tc}(a_0, a_1) + \dots + \text{tc}(a_{n-1}, a_n), \quad \text{incost}[a_0, \dots, a_n] = \text{tc}(a_1, a_0) + \dots + \text{tc}(a_n, a_{n-1}).$$

```
next $\hat{=}$ tail head, next2 $\hat{=}$ next $\times$ next, head2 $\hat{=}$ head $\times$ head, R $\hat{=}$ cost $\leq$ cost $^\circ$ , Q $\hat{=}$ Rn(next2 next2 $^\circ$ )n(head2 head2 $^\circ$ ), P $\hat{=}$ T,
g1 $\hat{=}$ list([start,dropl]), g2 $\hat{=}$ list([start,dropr]).
```

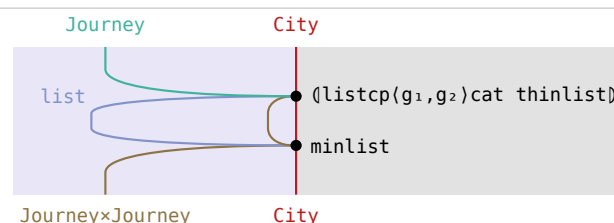
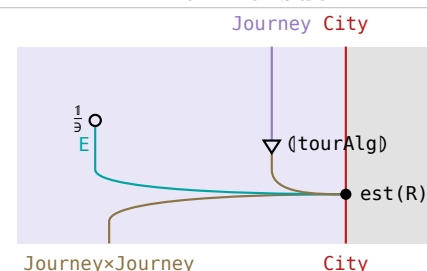
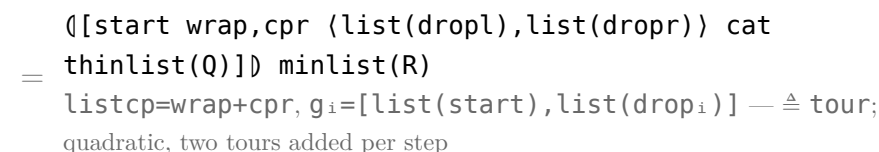
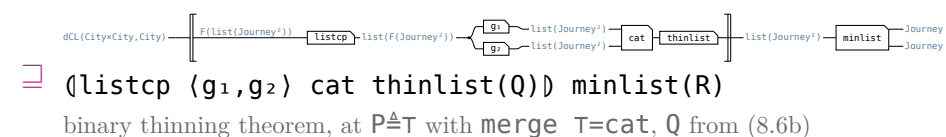
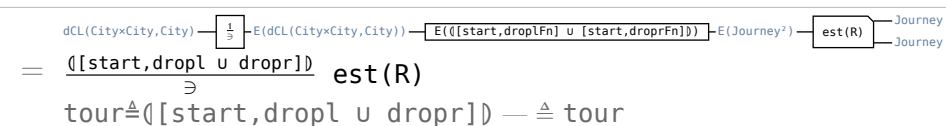
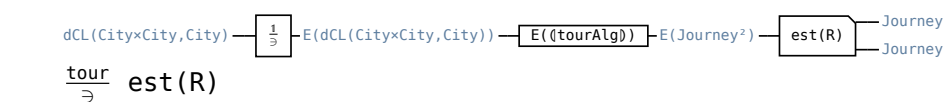
the law	what it says
$(1 \times R) \text{ drop}l \sqsubseteq \text{drop}l \text{ } R$ FALSE $(1 \times R) \text{ drop}r \sqsubseteq \text{drop}r \text{ } R$ FALSE	neither drop is monotonic on R : the two edges it adds and removes depend on head and next of both lists
$(1 \times Q) \text{ drop}l \sqsubseteq \text{drop}l \text{ } Q$ $(1 \times Q) \text{ drop}r \sqsubseteq \text{drop}r \text{ } Q$,	both are, once ties in cost are broken by the two second cities — the heads already agree among tours of one input

$$\begin{aligned}
& (\forall X. \forall Y. X \leq Y \Rightarrow \text{sortF}(X) \leq \text{sortF}(Y)) \wedge \\
& \text{sortF}([start, dropLfn] \tau [start, dropLfn]^\circ) g_1 \leq P([start, dropLfn]) \text{sortP} \wedge \\
& \text{sortF}([start, dropRfn] \tau [start, dropRfn]^\circ) g_2 \leq P([start, dropRfn]) \text{sortP} \wedge F(\text{sortP}) \text{listcp} \leq \text{cp} \\
& \text{sortF}(F(\tau)) \wedge (\text{sortP} \times \text{sortP}) \text{cat} \leq \text{cup} \text{sortP} \wedge \text{sortP} \text{thinlist} \leq \text{thin}(Qc(tc)) \text{sortP} \wedge \text{sortP} \\
& \text{minlist} \leq \text{est}(R) \Rightarrow (\text{listcp}(g_1, g_2) \text{cat} \text{thinlist}) \text{minlist} \leq \frac{(\text{tourAlg})}{3} \text{est}(R)
\end{aligned}$$

a least-cost bitonic tour, as a fold that thins the tours kept at each city

circuit

Hinze–Marsden



§9. Dynamic Programming

§9.1. Theory

§9.1.1. Example: segmenting a list

- $(\mathsf{T}) = \text{concat}$
 $\mathsf{T} = [\text{nil}, \text{cat}]$ takes a non-empty first list, so T° cuts a non-empty prefix off a list every way, and folding with T flattens a list of segments
- $(\mathsf{h}) = \text{list}(\text{sum})$
 $\mathsf{h} = [\text{nil}, \text{cons}(\text{sum} \times 1)]$ puts a segment's sum in front, so folding with h sums every segment
- $\mathsf{H} = \text{partition list}(\text{sum})$
 H segments a list every way, then sums each segment; with R comparing the largest entry, M cuts the list so that the largest segment sum is as small as possible

(9.1.1b)

x	h(x)
$\text{inl}(())$	$[]$
$\text{inr}([c], [])$	$[c]$
$\text{inr}([a, b], [c])$	$[a + b, c]$
$\text{inr}([a], [b + c])$	$[a, b + c]$

- $\mathsf{H}([a, b, c], \mathsf{ys}) \iff \mathsf{ys} = [a + b + c] \vee \mathsf{ys} = [a, b + c] \vee \mathsf{ys} = [a + b, c] \vee \mathsf{ys} = [a, b, c]$
 H sends $[a, b, c]$ to the sums of its four segmentations and nothing else
- $\mathsf{T}^\circ \mathsf{F}(\mathsf{H}) \mathsf{h} = \mathsf{H}$
cut once, solve the rest, put the pieces back together

(9.1.1c)

(9.1.1d)

$\mathsf{h} : \mathsf{FB} \rightarrow \mathsf{B}$ a map, $\mathsf{T} : \mathsf{FA} \rightarrow \mathsf{A}$ an F-algebra, $\mathsf{R} : \mathsf{B} \rightarrow \mathsf{B}$.

$\mathsf{H} \triangleq (\mathsf{T})^\circ (\mathsf{h}) : \mathsf{A} \rightarrow \mathsf{B}$, $\mathsf{M} \triangleq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$ the problem to be solved, $(\mu \mathsf{X} : \mathsf{G}(\mathsf{X}))$ as in $\triangleq \mu$.

$\mathsf{DPSetting}(\mathsf{h}, \mathsf{R}) \triangleq \text{Map}(\mathsf{h}) \wedge \text{MonoAlg}(\mathsf{h}, \mathsf{R}^\circ) \wedge \mathsf{R}^\circ \mathsf{R}^\circ \sqsubseteq \mathsf{R}^\circ$

h is a map, h is monotonic on R° , and R° is transitive — assumed by every theorem below

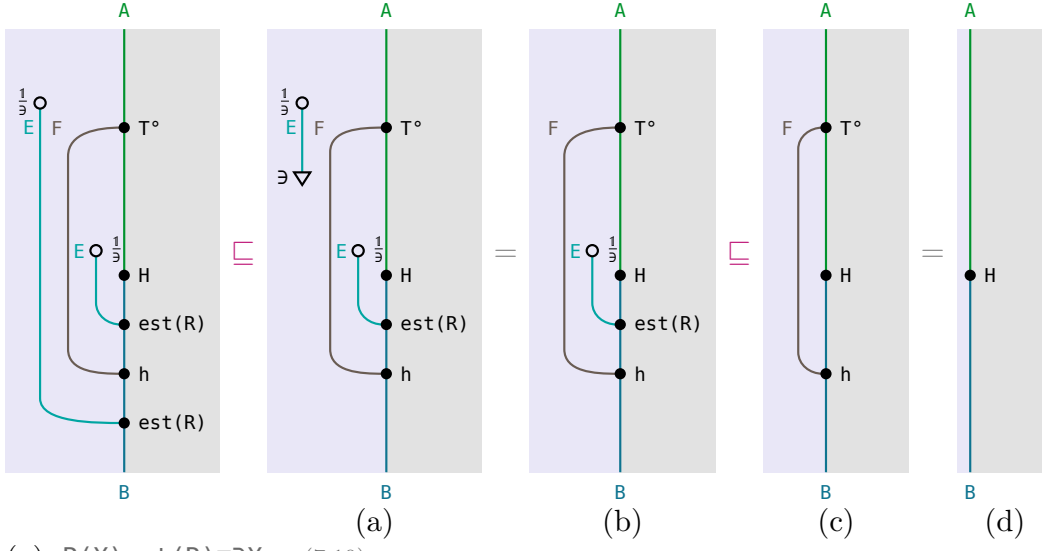
$(\mu \mathsf{X} : \frac{\mathsf{T}^\circ}{\mathsf{S}} \mathsf{P}(\mathsf{F}(\mathsf{X}) \mathsf{h}) \text{est}(\mathsf{R})) \sqsubseteq \frac{\mathsf{H}}{\mathsf{S}} \text{est}(\mathsf{R})$
every answer the recursion returns is an optimal one

Theorem 9.1

- $\frac{\mathsf{T}^\circ}{\mathsf{S}}$ takes the input apart one step every way; $\mathsf{F}(\mathsf{X})$ solves each part by the recursion X ; h assembles each candidate; $\text{est}(\mathsf{R})$ keeps a best one
- $\sqsubseteq \mathsf{M}$: that never misses the optimum, though all candidates are never listed as H would
- h monotonic on R : a better part never makes the whole worse
- $\sqsubseteq$, not $=$: some optima may not be returned; that the recursion returns anything at all needs T° to stop taking apart — dynamic programming theorem

$$T^\circ F(H)h=H \Rightarrow \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq H \quad (9.2)$$

taking the input apart every way T allows ($\frac{T^\circ}{\exists}$), solving each part by M and keeping an optimum ($P(F(M)h) \text{est}(R)$) returns only what H returns



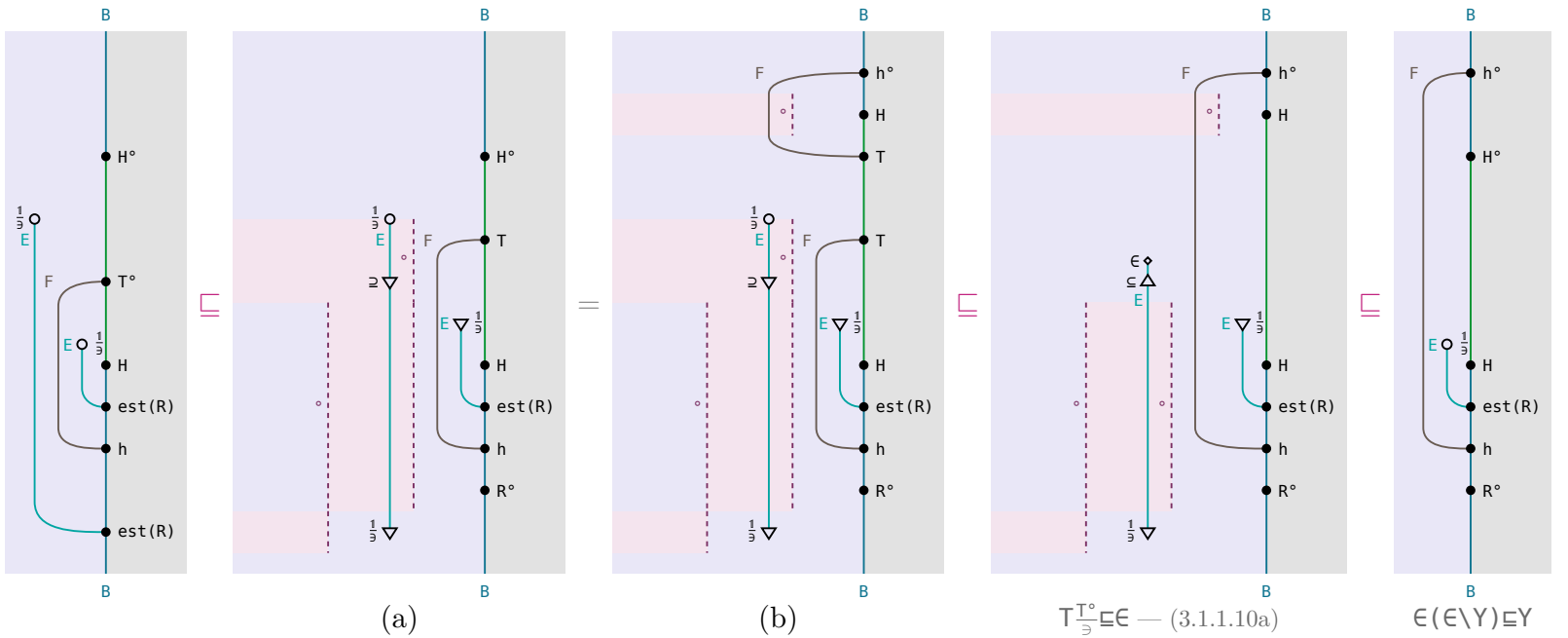
- (a) $P(X)\text{est}(R) \sqsubseteq \exists X$ — (7.10)
 (b) $\frac{T^\circ}{\exists} \exists = T^\circ$ — (3.1.1.10a)
 (c) $M \sqsubseteq \frac{H}{\exists} \exists = H$ — (7.1.1a)
 (d) $T^\circ F(H)h=H$ — hylomorphism theorem

$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq R^\circ \quad (9.3)$$

for every b that H returns from an input, the step $\frac{T^\circ}{\exists} P(F(M)h) \text{est}(R)$ returns from that input only b' with R relating b' to b

$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq h^\circ F(H^\circ)F(\frac{H}{\exists} \text{est}(R))hR^\circ$$

from an answer of H , the step's answer is reached by taking H 's parts back, rebuilding them by $F(M)h$, then one R°



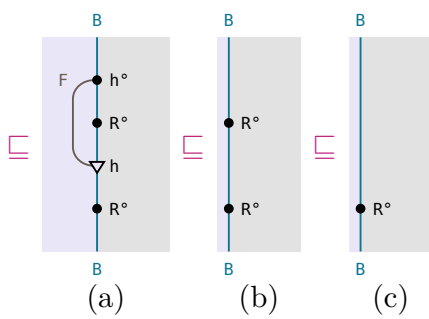
- (a) $P(X)\text{est}(R) \sqsubseteq \exists \setminus (XR^\circ)$ — (7.10)
 (b) $H^\circ = h^\circ F(H^\circ)T$ — hylomorphism theorem

$$T \frac{T^\circ}{\exists} \sqsubseteq \exists \text{ — (3.1.1.10a)}$$

$$\exists(\exists \setminus Y) \sqsubseteq Y$$

$$h^\circ F(H^\circ)F(\frac{H}{\exists} \text{est}(R))hR^\circ \sqsubseteq R^\circ$$

rebuilding an answer of H from its parts, each replaced by M 's, and one R° stays within R°

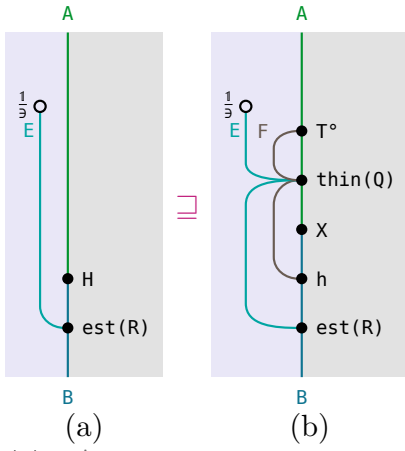


- (a) $H^\circ M \sqsubseteq R^\circ$ — (7.1.1a)
 (b) h monotonic on R°
 (c) R transitive

$$\text{ThinCondition}(\mathbf{T}, \mathbf{h}, \mathbf{R}, \mathbf{Q}) \Rightarrow (\mu \mathbf{X} : \frac{\mathbf{T}^\circ}{\mathbf{Q}} \text{thin}(\mathbf{Q}) \mathbf{P}(\mathbf{F}(\mathbf{X}) \mathbf{h}) \text{est}(\mathbf{R})) \sqsubseteq_{\frac{\mathbf{H}}{\mathbf{Q}}} \text{est}(\mathbf{R})$$

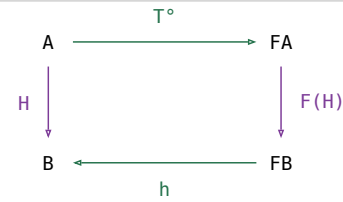
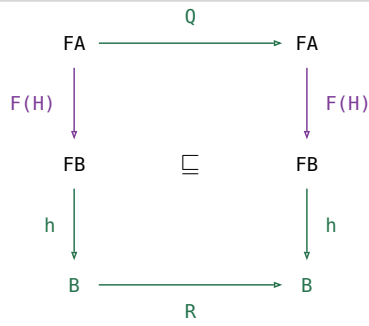
Theorem 9.2

an optimum over everything $\mathbf{H}$ returns is reached by taking the input apart every way $\mathbf{T}$ allows, dropping the parts that can never win, solving each of the rest and keeping one optimum



$$(a) \quad \mathbf{H} \triangleq (\mathbf{T})^\circ (\mathbf{h}) \triangleq \mathbf{H}$$

$$(b) \quad \mathbf{Q} \mathbf{F}(\mathbf{H}) \mathbf{h} \sqsubseteq \mathbf{F}(\mathbf{H}) \mathbf{h} \mathbf{R}, \mathbf{h} \text{ monotonic on } \mathbf{R} \text{ — (8.1a), (7.1.1a)}$$



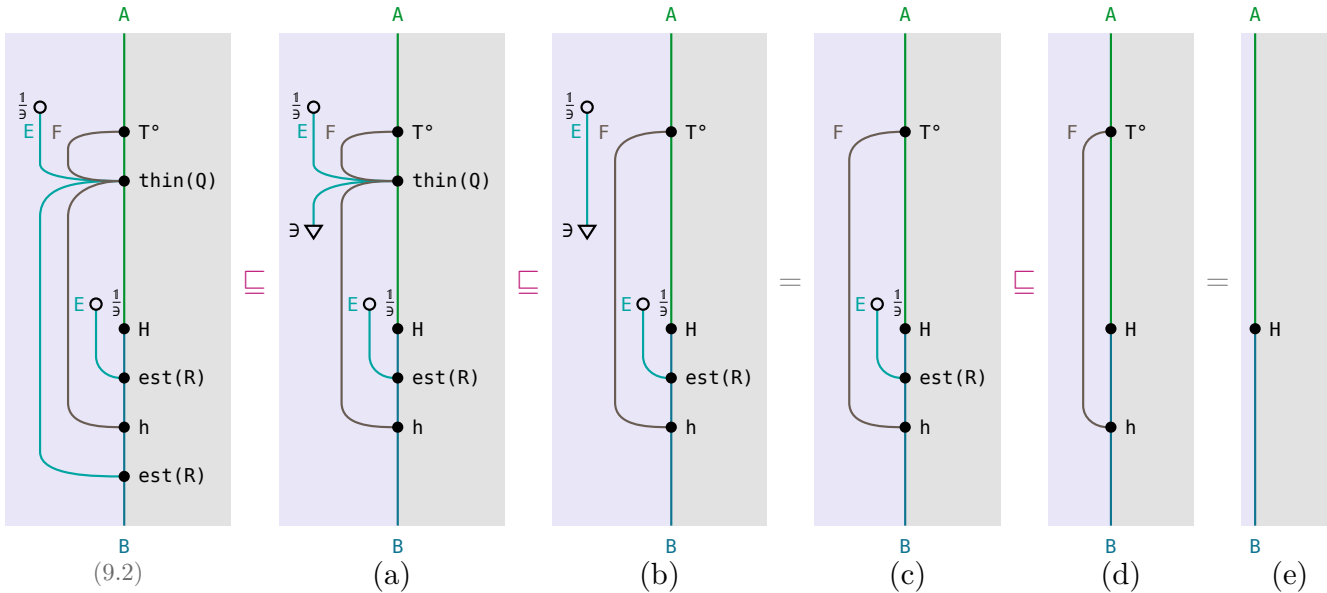
$\mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H}$: taking the input apart once by $\mathbf{T}^\circ$, solving the parts by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ returns exactly what $\mathbf{H}$ returns

$\mathbf{Q} \mathbf{F}(\mathbf{H}) \mathbf{h} \sqsubseteq \mathbf{F}(\mathbf{H}) \mathbf{h} \mathbf{R}$: every result of thinning by $\mathbf{Q}$, solving the parts by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ is reached by $\mathbf{R}$ from a result of solving by $\mathbf{F}(\mathbf{H})$ and assembling by $\mathbf{h}$ without $\mathbf{Q}$

$$\mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H} \Rightarrow \frac{\mathbf{T}^\circ}{\mathbf{Q}} \text{thin}(\mathbf{Q}) \mathbf{P}(\mathbf{F}(\frac{\mathbf{H}}{\mathbf{Q}} \text{est}(\mathbf{R})) \mathbf{h}) \text{est}(\mathbf{R}) \sqsubseteq_{\frac{\mathbf{H}}{\mathbf{Q}}} \mathbf{H}$$

(9.1.1e)

taking the input apart every way $\mathbf{T}$ allows ($\frac{\mathbf{T}^\circ}{\mathbf{Q}}$), dropping the parts $\text{thin}(\mathbf{Q})$ rejects, solving each rest by $\mathbf{M}$ and keeping an optimum returns only what $\mathbf{H}$ returns



$$(a) \quad \mathbf{P}(\mathbf{X}) \text{est}(\mathbf{R}) \sqsubseteq \mathbf{X} \text{ — (7.10)}$$

$$(b) \quad \text{thin}(\mathbf{Q}) \exists \exists \text{ — (8.1a)}$$

$$(c) \quad \frac{\mathbf{T}^\circ}{\mathbf{Q}} \exists = \mathbf{T}^\circ \text{ — (3.1.1.10a)}$$

$$(d) \quad \mathbf{M} \sqsubseteq_{\frac{\mathbf{H}}{\mathbf{Q}}} \exists = \mathbf{H} \text{ — (7.1.1a)}$$

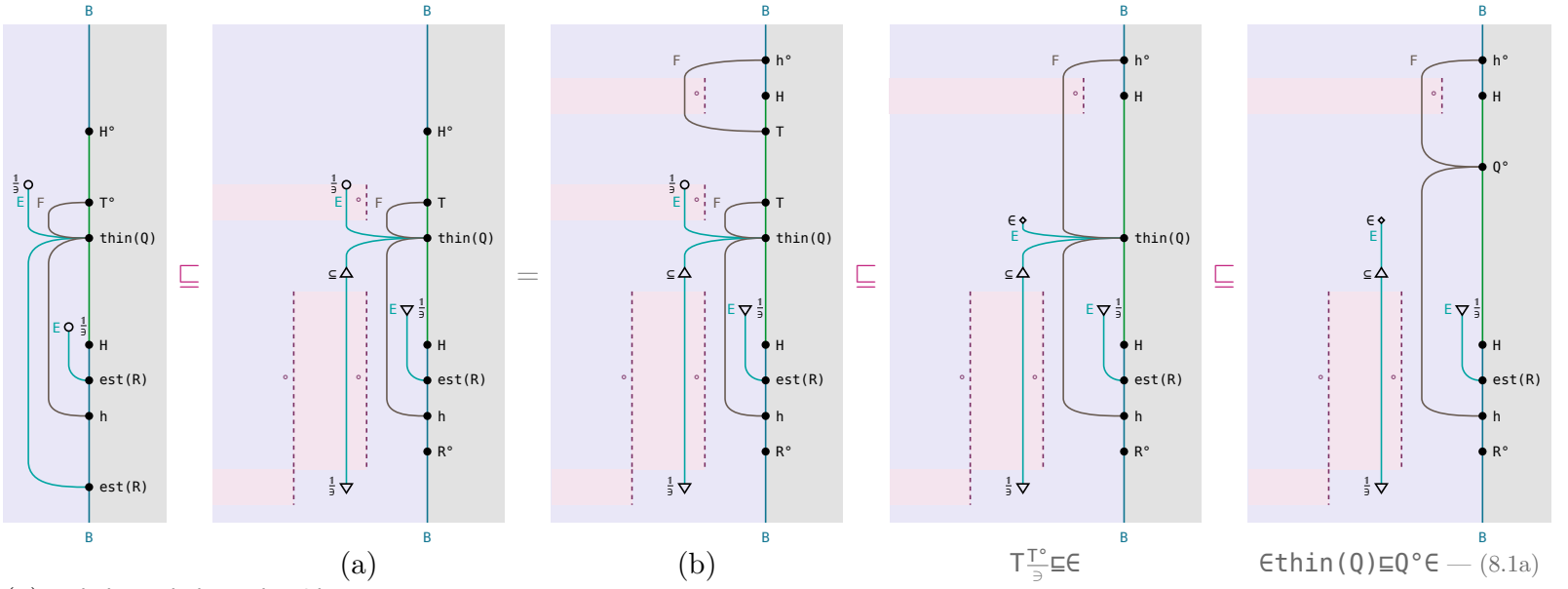
$$(e) \quad \mathbf{T}^\circ \mathbf{F}(\mathbf{H}) \mathbf{h} = \mathbf{H} \text{ — hylomorphism theorem}$$

$$T^\circ F(H)h=H \wedge QF(H)h \sqsubseteq F(H)hR \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{thin}(Q)P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq R^\circ \quad (9.1.1f)$$

H° followed by the body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input

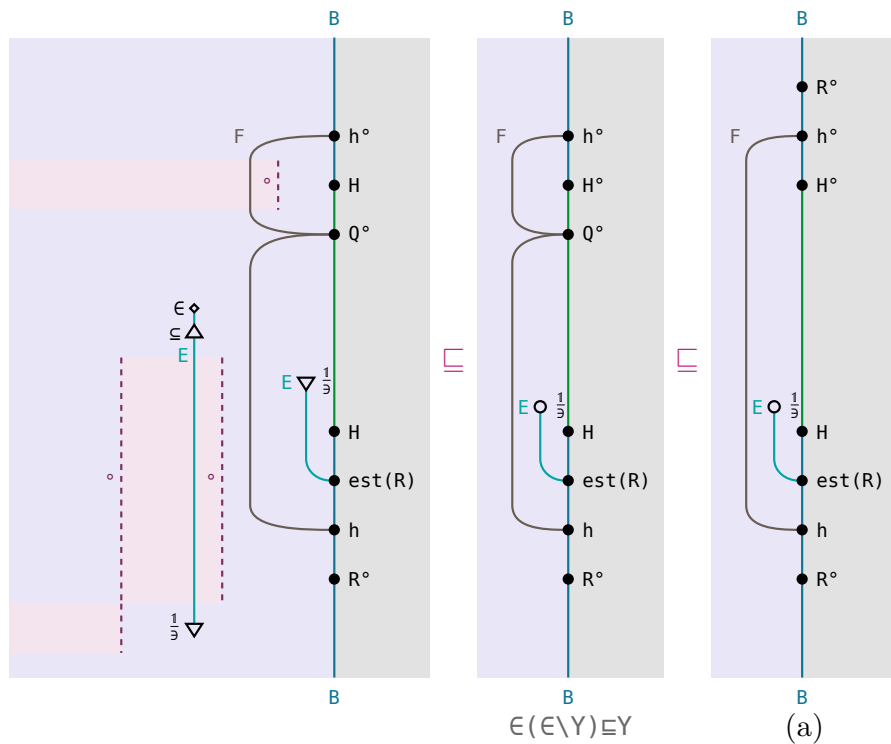
$$T^\circ F(H)h=H \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{thin}(Q)P(F(\frac{H}{\exists} \text{est}(R))h)\text{est}(R) \sqsubseteq h^\circ F(H^\circ)Q^\circ \in (\in \backslash F(\frac{H}{\exists} \text{est}(R))hR^\circ)$$

from an answer of H , the thinned step's answer is reached through H 's parts, a Q° step to a candidate, and that candidate's answer



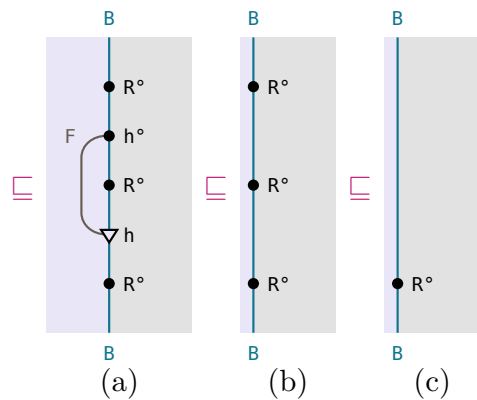
$$QF(H)h \sqsubseteq F(H)hR \implies h^\circ F(H^\circ)Q^\circ \in (\in \setminus F(\frac{H}{3} \text{ est}(R))hR^\circ) \sqsubseteq R^\circ h^\circ F(H^\circ)F(\frac{H}{3} \text{ est}(R))hR^\circ$$

the hypothesis on Q turns the Q° step into an R° step in front of rebuilding by $F(M)h$



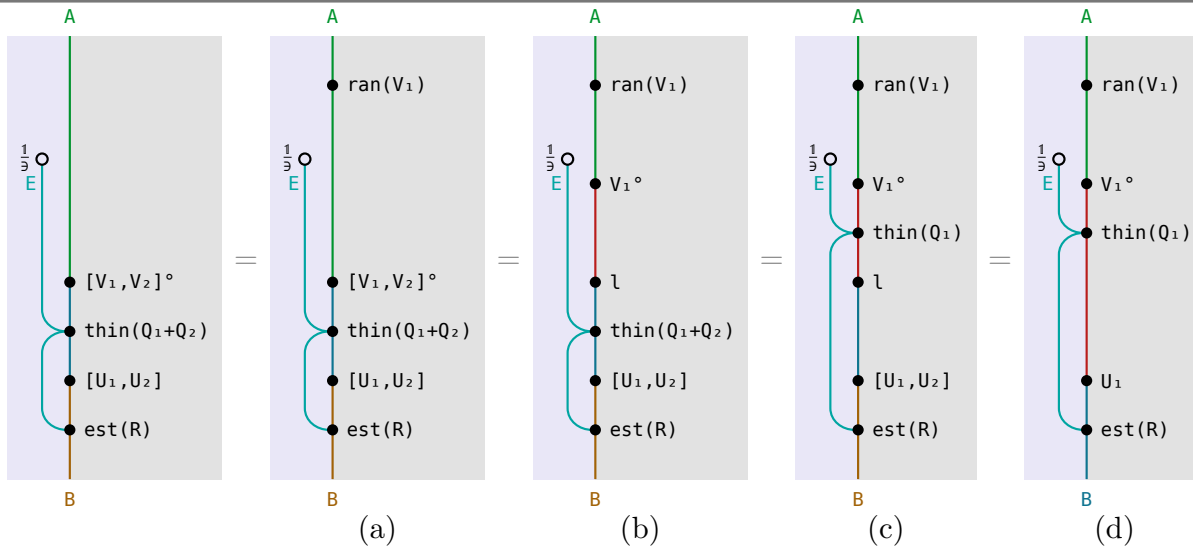
$$R^\circ h^\circ F(H^\circ)F(\frac{H}{3} \text{ est}(R))hR^\circ \sqsubseteq R^\circ$$

rebuilding from parts replaced by M 's, between two R° steps, stays within R°



$$V_2 \ V_1^\circ = \emptyset \implies \frac{[V_1, V_2]^\circ}{\exists} \text{thin}(Q_1+Q_2)P([U_1, U_2])\text{est}(R) = \text{ran}(V_1) \frac{V_1^\circ}{\exists} \text{thin}(Q_1)P(U_1)\text{est}(R) \cup \text{ran}(V_2) \frac{V_2^\circ}{\exists} \text{thin}(Q_2)P(U_2)\text{est}(R) \quad \text{Proposition 9.1}$$

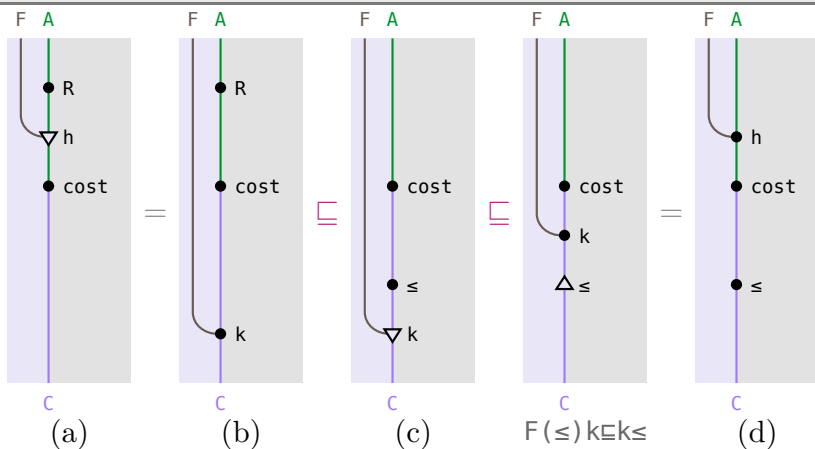
when V_1 and V_2 have disjoint ranges, the body over $[V_1, V_2]$ runs the V_1 problem on inputs V_1 reaches and the V_2 problem on inputs V_2 reaches



- (a) empty off $\text{ran } V_1 \cup \text{ran } V_2$
 (b) $\frac{[V_1, V_2]^\circ}{\exists} = \frac{V_1^\circ}{\exists} P(\text{inl})$ on $\text{ran } V_1$
 (c) $P(\text{inl}) \text{thin}(Q_1+Q_2) = \text{thin}(Q_1) P(\text{inl})$
 (d) P a relator, $\text{inl}[U_1, U_2] = U_1$

$$\text{Map}(\text{cost}) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge h \text{cost} = F(\text{cost})k \wedge F(\leq)k \sqsubseteq k \leq \implies \text{MonoAlg}(h, R) \quad \text{Proposition 9.2}$$

if R compares two values by comparing their **costs** under $\leq$, and h then **cost** equals $F(\text{cost})$ then a k monotonic on $\leq$, then h is monotonic on R



- (a) definition of R , shunting
 (b) $h \text{cost} = F(\text{cost})k$
 (c) $R \text{cost} \sqsubseteq \text{cost} \leq$
 (d) $h \text{cost} = F(\text{cost})k$

(9.1.1h)

the condition	how it is discharged
$F(R)h \sqsubseteq hR$ Proposition 9.2, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{cost} = F(\text{cost})k$, $F(\leq)k \sqsubseteq k \leq$	monotonicity when the cost is itself a fold with a step k monotonic on $\leq$
$F(Rn(H^\circ H))h \sqsubseteq hR$ Proposition 9.3, $R \triangleq \text{cost} \leq \text{cost}^\circ$, $h \text{cost} = F(\langle \text{cost}, H^\circ \rangle)k$, $F(\leq \times 1)k \sqsubseteq k \leq$, H° simple;	monotonicity in context : k may also read the input the part was built from
$QF(H)h \sqsubseteq F(H)hR$ at $Q \triangleq F(U, V)$ Proposition 9.4, U, V preorders, $F(U, R)h \sqsubseteq hR$, $VH \sqsubseteq HR$	both conditions at once, split along the two arguments of a bifunctor

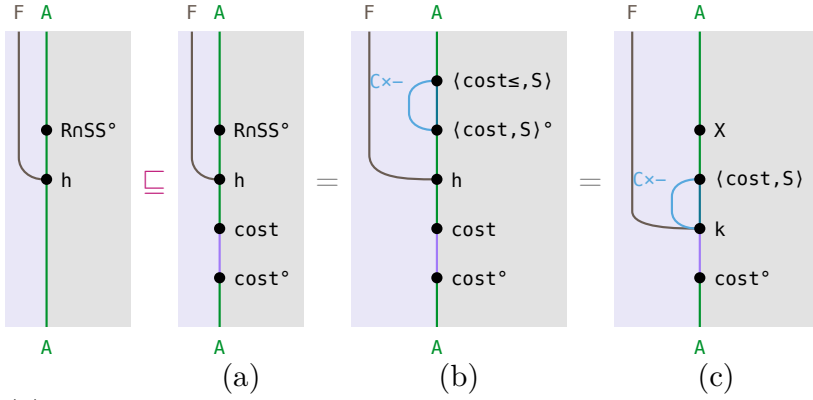
$$\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \wedge F(\leq \times 1)k \sqsubseteq k \leq \Rightarrow F(RnSS^\circ)h \sqsubseteq hR$$

Proposition 9.3

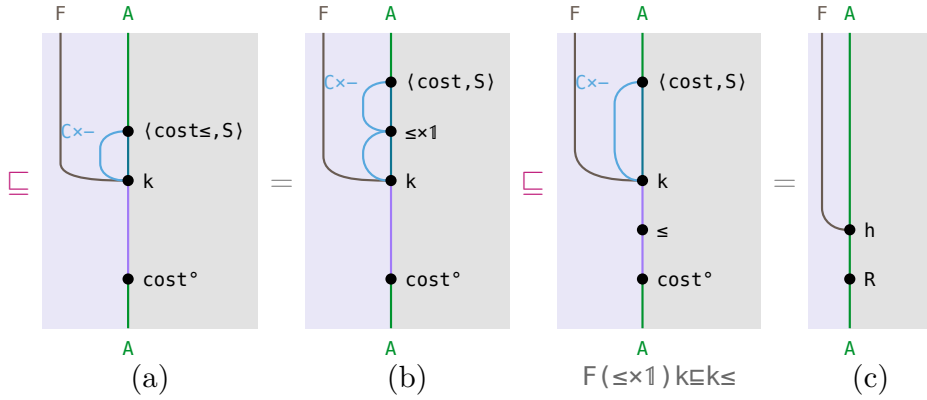
improving each part by R within its S -context, then assembling by h , is below hR

$$\text{Map}(\text{cost}) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \Rightarrow F(RnSS^\circ)h \sqsubseteq F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) F(\langle \text{cost}, S \rangle)k \text{cost}^\circ$$

improving the parts within their S -context and assembling by h is below the same improvement on the $(\text{cost}, \text{context})$ bundles, assembled by k



$$\text{Map}(\text{cost}) \wedge \text{Simple}(S) \wedge R = \text{cost} \leq \text{cost}^\circ \wedge \text{hcost} = F(\langle \text{cost}, S \rangle)k \wedge F(\leq \times 1)k \sqsubseteq k \leq \Rightarrow F(\langle \text{cost} \leq, S \rangle \langle \text{cost}, S \rangle^\circ) F(\langle \text{cost}, S \rangle)k \text{cost}^\circ \sqsubseteq hR$$

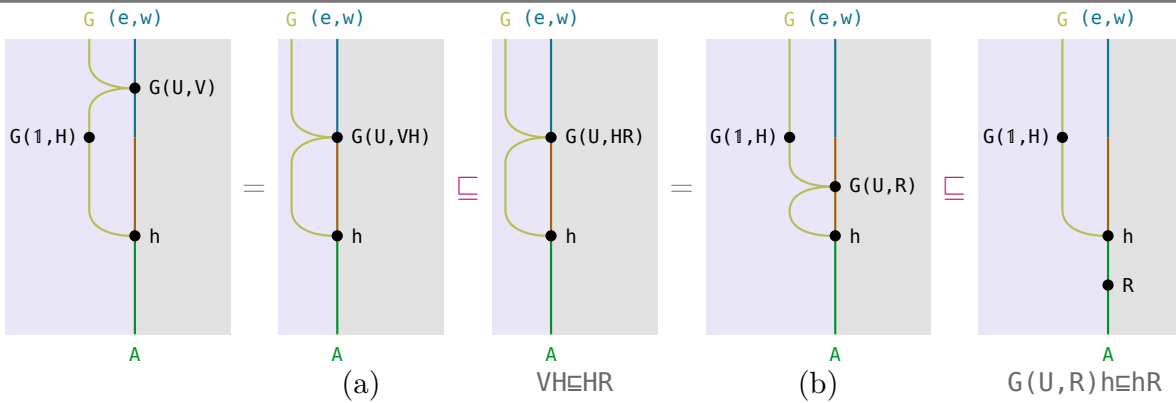
the improvement on the bundles, assembled by k , is below assembling by h and improving by R 

$$G(U, R)h \sqsubseteq hR \wedge Vh \sqsubseteq hR \Rightarrow G(U, V)G(1, H)h \sqsubseteq G(1, H)hR$$

Proposition 9.4

thinning the parts by U in the first argument and by V in the second $(G(U, V))$, then solving by H and assembling by h , gives only what solving and assembling and then improving by R gives

$(G(1, H)hR)$



§9.2. The string edit problem

(9.2a)

- $(\exists a. o = \text{cpy}(a)) \vee (\exists a. o = \text{del}(a)) \vee (\exists a. o = \text{ins}(a))$
an edit operation copies, deletes or inserts one character
- $\text{FX} = 1 + (0p \times X)$
 $F(0p, X)$ is the empty sequence or one operation paired with an X
- $\text{edit} = ([\text{base}, \text{step}])$
 edit is the fold of $[\text{base}, \text{step}]$ over an edit sequence, returning the two strings it reconstitutes
- $\text{length} = ([\text{zero}, \pi_2 \text{ succ}])$
the fold of $[\text{zero}, \pi_2 \text{ succ}]$ counts the operations of a sequence
- $R = \text{length} \leq \text{length}^\circ$
 R relates es to fs when es is no longer than fs
- $V = \text{suffix}^\circ \times \text{suffix}^\circ$
 V relates two pairs of strings when each string of the first is a suffix of the matching string of the second
- $Q(\text{inl}(u), \text{inl}(u))$
 Q is the identity on the **base** summand
- $Q(\text{inr}(p), \text{inr}(q)) \iff V(\pi_2(p), \pi_2(q))$
on the **step** summand Q compares any two operations ($U \triangleq T$) and the two pairs of strings by V
- $\text{empty}(p, q) \iff p = q \wedge p = ([], [])$
 empty is the coreflexive holding only at the pair of two empty strings
- $q \in \text{unstep}(p) \implies \text{step}(\text{inr}(q)) = p$
every decomposition unstep returns is sent back to its input by step

(9.2b)

x	$\alpha(x)$
$\text{inl}(u)$	$\text{wrap}(u)$
$\text{inr}((o, \text{es}))$	$[o] \# \text{es}$

(9.2c)

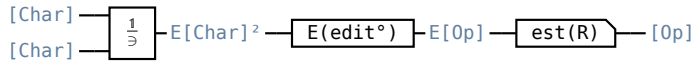
x	$[\text{base}, \text{step}](x)$
$\text{inl}(u)$	$([], [])$
$\text{inr}((\text{cpy}(a), (xs, ys)))$	$([a] \# xs, [a] \# ys)$
$\text{inr}((\text{del}(a), (xs, ys)))$	$([a] \# xs, ys)$
$\text{inr}((\text{ins}(a), (xs, ys)))$	$(xs, [a] \# ys)$

(9.2d)

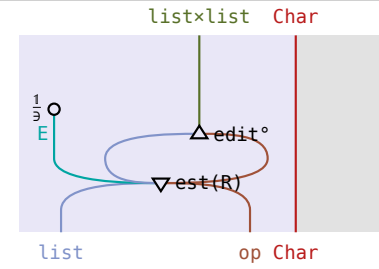
p	$\text{unstep}(p)$
$([], [])$	$[]$
$([a] \# xs, [])$	$[(\text{del}(a), (xs, []))]$
$([], [b] \# ys)$	$[(\text{ins}(b), ([], ys))]$
$([a] \# xs, [b] \# ys)$	if $a = b$ then $[(\text{cpy}, xs, ys)]$ else $[(\text{del}, xs, [b] \# ys), (\text{ins}, [a] \# xs, ys)]$

$$(\mu X : \frac{[base, step]^{\circ}}{\exists} \text{thin}(Q)P(F(X)[nil, cons])est(R)) \sqsubseteq \frac{edit^{\circ}}{\exists} est(R), \text{ mle} = (\text{empty} \rightarrow nil, \text{unstep} \text{list}((1 \times \text{mle}) \text{cons}) \text{minlist}(R)) \quad (9.2e)$$

a shortest edit sequence from which both strings can be reconstituted is one pass over the two of them, each step copying, deleting or inserting one character and the best sequence for what is left taken from the entries already computed

circuit

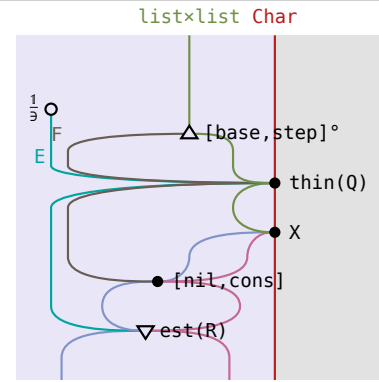
the specification — $\triangleq$ edit

Hinze–Marsden

list op Char



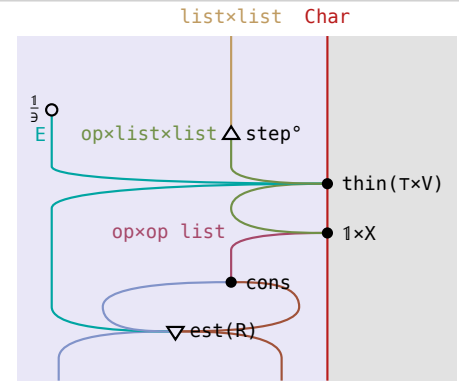
at $Q \triangleq 1 + (U \times V)$. Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is Proposition 9.2 at $\text{length} \triangleq ([zero, \pi_2 \text{succ}])$ with succ monotonic on $\leq$, so cons is monotonic on R . The thinning condition $\sqsubseteq QF(1, \text{edit}^{\circ}) \alpha \sqsubseteq F(1, \text{edit}^{\circ}) \alpha R$ over $F(0p, [Char] \times [Char])$ is Proposition 9.4 at $U \triangleq T - F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise, so any two operations may be compared — and $V \triangleq \text{suffix}^{\circ} \times \text{suffix}^{\circ}$, — $\text{edit}(1 \times \text{suffix}) \sqsubseteq R^{\circ} \text{edit}$, $\text{edit}(\text{suffix} \times 1) \sqsubseteq R^{\circ} \text{edit}$. $\text{edit.suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \Rightarrow BA^* \sqsubseteq C^*B$ cut those to one step each: drop the operation that produced the head, or weaken its cpy to a del , never lengthening the sequence



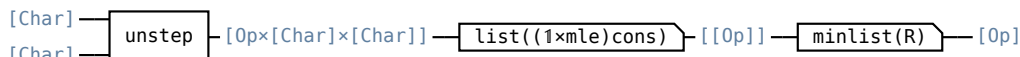
list op Char



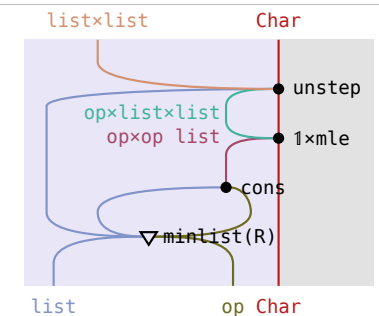
= Proposition 9.1: **base** and **step** have disjoint ranges, so the fixed point splits into one branch per summand — **empty**, the coreflexive on (xs, ys) with both lists empty, is where **base** returns



list op Char



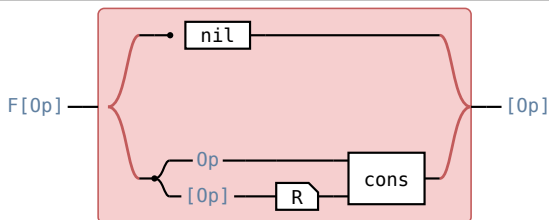
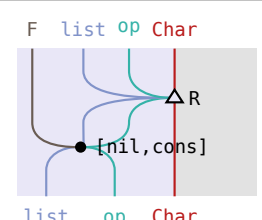
$\sqsubseteq$ **unstep** implements $\frac{step^{\circ}}{\exists} \text{thin}(U \times V)$ — at most two decompositions survive, a copy beating a delete or an insert wherever it is available — and **minlist**(**R**) implements **est**(**R**). The same subproblem is solved many times over, so the running time is exponential in the two lengths



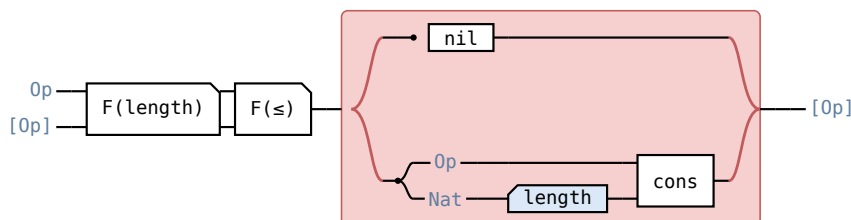
list op Char

$$F(R)[nil, cons] \sqsubseteq [nil, cons]R \quad (9.2f)$$

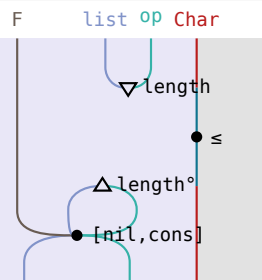
comparing the tails by **R** and then putting an operation in front ($F(R)\alpha$) relates only sequences that **R** also relates once the operation is in front (αR)

circuit**Hinze–Marsden**

list op Char



$R \triangleq \text{length} \leq \text{length}^{\circ} \triangleq \text{edit}$; F preserves composition

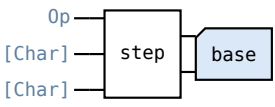
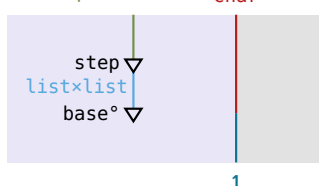
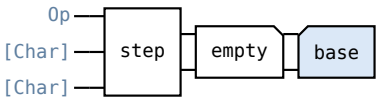
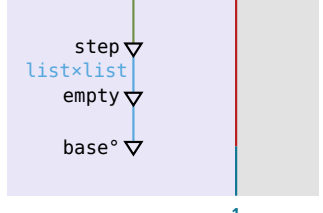
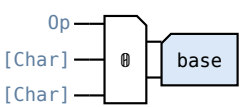
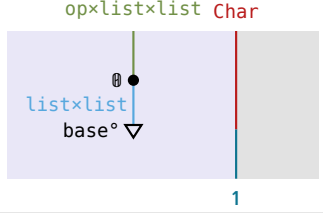
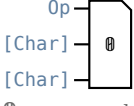
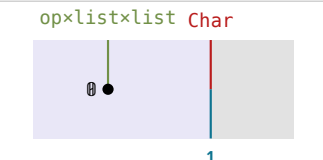


list op Char

circuit	Hinze–Marsden
length is a map, so entire: $1 \sqsubseteq \text{length} \text{ length}^\circ$	
$\alpha \text{ length} = F(\text{length})[\text{zero}, \pi_2 \text{ succ}] : \text{length} \triangle ([\text{zero}, \pi_2 \text{ succ}]) \text{ — edit}$	
length is a map, so simple, and F is monotonic: $F(\text{length}^\circ)F(\text{length}) \sqsubseteq F(1) = 1$	
succ is monotonic on $\leq$: $F(\leq)[\text{zero}, \pi_2 \text{ succ}] \sqsubseteq [\text{zero}, \pi_2 \text{ succ}] \leq$	
$F(\text{length})[\text{zero}, \pi_2 \text{ succ}] = \alpha \text{ length}$ again	
$R \triangle \text{length} \leq \text{length}^\circ \text{ — edit}$	

$QF(edit^\circ)[nil, cons] \sqsubseteq F(edit^\circ)[nil, cons]R$ (9.2g)	
every edit sequence built from a decomposition Q puts above a given one — any operation, each string lengthened at the front — is at least as long as one built from the given one	
circuit	Hinze–Marsden
$=$ $Q \triangleq 1 + (U \times V)$ is $F(U, V)$ at $U \triangleq T \triangleq \text{edit}$; and the bifunctor F preserves composition: $F(U, V)F(1, \text{edit}^\circ) = F(U, V \text{ edit}^\circ)$	
$\sqsubseteq$ Proposition 9.4's second condition $V \text{ edit}^\circ \sqsubseteq \text{edit}^\circ R$ — (9.2h)	
$=$ F preserves composition: $F(U, \text{edit}^\circ R) = F(1, \text{edit}^\circ)F(U, R)$	
$\sqsubseteq$ Proposition 9.4's first condition $F(T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: CONS adds one to both lengths whatever the two operations are	

$\text{edit } V^\circ \sqsubseteq R^\circ \text{ edit}$ (9.2h) cutting a front off either string an edit sequence produces ($\text{edit } V^\circ$) leaves a pair that a sequence no longer produces ($R^\circ \text{ edit}$)	
circuit	Hinze–Marsden
[0p] — edit — V — [Char]	list op Char ∇edit ∇V° list × list Char
[0p] — edit — suffix — [Char] $V \triangleq \text{suffix}^\circ \times \text{suffix}^\circ$, and $\times$ preserves composition	list op Char ∇edit $\nabla \text{suffix} \times 1$ $\nabla 1 \times \text{suffix}$ list × list Char
[0p] — R — edit — suffix — [Char] $\sqsubseteq \text{edit } (\text{suffix} \times 1) \sqsubseteq R^\circ \text{ edit}$: drop the operation that produced the head, or weaken its cpy to an ins. Proved by induction at suffix directly, not through $\text{suffix} = \text{tail}^*$ and $BA \sqsubseteq CB \implies BA^* \sqsubseteq C^*B$ as the book does	list op Char ∇edit $\nabla 1 \times \text{suffix}$ R° list × list Char
[0p] — R — R — edit — [Char] $\sqsubseteq \text{edit } (1 \times \text{suffix}) \sqsubseteq R^\circ \text{ edit}$, the mirror image: cpy weakens to a del	list op Char ∇edit R° R° list × list Char
[0p] — R — edit — [Char] R° is transitive, $\leq$ being so	list op Char ∇edit R° list × list Char

$\text{step } \text{base}^\circ = \emptyset$ (9.2i)	
no pair of strings is both a result of step and the result $([], [])$ of base	
circuit	Hinze–Marsden
	
 base returns only $([], [])$, so $\text{base}^\circ = \text{empty } \text{base}^\circ$	
 $\text{step } \text{empty} = \emptyset$: cpy and del put a character on the left string, ins one on the right	
 $\emptyset$ composed with anything is $\emptyset$	

(9.2j)

$\text{mle}(xs, ys) = \text{head}(\text{column}(xs, ys))$, $\text{column}(xs, ys) = [\text{mle}(u, ys) \mid u \leftarrow \text{tails}(xs)]$ $= \text{column}(xs) = ([\text{fstcol}(xs), \text{nextcol}(xs)])$, $\text{fstcol} = \text{list}(\text{del}) \text{ tails}$ the tabulation: $\text{mle}(xs, ys)$ needs $\text{mle}(u, v)$ for every tail u of xs and v of ys , so the columns are built right to left	
$\text{column}(xs, [b] \# ys) = \text{nextcol}(xs, (b, \text{column}(xs, ys)))$ $= \text{nextcol}(xs)(b, us) = ([\text{base}(b, \text{last}(us)), \text{step}(b)])(xus)$, $xus = \text{zip}(xs, \text{zip}(\text{init}(us), \text{tail}(us)))$ each column is a fold built bottom to top, over xs zipped with the adjacent pairs of the column to its right	
$\text{base}(b, u) = [[\text{ins}(b)] \# u]$ $= \text{step}(b)((a, (u, v)), ws) = (a \rightarrow b \rightarrow [[\text{cpy}(a)] \# v] \# ws, [\text{bmin}(R)([\text{del}(a)] \# w, [\text{ins}(b)] \# u)] \# ws)$ $w = \text{head}(ws)$; these base , step are not edit 's. An entry depends on the one below it (a delete), the one to its right (an insert), and the one below that (a copy) — quadratic in the two lengths	

§9.3. Optimal bracketing

(9.3a)

tree $A ::= \text{tip } A \mid \text{bin } (\text{tree } A, \text{tree } A)$, $FX = A + X^2$, so $F(R) = 1 + R^2$; $h \triangleq [\text{tip}, \text{bin}]$, $\text{flatten} \triangleq ([\text{wrap}, \text{cat}]) : \text{tree } A \rightarrow \text{list}^+ A$, $H = \text{flatten}^\circ$.

$\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, $\text{opt} \triangleq (\text{zero}, \text{st})$, $\text{opb } ((cx, sx), (cy, sy)) = (\text{cb } (sx, sy) + cx + cy, \text{sb } (sx, sy))$.

sb associative, so $\text{size} = \text{flatten } \text{sz}$ for a map sz $R \triangleq \text{cost} \leq \text{cost}^\circ$, $g \triangleq [\text{zero}, (1 \times \text{sz})^2 \text{ opb } \pi_1]$, single the coreflexive on singleton lists.

$\text{splits} \triangleq (\text{inits}^+, \text{tails}^+)$ zip , an implementation of $\frac{\text{cat}^\circ}{\text{zip}}$; $\text{array} \triangleq \text{inits } \text{list}(\text{row})$, $\text{row} \triangleq \text{tails } \text{list}(\text{mct})$, $\text{col} \triangleq \text{inits } \text{list}(\text{mct})$.

$\text{mix} \triangleq \text{zip } \text{list}(\text{bin}) \text{ minlist}(R)$, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}$, $\text{process} \triangleq ((\text{tip } \text{wrap}) \times 1) \text{ loop}(\text{next})$.

$\text{Assoc}(\text{sb}) \Rightarrow F(\langle \text{cost}, \text{flatten} \rangle)g = [\text{tip}, \text{bin}] \text{cost}$ (9.5)	
pairing each subtree with its cost and its flattening and then applying g gives what building the tree by $[\text{tip}, \text{bin}]$ and taking its cost gives	
circuit	Hinze–Marsden
$F(\text{tree}(A)) \rightarrow \boxed{F(\langle \text{cost}, \text{flatten} \rangle)} \rightarrow F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \text{zero} \\ \text{tree}(A) \rightarrow \text{cost} \\ \text{tree}(A) \rightarrow \text{flatten} \rightarrow \text{list}^+(A) \rightarrow \text{sz} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ definition of g; coproducts and products — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \text{zero} \\ \text{tree}(A) \rightarrow \text{cost} \\ \text{tree}(A) \rightarrow \text{size} \\ \text{tree}(A) \rightarrow \text{cost} \\ \text{tree}(A) \rightarrow \text{size} \end{array} \right\} \rightarrow \text{opb} \rightarrow \text{Int}$ $\text{flatten sz} = \text{size}$, since sb is associative — $\triangleq \text{flatten}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \text{zero} \\ \text{tree}(A) \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{cost} \\ \text{tree}(A) \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{size} \end{array} \right\} \rightarrow \text{Int}$ $\langle \text{cost}, \text{size} \rangle \triangleq ([\text{opt}, \text{opb}])$, at a node $\text{bin}(\text{cost}, \text{size}) = \langle \text{cost}, \text{size} \rangle^2 \text{ opb}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \text{tip} \rightarrow \text{tree}(A) \rightarrow \text{cost} \\ \text{tree}(A) \rightarrow \text{bin} \rightarrow \text{tree}(A) \rightarrow \text{cost} \end{array} \right\} \rightarrow \text{Int}$ $\text{tip cost} = \text{zero}$; products, $\langle \text{cost}, \text{size} \rangle \pi_1 = \text{cost}$	
$A + \text{tree}(A)^2 \rightarrow \left\{ \begin{array}{l} A \rightarrow \text{tip} \\ \text{tree}(A) \rightarrow \text{bin} \end{array} \right\} \rightarrow \text{tree}(A) \rightarrow \text{cost} \rightarrow \text{Int}$ coproducts, $[\text{tip cost}, \text{bin cost}] = [\text{tip}, \text{bin}] \text{cost}$	

$F(\leq \times 1) g \sqsubseteq g \leq$ (9.6)	
raising the cost of either subtree ($\leq \times 1$ in both slots) and then applying g returns a value at least what g returns on the costs before raising ($g \leq$)	
circuit	Hinze–Marsden
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{F(\leq \times 1)} \rightarrow \boxed{g} \rightarrow \text{Int}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ definition of opb , and $+$ monotonic	
$A + \text{Int} \times \text{list}^+(A) \times \text{Int} \times \text{list}^+(A) \rightarrow \text{Int}$ $\sqsubseteq$ $\leq$ reflexive, so $\text{zero} \sqsubseteq \text{zero} \leq$; coproducts	
$F(\text{Int} \times \text{list}^+(A)) \rightarrow \boxed{g} \rightarrow \boxed{\leq} \rightarrow \text{Int}$ $=$ definition of $g \triangleq \text{flatten}$	

$$\text{Assoc}(\text{sb}) \Rightarrow (\mu X : \frac{[\text{wrap}, \text{cat}]^\circ}{3} P([\text{tip}, (X \times X) \text{bin}]) \text{est}(R)) \sqsubseteq \frac{\text{flatten}^\circ}{3} \text{est}(R), \text{ mct} = (\text{single} \rightarrow \text{head tip}, \text{ (init col, tail row) mix}) \quad (9.3b)$$

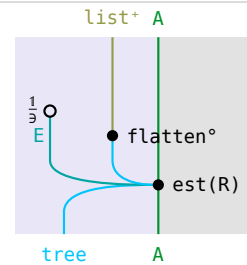
a least-cost bracketing of $a_1 \oplus \dots \oplus a_n$ is read off an array holding one best tree per non-empty segment, each entry built from the column to its left and the row below it

circuit

$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E(\text{flatten}^\circ) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

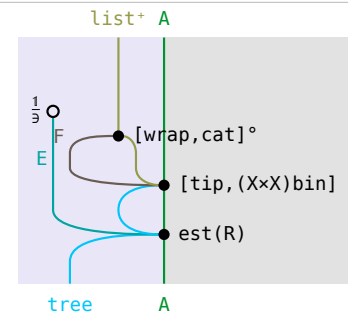
the specification — $\triangleq$ flatten

Hinze–Marsden



$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E([\text{wrap}, \text{cat}]^\circ) \rightarrow E(\text{list}^+(A)) \rightarrow P([\text{tip}, (X \times X) \text{bin}]) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

split the list in every way, bracket both halves, join. The condition is monotonicity **in context**,
 $\sqsubseteq F(Rn(\text{flatten} \text{ flatten}^\circ)) h \sqsubseteq h R$ — only trees with the same flattening are compared — which is Proposition 9.3 at $H^\circ = \text{flatten}$, a map, with (9.5) $[\text{tip}, \text{bin}] \text{ cost} = (1 + (\text{cost}, \text{flatten})^2) g$ (the cost of a node reads only the cost and the flattening of its two subtrees) and (9.6) $(1 + (\leq \times 1)^2) g \sqsubseteq g \leq (g \text{ monotonic on } \leq \text{ in its two cost arguments})$

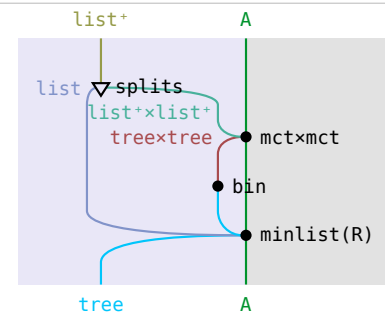


$$\text{list}^+(A) \rightarrow \frac{1}{3} \rightarrow E(\text{list}^+(A)) \rightarrow E(\text{cat}^\circ) \rightarrow E(\text{list}^+(A)^2) \rightarrow P((X \times X) \text{bin}) \rightarrow E(\text{tree}(A)) \rightarrow \text{est}(R) \rightarrow \text{tree}(A)$$

= Proposition 9.1: **wrap** and **cat** have disjoint ranges, and **single** is the coreflexive on singleton lists, where **wrap** returns

$$\text{list}^+(A) \rightarrow \text{splits} \rightarrow [\text{list}^+(A)^2] \rightarrow \text{list}((\text{mct} \times \text{mct}) \text{bin}) \rightarrow [\text{tree}(A)] \rightarrow \text{minlist}(R) \rightarrow \text{tree}(A)$$

$\sqsubseteq$ splits $\triangleq$ (inits⁺, tails⁺) zip implements $\frac{\text{cat}^\circ}{3}$ and minlist(R) implements est(R). Exponential, since the segments of one list overlap



$$\text{mct} = (\text{single} \rightarrow \text{head tip}, (\text{init col}, \text{tail row}) \text{ mix}) \quad (9.7)$$

= the tabulation: **mct** $\times$ S is needed for every non-empty segment XS , so the values are held as an array of rows, $\text{array} \triangleq \text{inits list}(\text{row}), \text{row} \triangleq \text{tails list}(\text{mct}), \text{col} \triangleq \text{inits list}(\text{mct}), \text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R)$

$$\text{col} = (\text{single} \rightarrow \text{head tip wrap}, (\text{init col}, \text{tail row}) \text{ next}) \quad (9.8)$$

$$\text{cons col} = (1 \times \text{array}) \text{ process} \quad (9.9)$$

$$\text{row} = (\text{single} \rightarrow \text{head tip wrap}, (\text{mct}, \text{tail row}) \text{ cons}) \quad (9.10)$$

a column extends the column to its left, a row the row below it; (9.9) is (9.8) rewritten as a loop, $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc}, \text{process} \triangleq ((\text{tip wrap}) \times 1) \text{ loop}(\text{next})$

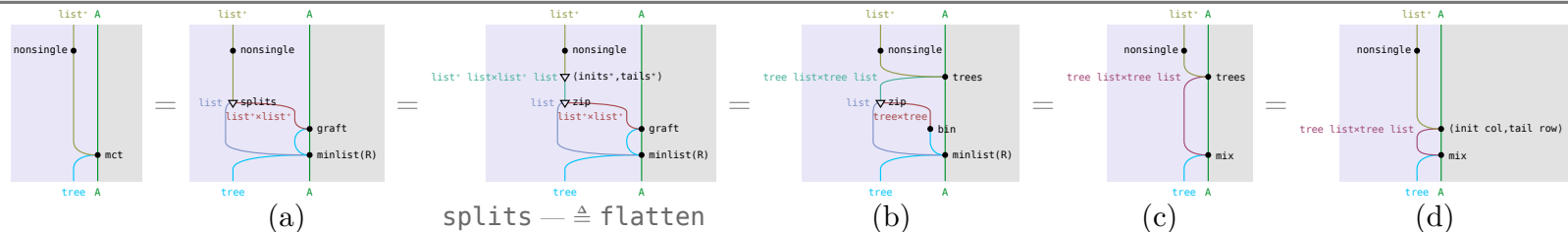
$$\text{array} = ([\text{fstcol}, \text{addcol}]), \text{fstcol} \triangleq \text{tip wrap wrap}$$

$$\text{addcol} \triangleq (\pi_1 \text{ tip wrap}, \text{step}) \text{ cons}, \text{step} \triangleq (\text{process tail}, \pi_2) \text{ zip list}(\text{cons})$$

the program: one fold building the array column by column, cubic in the length of the input

$$\text{nonsingle mct} = \text{nonsingle}(\text{init col}, \text{tail row}) \text{ mix} \quad (9.7)$$

on a list of two or more elements, **mct** gives what **mix** gives on the column of its **init** and the row of its **tail**



- (a) $\text{mct} \triangleq \text{flatten}$
 (b) $\text{graft}, \text{trees}; \text{zip list}(f \times g) = (\text{list}(f) \times \text{list}(g)) \text{ zip}$
 (c) $\text{mix} \triangleq \text{zip list}(\text{bin}) \text{ minlist}(R) \triangleq \text{flatten}$
 (d) $\text{inits}^+ = \text{init inits}, \text{tails}^+ = \text{tail tails}; \text{row}, \text{col} \triangleq \text{flatten}$

(9.8)

nonsingle col=nonsingle(init col,tail row)next

on a list of two or more elements, the column is what **next** makes of the column of its **init** and the row of its **tail**

(a) $\text{col} \triangleq \text{flatten}$
 (b) $\text{inits} = (\text{init inits}, 1)$ snoc on non-singletons
 (c) $\text{snoc list}(f) = (\text{list}(f) \times f) \text{ snoc}; \text{col}$
 (d) (9.7) — (9.7); $\text{next} \triangleq (\pi_1, \text{mix}) \text{ snoc} \triangleq \text{flatten}$

(9.10)

nonsingle row=nonsingle(mct,tail row)cons

on a list of two or more elements, the row is **mct** of the whole list consed onto the row of its **tail**

(a) $\text{row} \triangleq \text{flatten}$
 (b) $\text{tails} = (1, \text{tail tails})$ cons on non-singletons
 (c) $\text{cons list}(f) = (f \times \text{list}(f)) \text{ cons}; \text{row}$

(9.9)

cons col=(1×array)process

the column of **a** consed onto **x** is what **process** makes of **a** and the array of **x**

circuit	Hinze–Marsden
Exercise 9.13 with $k \triangleq \text{cons col}$, $g \triangleq \text{tip wrap}$, $h \triangleq \text{row}$, $f \triangleq \text{next}$: its two equations are (9.8) — (9.8) — at $a: [b]$ and at $a: (u++[b])$, with $[a] \text{ col} = [\text{tip}(a)]$	
products	
definition of process	

tops=(1×array)process tail (9.3c)	
the trees at the top of the new rows are what process makes of a and the array of x , less its first	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{tops}} \rightarrow [\text{tree}(A)]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list(mct)}} \boxed{\text{list(mct)}} \rightarrow [\text{tree}(A)]$ definition of tops$\triangleq$tic list(mct), our name	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{col}} \boxed{\text{tail}} \rightarrow [\text{tree}(A)]$ tail list(f)=list(f) tail; definition of col	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{array}} \rightarrow \boxed{\text{process}} \rightarrow \boxed{\text{tail}} \rightarrow [\text{tree}(A)]$ (9.9) — (9.9)	

rests=π₂ array (9.3d)	
the rest of each new row is the array of x	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{rests}} \rightarrow [[\text{tree}(A)]]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{init}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list(tail row)}} \boxed{\text{list(tail row)}} \rightarrow [[\text{tree}(A)]]$ definition of rests$\triangleq$tic list(tail row), our name	
$= \text{list}^+(A) \xrightarrow{A} \text{list}^+(A) \xrightarrow{\text{array}} \boxed{\text{array}} \rightarrow [[\text{tree}(A)]]$ tic list(tail)=π₂ init; definition of array	

newrows=(1×array)step (9.3e) the rows of the array of a:x below its first are what step makes of a and the array of x	
circuit	Hinze–Marsden
$\text{list}^+(A) \xrightarrow{A} \boxed{\text{newrows}} \rightarrow [[\text{tree}(A)]]$	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{inits}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [[\text{tree}(A)]]$ definition of $\text{newrows} \triangleq \text{tic } \text{list}(\text{row})$, our name	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{cons}} \rightarrow \text{list}^+(A) \xrightarrow{\text{inits}} \boxed{\text{tail}} \rightarrow [\text{list}^+(A)] \xrightarrow{\text{list}(\text{mct}, \text{tail row}) \text{cons}} [[\text{tree}(A)]]$ (9.10) — (9.10) — on the non-singleton tic lists	
$= \text{list}^+(A) \xrightarrow{A} \begin{cases} \boxed{\text{tops}} \\ \boxed{\text{rests}} \end{cases} \rightarrow \begin{cases} [\text{tree}(A)] \\ [[\text{tree}(A)]] \end{cases} \xrightarrow{\text{zip}} [\text{tree}(A) \times [\text{tree}(A)]] \xrightarrow{\text{list}(\text{cons})} [[\text{tree}(A)]]$ $\text{list}(f,g) = (\text{list}(f), \text{list}(g))$ zip; products; definitions of tops and rests	
$= \text{list}^+(A) \xrightarrow{A} \boxed{\text{array}} \xrightarrow{\text{step}} [[\text{tree}(A)]]$ (9.3c) and (9.3d); products; definition of step	

cons array=(1×array)addcol the array of a consed onto x is what addcol makes of a and the array of x (9.3f)	
circuit	Hinze–Marsden
$= \text{list}^+(A) \xrightarrow{\text{cons}} \text{list}^+(A) \xrightarrow{\text{inits}} [\text{list}^+(A)] \xrightarrow{\text{list}(\text{row})} [\text{tree}(A)]$ definition of array	
$= \text{cons } \text{inits} = (\pi_1 \text{ wrap, tic}) \text{ cons, abbreviating cons inits tail by tic}$	
$= \text{list}^+(A) \xrightarrow{\text{wrap}} \text{list}^+(A) \xrightarrow{\text{row}} [\text{tree}(A)] \xrightarrow{\text{cons}} [\text{tree}(A)]$ newrows $\text{cons } \text{list}(f) = (f \times \text{list}(f)) \text{ cons}$; definition of newrows	
$= \text{list}^+(A) \xrightarrow{\text{tip}} \text{tree}(A) \xrightarrow{\text{wrap}} [\text{tree}(A)] \xrightarrow{\text{cons}} [\text{tree}(A)]$ wrap row=tip wrap	
$= \text{list}^+(A) \xrightarrow{\text{array}} \text{addcol} \rightarrow [\text{tree}(A)]$ (9.3e); products; definition of addcol	

§9.4. Data compression

(9.4a)

$[A] ::= \text{nil} \mid \text{snoc } ([A], A), \text{ list}^+ A ::= \text{wrap } A \mid \text{snoc } (\text{list}^+ A, A), \text{ String} = [\text{Char}], \text{ Code} ::= \text{sym Char} \mid \text{ptr} (\text{String}, \text{String}^+), F(\text{Code}, \text{String}) = 1 + (\text{String} \times \text{Code}), \alpha \triangleq [\text{nil}, \text{snoc}].$

$\text{decode} \triangleq ([\text{nil}, \text{extend}]) : [\text{Code}] \rightarrow \text{String}, \text{ extend } (xs, \text{sym } a) = xs \# [a], \text{ extend } (xs, \text{ptr } (ys, zs)) = xs \# zs \text{ when } ys \# zs \text{ is a proper prefix of } xs \# zs; H = \text{decode}^\circ.$

$\text{size} \triangleq ([\text{zero}, \text{distr } [1 \times c, 1 \times p] \text{ plus}])$ with c, p the constant costs of a symbol and a pointer; $R \triangleq \text{size} \leq \text{size}^\circ, Q \triangleq F(\tau + \tau, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (\tau + \tau))$, the two τ on symbols and on pointers.

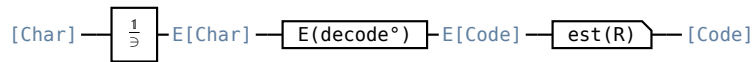
$\text{lrt } ws = \text{est}(\text{prefix}^\circ \times (\tau + \tau)) \{(xs, (ys, zs)) \mid xs \# zs = ws, ys \# zs \text{ a proper prefix of } ws\}$, the longest repeated tail; $\text{reduce } (ws \# [a]) = (zs \# [] \rightarrow [(ws, \text{sym } a), (xs, \text{ptr } (ys, zs))], [(ws, \text{sym } a)])$ with $(xs, (ys, zs)) = \text{lrt } (ws \# [a]).$

$$\theta \leq c \wedge \theta \leq p \Rightarrow (\mu X : \frac{[nil, extend]^\circ}{\exists} \text{thin}(Q)P([nil, (X \times 1)snoc])est(R)) \sqsubseteq \frac{decode^\circ}{\exists} est(R), \quad (9.4b)$$

$$\text{encode} = (\text{null} \rightarrow \text{nil}, \text{reduce list}((\text{encode} \times 1)snoc) \text{minlist}(R))$$

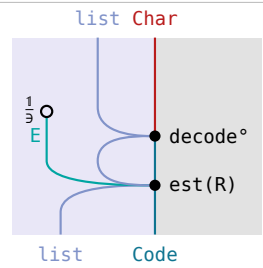
a smallest code sequence decoding to the given string is built from the right, each step emitting the last character as a symbol or ending with a pointer back into what has already been decoded

circuit



the specification $\dashv\equiv$ decode

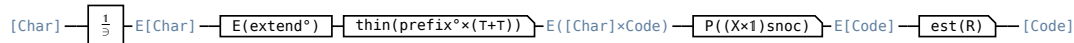
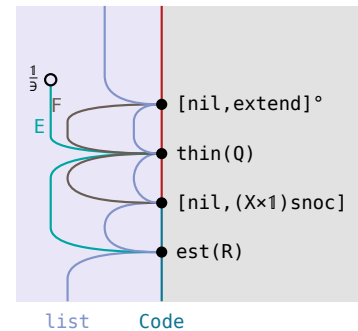
Hinze–Marsden



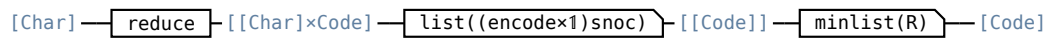
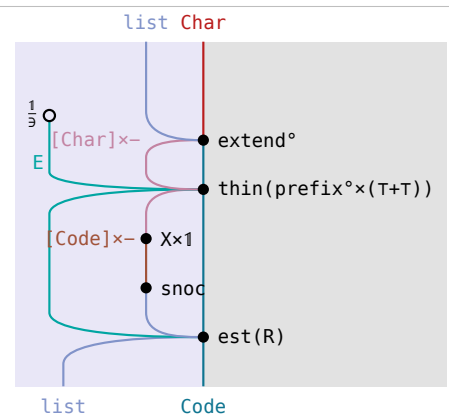
at $Q \triangleq F(T+T, \text{prefix}^\circ) = 1 + (\text{prefix}^\circ \times (T+T))$, the two T on symbols and on pointers.

Monotonicity $F(R) \alpha \sqsubseteq \alpha R$ is routine, the two costs being constants: $(T+T)[c, p] = [c, p]$.

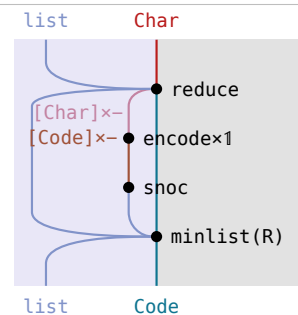
Proposition 9.4 splits the thinning condition in two — $F(T+T, R) \alpha \sqsubseteq \alpha R$, and $\text{decode prefix} \sqsubseteq R^\circ \text{ decode}$, for which $\text{decode init} \sqsubseteq R^\circ \text{ decode}$ is enough: dropping the last character of the output shortens or removes the last code element and never raises the cost. So between a symbol and a pointer nothing can be decided in advance, and between two pointers the longer match wins



Proposition 9.1: nil and extend have disjoint ranges. The decompositions of one string are $\frac{\text{extend}^\circ}{\exists} (ws \# [a]) = \{(ws, \text{sym } a)\} \cup \{(xs, \text{ptr } (ys, zs)) \mid xs \# zs = ws \# [a], ys \# zs \text{ a proper prefix of } ws\}$ — take the last character as a symbol, or end with a pointer



reduce implements $\frac{\text{extend}^\circ}{\exists} \text{thin}(\text{prefix}^\circ \times (T+T))$: thinning leaves at most two, the symbol and the pointer of the longest repeated tail lrt . Again exponential; the book gives no tabulation for it

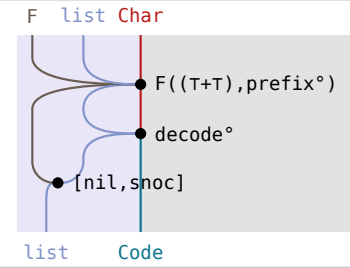
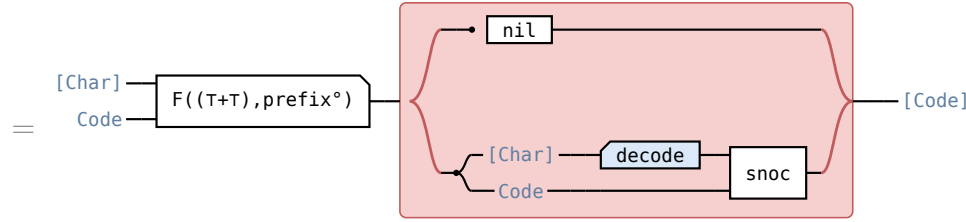
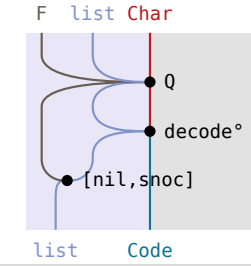
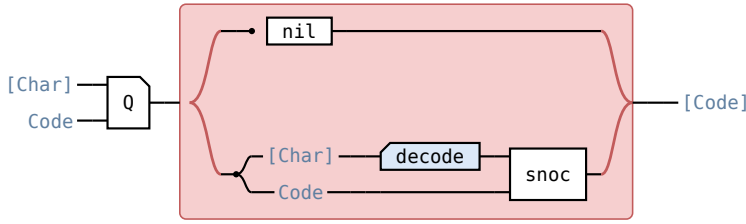


$$\theta \leq c \wedge \theta \leq p \Rightarrow QF(\text{decode}^\circ)[\text{nil}, \text{snoc}] \sqsubseteq F(\text{decode}^\circ)[\text{nil}, \text{snoc}]R \quad (9.4c)$$

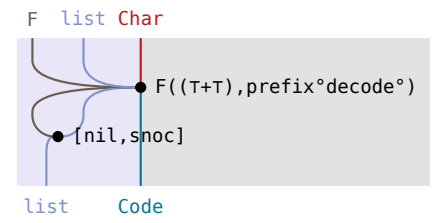
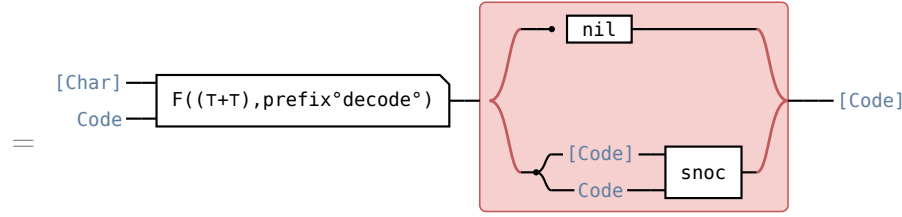
every code sequence built from a decomposition Q puts above a given one — a code element of the same kind, a longer front string — costs at least as much as one built from the given one

circuit

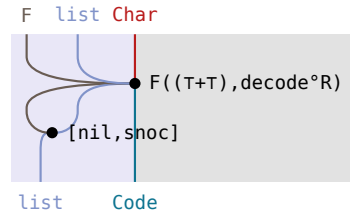
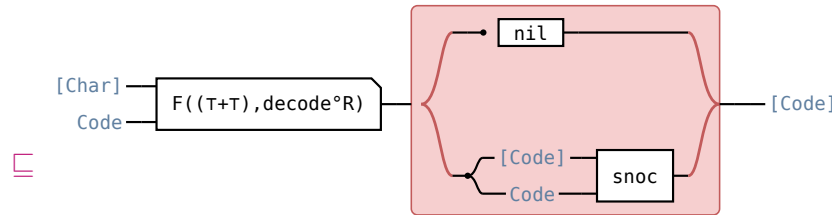
Hinze–Marsden



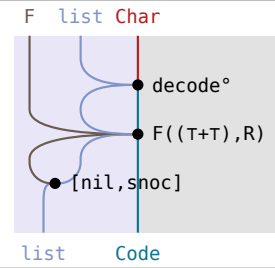
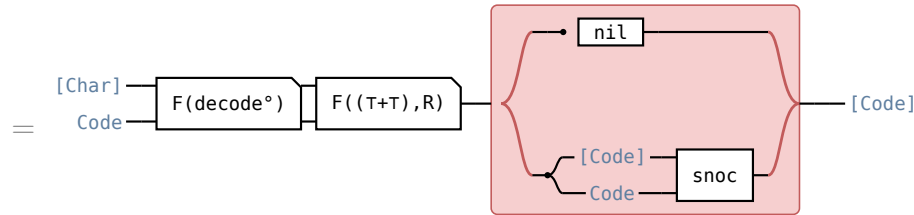
$Q \triangleq 1 + (\text{prefix}^\circ \times (T+T))$ is $F(T+T, \text{prefix}^\circ) \triangleq \text{decode}$



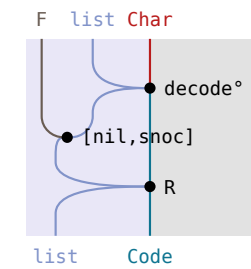
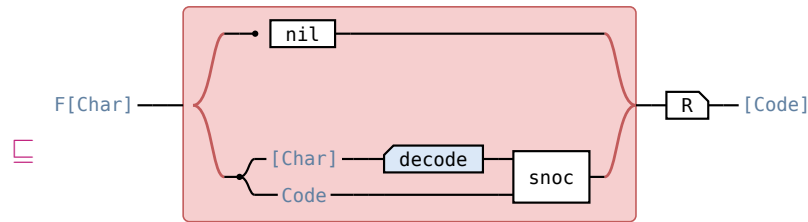
the bifunctor F preserves composition: $F(U, \text{prefix}^\circ)F(1, \text{decode}^\circ) = F(U, \text{prefix}^\circ \text{decode}^\circ)$



Proposition 9.4's second condition $\text{prefix}^\circ \text{decode}^\circ \sqsubseteq \text{decode}^\circ R$: dropping the last **sym**, or shortens or drops the last pointer. Proved at **prefix** directly by induction, not through **init**



F preserves composition: $F(U, \text{decode}^\circ R) = F(1, \text{decode}^\circ)F(U, R)$



Proposition 9.4's first condition $F(T+T, R) \alpha \sqsubseteq \alpha R$, left as an exercise in the book: **snoc** adds the cost of the last element to both sides, and $[c, p](T+T) = [c, p]$

extend nil^o=⊘ (9.4d) no string is both a result of extend and the result [] of nil	
circuit	Hinze–Marsden
nil returns only [], so $\text{nil}^o = \text{null } \text{nil}^o$	
extend null=⊘: a symbol ends the string with a character, a pointer with its non-empty ZS	
⊘ composed with anything is ⊘	

§10. Greedy Algorithms

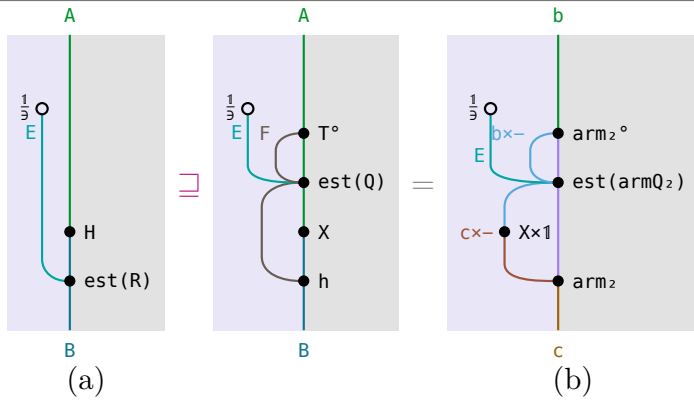
§10.1. Theory

(10.1a)

h, T, R, H, M as in $\triangleq H$; additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\exists}$ returns, so that $\frac{T^\circ}{\exists} \text{est}(Q)$ is entire.

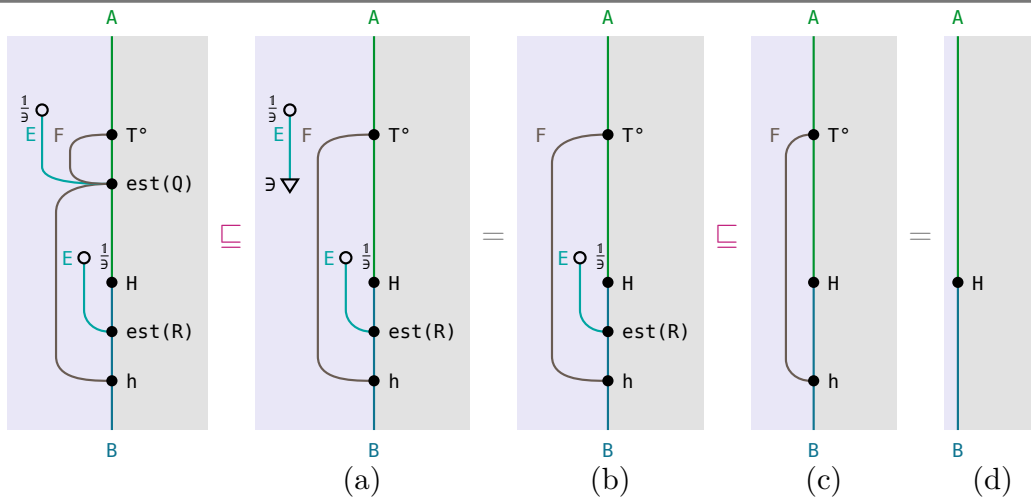
Map(h) $\wedge$ MonoAlg(h,R) $\wedge$ RRE R $\wedge$ ThinCondition(T,h,R,Q) $\Rightarrow$ ($\mu X : \frac{T^\circ}{\exists} \text{est}(Q)F(X)h \sqsubseteq \frac{H}{\exists} \text{est}(R)$) Theorem 10.1

the same optimum reached by keeping ONE decomposition at each step, so that no set is ever carried
and the recursion runs on values alone



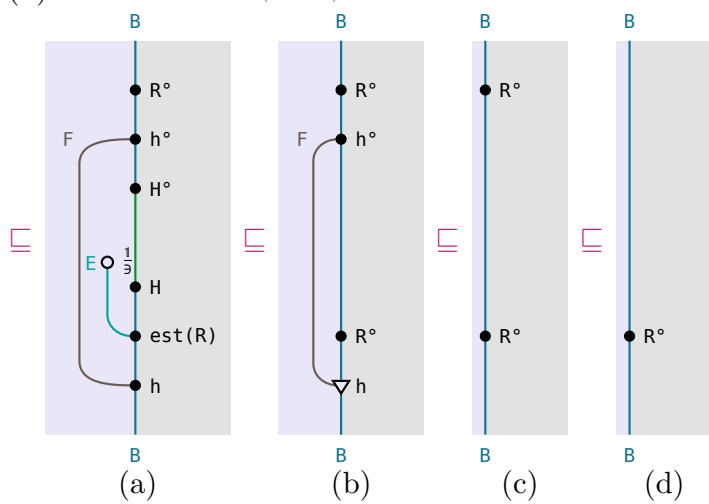
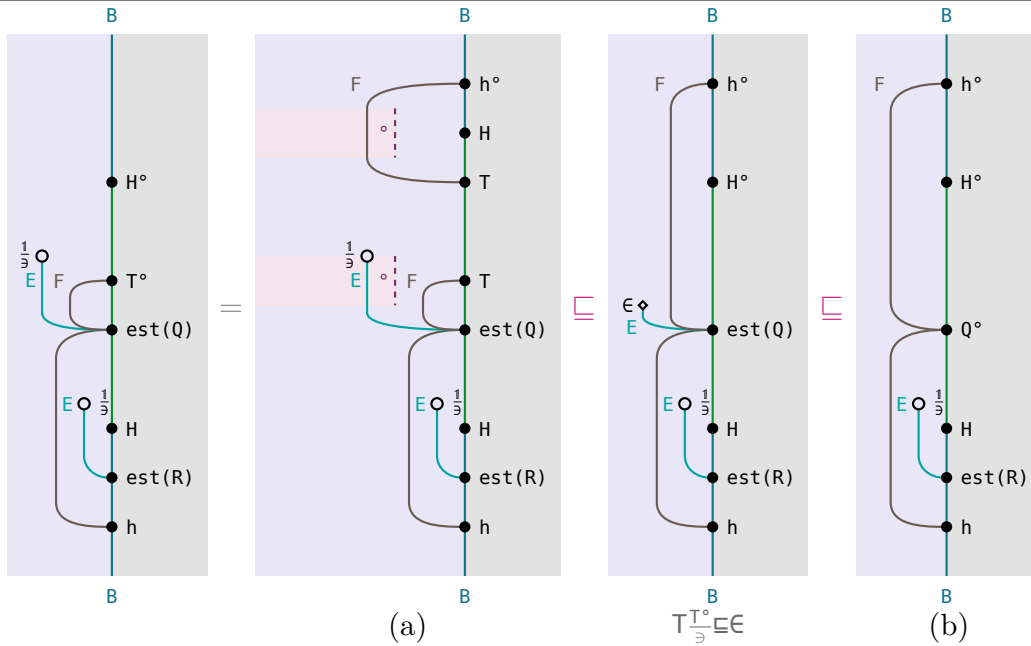
$T^\circ F(H)h=H \Rightarrow \frac{T^\circ}{\exists} \text{est}(Q)F(\frac{H}{\exists} \text{est}(R))h \sqsubseteq H$ (10.1b)

taking the input apart in one Q -extreme way, solving the parts by M and assembling by h returns only what H returns



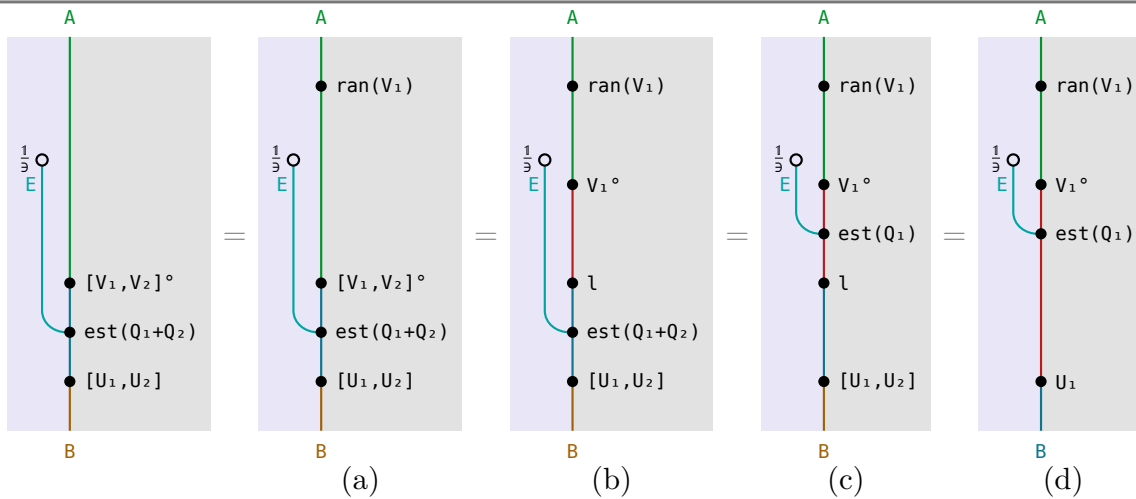
$$\text{Map}(h) \wedge \text{MonoAlg}(h, R^\circ) \wedge R^\circ R^\circ \sqsubseteq R^\circ \wedge T^\circ F(H)h = H \wedge QF(H)h \sqsubseteq F(H)hR \Rightarrow H^\circ \frac{T^\circ}{\exists} \text{est}(Q)F(\frac{H}{\exists} \text{est}(R))h \sqsubseteq R^\circ \quad (10.1c)$$

H° followed by the greedy body at M is $\sqsubseteq R^\circ$: an answer of the body is never worse than an answer of H to the same input



$$V_2 \quad V_1^\circ = \emptyset \Rightarrow \frac{[V_1, V_2]^\circ}{\exists} \text{est}(Q_1 + Q_2)[U_1, U_2] = \text{ran}(V_1) \frac{V_1^\circ}{\exists} \text{est}(Q_1)U_1 \cup \text{ran}(V_2) \frac{V_2^\circ}{\exists} \text{est}(Q_2)U_2 \quad \text{Proposition 10.1}$$

when V_1 and V_2 have disjoint ranges, the greedy step over $[V_1, V_2]$ runs the V_1 step on inputs V_1 reaches and the V_2 step on inputs V_2 reaches



§10.2. The detab-entab problem

(10.2a)

$\text{detab} \triangleq ([\text{nil}, \text{expand}]) : \text{String} \rightarrow \text{String}$ over snoc-lists, $\alpha \triangleq [\text{nil}, \text{snoc}]$, $H = \text{detab}^\circ$; $\text{expand } (xs, a) = (a = \text{TB} \rightarrow \text{fill } xs, xs \# [a])$, $\text{fill } xs = xs \# \text{blanks } (n - (\text{col } xs) \bmod n)$.

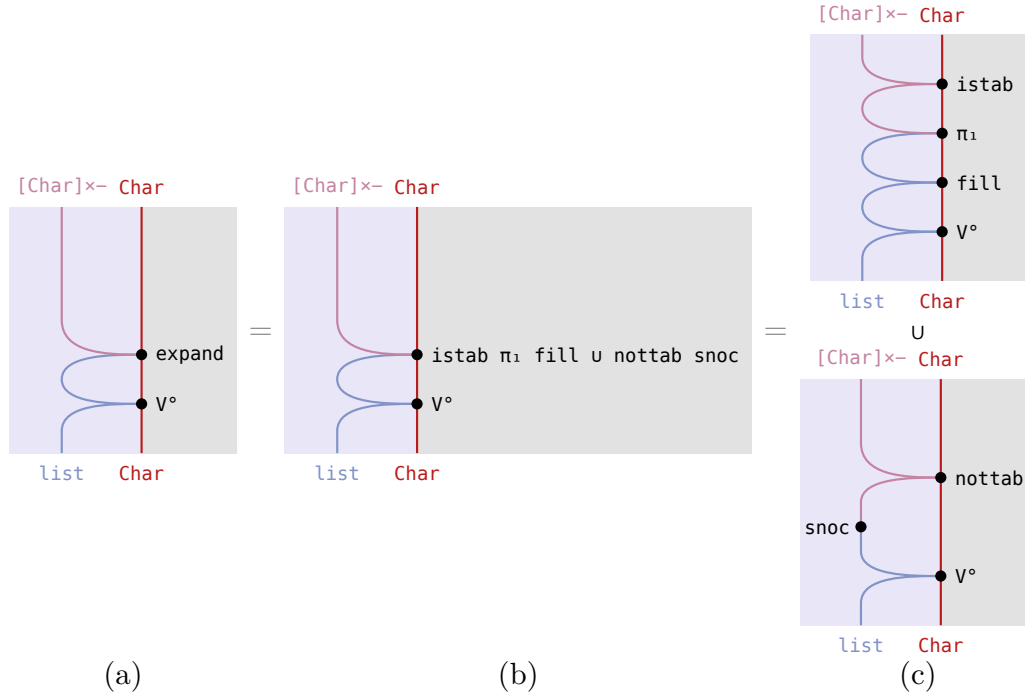
$\text{col} \triangleq ([\text{zero}, \text{count}])$, $\text{count } (c, a) = (a = \text{NL} \rightarrow 0, c + 1)$; TB the tab, BL the blank, NL the newline, tab stops every n columns.

$R \triangleq \text{length} \leq \text{length}^\circ$, U the preorder with $a \cup b \iff a = \text{TB} \vee a = b$, $V \triangleq \text{prefix}^\circ n (\text{fill } \text{fill}^\circ)$, $Q \triangleq 1 + (V \times U)$.

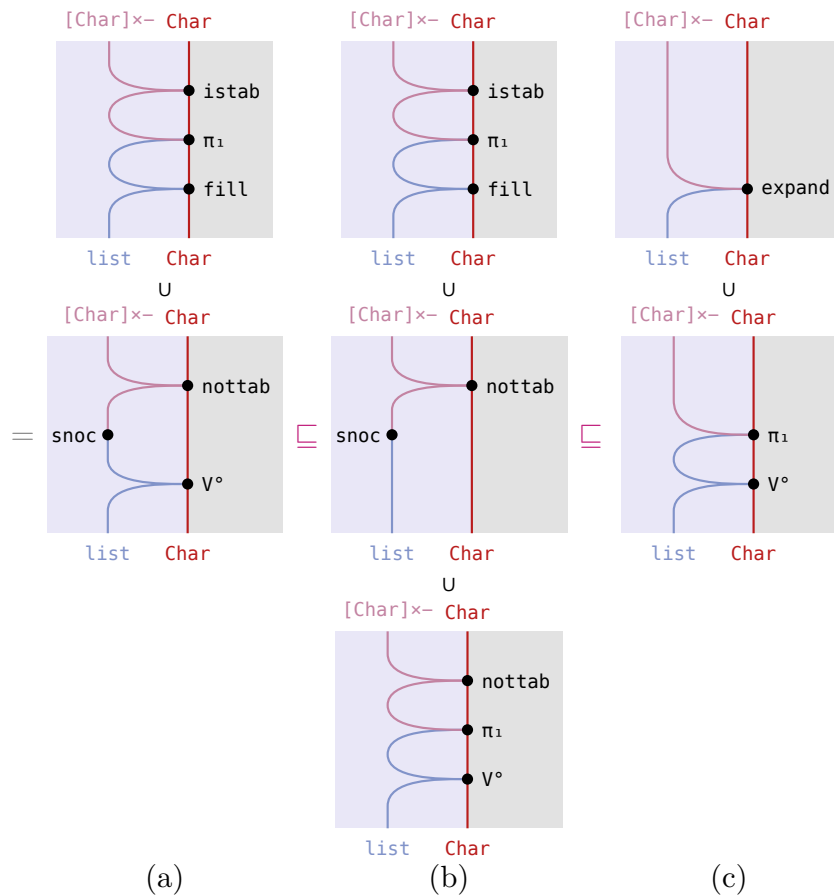
$\text{unfill } xs$ the shortest prefix of xs with fill ($\text{unfill } xs = \text{fill } xs$); tbc the trailing blank count, $\text{triple} \triangleq (\text{unfill } \text{entab}, (\text{tbc}, \text{col}))$.

$$0 < n \wedge \text{blank} \neq \text{nl} \implies \text{expand } V^\circ \sqsubseteq \text{expand} \cup \pi_1 V^\circ \quad (10.2b)$$

shortening the output of one **expand** step to a V -smaller string either keeps the whole step (**expand**) or drops its character and shortens its input string ($\pi_1 V^\circ$)



(a)	$\text{expand } V^\circ$	
(b)	$(\text{istab } \pi_1 \text{ fill } \cup \text{nottab snoc}) V^\circ$	$\text{expand} = \text{istab outl fillunottab snocR}$ — definition of expand
(c)	$\text{istab } \pi_1 \text{ fill } V^\circ \cup \text{nottab snoc } V^\circ$	$(\text{SuT})R = \text{SRuTR}$ — conditionals distribute



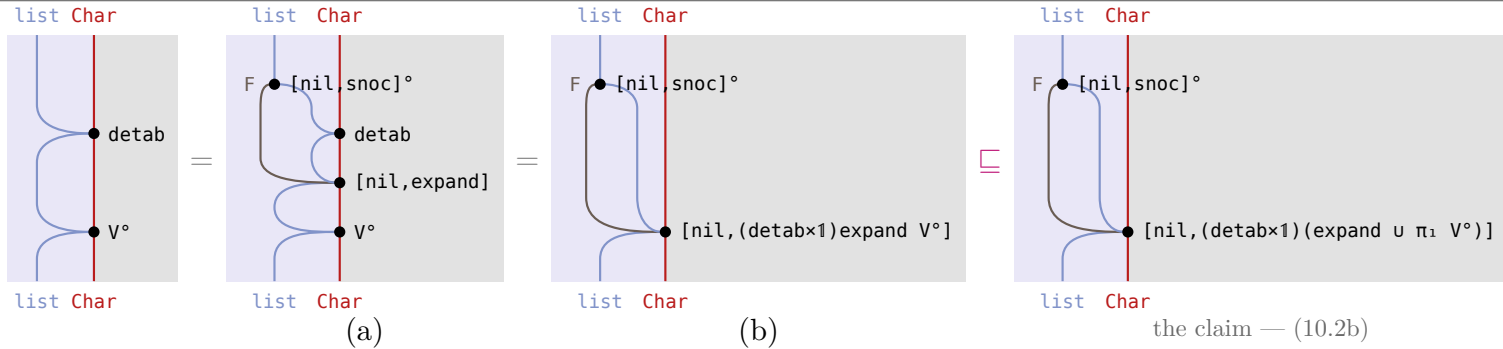
(a)	$\text{istab } \pi_1 \text{ fill } \cup \text{nottab snoc } V^\circ$	$\text{fill } V^\circ = \text{fill}$ (Ex. 10.4)
(b)	$\text{istab } \pi_1 \text{ fill } \cup \text{nottab } (\text{snoc } \cup \pi_1 V^\circ)$	$\text{snoc } V^\circ \sqsubseteq \text{snoc} \cup (\pi_1 V^\circ)$ (Ex. 10.4)
(c)	$\text{expand} \cup \pi_1 V^\circ$	$\text{expand} = \text{istab outl fillunottab snocR}$ — definition of expand; $\text{nottab } R \sqsubseteq R$ — guard dropped

(10.2a)

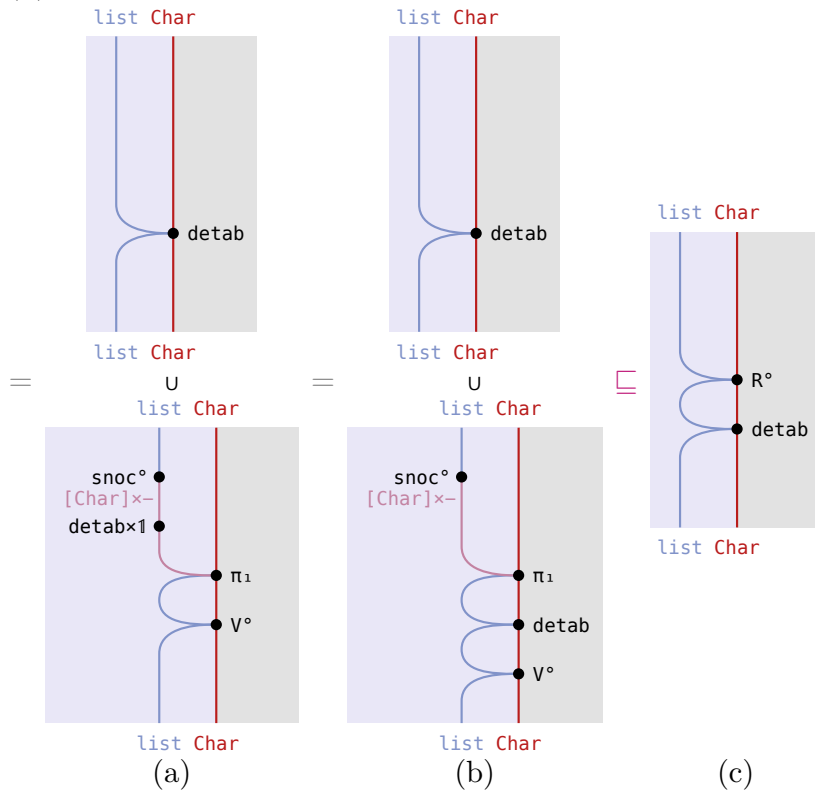
$$0 < n \wedge \text{blank} \neq n1 \Rightarrow \text{detab } V^\circ \sqsubseteq R^\circ \text{detab}$$

(10.2c)

any V -shortening of **detab**'s output is **detab**'s output on an input no longer than the given one



- (a) **detab** is a fold: $\text{detab} = \alpha^\circ F(\text{detab})[\text{nil}, \text{expand}] \triangleq \text{detab}$
 (b) coproducts, and $\text{nil } V^\circ = \text{nil}$ (Exercise 10.4)

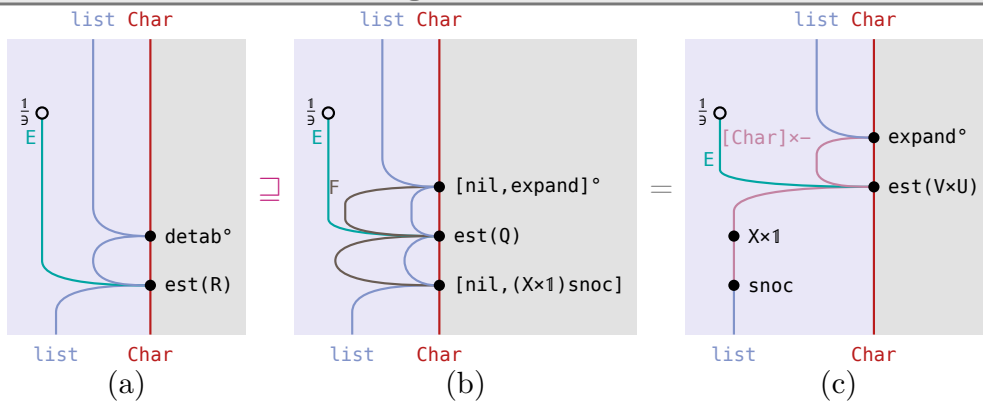


(a)	$\text{detab} \cup \text{snoc}^\circ(\text{detab} \times 1) \pi_1 V^\circ$	distributing $\cup$; the fold again, and the definition of F
(b)	$\text{detab} \cup \text{snoc}^\circ \pi_1 \text{detab } V^\circ$	naturality of π_1 : $(\text{detab} \times 1) \pi_1 = \pi_1 \text{detab}$; $\text{snoc}^\circ \pi_1$ is init
(c)	$R^\circ \text{detab}$	init is inductive, so $X \triangleq \text{detab } V^\circ$, a solution of $X \sqsubseteq \text{detab} \cup (\text{init } X)$, lies below the greatest one, prefix detab (induction on the input); $\text{prefix} \sqsubseteq R^\circ$

$$0 < n \wedge \text{blank} \neq n1 \Rightarrow (\mu X : \frac{[\text{nil}, \text{expand}]^\circ}{3} \text{est}(Q)F(X)[\text{nil}, \text{snoc}]) \sqsubseteq \frac{\text{detab}^\circ}{3} \text{est}(R)$$

(10.2d)

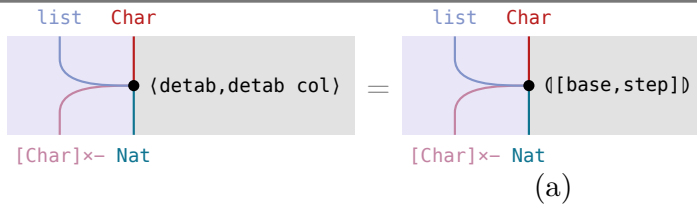
the shortest input **detab** expands to the given output is one pass along that output, holding each blank back and cashing the held blanks in for a tab wherever the column reaches a tab stop



- (a) the specification — $\triangleq \text{detab}; \text{detab entab} = 1$ and nothing shorter does
 (b) at $Q \triangleq 1 + (V \times U)$: one character of input is decided at each step. $F(T, R) \alpha \sqsubseteq \alpha R$. $\text{detab prefix} \sqsubseteq R^\circ \text{detab}$ is FALSE, — at $n=8$, $\text{detab } [a, b, c, d, e, \text{TB}] = [a, b, c, d, e, \text{BL}, \text{BL}, \text{BL}]$, whose prefix $[a, b, c, d, e, \text{BL}, \text{BL}]$ is longer than any input giving it, and $\text{detab } V^\circ \sqsubseteq R^\circ \text{detab}$ holds. $\text{expand } V^\circ \sqsubseteq \text{expand} \cup (\pi_1 V^\circ)$ — shortening the output either leaves the last step alone or discards it
 (c) Proposition 10.1: nil and expand have disjoint ranges. The greedy step is to emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop

$$\text{blank} \neq \text{nl} \wedge \text{tb} \neq \text{nl} \Rightarrow \langle \text{detab}, \text{detab col} \rangle = \langle [\text{base}, \text{step}] \rangle \quad (10.2e)$$

one pass over the input carries the output so far together with its column



(a) both components are folds over the same input; `step` on $(\text{detab } x, \text{col}(\text{detab } x))$ is $(\text{detab}, \text{col detab})$ one character further — not a tabulated row

Exercise 10.1

- $\text{blank} \neq \text{nl} \wedge \text{tb} \neq \text{nl} \Rightarrow \langle [\text{base}, \text{step}] \rangle (\text{convert}(l), r) \Leftrightarrow r = \text{loop}(\text{step}, (([], 0), l))$
the fold of $[\text{base}, \text{step}]$ over the input converted to a snoc-list is the left loop of `step` from `base`

- $\pi_1(\text{loop}(\text{step}, (([], c_0), x))) = \text{loop}'((f, g), (c_0, x))$

Exercise 10.1: when `step` only appends $f(c, a)$ to the output and moves the state by g , the output of the loop is $\text{loop}'(f, g)$, which never carries the output

- $a \neq \text{tb} \Rightarrow \text{expand}(y, b) = \text{snoc}(x, a) \Leftrightarrow \text{fill}(y) = \text{snoc}(x, a) \wedge b = \text{tb} \vee (y, b) = (x, a)$
`expand` produces $x \# [a]$ from a tab after a string that fills to it, or from x and a
- $0 < n \wedge \text{blank} \neq \text{nl} \Rightarrow (\exists y. \text{fill}(y) = \text{snoc}(x, a)) \Leftrightarrow a = \text{blank} \wedge \text{col}(\text{snoc } x \text{ } a) \% n = 0$
a string is a `fill` exactly when it ends in a blank on a tab stop

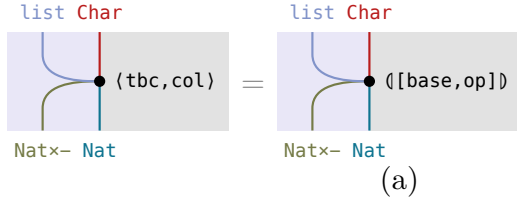
(10.2f)

- $\text{entab}(x) = \text{entab}(\text{unfill } x) ++ \text{blanks}(\text{tbc } x)$
(10.1): the output of `entab` is that of `unfill` x followed by the trailing blanks held back
- $\text{tbc}[] = 0$
the empty string has no trailing blanks
- $\text{tbc}(\text{snoc}(x, a)) = \text{if } a = \text{blank} \wedge \text{col}(\text{snoc } x \text{ } a) \% n \neq 0 \text{ then } \text{tbc } x + 1 \text{ else } 0$
a blank off a tab stop adds one held blank; anything else releases them all

(10.1)

$$\text{blank} \neq \text{nl} \Rightarrow \langle \text{tbc}, \text{col} \rangle = \langle [\text{base}, \text{op}] \rangle \quad (10.2g)$$

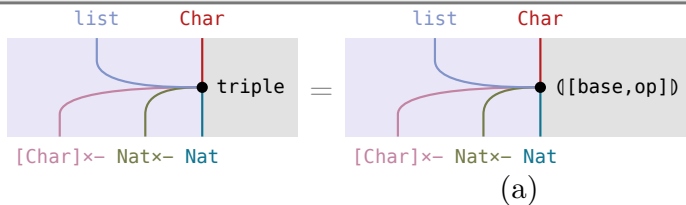
one pass counts the held blanks and the column together



(a) `op` on $(\text{tbc } x, \text{col } x)$ is (tbc, col) one character further, from the equations for `tbc` — (10.1)

$$\text{blank} \neq \text{nl} \Rightarrow \text{triple} = \langle [\text{base}, \text{op}] \rangle \quad (10.2h)$$

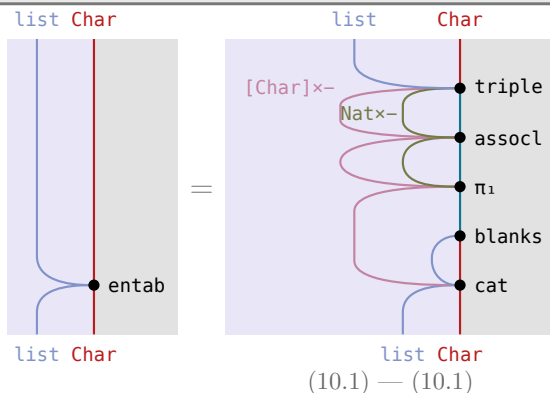
one pass carries the output up to the held blanks, their count and the column



(a) `op` on `triple x` is `triple` one character further: (10.1) where the blanks are released — (10.1)

$$\text{entab} = \text{triple} \text{ assocl } \pi_1 (1 \times \text{blanks}) \text{ cat} \quad (10.2i)$$

`entab` is `triple` with the held blanks appended to its output



§10.3. The minimum tardiness problem

(10.3a)

$FX=1+(X \times \text{Job})$, $\alpha \triangleq [\text{nil}, \text{snoc}]$ on schedules, $\beta \triangleq [\text{nil}, \text{snag}]$ on bags, snag putting a job into a bag; $\text{bagify} \triangleq \langle \beta \rangle : [\text{Job}] \rightarrow \text{Bag Job}$, $H = \text{bagify}^\circ$.

$\text{ct}, \text{dt}, \text{wt} : \text{Job} \rightarrow \text{Real}$ the completion, due and weighting quantities of a job; $\text{penalty}(xs, j) = (\text{sum}(\text{list}(\text{ct}))(xs)) + \text{ct}(j) - \text{dt}(j)) \times \text{wt}(j)$.

$\text{cost} \triangleq \frac{\text{prefix}}{\sup} P(\alpha^\circ [\text{zero}, \text{penalty}]) \text{ est}(\geq)$, $\text{cost} [] = 0$, $\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$, $R \triangleq \text{cost} \leq \text{cost}^\circ$.

$\text{perm} \triangleq \text{bagify } \text{bagify}^\circ = \langle [\text{nil}, \text{add}] \rangle$, $\text{add}(xs, j) = ys \# [j] \# zs$ for some $xs = ys \# zs$.

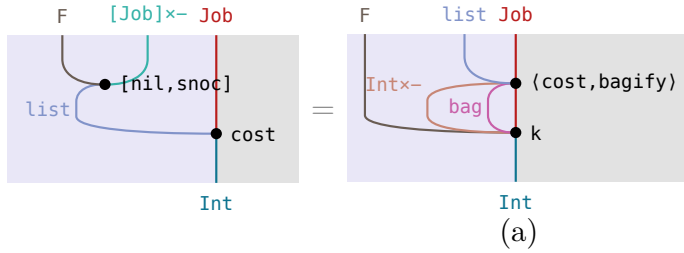
$k \triangleq [\text{zero}, \text{assocr} (1 \times ((\text{bagify}^\circ \times 1) \text{ penalty})) \text{ bmax}]$, $f \triangleq [\text{zero}, (\text{bagify}^\circ \times 1) \text{ penalty}]$, $Q \triangleq f \leq f^\circ$, $Q' \triangleq (\text{bagify}^\circ \times 1) \text{ penalty} \leq \text{penalty}^\circ (\text{bagify} \times 1)$.

$g \triangleq [\text{zero}, \text{penalty}]$ (10.5), $m \triangleq [\text{zero}, \pi_1 \text{ cost}]$ (B&dM's h , renamed as in (10.3e)) (10.6).

$[\text{nil}, \text{snoc}] \text{ cost} = F(\langle \text{cost}, \text{bagify} \rangle) k$

(10.3b)

the cost of a schedule is got from the cost and the bag of the schedule before its last job, and that job

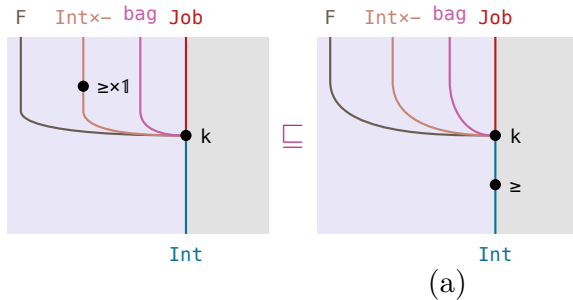


(a) $\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$, and $\text{penalty}(xs, j)$ reads only the bag of xs — $\triangleq \text{bagify}$

$F(\geq \times 1) k \leq k \geq$

Exercise 10.5

a larger cost before the last job gives a larger cost after it

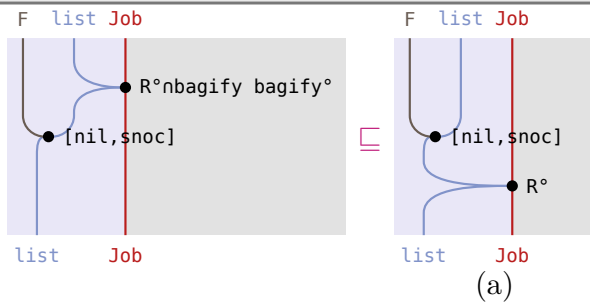


(a) bmax is monotone in its first argument

$F(R^\circ \text{nbagify } \text{bagify}^\circ) [\text{nil}, \text{snoc}] \leq [\text{nil}, \text{snoc}] R^\circ$

(10.2)

improving the schedule before the last job, among schedules of the same bag, improves the whole schedule

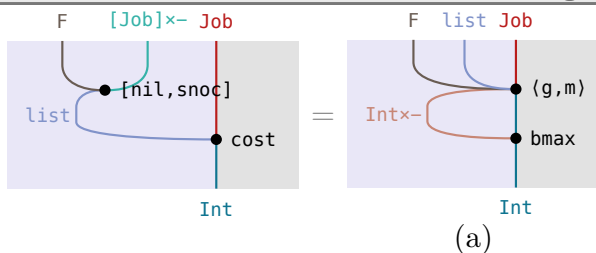


(a) Proposition 9.3 at $S \triangleq \text{bagify}$, $\leq \triangleq$ — Proposition 9.3, with (10.3b) and Exercise 10.5

$[\text{nil}, \text{snoc}] \text{ cost} = \langle g, m \rangle \text{bmax}$

(10.4)

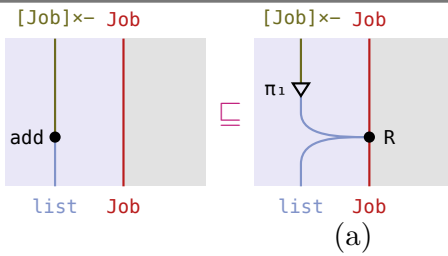
the cost of a schedule is the larger of the penalty of its last job and the cost of the schedule before it



(a) $\text{cost}(xs \# [j]) = \text{bmax}(\text{cost } xs, \text{penalty}(xs, j))$ and bmax commutes — $\triangleq \text{bagify}$

$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow \text{add} \sqsubseteq \pi_1 R \quad (10.7)$$

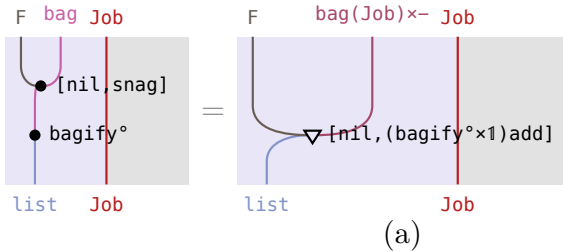
putting one more job anywhere into a schedule never lowers its cost



(a) induction on where j goes: every job after it starts later, so no penalty falls

$$[\text{nil}, \text{snag}] \text{bagify}^\circ = [\text{nil}, (\text{bagify}^\circ \times 1) \text{add}] \quad (10.8)$$

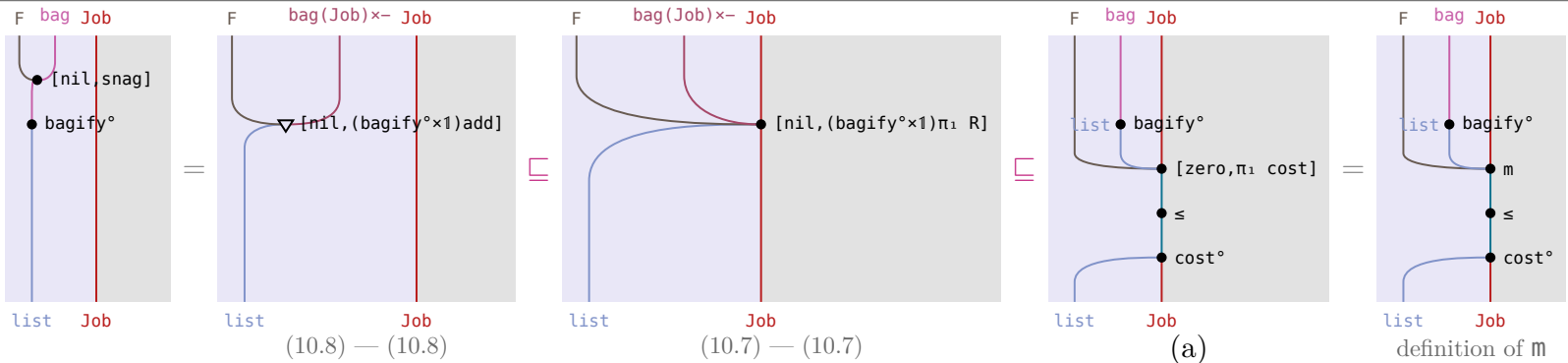
the orderings of a bag with one more job are the orderings of the bag with that job put in anywhere



(a) take out the last copy of j and put it back $\triangleq \text{bagify}$

$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow [\text{nil}, \text{snag}] \text{bagify}^\circ \sqsubseteq F(\text{bagify}^\circ) m \leq \text{cost}^\circ \quad (10.3c)$$

an ordering of a bag with one more job costs at least the cost of the ordering of the bag before its last job

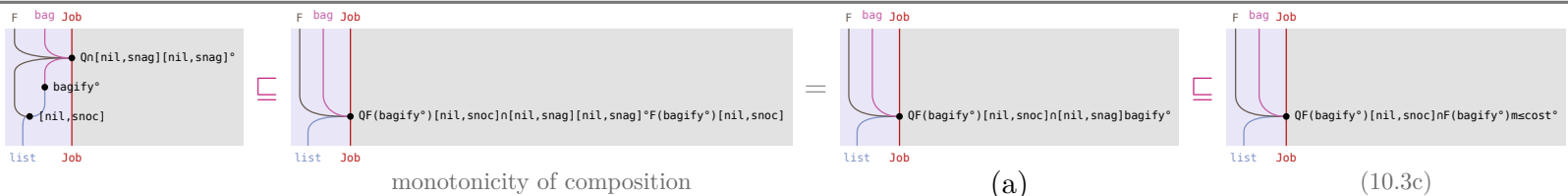


(a) definition of R , and $\text{nil} \sqsubseteq \text{zero} \leq \text{cost}^\circ$

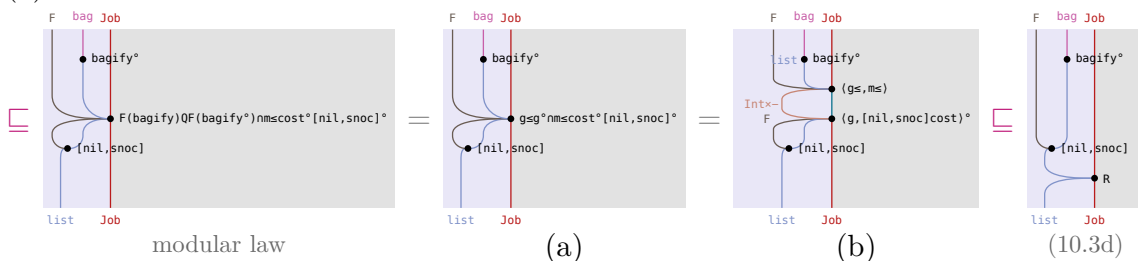
$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow (Qn[\text{nil}, \text{snag}][\text{nil}, \text{snag}]^\circ) F(\text{bagify}^\circ) [\text{nil}, \text{snoc}] \sqsubseteq F(\text{bagify}^\circ) \quad (10.3)$$

$$[\text{nil}, \text{snoc}] R$$

a schedule of a bag ending in a job of least penalty is no worse than any schedule of the same bag



(a) $\beta^\circ F(\text{bagify}^\circ) \alpha = \text{bagify}^\circ$, since $\text{bagify} = (\beta)$

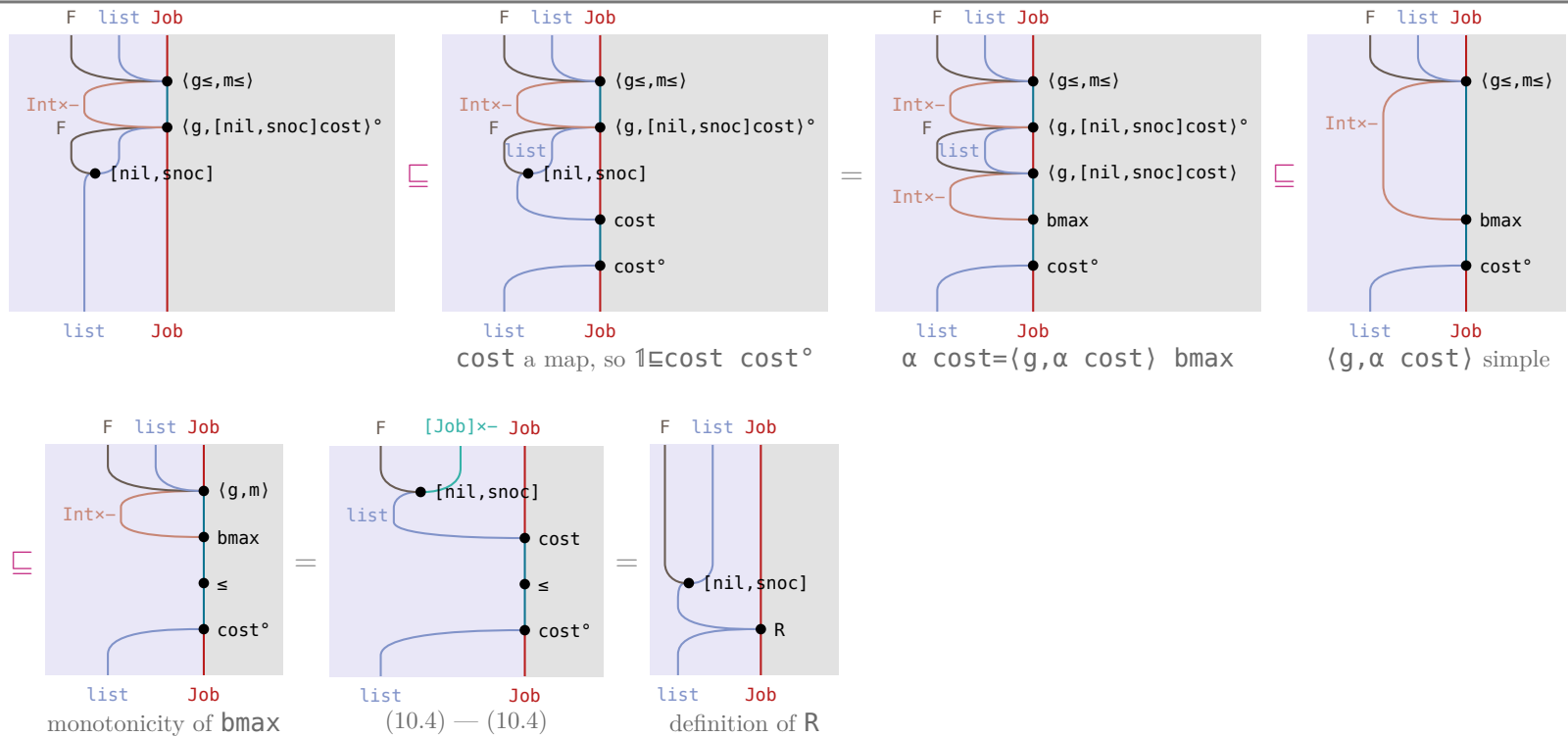


(a) choice of Q : $F(\text{bagify}) \ Q \ F(\text{bagify}^\circ) = g \leq g^\circ$

(b) products: $\langle R, S \rangle \langle T, U \rangle^\circ = RT^\circ \circ nSU^\circ$

$$\langle g \leq, m \leq \rangle \langle g, [\text{nil}, \text{snoc}] \text{cost} \rangle^\circ [\text{nil}, \text{snoc}] \in [\text{nil}, \text{snoc}] R \quad (10.3d)$$

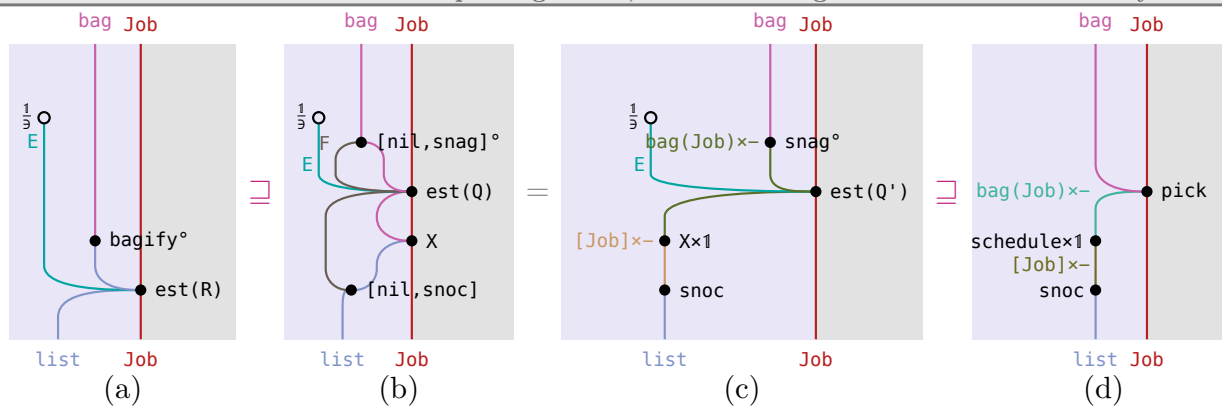
a job whose penalty is at most the last penalty, put after a schedule costing at most the cost before the last job,
gives a schedule costing at most the whole



$$(\forall j. 0 \leq \text{ct}(j)) \wedge (\forall j. 0 \leq \text{wt}(j)) \Rightarrow (\mu X : \frac{[\text{nil}, \text{snag}]^\circ}{\exists} \text{est}(Q)F(X)[\text{nil}, \text{snoc}] \sqsubseteq \frac{\text{bagify}^\circ}{\exists} \text{est}(R), \quad (10.3e)$$

$$\text{schedule} = (\text{null} \rightarrow \text{nil}, \text{pick}(\text{schedule} \times 1) \text{ snoc})$$

an ordering of the given bag with least maximum penalty is got by taking a job of least penalty out of the bag, putting it last, and scheduling what is left the same way



- (a) the specification $\triangleq \mathbf{bagify}$
- (b) No greedy **reduce** exists — one would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole
- (c) Proposition 10.1: **nil** and **snag** have disjoint ranges
- (d) $\mathbf{pick} \sqsubseteq_{\frac{\mathbf{snag}^\circ}{3}} \mathbf{est}(Q')$, a partial function, quadratic in the number of jobs

§10.4. The TeX problem

(10.4a)

```
internAval round : Decimal $\rightarrow$ [0,216), valA([zero,shift]), shift (d,r)=(d+r)/10, round r rounds 216r to the nearest integer: round r=n $\Leftrightarrow$ 2n-1<217r<2n+1.
```

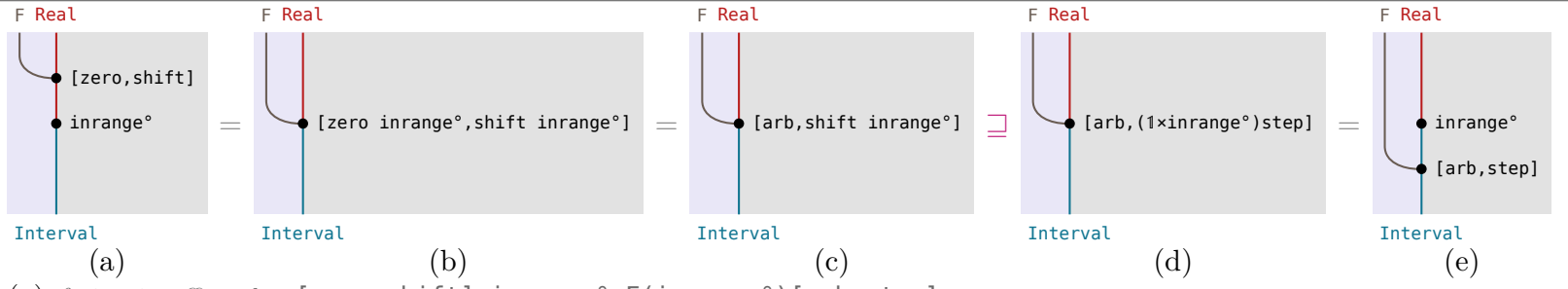
interval $n=((2n-1)/2^{17}, (2n+1)/2^{17})$, $r \text{ inrange } (a,b) \iff a < r < b$, $\text{round}^\circ = \text{interval inrange}$, $R \triangle \text{length} \leq \text{length}^\circ$.

Interval the pairs (a,b) with $\text{Legal}(a,b) \iff 0 < b < 1$ and $a < b$ (10.9); `step_legal` is what types `step [arb,step] : 1+(Digit×Interval)→Interval`, `step (d,(a,b))=((d+a)/10,(d+b)/10)`.

$FX = 1 + (\text{Digit} \times X)$, $\alpha \in [\text{nil}, \text{cons}]$, $H \in ([\text{arb}, \text{step}])^\circ$, $! : \text{Digit} \times \text{Interval} \rightarrow 1$, $Q \in (l^\circ !^\circ r) \cup 1$ l, r are (5.3a)'s injections into $FX = 1 + (\text{Digit} \times X)$, so $l : 1 \rightarrow FX$ and $r : \text{Digit} \times X \rightarrow FX$, $w \in 2^{17}$.

$([arb, step]) \text{Eval } \text{inrange}^\circ$

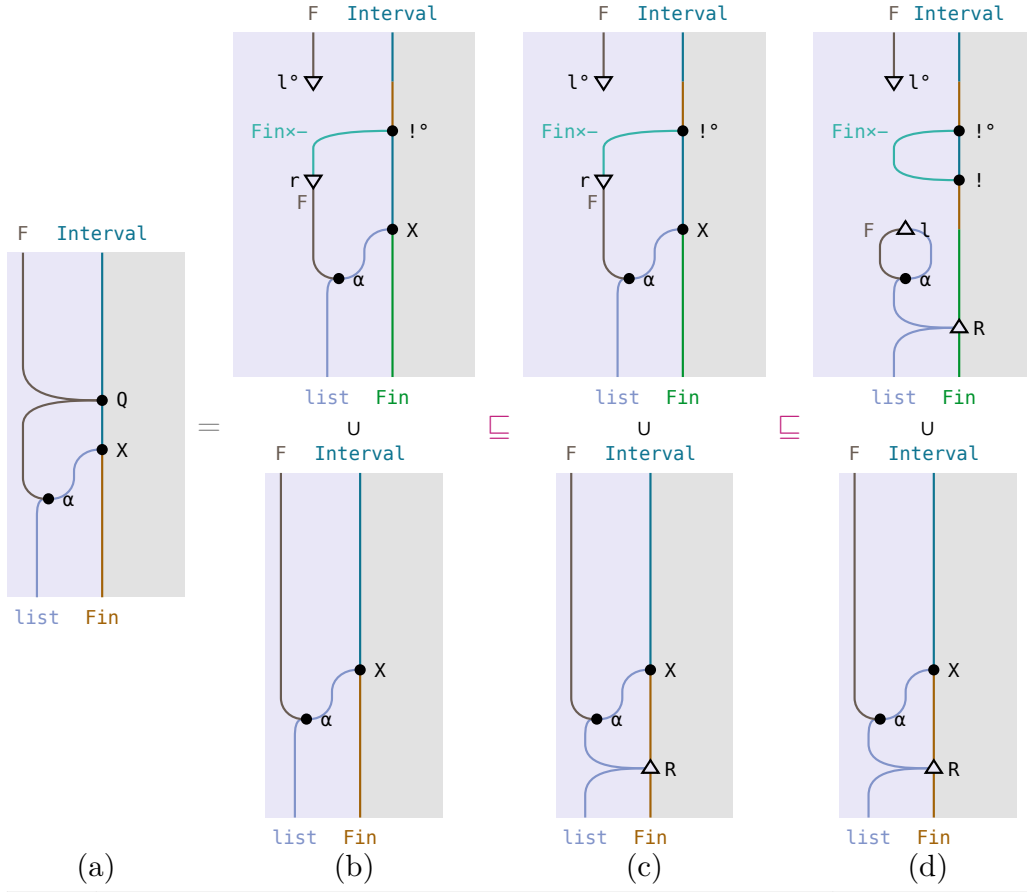
(10.4b)

every interval got by folding a decimal's digits with $[arb, step]$ has the decimal's value strictly inside it(a) fusion: it suffices that $[zero, shift] \text{ inrange}^\circ \sqsupseteq F(\text{inrange}^\circ)[arb, step]$ (b) $[T, U]Z = [TZ, UZ]$, a coproduct law not tabulated in the note(c) $zero \text{ inrange}^\circ = arb$: the first condition, which determines arb (d) arithmetic: $10a - d < r < 10b - d \implies a < (d+r)/10 < b$ for $(a, b) = \text{step}(d, (10a-d, 10b-d))$; only $\exists$, since $(10a-d, 10b-d)$ satisfies (10.9) only when $d < 10b < d+1$ (e) $F(S)[T, U] = [T, (1 \times S)U]$ read right to left: definition of F

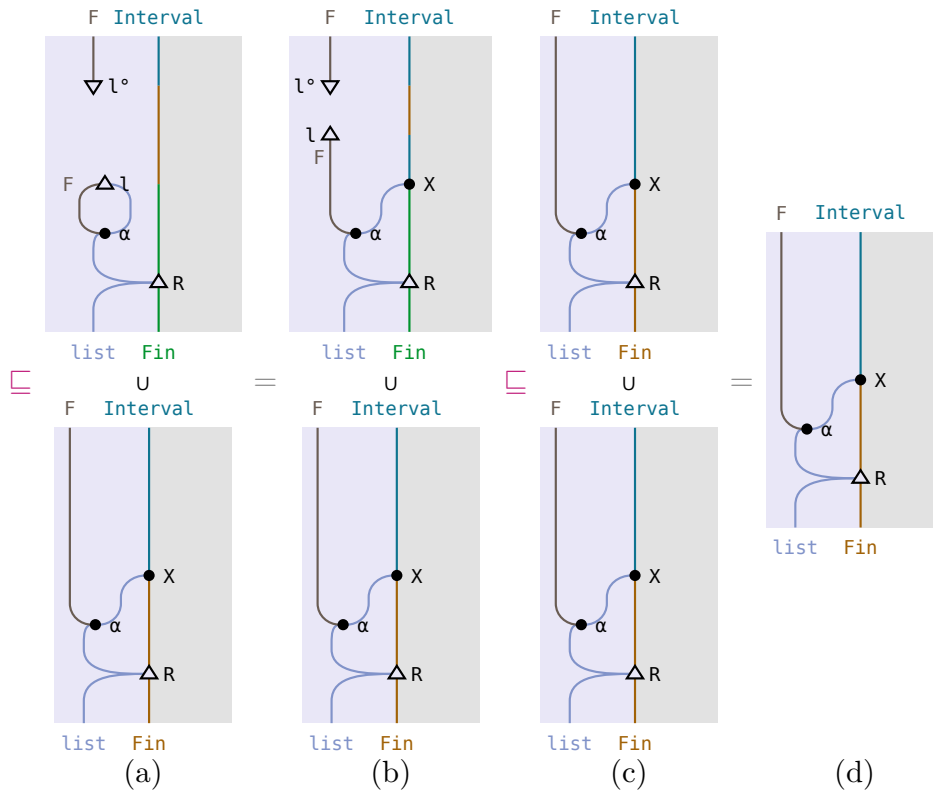
$$QF(X)\alpha \sqsubseteq F(X)\alpha R$$

(10.4c)

choosing by Q among the one-step decompositions before building with $F(X)$ and α yields only decimals that are R -related to one built without choosing



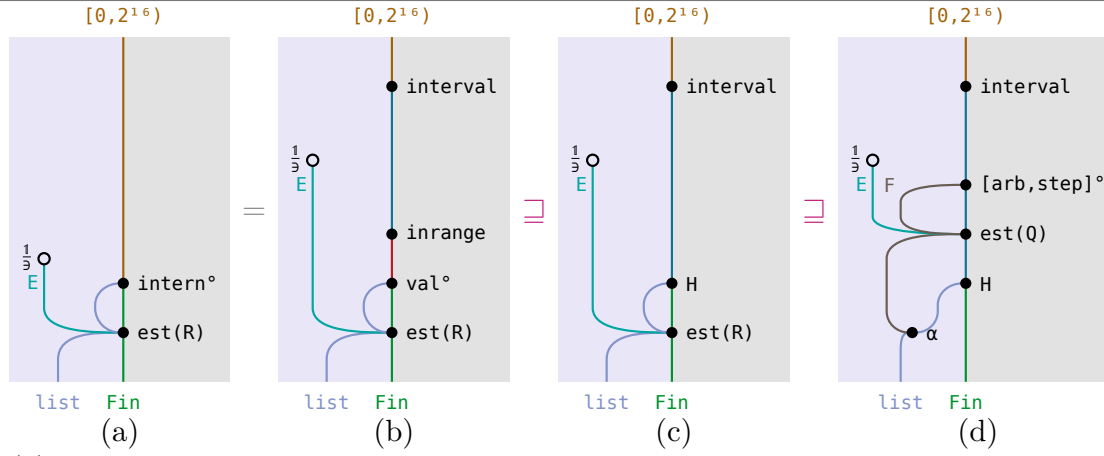
(a)	$QF(X)\alpha$	
(b)	$l^\circ !^\circ r F(X)\alpha \sqcup F(X)\alpha$	$(S \sqcup T)U = SU \sqcup TU$ — definition of Q ; composition distributes over U
(c)	$l^\circ !^\circ r F(X)\alpha \sqcup F(X)\alpha R$	$1 \sqsubseteq R$ — R is reflexive
(d)	$l^\circ !^\circ ! l \alpha R \sqcup F(X)\alpha R$	$r F(X)\alpha \sqsubseteq ! l \alpha R$ — $l \alpha = \text{nil}$, and $\text{length}(\text{nil}) = 0$ is at most any length



(a)	$l^\circ l \alpha R \sqcup F(X)\alpha R$	$!^\circ ! \sqsubseteq 1$ on 1 — universal property of $!$
(b)	$l^\circ l F(X)\alpha R \sqcup F(X)\alpha R$	$l F(X) = l$ — definition of F
(c)	$F(X)\alpha R \sqcup F(X)\alpha R$	$l^\circ l \sqsubseteq 1$ — l is simple
(d)	$F(X)\alpha R$	$S \sqcup S = S$ — U is idempotent

$$\text{interval}(\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q)F(X)\alpha) \sqsubseteq \frac{\text{intern}^\circ}{3} \text{est}(R) , \quad \text{extern}(n)=f(2n-1, 2n+1) \quad (10.4d)$$

a shortest decimal whose internal representation is the given multiple of 2^{-16} is got by emitting the one digit the interval of admissible reals allows, until that interval contains zero and the empty decimal will do



(a) the specification — $\triangleq$ **intern**

(b) **round**[°] is not a map, but **interval** is, so it comes out of the transpose

(c) the type restriction (10.9): a shortest decimal **H** gives an interval is a shortest one among all decimals inside it, and is inside it by (10.4b)

(d) $\frac{[\text{arb}, \text{step}]^\circ}{3}$ returns at most two elements — stop, or take one more digit — and ! **nil** $\sqsubseteq$ **cons** R° makes it stop whenever stopping is legal

- $f(a,b)=x \iff a < 0 \wedge x=[] \vee \neg a < 0 \wedge (\exists d. \exists y. d = \lfloor 10b \rfloor \wedge f(10a-d, 10b-d)=y \wedge x=[d]\#y)$
f gives (a,b) the empty decimal when $a < 0$, and otherwise the digit $d = \lfloor 10b \rfloor$ followed by what it gives $(10a-d, 10b-d)$

(10.4e)

- $(\mu X : \frac{[\text{arb}, \text{step}]^\circ}{3} \text{est}(Q)F(X)\alpha) = f$

the least solution of the greedy recursion of (10.4d) is f

- $d = \lfloor 10b \rfloor \wedge e = \lfloor 10b \rfloor \implies d=e$

Exercise 10.12: two digits d and e with $0 < 10b-d < 1$ and $0 < 10b-e < 1$ are equal

- $(0 < q-p \wedge q < w) \wedge p < 0 \implies f(p,q)=[]$

the program's first clause: $f(p,q)=[]$ when $p < 0$

- $(0 < q-p \wedge q < w) \wedge \neg p < 0 \wedge d = \lfloor 10q/w \rfloor \implies f(p,q)=[d]\#f(10p-wd, 10q-wd)$

the program's second clause: with $d = (\lfloor 10q \rfloor) \text{ div } w$, $f(p,q)$ is d followed by $f(10p-w \cdot d, 10q-w \cdot d)$

- $(0 < q-p \wedge q < w) \wedge \text{Reach}(p,q) \implies f(p/w, q/w) = f(p,q)$

on the pairs the program reaches from **interval** n, the integer f at (p,q) is a value of the rational f at $(p/w, q/w)$

- **interval** f = **extern**

f after **interval** is the program **extern**, whose value at n is $f(2n-1, 2n+1)$

§11. Appendix

§11.1. $P(S) \text{ est}(R) = (\exists S) n(\epsilon \setminus (SR^\circ))$

(11.1a)

(p, q) tabulates $W \triangleq (\exists S) n(\epsilon \setminus (SR^\circ))$, and $y \triangleq \frac{(p \exists S) n(qR)}{\exists}$, a map.

(11.1b)

$$\begin{array}{c} \boxed{\begin{array}{c} W \\ P(S) \text{ est}(R) \end{array}} \xleftarrow{p^\circ q = W, 1 \sqsubseteq yy^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \\ \boxed{\begin{array}{c} y^\circ q \\ \text{est}(R) \end{array}} \end{array}$$

(11.1c)

$$\begin{array}{c} \boxed{\begin{array}{c} y^\circ q \\ \exists \end{array}} \xleftarrow{f^\circ \cdot \neg f^\circ, \cdot \exists \neg \exists} \boxed{\begin{array}{c} q \\ (p \exists S) n(qR) \end{array}} \xleftarrow{\Delta \neg n, f^\circ \cdot \neg f^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ q \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} 1 \\ R \end{array}} \end{array}$$

R reflexive

(11.1d)

$$\begin{array}{c} \boxed{\begin{array}{c} \epsilon y^\circ q \\ R^\circ \end{array}} \xleftarrow{\cdot \exists \neg \exists, \circ, \text{meet}} \boxed{\begin{array}{c} R^\circ q^\circ q \\ R^\circ \end{array}} \quad \text{q simple} \\ \\ \boxed{\begin{array}{c} p^\circ y \\ P(S) \end{array}} \xleftarrow{\Delta \neg n, \cdot T \neg / T, \circ, f^\circ \cdot \neg f^\circ} \begin{array}{c} \boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \\ \boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \end{array} \\ \\ \boxed{\begin{array}{c} p^\circ y \exists \\ \exists S \end{array}} \xleftarrow{\cdot \exists \neg \exists, \text{meet}} \boxed{\begin{array}{c} p^\circ p \exists S \\ \exists S \end{array}} \quad \text{p simple} \\ \\ \boxed{\begin{array}{c} p \exists \\ y \exists S^\circ \end{array}} \xleftarrow{\text{modular law}} \boxed{\begin{array}{c} p \exists \\ (p \exists) n(qRS^\circ) \end{array}} \xleftarrow{1 \sqsubseteq qq^\circ} \boxed{\begin{array}{c} q^\circ p \exists \\ RS^\circ \end{array}} \xleftarrow{\circ, T \cdot \neg T \setminus} \boxed{\begin{array}{c} W \\ \epsilon \setminus (SR^\circ) \end{array}} \quad \text{W's right half} \end{array}$$

§11.2. $P(\text{est}(R)) \text{ est}(R) = P(\text{dom}(\text{est}(R))) \text{ union est}(R)$

(11.2a)

$$\begin{array}{c} \boxed{P(\text{est}(R)) \text{ est}(R)} \xrightarrow{\text{dom}(\text{est}(R)) \text{ est}(R) = \text{est}(R)} \boxed{P(\text{dom}(\text{est}(R)) \text{ est}(R)) \text{ est}(R)} \\ \\ \boxed{P(\text{dom}(\text{est}(R))) P(\text{est}(R)) \text{ est}(R)} \xrightarrow{(7.11)} \boxed{P(\text{dom}(\text{est}(R))) \text{ union est}(R)} \end{array}$$

P a relator

(11.2b)

$$\begin{array}{c} \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ P(\text{est}(R)) \text{ est}(R) \end{array}} \xleftarrow{(11.1b) \text{ at } S := \text{est}(R), \Delta \neg n, T \cdot \neg T \setminus} \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \exists \text{est}(R) \end{array}} \\ \\ \boxed{\begin{array}{c} P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \exists \text{est}(R) \end{array}} \xleftarrow{P(\text{dom}(\text{est}(R))) \sqsubseteq 1} \boxed{\begin{array}{c} \text{union est}(R) \\ \exists \text{est}(R) \end{array}} \\ \\ \boxed{\begin{array}{c} (\exists \exists) n(\epsilon \setminus (\epsilon \setminus R^\circ)) \\ \exists \text{est}(R) \end{array}} \xleftarrow{\text{modular law}} \boxed{\begin{array}{c} \exists (\exists n(\epsilon \setminus (\epsilon \setminus R^\circ))) \\ \exists (\exists n(\epsilon \setminus R^\circ)) \end{array}} \quad \text{count of } T \cdot \neg T \setminus \\ \\ \boxed{\begin{array}{c} \epsilon P(\text{dom}(\text{est}(R))) \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \xleftarrow{\epsilon \text{ lax natural}} \boxed{\begin{array}{c} \text{dom}(\text{est}(R)) \epsilon \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \\ \\ \boxed{\begin{array}{c} \text{est}(R) \text{ est}(R)^\circ \epsilon \text{ union est}(R) \\ \text{est}(R) R^\circ \end{array}} \xleftarrow{\text{dom}(\text{est}(R)) \sqsubseteq \text{est}(R) \text{ est}(R)^\circ} \boxed{\begin{array}{c} \text{est}(R) \epsilon \text{est}(R) \\ \text{est}(R) R^\circ \end{array}} \quad \text{UP of est} \end{array}$$

§11.3. Shortest paths on a cylinder, on lists

(11.3a)

definition	type	note
$F(A, X) = A + A \times X$	$\mathcal{A} \rightarrow \mathcal{A}$	The base functor: one square of the new column, alone or in front of the path so far.
$L = \text{list}^+$	$\mathcal{A} \rightarrow \mathcal{A}$	A path is a non-empty list of squares, one per column crossed.
α the initial algebra	$F(\text{dCL}(L, E)) \rightarrow \text{dCL}(L, E)$	$\alpha(5) = [5]$ and $\alpha(1, [5]) = [1, 5]$: a path is started by one square, or extended by one.
N the n-tuple relator	$\mathcal{A} \rightarrow \mathcal{A}$	One component per row of a column, so the fold carries n answers at once.
$R \triangleq \text{sum} \leq \text{sum}^\circ$	$[\text{Nat}] \rightarrow [\text{Nat}]$	The cost of a path, which the cheapest minimises.
setify	$A[n] \rightarrow EA$	$\text{setify}(1, 2, 3, 4) = \{1, 2, 3, 4\}$ — which row a component came from is forgotten.
moves	$A[n] \rightarrow E(A[n])$	$\text{moves}(x) = \{\text{up}(x), x, \text{down}(x)\}$ — rotated up, unrotated, rotated down.
trans	$E(A[n]) \rightarrow (E A)[n]$	$\text{trans}\{(a, b, c), (x, y, z)\} = (\{a, x\}, \{b, y\}, \{c, z\})$ — component k of the result is the set of the k-th components.
zip	$F(a[n]) \rightarrow (F a)[n]$	$\text{zip}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) = ((1, \{[5]\}), (2, \{[6]\}), (3, \{[7]\}), (4, \{[8]\}))$.
$\text{cp} \triangleq \frac{F(1, \exists)}{\exists}$	$F(EA) \rightarrow EFA$	$\text{cp}(1, \{[5], [6], [8]\}) = \{(1, [5]), (1, [6]), (1, [8])\}$.
$\text{gen} \triangleq F(1, \text{moves trans } N(\text{union})) \text{zip } N(\text{cp } E(\alpha))$	$F(NA, N(E(TA))) \rightarrow N(E(TA))$	$\text{gen}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\}))$ is worked out in (11.3.1b).
$\text{paths} \triangleq (\text{gen}) \text{setify union}$	$T(NA) \rightarrow E(TA)$	$\text{paths}[(1, 2, 3, 4), (5, 6, 7, 8)]$ is the union of (11.3.1b)'s four sets: 12 paths, 3 from each entry row.
the specification $\text{paths est}(R)$	$[\text{Nat}[n]] \rightarrow [\text{Nat}]$	A cheapest path from the entry side to the exit side.

§11.3.1. $N(E(L A))$

$$\begin{array}{l} 1 \quad 5 \\ 2 \quad 6 \\ 3 \quad 7 \\ 4 \quad 8 \end{array} \quad [(1, 2, 3, 4), (5, 6, 7, 8)] : L \ N \ \text{Nat}$$

(11.3.1a)

$(\text{gen})[(5, 6, 7, 8)] = (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})$

(11.3.1b)

$$\begin{aligned} \text{gen}((1, 2, 3, 4), (\{[5]\}, \{[6]\}, \{[7]\}, \{[8]\})) = \\ (\{ [1, 5], [1, 6], [1, 8] \}, \\ \{ [2, 5], [2, 6], [2, 7] \}, \\ \{ [3, 6], [3, 7], [3, 8] \}, \\ \{ [4, 5], [4, 7], [4, 8] \}) \end{aligned}$$

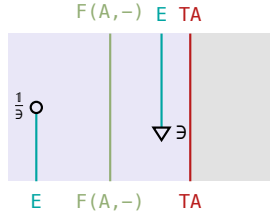
(11.3.1c)

N	the 4-tuple, one component per row of the entry column
E	the set of paths that can start in that row
L	a path is the non-empty list of the squares it crosses

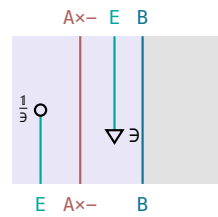
row 1's set has no $[1,7]$ — from row 1 only rows 4, 1 and 2 are reachable — and it does have $[1,8]$, because the cylinder glues the bottom row to the top. That is **moves trans N(union)** of $\triangleq$ **paths**: component k collects the paths of rows $k-1, k, k+1$.

§11.3.2. The cross product $cp \triangleq \frac{F(1, \exists)}{\exists}$

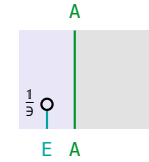
(11.3.2a)



cp on all of $F(A, E(L A))$



the $A \times -$ summand: $\exists$ picks one path, $1\% \exists$ collects the results



the A summand, no E to distribute: $1\% \exists$ alone, $a \mapsto \{a\}$

(11.3.2b)

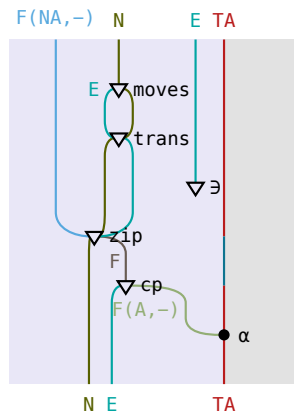
type	note
$F(A, -) = A + A \times -$	one wire carries the whole functor, both summands with it; the $\exists$ under it is $F(1, \exists)$
$\exists : E(L A) \rightarrow L A$	the $\exists$ inside $F(1, \exists)$: a set of paths, one of them — $L A$ is one path, A one square
$F(1, \exists) = 1 + 1 \times \exists$	$: A + A \times E(L A) \rightarrow A + A \times L A$, the relator acting on each summand
$\exists : E(A + A \times L A) \rightarrow A + A \times L A$	the $\exists$ under the bar: a set of cells, one of them — a different $\exists$
$cp : A + A \times E(L A) \rightarrow E(A + A \times L A)$	$\frac{R}{\exists}$ turns $R : X \rightarrow Y$ into $X \rightarrow E Y$
$cp = \frac{1}{\exists} E(F(1, \exists))$	what the picture draws: the unit makes the outer E , leaving one $\exists$ — (3.1.1.10a)

(11.3.2c)

$w = (1, \{[5], [6], [8]\}) : A \times E(L A)$ the right summand
 $1 \times \exists : A \times E(L A) \rightarrow A \times L A$ 1 keeps the square 1 , $\exists$ picks one path
 $w \mapsto (1, [5]), (1, [6]), (1, [8]) : A \times L A$ one output per path in w
 $cp(w) = \{(1, [5]), (1, [6]), (1, [8])\} : E(A + A \times L A)$ the three of them, collected

§11.3.3. $gen = F(1, \text{moves trans } N(\text{union})) \text{ zip } N(cp P(\alpha))$

(11.3.3a)



(11.3.3b)

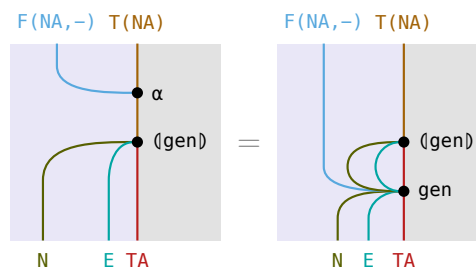
```

u = ((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) : F(N A,N(E(L A)))
F(1,moves trans N(union))                  1 keeps the column, the path SETS move
  moves({[5]},{[6]},{[7]},{[8]})
    = ({[6]},{[7]},{[8]},{[5]}),
      ({[5]},{[6]},{[7]},{[8]}),
      ({[8]},{[5]},{[6]},{[7]})}          : E(N(E(L A)))  down, unmoved, up
  trans(that)
    = ({[6]},{[5]},{[8]},{[7]},{[6]},{[5]}},
      {[8]},{[7]},{[6]},{[5]},{[8]},{[7]}) : N(E(E(L A)))  row k gets rows k-1, k, k+1
  N(union)(that)
    = ({[5],[6],[8]},{[5],[6],[7]},
      {[6],[7],[8]},{[5],[7],[8]})        : N(E(L A))      every path into row k, none dropped
zip(that)
  = ((1,{[5],[6],[8]}),(2,{[5],[6],[7]}),
    (3,{[6],[7],[8]}),(4,{[5],[7],[8]})) : N(F(A,E(L A)))  each row: its square, and the
                                                                paths it may be put in front of
N(cp P(α))(that)
  = ({[1,5],[1,6],[1,8]},{[2,5],[2,6],[2,7]},
    {[3,6],[3,7],[3,8]},{[4,5],[4,7],[4,8]}) : N(E(L A))    cp pairs the square with each path,
                                                                α prefixes it: α(1,[5])=[1,5]

```

§11.3.4. gen is an F-algebra; (gen): α(gen)=F(1,(gen))gen

(11.3.4a)



α puts the column back on the list, then the fold reads all of it

the fold reads the rest under F, then one gen puts the column in front

$$xs = [(1,2,3,4),(5,6,7,8)] = \alpha((1,2,3,4),[(5,6,7,8)]) : L(N A)$$

(11.3.4b)

$$(\text{gen})(xs) = \text{gen}((1,2,3,4), (\text{gen})[(5,6,7,8)])$$

both halves are in (11.3.1b): the tail folds to $(\{[5]\},\{[6]\},\{[7]\},\{[8]\})$, one path per row and each of them one square long, and gen on it is (11.3.3b)'s walk.

§11.3.5. Q : F(N A,N(L A))→N(L A), an F(N A,-)-algebra: Q=F(1,moves trans N(est(R))) zip N(α)

(11.3.5a)

```

Q : F(N A,N(L A)) → N(L A)                the fold's algebra: a new column, one path per row
u = ((1,2,3,4),([5],[6],[7],[8]))          : F(N A,N(L A))
F(1,moves trans N(est(R)))                  1 keeps the column, the paths move
  moves([5],[6],[7],[8])
    = ({[6],[7],[8],[5]},
      {[5],[6],[7],[8]},
      {[8],[5],[6],[7]})}                  : E(N(L A))      down, unmoved, up
  trans(that)
    = ({[6],[5],[8]},{[7],[6],[5]},
      {[8],[7],[6]},{[5],[8],[7]})        : N(E(L A))      row k gets rows k-1, k, k+1
  N(est(R))(that)
    = ([5],[5],[6],[5])                    : N(L A)          the cheapest into each row –
                                                                chosen BEFORE the new square
zip(that)
  = ((1,[5]),(2,[5]),(3,[6]),(4,[5]))      : N(F(A,L A))    each row: its square, and the one
                                                                predecessor that survived
N(α)(that)
  = ([1,5],[2,5],[3,6],[4,5])              : N(L A)          α(1,[5])=[1,5]

```

$Q = [N(\text{wrap}), (1 \times \text{moves trans } N(\text{est}(R))) \text{ zip}' N(\text{cons})]$ (11.3.5b)	
at the last column Q starts one path per row, and at each earlier one it puts each square in front of the cheapest of the three kept paths it can step to — the algebra (11.3.5c)'s last step folds	
circuit — the fork is $F(NA, N(LA)) = NA + NA \times N(LA)$	Hinze–Marsden
$F(N A) (N (T A)) \text{---} \boxed{Q} \text{---} N(T A)$	
$= FN(T A) \text{---} \boxed{F(1, \text{moves trans } N(\text{est}(R)))} \text{---} FN(T A) \text{---} \boxed{\text{zip}} \text{---} N(F A (T A)) \text{---} \boxed{N(\alpha)} \text{---} N(T A)$ the fusion condition (11.3.5d) read as a definition,	
$= F(N A) (N (T A)) \text{---} \left\{ \begin{array}{l} N(p_{\circ}) \text{---} \boxed{N(\text{wrapA})} \\ q \text{---} \boxed{\text{pm}} \text{---} q \text{---} \boxed{\text{zip}'} \text{---} N(q_{\circ}) \text{---} \boxed{N(\text{consA})} \end{array} \right\} \text{---} N(T A)$ $\text{zip} = 1 + \text{zip}', \alpha = [\text{wrap}, \text{cons}]$	

